
УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ
ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ, ВСЕ

Дата генерации: 14.09.2026, 14:55:19
Файлов исходного кода: 96
Суммарный объём кода: 1.41 МВ
Глав/билетов в пособии: 30

ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ

1. 1. Дерево отрезков с операциями на отрезках [id: seg-tree]
2. 2. Двумерная разреженная матрица (Sparse Table)
3. 3. Декартово дерево (Treap) [id: treap]
4. 4. Splay-дерево [id: splay-tree]
5. 5. Двумерная матрица префиксных сумм [id: pref-sum]
6. Планарность: K5, K3,3 и формула Эйлера [id: planarity]
7. 6. Графы. Представление, DFS, BFS [id: graph-dfs]
8. 7. Графы. Планарные. Покраска [id: graph-planarity]
9. 8. Графы. Компоненты связности [id: graph-components]
10. 9. Графы. Топологическая сортировка [id: topological-sort]
11. 10. Графы. Компоненты сильной связности (SCC)
12. 11. Графы. Мосты [id: graph-bridges]
13. 12. Графы. Точки сочленения [id: graph-articulation-points]
14. 13. Графы. Эйлеров цикл [id: graph-euler]
15. Билет 14. Дейкстра [id: dijkstra]
16. Билет 15. Форд-Беллман [id: bellman-ford]
17. Билет 16. Флойд-Уоршелл [id: floyd]
18. 17. Графы. Ускорения поиска [id: pathfinding-accelerations]
19. 17. Графы. Алгоритм Джонсона (Фокус с перевзвешиванием)
20. 18. Графы. Остовное дерево. Краскал [id: mst-kruskal]
21. 19. Графы. Остовное дерево. Прима [id: mst-prims]
22. 20. Графы. Остовное дерево. Борушка [id: mst-borushka]
23. 21. Алгоритм Кнута-Морриса-Патта (KMP) [id: kmp]
24. 22. Z-функция [id: string-z-func]
25. Билет 23. Алгоритм Ахо-Корасик [id: aho-corasick]

24. src/data/vizStepBus.ts
25. src/data/vizSync.ts

Билеты (учебный контент)

26. src/data/tickets/ticket1_5.ts
27. src/data/tickets/ticket14_20.ts
28. src/data/tickets/ticket21_24.ts
29. src/data/tickets/ticket6_13.ts

Интерактивные компоненты и симуляторы

30. src/components/ArticulationPointsViz.tsx
31. src/components/BellmanFordViz.tsx
32. src/components/BfsViz.tsx
33. src/components/ChapterImage.tsx
34. src/components/ChapterNav.tsx
35. src/components/ChapterView.tsx
36. src/components/ColoringViz.tsx
37. src/components/ComplexityClassesViz.tsx
38. src/components/ComponentsViz.tsx
39. src/components/DfsBridgesSimulator.tsx
40. src/components/DijkstraViz.tsx
41. src/components/DPVisualizer.tsx
42. src/components/EulerSimulator.tsx
43. src/components/ExpressQuiz.tsx
44. src/components/FloydViz.tsx
45. src/components/GraphTraversalViz.tsx
46. src/components/HeapViz.tsx
47. src/components/hints/demoKit.tsx
48. src/components/hints/demosExtra.tsx
49. src/components/hints/demosGraph.tsx
50. src/components/hints/demosStruct.tsx
51. src/components/hints/MiniDemo.tsx
52. src/components/hints/SimulatorModal.tsx
53. src/components/hints/TermPopover.tsx
54. src/components/JohnsonViz.tsx
55. src/components/KosarajuViz.tsx
56. src/components/KruskalSimulator.tsx
57. src/components/MnemonicCards.tsx

```
88. scripts/export/common.mjs
89. scripts/export/guide-pdf-entry.tsx
90. scripts/export/render-pages.mjs
91. scripts/selftest/bundle.ts
92. scripts/selftest/harness-dump.ts
93. scripts/selftest/harness-entry.ts
94. scripts/selftest/lazy.ts
```

CI / автоматизация

```
95. .github/workflows/deploy-pages.yml
96. .github/workflows/rebuild-pdf.yml
```

РАЗДЕЛ: КОНФИГУРАЦИЯ ПРОЕКТА

ФАЙЛ 1/96: add.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 1224 | РАЗМЕР:

Изменённые файлы

```text

```
A .github/workflows/deploy-pages.yml
M .github/workflows/rebuild-pdf.yml
M index.html
M public/export/full_code_all_pages.pdf
M public/export/full_code_all_pages.txt
A public/favicon.svg
M src/App.tsx
M src/components/Navbar.tsx
A src/components/PythonCompiler.tsx
M vite.config.ts
```

```

` .github/workflows/deploy-pages.yml`

```
      run: npm ci --no-audit --no-fund

    - name: Build for the repository subpath
      env:
        VITE_BASE: /algv0/
      run: npm run build

    - name: Make GitHub Pages fallback
      run: cp dist/index.html dist/404.html

    - name: Upload Pages artifact
      uses: actions/upload-pages-artifact@v5
      with:
        path: ./dist

  deploy:
    name: Publish to GitHub Pages
    needs: build
    runs-on: ubuntu-latest
    environment:
      name: github-pages
      url: ${ steps.deployment.outputs.page_url }
    steps:
      - name: Deploy
        id: deployment
        uses: actions/deploy-pages@v5
  ....

### Patch относительно `main`

```diff
diff --git a/.github/workflows/deploy-pages.yml b/.github/workflows/deploy-pages.yml
new file mode 100644
index 00000000..17418e3
--- /dev/null
+++ b/.github/workflows/deploy-pages.yml
@@ -0,0 +1,60 @@
+name: Deploy app to GitHub Pages
```



- name: Setup Node.js  
uses: actions/setup-node@v7  
with:  
  node-version: "24"  
  cache: npm
- name: Install ImageMagick + DejaVu fonts  
run: |  
  sudo apt-get update -qq  
  sudo apt-get install -y --no-install-recommends  
  # На Ubuntu политика ImageMagick иногда запрещает экспорт в pdf  
  # разрешаем то, что нужно пайплайну (text -> pdf)  
  for policy in /etc/ImageMagick-6/policy.xml /etc/ImageMagick-6/policy.xml; do  
    if [ -f "\$policy" ]; then  
      sudo sed -i 's/rights="none" pattern="\{PDF,PSD\}" rights="write,delete" /rights="write,delete" /'  
    fi  
  done  
  convert -version
- name: Install dependencies  
run: npm ci --no-audit --no-fund
- name: Step 1 – код → txt  
run: npm run export:txt
- name: Step 2 – txt → png  
env:  
  EXPORT\_DENSITY: \${GITHUB\_EVENT\_INPUTS.density:-300}  
run: npm run export:png
- name: Step 3 – png → pdf  
run: npm run export:pdf
- name: Type-check приложения  
run: npx tsc --noEmit  
continue-on-error: true

```
 fi
 git commit -m "chore(export): пересборка full"
 git push origin "HEAD:${GITHUB_REF_NAME}"

```

### Patch относительно `main`

```
````diff
diff --git a/.github/workflows/rebuild-pdf.yml b/.github/workflows/rebuild-pdf.yml
index bdc3c33..ff49c0c 100644
--- a/.github/workflows/rebuild-pdf.yml
+++ b/.github/workflows/rebuild-pdf.yml
@@ -42,15 +42,15 @@ jobs:
```

steps:

```
      - name: Checkout
      - uses: actions/checkout@v4
+    uses: actions/checkout@v6
      with:
        fetch-depth: 0
        persist-credentials: true

      - name: Setup Node.js
      - uses: actions/setup-node@v4
+    uses: actions/setup-node@v7
      with:
        node-version: "22"
+      node-version: "24"
        cache: npm
```

```
      - name: Install ImageMagick + DejaVu fonts
@@ -97,7 +97,7 @@ jobs:
        } >> "$GITHUB_STEP_SUMMARY"
```

```
      - name: Upload artifacts (PDF + TXT)
      - uses: actions/upload-artifact@v4
+    uses: actions/upload-artifact@v7
      with:
```

Универсальное пособие • полный исходный код

```
-----  
--- a/index.html  
+++ b/index.html  
@@ -3,6 +3,7 @@  
    <head>  
        <meta charset="UTF-8" />  
        <meta name="viewport" content="width=device-width,  
+    <link rel="icon" type="image/svg+xml" href="%BASE_  
        <title> Дискретка для тупых – Интерактивный гид по  
    </head>  
    <body class="bg-slate-950 text-slate-100 min-h-screen  
....
```

```
## `public/favicon.svg`
```

```
### Полное содержимое после изменений
```

```
````xml
```

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64
 <rect width="64" height="64" rx="16" fill="#4f46e5"/>
 <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#
 <circle cx="47" cy="21" r="4" fill="#34d399"/>
</svg>
````
```

```
### Patch относительно `main`
```

```
````diff
```

```
diff --git a/public/favicon.svg b/public/favicon.svg
new file mode 100644
```

```
index 00000000..3bbbba0
```

```
--- /dev/null
```

```
+++ b/public/favicon.svg
```

```
@@ -0,0 +1,5 @@
```

```
+<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64
+ <rect width="64" height="64" rx="16" fill="#4f46e5"/>
+ <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#
+ <circle cx="47" cy="21" r="4" fill="#34d399"/>
+</svg>
```



```

const onKey = (e: KeyboardEvent) => {
 const target = e.target as HTMLElement | null;
 if (target && /^(INPUT|TEXTAREA|SELECT)$/.test(target)) return;
 if (e.metaKey || e.ctrlKey || e.altKey) return;
 if (e.key === "ArrowRight" && next) goTo(next.id);
 if (e.key === "ArrowLeft" && prev) goTo(prev.id);
};
window.addEventListener("keydown", onKey);
return () => window.removeEventListener("keydown", onKey);
}, [goTo, next, prev]);

const progress = useMemo(() => ((index + 1) / chapters.length) * 100, [index, chapters]);

return (
 <div className="min-h-screen bg-slate-950 text-slate-50">
 <Navbar activeTab={activeTab} setActiveTab={setActiveTab}>
 {activeTab === "guide" ? (
 <>
 <MobileToc
 selectedChapterId={activeChapterId}
 setSelectedChapterId={goTo}
 currentTitle={activeChapter.title}
 index={index}
 total={chapters.length}
 />
 </>
) : (
 <div className="flex flex-col lg:flex-row max-w-7xl mx-auto" >
 <div className="hidden lg:block lg:w-80 shrink-0" >
 <Sidebar selectedChapterId={activeChapterId} />
 </div>
 <main id="main-content" className="flex-1 min-h-screen" >
 <div className="flex items-center justify-between" >
 <div className="min-w-0">
 <div className="text-[10px] uppercase">

```

```

 </main>
) : (
 <PythonCompiler />
)}

 <SimulatorModal vizId={modalVizId} onClose={() =>

 <footer className="p-8 text-center text-slate-600"
 © 2026 Universal Educational Guide. Интерактивны
 </footer>
 </div>
);
}
....

```

### Patch относительно `main`

```

````diff
diff --git a/src/App.tsx b/src/App.tsx
index 2091f33..4583756 100644
--- a/src/App.tsx
+++ b/src/App.tsx
@@ -7,9 +7,10 @@ import { ChapterView } from "../components/ChapterView";
import { ChapterNav } from "../components/ChapterNav";
import { SimulatorModal } from "../components/hints/SimulatorModal";
import { SimulatorHub } from "../components/SimulatorHub";
+import { PythonCompiler } from "../components/PythonCompiler";

export default function App() {
-  const [activeTab, setActiveTab] = useState<"guide" | "simulator"> "guide";
+  const [activeTab, setActiveTab] = useState<"guide" | "simulator"> "guide";
  const [activeChapterId, setActiveChapterId] = useState<string>("");
  const [modalVizId, setModalVizId] = useState<string>("");

@@ -95,7 +96,7 @@ export default function App() {
    </main>
  </div>
</>

```

```

    } finally {
      setBusy(false);
    }
  };

  return (
    <header className="bg-slate-900 border-b border-slate-800">
      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
        <div className="flex items-center gap-3 w-full">
          <div className="bg-gradient-to-tr from-indigo-500 to-indigo-600 w-6 h-6">
            <Sparkles className="w-6 h-6" />
          </div>
          <div className="min-w-0 flex-1">
            <h1 className="text-xl font-extrabold text-white">
              Универсальное пособие
            </h1>
            <p className="text-xs text-indigo-400 font-medium">
              Подготовка к экзаменам: Алгоритмы, Структуры
            </p>
          </div>
        </div>
      </div>

      <div className="flex items-center w-full lg:w-auto">
        <button
          id="tab-guide-btn"
          onClick={() => setActiveTab('guide')}
          className={`flex-1 lg:flex-none flex items-center justify-center gap-2 text-sm font-medium transition-colors duration-200 whitespace-nowrap ${
            activeTab === 'guide'
              ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500/50'
              : 'text-slate-400 hover:text-white'
          }`}
        >
          <BookOpen className="w-4 h-4" /> Учебное по
        </button>
      </div>
    </header>
  );
}

```

```
      href={` ${import.meta.env.BASE_URL}export/full_
      download="full_code_all_pages.txt"
      target="_blank"
      rel="noreferrer"
      className="flex items-center justify-center
      :bg-sky-600/20 border border-sky-500/30 tra
      title="Скачать полный TXT файл со всем кодом
    >
      <FileText className="w-4 h-4" /> TXT
    </a>
    <button
      id="download-html-btn"
      onClick={saveBook}
      disabled={busy}
      className="flex items-center justify-center
      er:bg-amber-600/20 border border-amber-500/
      title="Скачать всё пособие одним автономным
    >
      {busy ? <Loader2 className="w-4 h-4 animate
    </button>
  </div>
</div>
</header>
);
};
....
```

Patch относительно `main`

```
````diff
diff --git a/src/components/Navbar.tsx b/src/components
index a528778..6f8cd36 100644
--- a/src/components/Navbar.tsx
+++ b/src/components/Navbar.tsx
@@ -1,11 +1,11 @@
 import React, { useState } from 'react';
 -import { BookOpen, Cpu, Sparkles, FileDown, FileText,
 +import { BookOpen, Cpu, Sparkles, FileDown, FileText,
```

## Универсальное пособие • полный исходный код

```

- href="/export/full_code_all_pages.txt"
+ href={` ${import.meta.env.BASE_URL}export/full_code_all_pages.txt`
 download="full_code_all_pages.txt"
 target="_blank"
 rel="noreferrer"
.....
```

```
`src/components/PythonCompiler.tsx`
```

```
Полное содержимое после изменений
```

```
````tsx  
import { useCallback, useState } from "react";  
import { CheckCircle2, ExternalLink, Info, Loader2, Play } from "lucide-react";  
  
const PYODIDE_VERSION = "0.27.7";  
const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/bin/pyodide.js`;  
const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/index.html`;  
  
const STARTER_CODE = `# Первый запуск загрузит Python в интерпретатор  
name = "алгоритмы"  
  
for number in range(1, 4):  
    print(f"{number}. Учим {name}!")  
`;  
  
type PyodideRuntime = {  
  runPythonAsync: (source: string) => Promise<unknown>;  
  setStdout: (options: { batched: (message: string) => void }) => void;  
  setStderr: (options: { batched: (message: string) => void }) => void;  
  setStdin: (options: { stdin: () => string | number | null }) => void;  
};  
  
type PyodideModule = {  
  loadPyodide: (options: { indexURL: string }) => Promise<PyodideRuntime>;  
};  
  
let runtimePromise: Promise<PyodideRuntime> | null = null;
```

```

runtime.setStdout({ batched: (message) => stdout.write(message) });
runtime.setStderr({ batched: (message) => stderr.write(message) });
runtime.setStdin({ stdin: () => (inputLines.length > 0 ? inputLines.shift() : null) });
await runtime.runPythonAsync(code);

const text = [...stdout, ...stderr].join("");
setOutput(text || "Готово: программа ничего не вывела");
} catch (error) {
const captured = [...stdout, ...stderr].join("");
setOutput(`${captured}${captured && !captured.endsWith("\n") ? "\n" : ""}`);
setRuntimeMessage("Не удалось выполнить код – подробности в консоли");
} finally {
setRunning(false);
}
}, [code, stdin, running, runtimeReady]);

const reset = () => {
setCode(STARTER_CODE);
setStdin("");
setOutput("Нажмите «Запустить», чтобы увидеть результат");
};

return (
<main className="max-w-screen-2xl mx-auto px-4 sm:px-6">
<div className="max-w-6xl mx-auto">
<div className="flex flex-col sm:flex-row sm:items-center">
<div>
<div className="inline-flex items-center gap-2">
<Terminal className="w-4 h-4" /> Боковая панель
</div>
<h2 className="text-2xl sm:text-3xl font-extrabold">
<p className="text-slate-400 mt-2 max-w-2xl">
Пишите небольшой код и запускайте его прямо в браузере,
который переводит CPython в WebAssembly.
</p>
</div>
<a href="#">

```



```

-----
new file mode 100644
index 00000000..1113e0f
--- /dev/null
+++ b/src/components/PythonCompiler.tsx
@@ -0,0 +1,205 @@
+import { useCallback, useState } from "react";
+import { CheckCircle2, ExternalLink, Info, Loader2, Plus } from "lucide-react";
+
+const PYODIDE_VERSION = "0.27.7";
+const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/bin/pyodide.js`;
+const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/index.html`;
+
+const STARTER_CODE = `# Первый запуск загрузит Python
+name = "алгоритмы"
+
+for number in range(1, 4):
+    print(f"{number}. Учим {name}!")
+`;
+
+type PyodideRuntime = {
+  runPythonAsync: (source: string) => Promise<unknown>;
+  setStdout: (options: { batched: (message: string) => void }) => void;
+  setStderr: (options: { batched: (message: string) => void }) => void;
+  setStdin: (options: { stdin: () => string | number | null }) => void;
+};
+
+type PyodideModule = {
+  loadPyodide: (options: { indexURL: string }) => Promise<PyodideRuntime>;
+};
+
+let runtimePromise: Promise<PyodideRuntime> | null = null;
+
+function getPythonRuntime() {
+  if (!runtimePromise) {
+    runtimePromise = import(/* @vite-ignore */ PYODIDE_MODULE_URL, {
+      (module as PyodideModule).loadPyodide({ indexURL: PYODIDE_INDEX_URL })
+    });
+  }
+}

```



```

+     setOutput(text || "Готово: программа ничего не в
+   } catch (error) {
+     const captured = [...stdout, ...stderr].join("");
+     setOutput(`${captured}${captured && !captured.en
+     setRuntimeMessage("Не удалось выполнить код – пр
+   } finally {
+     setRunning(false);
+   }
+ }, [code, stdin, running, runtimeReady]);
+
+ const reset = () => {
+   setCode(STARTER_CODE);
+   setStdin("");
+   setOutput("Нажмите «Запустить», чтобы увидеть резу
+ };
+
+ return (
+   <main className="max-w-screen-2xl mx-auto px-4 sm:p
+     <div className="max-w-6xl mx-auto">
+       <div className="flex flex-col sm:flex-row sm:i
+         <div>
+           <div className="inline-flex items-center g
+             <Terminal className="w-4 h-4" /> Боковая
+           </div>
+           <h2 className="text-2xl sm:text-3xl font-e
+           <p className="text-slate-400 mt-2 max-w-2x
+             Пишите небольшой код и запускайте его пр
+             который переводит CPython в WebAssembly.
+           </p>
+         </div>
+         <a
+           href="https://pyodide.org/"
+           target="_blank"
+           rel="noreferrer"
+           className="inline-flex items-center gap-1.5
+         >
+           Как это работает <ExternalLink className="
+         </a>

```

```

+         void runCode();
+     }
+ }}
+     spellCheck={false}
+     aria-label="Код Python"
+     className="block w-full min-h-[360px] re
e-none focus:ring-2 focus:ring-inset focus:ring-emera
+     placeholder="Напишите Python-код..."
+     />
+
+     <div className="border-t border-slate-800
+       <label className="block px-4 pt-3 text-[
+         Ввод для input() • по одной строке на
+       </label>
+       <textarea
+         id="python-stdin"
+         value={stdin}
+         onChange={(event) => setStdin(event.ta
+         spellCheck={false}
+         rows={2}
+         placeholder={'Например:\nАлиса'}
+         className="block w-full resize-y bg-tr
ceholder:text-slate-700"
+       />
+     </div>
+   </section>
+
+   <section className="rounded-2xl border borde
+     <div className="flex items-center justify-l
+       <div className="flex items-center gap-2
+         <Terminal className="w-4 h-4 text-emera
+       </div>
+       <span className={`inline-flex items-centr
+         {runtimeReady && <CheckCircle2 classNa
+         {runtimeReady ? "готов" : "не загружен
+       </span>
+     </div>
+   <pre className="min-h-[360px] max-h-[560px]

```

```
const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

// https://vite.dev/config/
export default defineConfig({
  // На GitHub Pages приложение живёт в /algv0/, локально
  // VITE_BASE можно переопределить, если репозиторий б
  base: process.env.VITE_BASE || "/",
  plugins: [react(), tailwindcss(), viteSingleFile()],
  resolve: {
    alias: {
      "@": path.resolve(__dirname, "src"),
    },
  },
  server: {
    host: "0.0.0.0",
    port: 5173,
    strictPort: true,
    // предпросмотр может открываться через внешний про
    allowedHosts: true,
  },
  preview: {
    host: "0.0.0.0",
    port: 4173,
    strictPort: true,
    allowedHosts: true,
  },
});

```

Patch относительно `main`

```
````diff
diff --git a/vite.config.ts b/vite.config.ts
index c0456e7..e19d945 100644
--- a/vite.config.ts
+++ b/vite.config.ts

```

## # Gemini App Canvas Instructions

Ты – AI-ассистент, работающий в Gemini App Canvas (или Gemini App Canvas).  
При запросе пользователя создать или сгенерировать новую страницу:  
\*\*строго\*\* придерживаться готовых шаблонов разметки, не добавляя ничего нового.

## ## Правила для малых страниц (Билетов / Глав)

- \*\*Единый стиль:\*\*** Всегда используй Tailwind CSS и HTML.
- \*\*Структура JSON/TS:\*\*** Каждая новая малая страница должна быть в формате JSON/TS (например, `{ "id": "...", "content": "...", "type": "..." }`) для вставки в ``src/data/content.ts``.
- \*\*Строгая структура контента:\*\***
  - Заголовок с цветным бейджем (например, ``bg-indigo-100``).
  - Определение/правило в центре (``border-blue-500`` или ``border-purple-500``).
  - Обязательная сетка аналогий (2 колонки: неформальный и формальный).
  - "Душные" детали (код, формулы, сложные доказательства).
- \*\*Не ломай верстку:\*\*** Не придумывай свои CSS-классы. Используй только те, что есть в файлах ``src/data/content.ts``. Не используй чистый markdown.

---

ФАЙЛ 4/96: index.html

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 14 | РАЗМЕР: 600 байт

---

```
<!doctype html>
<html lang="ru" class="dark bg-slate-950 text-slate-100">
 <head>
 <meta charset="UTF-8" />
 <meta name="viewport" content="width=device-width, height=device-height, initial-scale=1.0" />
 <link rel="icon" type="image/svg+xml" href="%BASE_URL%/icon.svg" />
 <title> Универсальное пособие по алгоритмам – билет №4
 </head>
 <body class="bg-slate-950 text-slate-100 min-h-screen">
 <div id="root"></div>
 <script type="module" src="/src/main.tsx"></script>
 </body>
</html>
```

```
 "selftest": "npx esbuild scripts/selftest/bundle.ts
 el=error --loader:.jpg=file && node /tmp/bundle-s
 at=cjs --outfile=/tmp/lazy-selftest.cjs --jsx=auto
 esbuild scripts/selftest/harness-entry.ts --bundl
 on3 scripts/selftest/harness_check.py"
 },
 "dependencies": {
 "clsx": "2.1.1",
 "lucide-react": "^1.21.0",
 "motion": "^12.40.0",
 "pdfkit": "^0.20.1",
 "react": "19.2.6",
 "react-dom": "19.2.6",
 "svg-to-pdfkit": "^0.1.8",
 "tailwind-merge": "3.4.0"
 },
 "devDependencies": {
 "@tailwindcss/vite": "4.1.17",
 "@types/node": "22.19.17",
 "@types/pdfkit": "^0.17.6",
 "@types/react": "19.2.7",
 "@types/react-dom": "19.2.3",
 "@vitejs/plugin-react": "5.1.1",
 "esbuild": "^0.24.0",
 "tailwindcss": "4.1.17",
 "typescript": "5.9.3",
 "vite": "7.3.2",
 "vite-plugin-singlefile": "2.3.0"
 },
 "description": "Интерактивное пособие по алгоритмам и
}
```

### ### Автоматизация (CI)

`github/workflows/rebuild-pdf.yml` запускается \*\*на каждом коммите\*\* и пересобирает TXT → PNG → PDF, кладёт PNG-страницы и документы в репозиторий и коммитит обновлённые `public/export/full\_code\_all\_pages.pdf` и `public/export/full\_code\_all\_pages.png` (с меткой `[skip ci]`, чтобы не заиклиться).

Приложение отдаёт эти файлы по ссылкам `/export/full\_code\_all\_pages.pdf` и `/export/full\_code\_all\_pages.png`.

### ## Аудит контента

```
```bash
node scripts/audit-coverage.mjs          # какие темы без кода
node scripts/audit-coverage.mjs --md     # markdown-таблицы
```
```

Актуальный отчёт: [`TOPICS\_AUDIT.md`](./TOPICS\_AUDIT.md)

### ## Как устроен интерфейс

- \* **Темы 1–24** — канонический список билетов; одна страница на тему (`src/data/content.ts`, таблица `CANON` задаёт точные названия);
- \* **Python-компилятор** — боковая панель (кнопка «Python-компилятор», отладчик. Список «переменных темы» для каждой страницы в `src/data/vizSync.ts`; значения из выполняемого Python-кода — объекты демонстрации (матрицы, графы, деревья) напрямую в код, инициализация подставляет заранее заданные значения);
- \* **Содержание** — на десктопе боковая панель с поиском, на телефоне — липкая строка «Содержание · N из M» и вкладки «Содержание» и «Поиск»;
- \* **Переходы между темами** — кнопки «Назад / Вперёд» в боковой панели, «Предыдущая / Следующая тема» под текстом; на клавиатуре — «←» и «→»;
- \* **Всплывающие подсказки по терминам** — текст главы и известные термины (BFS, куча, бор, суффиксная ссылка, подчёркиваются, а по наведению/тапу показывается карта термина-анимацией и кнопкой «Показать симулятор» (симулятор — `src/data/glossary.ts`, мини-анимации — `src/data/vizRegistry.ts`);
- \* **Реестр интерактивов** — `src/components/vizRegistry`.

Универсальное пособие • полный исходный код

-----  
Вместо обычных `` в приложении используются векторы

\* Они помещены внутрь оберток с классом `

\* Все отрисовки находятся внутри векторного тега ``

\* Подпись к картинке, если она есть, идет сразу после SVG

## Списки

\* Стандартные списки оформлены через `



## Валидный экспорт в Markdown (встроенный)

Экспорт перехватывает основные структурные компоненты HTML

1. Заголовки `

## ` и `` заменяются на `##` и `###`

2. Элементы ``.analogy-badge` получают выделение маркера`

3. Теги `` заменяются строкой-заглушкой `[ Иллюстрация]`.

4. Элементы списка `- ` получают маркер `-`.

5. Результат извлекается в виде текста и чистится от лишнего

-----  
ФАЙЛ 8/96: TOPICS\_AUDIT.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 106 | РАЗМЕР: 1.5 КБ

-----  
# Аудит покрытия тем: где нет «объяснения на пальцах» и

> Сгенерировано скриптом `node scripts/audit-coverage.m`

> Критерии:

> \* **Аналогия** — в HTML главы есть блок «Аналогия ...»

> \* **2D** — к главе привязан интерактивный визуализатор

>

> Всего глав в пособии: **25**.

---

## Уровень 1. Нет вообще ничего: ни аналогии, ни 2D

| Глава | id | Что делать |

| --- | --- | --- |

## Уровень 4. Есть аналогия, но НЕТ 2D-визуализации (

> Замкнуто: «5. Двумерная матрица префиксных сумм» полн

| Глава                                | id                 | Чего не хватает     |
|--------------------------------------|--------------------|---------------------|
| 3. Декартово дерево (Trear)          | `trear`            | Нет ни визуализации |
| 8. Графы. Компоненты связности       | `graph-components` |                     |
| 24. Классы сложности, сведение задач | `complexity-cl     |                     |

## Сводная таблица по всем главам

| #  | Глава                                           | Аналогий | 2D-виз. | SVG | Картинка |
|----|-------------------------------------------------|----------|---------|-----|----------|
| 2  | 2. Двумерная разреженная матрица (Sparse Table) |          |         |     |          |
| 3  | 3. Декартово дерево (Trear)                     | 2        | —       | —   | —        |
| 4  | 4. Splay-дерево                                 | 2        | —       | —   |          |
| 5  | 5. Двумерная матрица префиксных сумм            | 1        |         |     | —        |
| —  | Планарность: K5, K3,3 и формула Эйлера          | —        |         |     |          |
| 6  | 6. Графы. Представление, DFS, BFS               | 4        |         | —   | —        |
| 8  | 8. Графы. Компоненты связности                  | 1        | —       | —   | —        |
| 9  | 9. Графы. Топологическая сортировка             | 1        |         |     | —        |
| 10 | 10. Графы. Компоненты сильной связности (SCC)   |          |         |     |          |
| 12 | 12. Графы. Точки сочленения                     | 1        |         | —   | —        |
| 14 | 14. Графы. Кратчайший путь. Дейкстра            | 3        |         |     | —        |
| 15 | 15. Графы. Кратчайший путь. Форд–Беллман        | 1        |         |     |          |
| 16 | 16. Графы. Кратчайший путь. Флойд               | —        |         |     | —        |
| 17 | 17. Графы. Алгоритм Джонсона                    | —        |         | —   | —        |
| 18 | 18. Графы. Остовное дерево. Краскал             | 1        |         |     | —        |
| 19 | 19. Графы. Остовное дерево. Прима               | 1        |         |     | —        |
| 20 | 20. Графы. Остовное дерево. Борувка             | 1        |         |     | —        |
| 21 | 21. Префикс-функция (π), КМП                    | 4        |         | —   | —        |



```
"isolatedModules": true,
"noEmit": true,
"jsx": "react-jsx",

/* Path mapping */
"baseUrl": ".",
"paths": {
 "@/*": ["src/*"]
},

/* Linting */
"strict": true,
"noUnusedLocals": true,
"noUnusedParameters": true,
"noFallthroughCasesInSwitch": true
},
"include": ["src", "vite.config.ts"]
}
```

---

ФАЙЛ 10/96: vite.config.ts

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 36 | РАЗМЕР: 1

---

```
import path from "path";
import { fileURLToPath } from "url";
import tailwindcss from "@tailwindcss/vite";
import react from "@vitejs/plugin-react";
import { defineConfig } from "vite";
import { viteSingleFile } from "vite-plugin-singlefile"

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

// https://vite.dev/config/
export default defineConfig({
 // На GitHub Pages приложение живёт в /algv0/, локаль
```

```

import { MobileToc } from "../components/MobileToc";
import { ChapterView } from "../components/ChapterView";
import { ChapterNav } from "../components/ChapterNav";
import { SimulatorModal } from "../components/hints/Simu
import { PythonCompiler } from "../components/PythonComp

export default function App() {
 const [activeChapterId, setActiveChapterId] = useState<string | null>();
 const [modalVizId, setModalVizId] = useState<string | null>();
 /** Боковая (на мобильном – нижняя) панель компилятора */
 const [compilerOpen, setCompilerOpen] = useState(false);
 /** Ширина панели компилятора (деSKTOP), растягивается */
 const [compilerWidth, setCompilerWidth] = useState<number>();
 const cached = typeof window !== "undefined" ? Number(window.innerWidth) : null;
 return Number.isFinite(cached) && cached >= 380 ? cached : null;
});
useEffect(() => {
 window.localStorage.setItem("compilerWidth", String(compilerWidth));
}, [compilerWidth]);
/** Высота терминала: null оставляет адаптивную высоту */
const [compilerHeight, setCompilerHeight] = useState<number>();
const cached = typeof window !== "undefined" ? Number(window.innerHeight) : null;
return Number.isFinite(cached) && cached >= 260 ? cached : null;
});
useEffect(() => {
 if (compilerHeight !== null) window.localStorage.setItem("compilerHeight", String(compilerHeight));
}, [compilerHeight]);
/** Содержание свернуто (по умолчанию сворачиваем sidebar) */
const [sidebarCollapsed, setSidebarCollapsed] = useState<boolean>();
const cached = typeof window !== "undefined" ? window.localStorage.getItem("sidebarCollapsed") : null;
if (cached !== null) return cached === "1";
return typeof window !== "undefined" && window.innerWidth < 380;
});
const toggleSidebar = useCallback(() => {
 setSidebarCollapsed((c) => {
 window.localStorage.setItem("sidebarCollapsed", c ? "1" : null);
 return !c;
 });
});

```

```

// Стрелки ← → листают темы (как на обучающих сайтах)
useEffect(() => {
 const onKey = (e: KeyboardEvent) => {
 const target = e.target as HTMLElement | null;
 if (target && /^(INPUT|TEXTAREA|SELECT)$/.test(target)) return;
 if (e.metaKey || e.ctrlKey || e.altKey) return;
 if (e.key === "ArrowRight" && next) goTo(next.id);
 if (e.key === "ArrowLeft" && prev) goTo(prev.id);
 };
 window.addEventListener("keydown", onKey);
 return () => window.removeEventListener("keydown", onKey);
}, [goTo, next, prev]);

const progress = useMemo(() => ((index + 1) / chapters.length) * 100, [index, chapters]);

return (
 <div className="min-h-screen bg-slate-950 text-slate-50">
 <Navbar
 compilerOpen={compilerOpen}
 onToggleCompiler={() => setCompilerOpen((o) => !o)}
 />

 <div ref={contentRef} className={compilerOpen ? "h-screen" : "h-[calc(100vh-50px)]"}>
 <MobileToc
 selectedChapterId={activeChapterId}
 setSelectedChapterId={goTo}
 currentTitle={activeChapter.title}
 index={index}
 total={chapters.length}
 />

 <div className="flex flex-col lg:flex-row min-h-[calc(100vh-50px)]">
 <div
 className={`hidden lg:block shrink-0 border-right border-slate-800 p-4 transition-all duration-200 ${sidebarCollapsed ? "hidden" : ""}`}
 />

```

```

 key={activeChapter.id}
 chapter={activeChapter}
 prev={prev}
 next={next}
 index={index}
 total={chapters.length}
 onGo={goTo}
 onOpenSimulator={setModalVizId}
 onOpenCompiler={openCompilerWithViz}
 />
 </main>
 </div>
</>

<footer className="p-8 text-center text-slate-600">
 © 2026 Universal Educational Guide. Интерактивное
</footer>
</div>

<SimulatorModal vizId={modalVizId} onClose={() =>
 setModalVizId(null)}>
 {compilerOpen && (
 <PythonCompiler
 chapterId={activeChapter.id}
 chapterTitle={activeChapter.title}
 width={compilerWidth}
 height={compilerHeight}
 onWidthChange={setCompilerWidth}
 onHeightChange={setCompilerHeight}
 onOpenGuide={() =>
 document.getElementById("chapter-viz")?.scrollTo(0,0)}
 onClose={() => setCompilerOpen(false)}
 />
)}
</div>
);
}

```

```

*/
export function annotateTerms(root: HTMLElement): void {
 if (!cachedRegex) cachedRegex = buildRegex();
 const re = cachedRegex;

 const perTerm = new Map<string, number>();
 const perSurface = new Map<string, number>();
 const walker = document.createTreeWalker(root, NodeFilter.FILTER_ACCEPT,
 acceptNode(node) {
 const parent = (node as Text).parentElement;
 if (!parent) return NodeFilter.FILTER_REJECT;
 if (parent.closest(SKIP_SELECTOR)) return NodeFilter.FILTER_REJECT;
 if (!node.nodeValue || node.nodeValue.trim().length === 0) return NodeFilter.FILTER_REJECT;
 return NodeFilter.FILTER_ACCEPT;
 },
);

 const targets: Text[] = [];
 let current = walker.nextNode();
 while (current) {
 targets.push(current as Text);
 current = walker.nextNode();
 }

 for (const textNode of targets) {
 const text = textNode.nodeValue ?? "";
 re.lastIndex = 0;
 if (!re.test(text)) continue;
 re.lastIndex = 0;

 const frag = document.createDocumentFragment();
 let last = 0;
 let m: RegExpExecArray | null;

 while ((m = re.exec(text)) !== null) {
 const surface = m[1].toLowerCase();
 const id = labelToId.get(surface);
 if (!id) continue;

```

```
 } catch {
 /* если браузер не умеет lookbehind – просто оста
 }
 };

 // при смене главы
 // eslint-disable-next-line react-hooks/exhaustive-dep
 useEffect(run, deps);

 // страховка: контент перерисовали, а подсказок в нём
 useEffect(() => {
 const el = ref.current;
 if (!el) return;
 if (!el.querySelector(".term-hint")) run();
 });
}
```

---

ФАЙЛ 13/96: src/index.css

КАТЕГОРИЯ: Ядро приложения | СТРОК: 175 | РАЗМЕР: 3.3 K

---

```
@import "tailwindcss";

@layer utilities {
 .neon-glow-blue {
 box-shadow: 0 0 15px rgba(59, 130, 246, 0.5);
 }
 .neon-glow-emerald {
 box-shadow: 0 0 15px rgba(16, 185, 129, 0.5);
 }
 .neon-glow-rose {
 box-shadow: 0 0 15px rgba(244, 63, 94, 0.5);
 }
 .neon-glow-yellow {
 box-shadow: 0 0 20px rgba(234, 179, 8, 0.8);
 }
}
```

```

 transform: translateY(4px);
 }
 to {
 opacity: 1;
 transform: translateY(0);
 }
}

.term-popover {
 animation: hintIn 0.14s ease-out;
}

/* Интерактивные термины в тексте главы (превью как в Википедии)
.term-hint {
 cursor: help;
 border-bottom: 1px dashed rgba(129, 140, 248, 0.75);
 color: #c7d2fe;
 border-radius: 3px;
 padding: 0 1px;
 transition:
 background-color 0.15s ease,
 color 0.15s ease;
}
.term-hint:hover,
.term-hint:focus-visible {
 background: rgba(99, 102, 241, 0.22);
 color: #fff;
 outline: none;
}
@media (hover: none) {
 .term-hint {
 border-bottom-style: solid;
 background: rgba(99, 102, 241, 0.12);
 }
}

/* Контент главы приходит готовым HTML — чиним его под себя
.chapter-body {

```

```
 line-height: 1.3 !important;
 }
 .chapter-body :where(h3) {
 font-size: 1.05rem !important;
 line-height: 1.35 !important;
 }
 .chapter-body :where(.p-6, .p-8) {
 padding: 0.9rem !important;
 }
 .chapter-body :where(.text-3xl) {
 font-size: 1.4rem !important;
 }
 .chapter-body :where(.text-2xl) {
 font-size: 1.2rem !important;
 }
 .chapter-body :where(.text-xl) {
 font-size: 1.05rem !important;
 }
}

/* Custom scrollbar styling for dark theme */
::-webkit-scrollbar {
 width: 8px;
 height: 8px;
}
::-webkit-scrollbar-track {
 background: #0f172a;
}
::-webkit-scrollbar-thumb {
 background: #334155;
 border-radius: 4px;
}
::-webkit-scrollbar-thumb:hover {
 background: #475569;
}

/* Индикатор длительности кадра в пошаговых визуализаци.
@keyframes demoProgress {
```



```
}
```

---

ФАЙЛ 16/96: src/vite-env.d.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 19 | РАЗМЕР: 324 В

---

```
/// <reference types="vite/client" />
```

```
declare module '*.jpg' {
 const src: string;
 export default src;
}
declare module '*.jpg?url' {
 const src: string;
 export default src;
}
declare module '*.jpeg' {
 const src: string;
 export default src;
}
declare module '*.png' {
 const src: string;
 export default src;
}
```

---

РАЗДЕЛ: КОНТЕНТ И ДАННЫЕ ПОСОБИЯ

---

---

ФАЙЛ 17/96: src/data/additionalTickets.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 886 | РАЗМЕР: 10 КБ

---

```
import { Chapter } from "../types";
```

## Аналогия 2: Ступени

```
</div>
<p class="text-slate-300 text-sm mb-5">
 нужно покрыть отрезок, мы берем две самые бо
</div>
</div>
<details class="bg-slate-900/40 rounded-lg border
 <summary class="p-4 font-bold text-slate-300
 -b border-slate-700/50">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300"
 <pre class="bg-slate-950 p-4 rounded"
int kx = log2(R2 - R1 + 1);
int ky = log2(C2 - C1 + 1);
int ans1 = min(st[R1][C1][kx][ky], st[R1][C2 - (1 << ky)
int ans2 = min(st[R2 - (1 << kx) + 1][C1][kx][ky], st[R
int minimum = min(ans1, ans2);</pre>
 </div>
 </details>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl border
 <h3 class="text-xl font-bold text-amber-400"
 <p class="text-slate-300 text-sm mb-5">Част
 а ту же скорость». <b class="text-rose-300"
 олько для суммы, разреженная – огромный
 посчитают клетки дважды), минимум нельзя ч
 <div class="grid grid-cols-1 xl:grid-cols-2"
 <div class="bg-slate-800 p-5 rounded-lg"
 <div class="absolute -top-3 left-4
border border-emerald-500 shadow-md">
 Сетка из вафельных трубочек –
 </div>
 по включениям-исключениям – как
```

```

 <td class="py-2 pr-4 text-s
 <td class="py-2 pr-4 font-m
 <td class="py-2 font-mono t
 </tr>
 <tr class="border-t border-slate
 <td class="py-2 pr-4 text-s
 <td class="py-2 pr-4">только
 <td class="py-2">min / m
 </tr>
 <tr class="border-t border-slate
 <td class="py-2 pr-4 text-s
 <td class="py-2 pr-4 text-e
 <td class="py-2 text-rose-3
 </tr>
 </tbody>
 </table>
</div>
<div class="bg-slate-900/40 rounded-lg bord
 <p class="text-slate-300 text-sm mb-1">
 <p class="text-slate-400 text-sm mb-1">
летки дважды. Сумма ≠ идемпотентна.</p>
 <p class="text-slate-400 text-sm mb-1">
ации.</p>
 <p class="text-slate-400 text-sm">• Выб
>
</div>
</div>
</div>
</section>`,
 }, {
 id: "treap",
 title: "3. Декартово дерево (Treap)",
 type: "html",
 description: "Дерево поиска по ключу и Куча по приор
 category: "Продвинутые структуры",
 content: `
<section id="treap" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

```



```

def merge(a, b): # все x в a < все x в b
 if not a or not b: return a or b
 if a.pri > b.pri:
 a.right = merge(a.right, b); return a
 b.left = merge(a, b.left); return b

def erase(root, key): # удалить произвольный элемент
 if key < root.key: root.left = erase(root.left, key)
 elif key > root.key: root.right = erase(root.right, key)
 else: root = merge(root.left, root.right)
 return root
</pre>
</div>
</details>
</div>
</div>
</section>`,
},
{
 id: "splay-tree",
 title: "4. Splay-дерево",
 type: "html",
 description: "Самонастраивающееся дерево: поднимаем элемент x в корень.",
 category: "Продвинутые структуры",
 content: `
<section id="splay-tree" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 4
 <h2 class="text-2xl sm:text-3xl font-bold text-indigo-600">4. Splay-дерево</h2>
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-700">
 <h3 class="text-xl font-bold text-blue-400">4. Splay-дерево</h3>
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">Самонастраивающееся дерево: поднимаем элемент x в корень. Единственный инвариант: для любого узла x все элементы в левом поддереве меньше x, а в правом больше x.</p>
 <p class="text-xl font-mono text-white">def splay(x):
 # поднимаем x в корень
 while x.pri < root.pri:
 # если x - левый ребенок, то поворачиваем вокруг родителя
 if x == root.left:
 # поворот вокруг родителя
 root = rotate_right(root)
 else:
 # поворот вокруг дедушки
 root = rotate_left(rotate_right(root))
 # теперь x - левый ребенок
 x = root.left
 while x.pri > root.pri:
 # если x - правый ребенок, то поворачиваем вокруг родителя
 if x == root.right:
 # поворот вокруг родителя
 root = rotate_left(root)
 else:
 # поворот вокруг дедушки
 root = rotate_right(rotate_left(root))
 # теперь x - правый ребенок
 x = root.right
 # теперь x - корень
 root = x
 x.left.right = x
 x.right.left = x
 return root
 </div>
 <p>Этот алгоритм работает за O(n log n) в худшем случае, но в среднем за O(log n).</p>
 </div>
 </div>
`

```

рно вдвое. Первый проход дорогой, зато опла  
ы.</p>

</div>

</div>

<h4 class="text-lg font-bold text-white mt-3">

<p class="text-slate-300 text-sm mb-3">Повор

Среднее поддерево  $\beta$  (между  $x$  и  $p$ ) – единств

$x$ , и под  $p$ .</p>

<pre class="bg-slate-950 p-4 rounded-lg overflow-x-clip font-mono text-sm leading-relaxed">

```

 / \ x выше / \ p
 x y -----> a p
 / \ p выше / \
 a b <----- b y</pre>
```

<p class="text-slate-300 text-sm mt-3">Обхо

омать дерево поиска поворотами невозможно в  
омент «отстёжки», а врезка «Механика одного

<h4 class="text-lg font-bold text-white mt-3">

<p class="text-slate-300 text-sm mb-3">Смотр

<div class="overflow-x-auto -mx-2 px-2">

<table class="w-full text-xs sm:text-sm">

<thead>

<tr class="text-left text-slate-300">

<th class="py-2 pr-3 font-bolder">

<th class="py-2 pr-3 font-bolder">

<th class="py-2 pr-3 font-bolder">

<th class="py-2 font-bolder">

</tr>

</thead>

<tbody class="text-slate-300">

<tr class="border-b border-slate-700">

<td class="py-3 pr-3 font-bolder">

<td class="py-3 pr-3">деда

<td class="py-3 pr-3">сам  $x$

<td class="py-3"> $x$  – корень

</tr>

```

 <p class="text-slate-300 text-sm mb-2">
 <p class="text-slate-400 text-sm mb-1">
га» – просто переворот.</p>
 <p class="text-slate-400 text-sm mb-1">
иза к верху; в симуляторе оно подсвечивается.
 <p class="text-slate-400 text-sm">• «Sp
о амортизированная.</p>
</div>

<p class="text-slate-300 text-sm mt-4">с
ое <code class="bg-slate-950 px-2 py-0.5 rounded">
одиночная операция – до <code class="bg-slate-950 px-2 py-0.5 rounded">
>0(n)</code>; n вставок подряд – амортизированная.
-300 border border-slate-800">0(n log n)</code>

<p class="text-slate-300 text-sm mt-4">Все insert (в
lay последнего на пути); insert – BST, в котором
муна левой половины не будет правого сына –
а под «горячие» ключи, дешёвые split/merge.
во меняется даже при чтении (плохо для многих операций).

<details class="bg-slate-900/40 rounded-lg border border-slate-800">
 <summary class="p-4 font-bold text-slate-300">
 -b border-slate-700/50">
 Код: rotate + splay + find (Скрытый код)
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <pre class="bg-slate-950 p-4 rounded">
узел: { key, left, right, parent }

def rotate_up(x):
 # x встаёт на место родителя
 p = x.parent
 g = p.parent
 if p.left is x:
 # правый поворот
 p.left = x.right
 if x.right: x.right.parent = p
 x.right = p
 else:
 # левый (зеркально)

```

```

 id: "prefix-sums-2d",
 title: "5. Двумерная матрица префиксных сумм",
 type: "html",
 description: "Сжатие суммы подматрицы за $O(1)$ ",
 category: "Алгоритмы матрицы",
 content: `
<section id="prefix-sums-2d" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 , y1...y2) = P[x2][y2] - P[x1-1][y2] - P[x2
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
 rder border-emerald-500 shadow-md">
 × Аналогия 1: Вырезание прямоу
 </div>
 <p class="text-slate-300 text-sm mb">
 з него маленький целевой прямоугольник в цел
 ый верхний угол отрезается дважды! Поэтому
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "graph-planar-colors",
 title: "7. Графы. Планарные. Покраска",
 type: "html",

```



```

<section id="top-sort" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-slate-800">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
перед.</p>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Учеба в вузе
 </div>
 <p class="text-slate-300 text-sm mb-4">
екая сортировка – это расписание, которое
р>
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Ограничения: Циклы
 </div>
 <p class="text-slate-300 text-sm mb-4">
получить опыт, нужна работа". Алгоритм выяв
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "scc-kosaraju",

```

```

 category: "Графы. Продвинутые",
 content: `
<section id="graph-bridges" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-rose-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-rose-300 italic">
 <p class="text-xl font-mono text-white">
и $f_{up}[v] > t_{in}[u]$.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
rder border-emerald-500 shadow-md">
 Аналогия 1: Единственный мост
 </div>
 <p class="text-slate-300 text-sm mb">
ЭТОТ МОСТ, ЭКОНОМИКОЙ ОБОИХ КУСКОВ ПРИДЕТ
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "graph-articulation",
 title: "12. Графы. Точки сочленения",
 type: "html",
 description: "Вершины, удаление которых убивает свя.",
 category: "Графы. Продвинутые",
 content: `
<section id="graph-articulation" class="mb-12 scroll-mt">
 <div class="flex items-center mb-6 flex-wrap gap-3">

```

```

<div class="bg-slate-700/50 p-6 rounded-xl border-2 border-emerald-500 shadow-md">
 <h3 class="text-xl font-bold text-indigo-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-indigo-300 italic">
 <p class="text-xl font-mono text-white">
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Рисование не отрывая
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Если сходится нечетное число линий, значит, из
 а везде должно быть четное число (зашел-вышел)
 </div>
 </div>
 </div>
</div>
</div>
</section>`,
 },
 {
 id: "pathfinding-accel",
 title: "17. Графы. Ускорения поиска",
 type: "html",
 description: "",
 category: "Графы. Пути",
 content: `
<section id="pathfinding-accel" class="mb-12 scroll-mt-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border-2 border-emerald-500 shadow-md">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">

```

```

<div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border-emerald-500 shadow-md">
 Аналогия: Прокладка интернет
 </div>
 <p class="text-slate-300 text-sm mb-6">
 с самых дешевых дорог в стране". Он строит
 единую сеть.</p>
 </div>
 </div>
</div>
</div>
</section>`,
 },
 {
 id: "mst-prima",
 title: "19. Графы. Остовное дерево. Прима",
 type: "html",
 description: "",
 category: "Графы. Остовные",
 content: `
<section id="mst-prima" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border-emerald-500">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 которое торчит из посещенных в непосещенных
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border-emerald-500 shadow-md">

```

```

<p class="text-slate-300 text-sm mb-4">
 изкому племени для объединения. К вечеру пле-
 ства шлют гонцов к ближайшему чужому царству.
 </div>
</div>
</div>
</div>
</section>`,
 },
 {
 id: "string-kmp",
 title: "21. Алгоритм Кнута-Морриса-Пратта (КМП)",
 type: "html",
 description: "",
 category: "Строки",
 content: `
<section id="string-kmp" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 i]), который одновременно является её суффикс-
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
rder border-emerald-500 shadow-md">
 Аналогия 1: Бумеранг (Префикс-
 </div>
 <p class="text-slate-300 text-sm mb-4">
 идишь "АБРА". Если ты ошибешься при поиске

```

```

<h3 class="text-xl font-bold text-blue-400">
<div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
щегося с i.</p>
</div>
<div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 Аналогия 1: Копипаста
 </div>
 <p class="text-slate-300 text-sm mb-4">
в тексте кричит: "Эй! Начиная с этой буквы
т "окно" [L, R] самой дальней найденной копи
 </div>
 </div>
<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate-300
-b border-slate-700/50">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <pre class="bg-slate-950 p-4 rounded
int l = 0, r = 0;
for (int i = 1; i < n; i++) {
 if (i <= r) z[i] = min(r - i + 1, z[i - l]);
 while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
 if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }
}</pre>
 </div>
 </details>
</div>
</div>
</section>`,
 },
 {
 id: "complexity-classes",

```

```
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
];
```

---

ФАЙЛ 18/96: src/data/content\_temp.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 425 | РАЗМ

---

```
export interface Chapter {
 id: string;
 title: string;
 type: "html";
 content: string;
}

export const chapters: Chapter[] = [
 {
 id: "euler-path-vs-cycle",
 title: "Эйлер: Путь ≠ Цикл (ты путаешь!)",
 type: "html",
 content: `<section id="euler-path-vs-cycle" class="
<div class="flex items-center mb-6">
 <span class="bg-indigo-600 text-white px-4 py-1
 <h2 class="text-3xl font-bold text-white">Эйлер
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-blue-400
 <div class="bg-slate-900 p-4 rounded-lg mb-
 <p class="text-lg text-blue-300 italic`
```

```

 </div>
 <p class="text-slate-300 text-sm mb-4">
 Гамильтон – это болезнь.
 </p>
 <p> ровно раз и вернуться домой.
 Ты обходишь все бары (вершины) ровно
 Главное – зайти и выйти, не заходя
 Никто не знает, как решать быстро.

 лноту).
 </p>
 </div>
 </div>

 <details class="bg-slate-900/40 rounded-lg border-1 border-slate-700">
 <summary class="p-4 font-bold text-slate-400">
 Душный режим: Формальное доказательство
 </summary>
 <div class="p-5 text-sm text-slate-300 space-y-4">
 <p class="text-blue-400">Теорема Эйлера
 <p>
 Связный граф содержит эйлеров цикл ⇔

Эйлеров путь (не цикл) ⇔ ровно 0 или 2 вершины
 имеют нечётную степень.
 </p>
 <p class="text-rose-400">Почему Гамильтон?
 <p>
 Задача коммивояжёра (TSP) – это взрыв мозга.
 TSP – NP-трудная: полиномиальный алгоритм не найден.
 В общем, не парься. Просто запомни!
 </p>
 </div>
 </details>
</div>
</section>`,
 },
 {
 id: "bridges-code",
 title: "Мосты в графах: код + визуализация",
 }
]

```



Из пещеры и нет других выходов — то  
Если верёвка (ребро  $v-u$ ) оборвётся —  
Если бы из пещеры и был чёрный ход

</p>

</div>

<div class="bg-slate-900 p-6 rounded-lg border-rose-500 shadow-md">  
 <div class="absolute -top-3 left-4 bg-rose-500 shadow-md">

Аналогия: Кровеносный сосуд

</div>

<p class="text-slate-300 text-sm mb-4">

Граф — это кровеносная система. Ребра (артерии и вены, т.е. каналы, по которым течёт кровь) — это не мост, живём.

Если же перерезать единственную артерию, то все вены перестанут работать.

Алгоритм DFS ищет такие «критические» ребра.

</p>

</div>

</div>

<details class="bg-slate-900/40 rounded-lg border-rose-500 shadow-md">  
 <summary class="p-4 font-bold text-slate-400">  
 order-slate-700/50">

Полный код на C++ (Скрыто)

</summary>

<div class="p-5 text-sm text-slate-300 space-y-4">

<p>Классическая реализация — <code class="text-slate-400">DFS</code> по графу. Если же перерезать единственную артерию, то все вены перестанут работать. Алгоритм DFS ищет такие «критические» ребра.</p>

<div class="bg-slate-950 p-4 rounded-lg border-rose-500 shadow-md">

<pre class="text-xs font-mono text-slate-400">

```
#include <vector>
```

```
#include <algorithm>
```

```
using namespace std;
```

```
int n; // число вершин
```

```
vector<vector<int>>> adj; // список смежных ребер
```

```

 <p class="text-xs text-slate-400">Запис
 </div>
 </details>
 </div>
</section>`,
 },
 {
 id: "planarity-euler-formula",
 title: "Планарность: K5, K3,3 и формула Эйлера",
 type: "html",
 content: `<section id="planarity-euler-formula" cla
<div class="flex items-center mb-6">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-3xl font-bold text-white">Плана
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-blue-400 m

 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic m
 <p class="text-3xl font-mono text-white"
 <div class="text-sm text-slate-400 spac
 <p><span class="text-blue-400 font-l
 <p><span class="text-emerald-400 fo
 <p><span class="text-rose-400 font-l
 </div>
 </div>
 </div>

 <p class="text-slate-300 text-sm">
 Следствие: Для планарн
ode>.
 Если ребер больше – граф гарантированно
 </p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap
```

```
<p>
 Пусть K_5 планарен. $V = 5$, $E = 10$.

 $F = E - V + 2 = 10 - 5 + 2 = 7$

Каждая грань ограничена минимум

Сумма длин граней $= 2E = 20$.

Отсюда: $2E \geq 3F \Rightarrow 20 \geq 21$ —

Значит, K_5 не может быть планарн
 </p>
</div>
</details>

<div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-fuchsia-400">

 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-fuchsia-300 italic">
 <p class="text-xl font-mono text-white">
ают</p>
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
rder border-fuchsia-500 shadow-md">
 Аналогия: карта на стене
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Красим карту регионов: соседние
 Регион = вершина, общая граница
 Для плоской карты (планарный граф)
 теорема о четырёх красках (Аппеля)
 </p>
 </div>

 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
er border-amber-500 shadow-md">
 Что нужно знать на экзамен
```

```

 <p class="text-lg text-rose-300 italic">
 <div class="text-left font-mono text-sm">
 <p><strong class="text-white">Точка
рой увеличивает число компонент связности.
 <div class="p-3 bg-slate-950 rounded">

 <p class="text-xl font-bold tex">
 <p class="text-xs text-slate-400">
 </div>
 </div>
</div>

<p class="text-slate-300 text-sm">
 Важно: Это работает то
еются к точкам сочленения. То есть,
ег - это тупик со степенью 1).
</p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap">
 <div class="bg-slate-800 p-6 rounded-lg bor">
 <div class="absolute -top-3 left-4 bg-s
rder-rose-500 shadow-md">
 Аналогия: Главный перекресток
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Представь город, где два района сое
её перекроют (удалят вершину) – районы буд
g>хрупкость системы.
 </p>
 </div>

 <div class="bg-slate-900 p-6 rounded-lg bor">
 <div class="absolute -top-3 left-4 bg-e
er border-emerald-500 shadow-md">
 Телеком: Центральный свитч
 </div>
 <p class="text-slate-300 text-sm mb-4">
```

```

 IS_CUTPOINT(v);
 }</pre>
 </div>
 </div>
</details>

<div class="bg-indigo-950/60 p-6 rounded-xl border-2 border-indigo-400">
 <h4 class="text-lg font-bold text-indigo-400">Слабые звенья
 <p class="text-slate-300 text-sm mb-4">
 Это глубокий дуальный мост между ориентацией и связностью.
 </p>
 <ul class="list-disc list-inside text-sm text-slate-300">
 <strong class="text-rose-400">Точки разрыва и показывают физическую уязвимость связности.
 <strong class="text-emerald-400">SCC и независимые "конгломераты".
 Как точка сочленения рушит связность? Косарайю, свернув каждую SCC в одну супер-вершину (сделаем неориентированным), то любая вершина с степенью 1 — это и есть критическая SCC! Если удалить эту вершину, одно слабое звено изолирует целые касты SCC.

</div>
</div>
</section>`,
 },
];

```

ФАЙЛ 19/96: src/data/content.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 129 | РАЗМЕР: 1.2 КБ

```

import { Chapter } from "../types";
import { graphChapters } from "../graphContent";
import { additionalTickets } from "../additionalTickets";
import { tickets1to5 } from "../tickets/ticket1 5";

```

```

 ...graphChapters,
 // билет 1 (segment-trees) есть только здесь; остальные
 ...tickets1to5,
 ...additionalTickets,
 ...tickets6to13,
 ...tickets14to20,
 ...tickets21to24,
 ...extraChapters,
].filter((c) => c && c.id);

// Старые версии страниц, у которых есть свежая замена
// graph-planar-colors → planarity-euler-formula, graph-bridges → planarity-bridges
// graph-euler → euler-path-vs-cycle, pathfinding-accel → pathfinding-accel
const stale = new Set([
 "graph-planar-colors",
 "graph-bridges",
 "graph-euler",
 "pathfinding-accel",
]);

// Дубликаты: берём последнюю (самую свежую) версию стр.
const dedup = new Map<string, Chapter>();
merged.forEach((c) => {
 if (c.id === "alg-map") return; // страница-заглушка
 if (stale.has(c.id)) return;
 dedup.set(c.id, c);
});

// Нормализуем названия по каноническому списку тем, а
// по учебным блокам: блоки идут строго в порядке чтения
// номерам билетов. Близкие темы (1–5, 14–17, 18–20, 21–24)
const BLOCKS: [number, number, string][] = [
 [1, 5, "Билеты 1–5 • Структуры и матрицы"],
 [6, 13, "Билеты 6–13 • Графы: обходы и связность"],
 [14, 17, "Билеты 14–17 • Кратчайшие пути"],
 [18, 20, "Билеты 18–20 • Остовные деревья"],
 [21, 23, "Билеты 21–23 • Строки"],
 [24, 24, "Билет 24 • Теория сложности"],
];

```

```
floyd: ["dijkstra", "johnson-algo"],
"johnson-algo": ["bellman-ford", "dijkstra"],
"mst-kruskal": ["mst-prima", "mst-boruvka"],
"mst-prima": ["mst-kruskal", "mst-boruvka"],
"mst-boruvka": ["mst-kruskal", "mst-prima"],
"string-kmp": ["string-z-func", "aho-corasick"],
"string-z-func": ["string-kmp", "aho-corasick"],
"aho-corasick": ["string-kmp", "string-z-func"],
"complexity-classes": [],
};
```

---

ФАЙЛ 20/96: src/data/debugRunner.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРOK: 184 | РАЗМ

---

```
/**
 * Пошаговый дебаггер поверх Pyodide: CPython выполняет
 * под sys.settrace и на каждой строке записывает фрейм
 * как панель Variables в обычном отладчике.
 */
```

```
export type DebugValue = null | boolean | number | string
```

```
export interface DebugStep {
 /** Номер строки в коде пользователя (1-based). */
 line: number;
 /** Имя функции-фрейма ("<module>" – верхний уровень) */
 func: string;
 /** Локальные переменные: имя → герг (усечённый), для
 locals: Record<string, string>;
 /**
 * JSON-снимок доступных имён (globals + locals). В о
 * можно напрямую отдать визуализации: i/j остаются ч
 */
 values?: Record<string, DebugValue>;
}
```

```

except Exception:
 r = "<...>"
 return r if len(r) <= 80 else r[:77] + "..."

def __snapshot(v, depth=0):
 """Небольшой JSON-снимок: UI использует его без парсера.
 if v is None or isinstance(v, (bool, int, str)):
 return v
 if isinstance(v, float):
 return v if v == v and v not in (float("inf"), float("-inf")) else "..."
 if depth >= 4:
 return __safe(v)
 limit = 80 if depth < 2 else 32
 if isinstance(v, (list, tuple)):
 return [__snapshot(x, depth + 1) for x in list(v)[:limit]]
 if isinstance(v, (set, frozenset)):
 return [__snapshot(x, depth + 1) for x in list(v)[:limit]]
 if isinstance(v, dict):
 # 8x8 Sparse Table содержит 64+49+25+1 = 139 элементов
 # предел 80 незаметно обрезал уровни 4x4 и 8x8
 dict_limit = 192 if depth == 0 else limit
 return {str(k): __snapshot(x, depth + 1) for k, x in list(v.items())[:dict_limit]}
 # deque, heap-подобные и другие итерируемые учебные
 if type(v).__name__ == "deque":
 return [__snapshot(x, depth + 1) for x in list(v)[:limit]]
 return __safe(v)

def __visible(items):
 return {
 k: v for k, v in items
 if not k.startswith("__") and type(v).__name__ != "function"
 }

def __capture(frame):
 local_values = __visible(frame.f_locals.items())
 # Внутри функции алгоритма важные структуры (методы, атрибуты)
 # глобальные. Локальные значения имеют приоритет над глобальными
 scope_values = __visible(frame.f_globals.items())

```



```

finally:
 __sys.settrace(None)

__OUT = __json.dumps({
 "steps": __steps,
 "finalLocals": __final,
 "finalValues": __final_values,
 "error": __err,
 "truncated": __was_truncated,
})
__OUT
\;
}

/**
 * Какие переменные изменились на этом шаге относительно
 * они подсвечиваются (как amber-подсветка изменений в
 * Новые переменные тоже считаются изменёнными.
 */
export function changedVars(
 prev: Record<string, string> | null | undefined,
 cur: Record<string, string>
): Record<string, boolean> {
 const out: Record<string, boolean> = {};
 for (const k of Object.keys(cur)) {
 out[k] = !prev || !(k in prev) || prev[k] !== cur[k]
 }
 return out;
}

/** Базовое имя из карточки переменной: "tin[v]" → "tin"
export const baseVarName = (name: string): string => name

/**
 * Порядок показа в панели переменных: сначала переменные
 * страницы (те, что подсвечивает визуализация), остальные
 */
export function orderVars(

```

```
 /** id полного симулятора (см. vizRegistry) – кнопка
 simulator?: string;
}

export const GLOSSARY: GlossaryEntry[] = [
 {
 id: "bfs",
 labels: ["BFS", "обход в ширину", "поиск в ширину",
 title: "BFS – обход в ширину",
 short:
 "Волна от источника: сначала все соседи, потом со
 formal:
 "Очередь (FIFO). Кладём старт, достаём вершину –
 у рёбер.",
 complexity: "O(V + E)",
 demo: "bfs",
 simulator: "queue-bfs",
 },
 {
 id: "dfs",
 labels: ["DFS", "обход в глубину", "поиск в глубину",
 title: "DFS – обход в глубину",
 short:
 "Прёшь вперёд до упора, упёрся – откатываешься на
 formal:
 "Стек (или рекурсия). Из DFS растут: времена вход
 complexity: "O(V + E)",
 demo: "dfs",
 simulator: "stack-dfs",
 },
 {
 id: "binsect",
 labels: ["binsect", "бинсект", "бинарный поиск", "д
 title: "Бинарный поиск (и его самозванец binsect)",
 short:
 "Бинарный поиск НАХОДИТ готовый элемент в уже отс
 о не сортирует. «binsect» из конспекта – шуточный
 quicksort.",
```

```
 title: "DSU — система непересекающихся множеств",
 short:
 "«Кто твой староста?» Каждый элемент знает своего",
 formal:
 "find со сжатием путей + union по рангу/размеру.",
 complexity: "почти $O(1)$ — обратная функция Аккермана",
 demo: "dsu",
 simulator: "mst-kruskal",
},
{
 id: "trie",
 labels: ["бор", "бора", "боре", "бору", "бором", "т"],
 title: "Бор (trie, префиксное дерево)",
 short:
 "Дерево, где путь от корня по буквам и есть слово",
 formal: "Из корня по символам; в вершине хранится ф.",
 complexity: "поиск слова — $O(|\text{слова}|)$ ",
 demo: "trie",
 simulator: "aho-corasick",
},
{
 id: "bfs-on-trie",
 labels: [
 "обход дерева",
 "обход бора",
 "обход в ширину по бору",
 "какой-то обход",
 "обходом дерева",
 "конечный автомат",
 "конечного автомата",
 "автомат",
 "автомата",
],
 title: "Обход бора в ширину (тот самый «какой-то об",
 short:
 "В Ахо–Корасике бор обходят именно BFS: суффиксн",
 "о есть строго по уровням.",
 formal:
```

```
demo: "relax",
simulator: "dijkstra",
},
{
id: "prefix-sum",
labels: ["префиксные суммы", "префиксная сумма", "префикс"],
title: "Префиксные суммы",
short:
"Заранее посчитанные «сколько всего набежало от нас до i»",
formal: "P[0]=0, P[i]=P[i-1]+a[i-1]. Сумма a[l..r] = P[r]-P[l-1].",
complexity: "препроцессинг O(n), запрос O(1)",
demo: "prefix-sum",
simulator: "prefix-sums-2d",
},
{
id: "sparse-table",
labels: [
"разреженная таблица", "разреженная таблица", "sparse-table",
"разреженная", "двоичные подъёмы", "двоичный подъём",
],
title: "Разреженная таблица (метод двоичных подъёмов)",
short:
"Заранее считаем ответ для всех отрезков длины 1, 2, 4, ..., m.",
formal: "ST[i][j] = op(ST[i][j-1], ST[i+2^(j-1)][j-1])",
complexity: "препроцессинг O(n log n), запрос O(1)",
demo: "sparse-table",
simulator: "sparse-table",
},
{
id: "segment-tree",
labels: ["дерево отрезков", "дерево отрезков", "дерево отрезков"],
title: "Дерево отрезков",
short:
"Матрёшка из отрезков: корень отвечает за весь массив, а потом делится на две части.",
formal: "Строится за O(n), запрос и обновление — O(log n)",
demo: "segment-tree",
simulator: "segment-tree",
}
```

```
{
 id: "amortized",
 labels: [
 "амортизированная", "амортизированный", "амортизи",
 "амортизированное", "амортизированного", "амортизи",
],
 title: "Амортизированная сложность",
 short:
 "Средняя цена операции «по абонементу»: одна опер.
 formal: "Метод потенциалов: дорогая операция оплачивае",
 ассив.",
 demo: "amortized",
},
{
 id: "np",
 labels: [
 "NP-полная", "NP-полнота", "NP-трудная", "NP-полн",
 "класс P", "полином", "полиномиальное",
],
 title: "Класс NP и NP-полнота",
 short:
 "NP — «решение проверить легко, найти трудно» (су",
 "е в NP.",
 formal: "Задача NP-полна, если она в NP и к ней полн",
 demo: "np",
},
{
 id: "potential",
 labels: ["потенциал", "потенциалы", "потенциалов",
 title: "Потенциалы (перевзвешивание Джонсона)",
 short:
 "Трюк «поднять все дороги на одинаковую высоту»:
 formal: " $w'(u,v) = w(u,v) + h(u) - h(v)$, где h — ра",
 к путей сохраняется.",
 demo: "relax",
 simulator: "johnson-algo",
},
{
```

```

 demo: "prefix-function",
 simulator: "string-z-func",
},
{
 id: "dp",
 labels: [
 "динамическое программирование", "динамики", "дин
 "мемоизация", "мемоизацию", "подзадач", "подзадач
],
 title: "Динамическое программирование",
 short:
 "Решаем задачу через ответы на её меньшие копии и
 formal:
 "Нужны: оптимальная подструктура + перекрывающиеся
 ция).",
 demo: "dp",
 simulator: "dynamic-programming",
},
{
 id: "shortest-path",
 labels: ["кратчайший путь", "кратчайшие пути", "кра
 title: "Кратчайший путь",
 short:
 "Самый дешёвый маршрут из A в B. «Дешёвый» – по с
 formal:
 "Дейкстра – неотрицательные веса, $O(E \log V)$. Фор
 ы, $O(V^3)$.",
 demo: "relax",
 simulator: "dijkstra",
},
{
 id: "negative-cycle",
 labels: ["отрицательный цикл", "отрицательного цикл
 title: "Отрицательный цикл",
 short:
 "Круг, пройдя по которому вы становитесь «богаче»
 formal:
 "Форд–Беллман: если на V -й итерации ещё удаётся с

```

```

 },
 {
 id: "splay",
 labels: [
 "splay", "splay-дерево", "AVL", "AVL-дерево", "вс
],
 title: "Splay и повороты",
 short:
 "Каждый раз, когда к узлу обратились, его «вытягивае
 formal:
 "Три случая: zig (родитель – корень), zig-zig (уз
 рацию.",
 demo: "amortized",
 simulator: "splay-tree",
 },
 {
 id: "adjacency",
 labels: ["матрица смежности", "список смежности", "
 title: "Способы хранить граф",
 short:
 "Матрица смежности – таблица «есть ли ребро», быс
 й, экономно для разреженных графов.",
 formal: "Матрица: проверка $O(1)$, память $O(V^2)$. Спис
 },
 {
 id: "dijkstra",
 labels: ["Дейкстра", "Дейкстры", "Дейкстре", "Дейкс
 title: "Алгоритм Дейкстры",
 short:
 "Жадный «навигатор»: всегда раскрываем ближайший
 льные.",
 formal:
 "Куча хранит пары (расстояние, вершина). Достали
 complexity: " $O(E \log V)$ с бинарной кучей",
 demo: "relax",
 simulator: "dijkstra",
 },
],
 {

```

```

 усок.",
 complexity: "O(E log V)",
 demo: "mst",
 simulator: "mst-prima",
 },
 {
 id: "boruvka",
 labels: ["Борувка", "Борувки", "Борувку", "Boruvka"],
 title: "Алгоритм Борувки",
 short:
 "Все компоненты одновременно шлют «гонца» к ближайшим",
 formal:
 "Фаза: для каждой компоненты ищем минимальное исх",
 complexity: "O(E log V)",
 demo: "mst",
 simulator: "mst-boruvka",
 },
 {
 id: "kruskal",
 labels: ["Краскал", "Краскала", "Краскалу", "Kruskal"],
 title: "Алгоритм Краскала",
 short:
 "Сортируем все рёбра по весу и берём подряд те, ч",
 formal:
 "Сортировка O(E log E) + E операций union/find. С",
 complexity: "O(E log E)",
 demo: "mst",
 simulator: "mst-kruskal",
 },
 {
 id: "components",
 labels: [
 "компонента связности", "компоненты связности", "к",
 "компоненте связности", "связности", "связный", "с",
],
 title: "Компоненты связности",
 short:
 "Граф-архипелаг: острова, между которыми нет мостов",
 }

```



```

 short:
 "Одно дерево живёт по двум правилам сразу: по ключу
 ча. Случайность и держит его сбалансированным."
 formal:
 "Базис – split(t, x) (разрезать по ключу) и merge
 я через них. Ожидаемая высота $O(\log n)$.",
 complexity: "ожидаемо $O(\log n)$ ",
 demo: "heap",
},
{
 id: "bst",
 labels: ["бинарное дерево поиска", "дерево поиска",
 title: "Бинарное дерево поиска",
 short:
 "Слева всё меньше корня, справа – всё больше. Поиск
 formal:
 "Без балансировки дерево может вырождаться в список
 complexity: " $O(\text{высоты})$: от $O(\log n)$ до $O(n)$ ",
 demo: "segment-tree",
},
{
 id: "inclusion-exclusion",
 labels: [
 "включений-исключений", "включения-исключения", "
 "вырезание прямоугольников",
],
 title: "Включения-исключения на префиксных суммах",
 short:
 "Чтобы получить сумму прямоугольника, берём больше
 рый отрезали дважды.",
 formal:
 " $\text{Sum}(x1..x2, y1..y2) = P[x2][y2] - P[x1-1][y2] -$
 complexity: "запрос $O(1)$ после $O(nm)$ предподсчёта",
 demo: "prefix-sum",
 simulator: "prefix-sums-2d",
},
{
 id: "reduction",

```

```

 "Локальная перевеска двух узлов: ребёнок встаёт на
 дившемся месту. Порядок ключей при этом не лом
formal:
 "Правый поворот вокруг p: $x = p.left$; $p.left = x$.
 g и Zig-Zag.",
complexity: "O(1)",
demo: "zig",
simulator: "splay-rotations",
},
{
 id: "zig",
 labels: ["Zig", "Zag", "зиг"],
 title: "Zig – одиночный поворот",
 short:
 "Самый простой случай: родитель x уже сидит в корн
 канчивается, если глубина была нечётной.",
 formal: "Применяется ровно один раз за всю операцию
demo: "zig",
simulator: "splay-rotations",
},
{
 id: "zig-zig",
 labels: ["Zig-Zig", "зиг-зиг", "ZigZig"],
 title: "Zig-Zig – x и родитель с одной стороны",
 short:
 "Дерево вытянулось в «бамбук»: $g \rightarrow p \rightarrow x$ идут в о
 м (p, x). Именно это вдвое сокращает глубину вс
 formal:
 "Если крутить наивно (два раза поднимать x), бамб
demo: "zig-zig",
simulator: "splay-rotations",
},
{
 id: "zig-zag",
 labels: ["Zig-Zag", "зиг-заг", "ZigZag", "зигзаг"],
 title: "Zig-Zag – x и родитель с разных сторон",
 short:
 "Узлы идут «змейкой»: p – левый сын g, а x – прав

```

```

 оезда). Алгоритмы поиска кратчайшего пути по
 <div class="mt-8 mb-4">
 <div id="slot-mnemonic-cards"></div>
 </div>
 </div>
</section>`
},
{
 id: "dijkstra",
 title: "Билет 14. Дейкстра",
 type: "html",
 category: "Графы",
 content: `<section id="dijkstra-content" class="mb-
<div class="flex items-center mb-6">

 <h2 class="text-3xl font-bold text-white">Дейкстра
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border
 <h3 class="text-xl font-bold text-emerald-400">

 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
 order border-emerald-500 shadow-md">
 Аналогия 1: Позитивное планирование
 </div>
 <p class="text-slate-300 text-sm mb-4">
 (нельзя поехать и заработать). Он всегда в
 <div id="slot-chapter-image-dijkstra">

```

```

</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-rose-400">

 <div class="bg-slate-900 p-4 rounded-lg mb-1">
 <p class="text-lg text-rose-300 italic">
 <p class="text-xl font-mono text-white">
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 ♂ Аналогия 1: Параноик
 </div>
 <p class="text-slate-300 text-sm mb-1">
 еряет ВСЕ возможные дороги V-1 раз подряд.
 <div id="slot-chapter-image-bellman"
 </div>

 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
 Отрицательный цикл
 </div>
 <p class="text-slate-300 text-sm mb-1">
 г станет ЕЩЕ короче – значит мы попали во в
 </div>
 </div>

 <div id="slot-bellman-viz" class="mt-8"></div>
 </div>
 </div>
</section>`

```

```

 </div>
 <p class="text-slate-300 text-sm mb-6">
 три вложенных цикла. Сложность $O(V^3)$. Если
 </div>
 </div>

 <div id="slot-floyd-viz" class="mt-8"></div>
</div>
</section>`
},
{
 id: "aho-corasick",
 title: "Билет 23. Алгоритм Ахо-Корасик",
 type: "html",
 category: "Строки",
 content: `<section id="aho-corasick" class="mb-12">
<div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border-l">
 <h3 class="text-xl font-bold text-blue-400 mb-4">
 <div class="bg-slate-900 p-4 rounded-lg mb-6 text-white">
 <p class="text-sm text-blue-300 italic mb-2">Осложнен
 <p class="text-lg font-mono text-white">Бор (Тр
 </div>
 <p class="text-slate-300 text-sm">Позволяет найти
 -mono text-amber-300 bg-black/30 px-1 rounded">
 y.</p>
 </div>

<!-- Аналогии -->
<div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg border bo

```

```

// Расчет ссылок идет через BFS (level-order traversal)
// так как для вычисления link вершины v на глубине d,
// её предки на глубине d-1 уже должны иметь вычисленные

```

```

void buildLinks() {
 queue<int> q;
 // Корни и их дети
 for (auto const& [ch, child] : nodes[0].trie) {
 nodes[child].link = 0;
 q.push(child);
 }

 while (!q.empty()) {
 int v = q.front(); q.pop();
 for (auto const& [ch, child] : nodes[v].trie) {
 // Суффиксная ссылка ребёнка – это переход к
 nodes[child].link = nodes[nodes[v].link].go

 // Если суффиксная ссылка сама является словом
 // Иначе наследуем terminal_link
 if (nodes[nodes[child].link].is_terminal) {
 nodes[child].term_link = nodes[child].link;
 } else {
 nodes[child].term_link = nodes[nodes[ch].term_link];
 }

 q.push(child);
 }
 }
}

```

```

</div>
</div>
</details>
</div>
</section>`

```

```

},
{
 id: "alg-map",
 title: "Алгоритмы на карте",

```

```

 "Есть эйлеров цикл (вернусь домой!)",
 "Есть эйлеров путь (из одной нечётной в другую)",
 "Ни пути, ни цикла нет – это полный провал"
],
 correctIndex: 1,
 explanation: "Если нечётных вершин ровно 2 – это эйлеров путь, если нечётных больше 2 – эйлерова пути нет, заканчиваем в другой."
},
{
 question: "Почему гамильтонов цикл – это NP-полная задача?",
 options: [
 "Потому что Гамильтон – болезнь, и лечить её тяжело",
 "Потому что для эйлерова цикла есть простой критерий",
 "Потому что Эйлер был умнее Гамильтона"
],
 correctIndex: 1,
 explanation: "Для эйлерова цикла работает критерий на количество нечётных вершин. Это NP-полная задача, которую не решить за полиномиальное время."
},
{
 question: "Куда ведёт эйлеров путь, если нечётных вершин больше 2?",
 options: [
 "Всё ещё можно пройти по всем рёбрам ровно раз",
 "Такого графа не существует",
 "Эйлерова пути нет – нужно удалять лишние рёбра"
],
 correctIndex: 2,
 explanation: "Если нечётных вершин больше 2, эйлерова пути нет. Если нечётных вершин ровно 2 нечётные или 0."
}
],
"bridges-code": [
 {
 question: "После DFS у нас low[u] = 4, а tin[v] = 2. Есть ли мост между u и v?",
 options: [
 "Да, так как low[u] (4) > tin[v] (2)",
 "Нет, low[u] должно быть меньше или равно",
 "Только если граф связный и неориентированный"
]
 }
]

```

```

 {
 question: "Сколько граней у планарного графа с V=10, E=17?",
 options: [
 "3 грани",
 "5 граней",
 "10 граней"
],
 correctIndex: 1,
 explanation: " $F = E - V + 2 = 10 - 7 + 2 = 5$. Не 3, а 5.",
 },
 {
 question: "Почему K5 непланарен?",
 options: [
 "Потому что $10 > 3*5 - 6 = 9$ — нарушение",
 "Потому что Эйлер был против",
 "Потому что 5 вершин — несчастливое число"
],
 correctIndex: 0,
 explanation: "У K5: V=5, E=10, $3V - 6 = 9$. $10 > 9$ — нарушение.",
 }
]
};

```

ФАЙЛ 24/96: src/data/vizStepBus.ts

КАТЕГОРИЯ: Контент и данные пособия | СТР0К: 158 | РАЗМЕР: 1.1 КБ

```

import { createContext, useContext, useEffect, useState } from "react";
import type { DebugValue } from "../debugRunner";

```

```

/**

```

```

 * Прямая связь «компилятор ↔ открытая визуализация».

```

```

 *

```

```

 * Компилятор передаёт не «магический номер строки эталонного

```

```

 * выполненного Python-кода. Визуализация читает захардкоженные

```

```

 * (i, j, k, v, dist, P, st...) и подсвечивает соответствующие

```



```

/** Текущий снимок компилятора для прямой отрисовки мас
export function useVizRuntime(): VizStateEventDetail | null {
 const chapterId = useContext(VizChapterContext);
 const [runtime, setRuntime] = useState<VizStateEventDetail>()
 chapterId ? latestByChapter.get(chapterId) ?? null : null
 };

 useEffect(() => {
 setRuntime(chapterId ? latestByChapter.get(chapterId) : null);
 if (!chapterId) return;
 const handler = (e: Event) => {
 const detail = (e as CustomEvent<VizStateEventDetail>).detail;
 if (detail?.chapterId === chapterId) {
 setRuntime(detail.variables || detail.line !== null ? detail : null);
 }
 };
 window.addEventListener(CHANNEL, handler);
 return () => window.removeEventListener(CHANNEL, handler);
 }, [chapterId]);

 return runtime;
}

export type VizStepSelector = {variables: Record<string, string>};

/**
 * Подписка старых покадровых демонстраций. Если передан номер кадра
 * номер кадра выбирается по именам/значениям переменных.
 */
export function useVizStepSync(
 currentStep: number,
 goToStep: (n: number) => void,
 maxStep: number,
 selectByVariables?: VizStepSelector
) {
 const chapterId = useContext(VizChapterContext);

```

-----  
/\* — Выбор демонстрации внутри страницы (вкладки 1D / 2D)

```
const DEMO_CHANNEL = "algo:viz-demo";
const latestDemoByChapter = new Map<string, string>();

export interface VizDemoEventDetail {
 chapterId: string;
 /** base-запись страницы – demoId пустой; подписанные */
 demoId: string;
}

/** Демонстрация сообщает, какая её вкладка сейчас открыта */
export function emitVizDemo(chapterId: string, demoId: string) {
 if (typeof window === "undefined") return;
 latestDemoByChapter.set(chapterId, demoId);
 window.dispatchEvent(new window.CustomEvent<VizDemoEventDetail>({
 detail: { chapterId, demoId }
 }));
}

/**
 * Подписка: панель компилятора следует за активной вкладкой
 * Сразу отдаём последнее значение: компилятор часто отстает
 * как пользователь переключился с 1D на 2D.
 */
export function onVizDemo(chapterId: string, cb: (demoId: string) => void) {
 if (typeof window === "undefined") return () => {};
 const latest = latestDemoByChapter.get(chapterId);
 if (latest !== undefined) cb(latest);
 const handler = (e: Event) => {
 const detail = (e as CustomEvent<VizDemoEventDetail>).detail;
 if (detail && detail.chapterId === chapterId) cb(detail.demoId);
 };
 window.addEventListener(DEMO_CHANNEL, handler);
 return () => window.removeEventListener(DEMO_CHANNEL, handler);
}
```



```

 { name: "case", role: "какой случай крутить: zig"},
 { name: "i", role: "номер кадра в разборе случая"},
 { name: "x, p, g", role: "тройка узлов: поднимаем"},
],
},
"sparse-table": {
 vizTitle: "Разреженная таблица (1D и 2D)",
 stepNote: "Шаг = одна ячейка $st[i][j] = \min(st[i][j - 1], st[i + (1 \ll (j - 1))][j - 1])$ ",
 code: `a = [2, 3, 5, 62, 3, 21, 1, 4]
n, LOG = 8, 4
i, j = 0, 0
st = [] # сюда складываем строки
for row in a:
 st.append([row] + [None] * (LOG - 1))

for j in range(1, LOG):
 for i in range(n - (1 << j) + 1):
 st[i][j] = min(st[i][j - 1], st[i + (1 << (j - 1))][j - 1])
 variables: [
 { name: "a", role: "исходный массив демо (вкладка 1D)" },
 { name: "n, LOG", role: "длина массива и число уровней" },
 { name: "i", role: "начало блока: $st[i][j]$ отвечает за блок длины 2^j " },
 { name: "j", role: "уровень таблицы: блок длины 2^j " },
 { name: "st", role: "таблица уровней – подсвечиваем" },
],
},
"prefix-sums-2d": {
 vizTitle: "Префиксные суммы (1D и 2D)",
 stepNote: "Шаг = заполнение очередного $P[i] = P[i - 1] + a[i - 1]$ ",
 code: `a = [3, 1, 4, 1, 5, 9, 2, 6]
i = 0

P = [0]
for i in range(1, len(a) + 1):
 P.append(P[i - 1] + a[i - 1])
print("P =", P)`
 variables: [
 { name: "a", role: "исходный массив демо (вкладка 1D)" },

```

```

kids = (2 * i + 1, 2 * i + 2) # дети
print(f"i={i} parent={parent} kids={kids}")`,
 variables: [
 { name: "n", role: "текущее число элементов в куче"},
 { name: "i", role: "индекс элемента, который сейчас обрабатываем"},
 { name: "(i-1)//2", role: "индекс родителя вершины"},
 { name: "2*i+1, 2*i+2", role: "индексы левого и правого ребенка"},
],
},
"stack-dfs": {
 vizTitle: "Стек и обход в глубину",
 stepNote: "Шаг = push новой тарелки на стек или pop",
 code: `st = [] # стек = рекурсия
st.append("A") # push — зашли в вершину
st.append("B")
top = st.pop() # pop — возврат на уровень
print("pop →", top, "стек:", st)`,
 variables: [
 { name: "st", role: "стек (массив): тарелки стран"},
 { name: "top", role: "что сняли с вершины — текущая вершина"},
],
},
"queue-bfs": {
 vizTitle: "Очередь и обход в ширину",
 stepNote: "Шаг = dequeue вершины из головы очереди",
 code: `from collections import deque

q = deque(["A"]) # очередь BFS (FIFO)
q.append("B") # enqueue соседа в хвост
v = q.popleft() # шаг: обрабатываем голову
print("v =", v, "очередь:", list(q))`,
 variables: [
 { name: "q", role: "очередь (массив): гости страны"},
 { name: "v", role: "вершина, извлечённая из головы очереди"},
],
},
"graph-dfs-bfs": {
 vizTitle: "DFS и BFS на графе",

```

```
демо-граф страницы: три «острова» A-D, E-G, H-J
adj = {"A": ["B", "C", "D"], "B": ["A", "C"], "C": ["A", "B", "D"],
 "E": ["F", "G"], "F": ["E", "G"], "G": ["E", "F"]}
comp = {} # comp[v] – номер компонента
count = 0 # сколько раз запускали обход
v, q = None, [] # текущая вершина и очередь

--- тело алгоритма ---
for s in adj:
 if s in comp:
 continue
 count += 1
 q = deque([s])
 comp[s] = count - 1
 while q:
 v = q.popleft()
 for to in adj[v]:
 if to not in comp:
 comp[to] = comp[v]
 q.append(to)
print(comp, "компонент:", count)`
variables: [
 { name: "v", role: "текущая вершина обхода – подсказка", range: "A-J",
 { name: "comp", role: "словарь меток: номер компонента", range: "0-3",
 { name: "q", role: "текущая очередь BFS", range: "0-4",
 { name: "count", role: "число запусков обхода = число компонент", range: "0-3"
],
},
"top-sort": {
 vizTitle: "Топологическая сортировка",
 stepNote: "Шаг = выход рекурсии из вершины v: она должна быть обработана",
 code: `adj = {0: [1, 2], 1: [3], 2: [3], 3: [4], 4: [5]}`
used, order = set(), []

def dfs(v):
 used.add(v)
 for to in adj[v]: # шаг: ребро v → to
```

```

used = set()
comp = {}
num = 0
for v in reversed(order): # проход 2: по order задом
 if v in used:
 continue
 num += 1
 stack = [v]
 used.add(v)
 while stack:
 u = stack.pop()
 comp[u] = num
 for to in adjT[u]:
 if to not in used:
 used.add(to)
 stack.append(to)
print("компонент:", num, comp)\,
 variables: [
 { name: "v", role: "текущая вершина DFS (проход 1"
 { name: "order", role: "порядок выхода прохода 1 -"
 { name: "adjT", role: "транспонированный граф: вс"
 { name: "comp", role: "номер SCC каждой вершины",
],
},
"graph-articulation": {
 vizTitle: "Точки сочленения",
 stepNote: "Шаг = обновление low[v] по ребру; вершин"
 code: `# демо-граф страницы: треугольник 0-1-2, мос
DFS из 0: tin/low после полного обхода; точки сочленен
tin = {0: 1, 1: 2, 2: 3, 3: 4, 4: 5, 5: 6, 6: 7}
low = {0: 1, 1: 1, 2: 1, 3: 4, 4: 5, 5: 5, 6: 5}

v, to = 3, 4 # шаг: возврат из 4 в 3
is_cut = low[to] >= tin[v] # 5 >= 4 → 3 – точка
v2, to2 = 1, 2
is_cut2 = low[to2] >= tin[v2] # 1 >= 2 → нет (1 с
print(f"v={v}: точка сочленения? {is_cut}; v={v2}: {is

```

```

for v in deg:
 if deg[v] % 2:
 odd.append(v)
print(deg, "нечётных:", len(odd))
0 нечётных → цикл · 2 → путь · иначе — нельзя`,
variables: [
 { name: "path", role: "маршрут списком меток вершин"},
 { name: "deg", role: "степень вершины: чётная вся"},
],
},
"planarity-euler-formula": {
 vizTitle: "Планарность: K5/K3,3 + раскраска",
 stepNote: "Шаг = проверка формулы Эйлера для K5/K3,3",
 code: `V, E = 5, 10 # K5 (K3,3: V=6, E=9)
F = 2 - V + E # формула: V - E + F = 2
not_planar = E > 3 * V - 6 # 10 > 9 → K5 непланарен

покраска (вторая демонстрация страницы):
смежности демо-графа из 7 вершин A..G
adj = [[1, 4, 5], [0, 2, 5], [1, 3, 5, 6], [2, 4, 5, 6],
 [3, 0, 5], [0, 1, 2, 3, 4, 6], [2, 3, 5]]
color = [-1] * 7 # color[v] — цвет вершины (0..6)
k, v = 0, 0 # сколько цветов использовано
for v in range(7): # жадная раскраска
 used = {color[u] for u in adj[v] if color[u] >= 0}
 c = 0
 while c in used: c += 1
 color[v] = c
 k = max(k, c + 1)`,
variables: [
 { name: "V", role: "число вершин графа", range: "1..7"},
 { name: "E", role: "число рёбер", range: "K5: 10"},
 { name: "F", role: "число граней в плоской укладке"},
 { name: "v", role: "вершина, которую красят (метка)"},
 { name: "color", role: "массив/словарь цветов вершин"},
 { name: "k", role: "сколько цветов уже использовано"},
],

```



## Универсальное пособие • полный исходный код

```

E = [("S", "A", 4), ("S", "B", 5), ("B", "C", -2), ("A",
 ("C", "D", 3), ("C", "E", 4), ("D", "F", 2), ("E",
n = 7 # вершин: S, A, B, C, D, E
dist = {"S": 0}
u, v, w = E[0] # шаг: очередное ребро (u
i = 1 # номер итерации

--- тело алгоритма ---
for i in range(1, n): # n-1 прохода по всем рёбрам
 for u, v, w in E:
 if dist.get(u, float("inf")) + w < dist.get(v, float("inf")):
 dist[v] = dist[u] + w # релаксация
print(dist)
n-й проход улучшил бы dist ⇒ отрицательный цикл (на э
variables: [
 { name: "E", role: "список рёбер (u, v, w) – демо",
 { name: "n", role: "число вершин: после n-1 итерации",
 { name: "i", role: "номер итерации: после i итерации",
 { name: "u, v, w", role: "текущее ребро: откуда, куда, вес",
 { name: "dist", role: "расстояния от S; улучшение",
],
},
"floyd": {
 vizTitle: "Флойд на матрице аэропортов",
 stepNote: "Шаг = проверка пары (i, j) при посреднике k",
 code: `INF = 999 # «бесконечность»
 # поэтому отдельные проверки
d = [[0, 4, INF, 2, INF, INF, INF], [INF, 0, 3, INF, INF, INF, INF],
 [INF, INF, INF, 0, INF, 1], [INF, INF, INF, INF, INF, INF, 0],
n = 7
k, i, j = 0, 0, 0 # шаг: посредник k, летим из i в j

--- тело алгоритма ---
for k in range(n): # посредник k
 for i in range(n):
 for j in range(n):
 if d[i][k] + d[k][j] < d[i][j]: # через k
 d[i][j] = d[i][k] + d[k][j]
```

```

w, u, v = E[0] # шаг: очередное ребро
i = 0

--- тело алгоритма ---
def find(x):
 while parent[x] != x:
 parent[x] = parent[x] # сжатие пути
 x = parent[x]
 return x

mst = []
for i, (w, u, v) in enumerate(E):
 if find(u) != find(v): # разные компоненты → берём
 parent[find(u)] = find(v)
 mst.append((w, u, v))
print("остов:", mst, "вес:", sum(w for w, _, _ in mst))
variables: [
 { name: "E", role: "рёбра демо-графа, отсортирован"},
 { name: "w, u, v", role: "текущее ребро: вес и ко"},
 { name: "i", role: "индекс текущего ребра в отсор"},
 { name: "parent", role: "DSU: представитель компо"},
 { name: "mst", role: "рёбра остова; в ответе равн"},
],
},
"mst-prima": {
 vizTitle: "Прим",
 stepNote: "Шаг = из кучи достаётся самое лёгкое ребро",
 code: `import heapq

смежность демо-графа страницы: 9 вершин A..I
adj = {"A": [(2, "B"), (5, "D")], "B": [(2, "A"), (4, "C"), (12, "H")],
 "C": [(4, "B"), (9, "F")], "D": [(5, "A"), (3, "E")],
 "E": [(7, "D"), (9, "C"), (13, "I")], "F": [(8, "C"), (9, "E"), (13, "I")],
 "G": [(8, "H"), (12, "I")], "H": [(12, "B"), (8, "G")], "I": [(13, "E"), (13, "F")]}
heap = [(0, "A", "-")] # (вес, вершина, откуда)
taken, mst = set(), []
w, v, parent = 2, "B", "A" # шаг: ребро из кучи
i = 0

```

```

 return x

while len({find(v) for v in comp}) > 1:
 cheapest = {}
 for w, u, v in E: # каждая компонента выбирает
 ru, rv = find(u), find(v)
 if ru == rv:
 continue
 if ru not in cheapest or cheapest[ru][0] > w:
 cheapest[ru] = (w, u, v)
 if rv not in cheapest or cheapest[rv][0] > w:
 cheapest[rv] = (w, u, v)
 for ru, (w, u, v) in cheapest.items(): # все слияния
 comp[find(u)] = find(v)
 print(f"фаза {phase}: осталось компонент", len({find(v) for v in comp}))
 phase += 1
print("готово за", phase - 1, "фазы (на демо — 2)")`,
variables: [
 { name: "E", role: "рёбра демо-графа", range: "12"},
 { name: "comp", role: "DSU: номер компоненты (племени)", range: "1"},
 { name: "cheapest", role: "самое дешёвое исходящее ребро", range: "1"},
 { name: "u, v, w", role: "ребро-кандидат: концы и вес", range: "1"},
 { name: "phase", role: "номер фазы параллельных слияний", range: "1"},
],
},
"string-kmp": {
 vizTitle: "Префикс-функция (КМП)",
 stepNote: "Шаг = обработка символа s[i]: откаты по префиксу",
 code: `pattern = "aba"
text = "abacaba"
s = pattern + "#" + text # демо-страница ищет pattern в text
i, j = 1, 0
pi = [None] * len(s) # пустые клетки: значения по умолчанию
pi[0] = 0

--- тело алгоритма ---
for i in range(1, len(s)): # шаг: считаем pi[i]
 j = pi[i - 1] # длина совпадения
 while s[i] != s[j]:
 j = pi[j - 1]
 pi[i] = j
print(pi)

```

```

 { name: "text", role: "текст, в котором ищем", ga
 { name: "s", role: "склейка pattern + \"#\" + tex
 { name: "i", role: "позиция, для которой считаем
 { name: "l, r", role: "правый Z-блок: отрезок, со
 { name: "z", role: "z[i] – длина общего префикса
],
},
"aho-corasick": {
 vizTitle: "Ахо–Корасик",
 stepNote: "Шаг = построение бора по образцам, затем
 code: `s = "USHERS" # текст, в котором
i, v = 0, 0 # позиция в тексте и вершина
trie = {"": {}} # бор: вершина → переходы
state, c = "", "u" # шаг: автомат идёт по символу

nxt = trie.get(state, {}).get(c, "") # go(state, c) с
print(f"шаг {i + 1}: v={v}, символ {c!r} → {nxt!r}")`,
 variables: [
 { name: "s", role: "текст для поиска (можно и tex
 { name: "i", role: "текущая позиция в тексте – ло
 { name: "v", role: "номер текущей вершины автомата
 { name: "c / state", role: "символ и состояние –
],
},
"complexity-classes": {
 vizTitle: "Классы сложности: матрёшка $P \subseteq NP \subseteq PSPACE$
 stepNote: "Шаг = подсветка класса/задачи по имени и
 code: `cls = "NP" # класс: "P", "NP"
task = "3-SAT" # задача: "3-SAT", "Vertex Cover"

сведение $A \leq_p B$: стрелка между задачами
reductions = [("3-SAT", "Vertex Cover")]
for a, b in reductions:
 print(f"{a} $\leq_p$ {b}")`,
 variables: [
 { name: "cls", role: "какой класс подсветить: P /
 { name: "task", role: "какую задачу подсветить: 3
 { name: "reductions", role: "список пар задач – п

```

```

 { name: "st2[{r,c,k,k})", role: "минимум квадратн
 },
],
},
"sparse-table#2d-query": {
 vizTitle: "Разреженная таблица 2D: запрос",
 stepNote: "Демонстрация кликабельна вручную: выбери
 code: `# st2[{r, c, kx, ky}] из вкладки «построение
r1, c1, r2, c2 = 1, 1, 6, 6

h, w = r2 - r1 + 1, c2 - c1 + 1
kx, ky = h.bit_length() - 1, w.bit_length() - 1
print(f"прямоугольник {h}x{w} кроется блоками уровня ({
 variables: [
 { name: "r1, c1, r2, c2", role: "углы запрашиваем
 { name: "kx, ky", role: "уровень самого крупного
],
},
"prefix-sums-2d#2d": {
 vizTitle: "Префиксные суммы 2D (прямоугольники)",
 stepNote: "Демонстрация кликабельна вручную: наведи
 code: `A = [[1, 2, 3, 4], [5, 6, 7, 8], [9, 1, 2, 3
n, m = 4, 4
i, j = 0, 0
S = [] # строки таблицы: рамка и
for rr in range(n + 1):
 S.append([0] + [None] * m)
S[0] = [0] * (m + 1)
for i in range(1, n + 1):
 for j in range(1, m + 1):
 S[i][j] = A[i - 1][j - 1] + S[i - 1][j] + S[i][

r1, c1, r2, c2 = 1, 1, 2, 2 # клетки 1-индексные, как
ans = S[r2 + 1][c2 + 1] - S[r1][c2 + 1] - S[r2 + 1][c1]
print("итого:", ans)`,
 variables: [
 { name: "i, j", role: "текущая ячейка таблицы S (
 { name: "S[i][j]", role: "сумма прямоугольника (0

```

```

const keep: string[] = [];
for (const line of code.split("\n")) {
 const t = line.trim();
 if (t === "# --- тело алгоритма ---" || /^(for|while)/.test(t)) {
 keep.push(line);
 }
}
// убрать пустые строки с конца
while (keep.length > 0 && keep[keep.length - 1].trim() === "") {
 keep.pop();
}

/**
 * Дефолтное содержимое редактора: шапка + ТОЛЬКО инициализация
 * переменные демо (без всего кода алгоритма).
 */
export function buildInitTemplate(chapterId: string, chapterTitle: string, sync: PageSync) {
 const vizTitle = sync.vizTitle;
 const head = `
 <div># «${chapterTitle}»\n# демо: ${vizTitle}\n#
 </div>
 <div># «${chapterTitle}»\n# демо на странице нет – св
 `;

 if (!sync.code) return head;
 const init = extractInit(sync.code);
 return `${head}\n${init}\n`;
}
}

```

## РАЗДЕЛ: БИЛЕТЫ (УЧЕБНЫЙ КОНТЕНТ)

-----  
ФАЙЛ 26/96: src/data/tickets/ticket1\_5.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 357 | РАЗМЕР: 1.5 КБ

```
import { Chapter } from '../../../types';
```







```

<td class="py-2 pr-4 text-s
<td class="py-2 pr-4 text-e
<td class="py-2 text-rose-3
</tr>
</tbody>
</table>
</div>
<div class="bg-slate-900/40 rounded-lg bord
 <p class="text-slate-300 text-sm mb-1">
 <p class="text-slate-400 text-sm mb-1">
летки дважды. Сумма ≠ идемпотентна.</p>
 <p class="text-slate-400 text-sm mb-1">
ации.</p>
 <p class="text-slate-400 text-sm">• Выб
>
</div>
</div>
</div>
</section>`
},
{
 id: "treap",
 title: "3. Декартово дерево (Treap)",
 type: "html",
 description: "Дерево поиска по ключу и Куча по приор
 category: "Продвинутые структуры",
 content: `
<section id="treap" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-v
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-1
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">

```

```

 }
 }</pre>
</div>
</details>
</div>
</div>
</section>`
},
{
 id: "splay-tree",
 title: "4. Splay-дерево",
 type: "html",
 description: "Дерево поиска, которое самобалансирует",
 category: "Продвинутые структуры",
 content: `
<section id="splay-tree" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 (Zig, Zig-Zig, Zig-Zag), гарантируя амортизацию
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
 border-emerald-500 shadow-md">
 Аналогия 1: VIP-лифт
 </div>
 <p class="text-slate-300 text-sm mb-4">
 вится корнем дерева (VIP-статус). Если ты член
 ка почти не требуется времени!</p>
 </div>

```

Оно поднимет узел, на котором  
ска. За счет сдвига этого листа в корень, д  
/p>

<div>

```
<strong class="text-white block
```

Если агент постоянно переключается между находящимися на разных концах дерева, его можно считать свинг-агентом. Постоянное свинг-переключение превратит

<div>

```
<strong class="text-white block
```

Хотя один поиск может стоить  
емах обладают прекрасным свойством: они не  
по пути. "Палочное" дерево прессуется в сб  
осов.

&lt;/section&gt;`

4.

3

```
id: "prefix-sums-2d",
```

```
title: "5. Двумерная матрица префиксных сумм".
```

```
type: "html",
```

```
description: "Сжатие суммы подматрицы за $O(1)$ ".
```

category: "Алгоритмы матрицы".

content: `

```
<section id="prefix-sums-2d" class="mb-12 scroll-mt-10":
```

```
<div class="flex items-center mb-6 flex-wrap gap-3":
```

`<span class="bg-indigo-600 text-white px-4 py-1`

## class="text-2xl sm:text-3xl font-bold text-violet-600">

&lt;/div&gt;

```
 </div>
 </div>
 </div>
 </section>`
 }
};
```

---

ФАЙЛ 27/96: src/data/tickets/ticket14\_20.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 248 | РАЗМ

---

```
import { Chapter } from '../types';
```

```
export const tickets14to20: Chapter[] = [
```

```
 {
 id: "dijkstra",
 title: "14. Графы. Поиск кратчайшего пути. Дейкстра",
 type: "html",
 category: "Графы. Пути",
 content: `
<section id="dijkstra-content" class="mb-12 scroll-mt-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 14
 <h2 class="text-2xl sm:text-3xl font-bold text-white">14. Графы. Поиск кратчайшего пути. Дейкстра
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-700">
 <h3 class="text-xl font-bold text-emerald-400">Аналогия 1
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">Дейкстра — это алгоритм поиска кратчайшего пути в графе.
 <p class="text-xl font-mono text-white">Аналогия 1
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4">
```

```

 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2"
 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 ♂ Аналогия 1: Параноик
 </div>
 <p class="text-slate-300 text-sm mb
еряет ВСЕ возможные дороги V-1 раз подряд.
 </div>
 <div class="bg-slate-900 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
 Отрицательный цикл
 </div>
 <p class="text-slate-300 text-sm mb
опали во временную петлю.</p>
 </div>
 </div>
 <div id="slot-bellman-viz" class="mt-8"></d
 </div>
 </div>
 </section>`
 },
 {
 id: "floyd",
 title: "16. Графы. Поиск кратчайшего пути. Флойд",
 type: "html",
 category: "Графы. Пути",
 content: `
<section id="floyd-content" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-2xl sm:text-3xl font-bold text-v
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-indigo-400

```

```


 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-amber-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-xl font-mono text-white">
 </div>
 <div class="grid grid-cols-1 gap-6 mb-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-amber-500 shadow-md">
 ⚡ Суть перевзвешивания
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Дейкстра (Билет 14) быстрая, но работает очень медленно. Мы хотим сделать Джонсона
 е пути остались теми же.
 </p>
 <ol class="text-sm text-slate-300 space-y-4">
 Добавляем фиктивную вершину
 Запускаем от неё медленный поиск
 «потенциалы» <code class="text-pink-400">
 Меняем старые веса: новое равенство
 Магия! Все веса теперь равны
 «потенциалы» сокращаются).
 Теперь смело запускаем быстрый поиск

 </div>
 </div>
 </div>
 <div id="slot-johnson-viz" class="mt-8"></div>
 </div>
</div>
</section>`
 },
 {
 id: "mst-kruskal",

```

```

<section id="mst-prima" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 которое торчит из посещенных в непосещенных
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
 order border-emerald-500 shadow-md">
 ✦ Аналогия: Заражение вирусом
 </div>
 <p class="text-slate-300 text-sm mb-4">
 ли вирус). Мы стартуем из одного города, и
 Сеть растет только из одного связного куска
 </div>
 </div>
 </div>
 </div>
 </div>
</section>`
 },
 {
 id: "mst-boruvka",
 title: "20. Графы. Остовное дерево. Борувка",
 type: "html",
 category: "Графы. Остовные",
 content: `
<section id="mst-boruvka" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">

```

```

type: "html",
category: "Строки",
content: `
<section id="string-kmp" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 сом, оканчивающимся в позиции i.</p>
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 ♂ Аналогия 1: Поиск без отка
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Мы идём по строке слева направо. К
 строки СЕЙЧАС закончился под моим пальцем?»
 Представь, ты ищешь в тексте слов
 х</code>. Тупой алгоритм начнет искать заново
 </code>, а это начало нашего слова! Не надо
 </p>
 </div>

 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Аналогия 2: LLM стриминг

```



```

<div class="p-5 text-sm text-slate-300" style="background-color: #e0e0e0; padding: 10px; border-radius: 10px;">
 <pre class="bg-slate-950 p-4 rounded" style="background-color: #2d3748; color: white; padding: 10px; border-radius: 10px;">
vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
 int j = pi[i-1];
 while (j > 0 && s[i] != s[j]) j = pi[j-1];
 if (s[i] == s[j]) j++;
 pi[i] = j;
}
 </pre>
</div>
</details>
</div>
</div>
</section>`
},
{
 id: "string-z-func",
 title: "22. Z-функция – Сдвиг и наложение",
 type: "html",
 category: "Строки",
 content: `
<section id="string-z-func" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 ca 0) и её суффиксом, начинающимся с i.</p>
 </div>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg" style="background-color: #2d3748; color: white; padding: 10px; border-radius: 10px;">

```

```


 <code>Z[3]</code> (abcd
? Нет, в начале 'a', тут 'd'). Совпало 3 бу
 <code>Z[4], Z[5], Z[6]</code>

</div>
</details>
<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
 Код C++ (Скрыто)
</summary>
<div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounded
int l = 0, r = 0;
for (int i = 1; i < n; i++) {
 if (i <= r) z[i] = min(r - i + 1, z[i - 1]);
 while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z
 if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }
}</pre>
 </div>
</details>
</div>
</div>
</section>`
},
{
 id: "aho-corasick",
 title: "23. Алгоритм Ахо-Корасик",
 type: "html",
 category: "Строки",
 content: `
<section id="aho-corasick" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-indigo-600 text-white px-4 py-1 roun
 <h2 class="text-2xl sm:text-3xl font-bold text-white
 </div>

```

```

<div class="p-5 text-sm text-slate-300 space-y-4">
 <pre class="bg-slate-950 p-4 rounded overflow-hidden">
int get_link(int v) {
 if (t[v].link == -1) {
 if (v == 0 || t[v].p == 0) t[v].link = 0;
 else t[v].link = go(get_link(t[v].p), t[v].pch);
 }
 return t[v].link;
}

int go(int v, char c) {
 if (t[v].go[c] == -1) {
 if (t[v].next[c] != -1) t[v].go[c] = t[v].next[c];
 else t[v].go[c] = v == 0 ? 0 : go(get_link(v), t[v].pch);
 }
 return t[v].go[c];
}</pre>
 </div>
</details>
</div>
</section>`
 },
 {
 id: "complexity-classes",
 title: "24. Классы сложности, сведение задач",
 type: "html",
 category: "Теория сложности",
 content: `
<section id="complexity-classes" class="mb-12 scroll-mt-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 24
 <h2 class="text-2xl sm:text-3xl font-bold text-purple-400">Классы сложности, сведение задач</h2>
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-purple-400">Введение</h3>
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-purple-300 italic">Введение в теорию сложности.

```

```
export const tickets6to13: Chapter[] = [
 {
 id: "graph-dfs-bfs",
 title: "6. Графы. Представление, DFS, BFS",
 type: "html",
 description: "Представление графов и базовые обходы",
 category: "Графы. База",
 content: `
<section id="graph-dfs-bfs" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 {/* База представлений */}
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-slate-300">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-slate-300 italic">
 <p class="text-xl font-mono text-white">
 (V + E).</p>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия: Матрица = Таблица
 </div>
 <p class="text-slate-300 text-sm mb-4">
 глупо, но проверка "знает ли Вася Петю" мгно
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Аналогия: Списки = Контакты
 </div>
 <p class="text-slate-300 text-sm mb-4">

```

Работает через Очередь (FIFO). Продвигается

</p>

<ul class="text-xs text-emerald-300">

<li><b>Где используется:</b></li>

<li>Кратчайший путь в Невзвешенном графе

<li>Алгоритм Ахо-Корасик (сборка текста)

<li>Алгоритм Форда-Фалкерсона / Алгоритм Эдмондса-Карпа

<li>Двунаправленный поиск для у

</ul>

</div>

</div>

<details class="bg-slate-900/40 rounded-lg border">

<summary class="p-4 font-bold text-slate-100">

-b border-slate-700/50">

Код обходов C++ (Основа)

</summary>

<div class="grid grid-cols-1 md:grid-cols-2">

<pre class="bg-slate-950 p-3 rounded">

// DFS (Рекурсия)

vector<bool> vis;

vector<vector<int>> adj;

void dfs(int v) {

vis[v] = true;

for (int u : adj[v]) {

if (!vis[u]) dfs(u);

}

</pre>

<pre class="bg-slate-950 p-3 rounded">

// BFS (Очередь)

queue<int> q;

q.push(start\_node);

vis[start\_node] = true;

while(!q.empty()) {

int v = q.front(); q.pop();

for (int u : adj[v]) {

```

 <p class="text-slate-300 text-sm mb-6">
 . Границы не пересекаются в одной точке по-
 ы не сливались цветами, Всегда можно всего
 </div>
 </div>
</div>
</div>
</section>`,
 },
 {
 id: "graph-components",
 title: "8. Графы. Компоненты связности",
 type: "html",
 description: "Нахождение кусков графа",
 category: "Графы. База",
 content: `
<section id="graph-components" class="mb-12 scroll-mt-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 ество компонент.</p>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 b
 rder border-emerald-500 shadow-md">
 Аналогия 1: Острова
 </div>
 <p class="text-slate-300 text-sm mb-6">
 карту – остались ли серые (неисследованные)
 ь через океан – столько и компонент.</p>

```

```
border-rose-500 shadow-md">
```

Ограничения: Циклы

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">
получить опыт, нужна работа". Алгоритм выяв
```

```
</div>
```

```
</div>
```

```
<details class="bg-slate-900/40 rounded-lg border-rose-500 shadow-md">
```

```
<summary class="p-4 font-bold text-slate-300">
-b border-slate-700/50">
```

Душный мод: Код на C++ (Скрыто)

```
</summary>
```

```
<div class="p-5 text-sm text-slate-300">
```

```
<p class="text-indigo-400 font-bold">
```

```
<div class="bg-slate-950 p-4 rounded">
```

```
<pre class="text-xs font-mono">
```

```
vector<int> adj[MAXN];
```

```
bool visited[MAXN];
```

```
vector<int> ans;
```

```
void dfs(int v) {
```

```
 visited[v] = true;
```

```
 for (int to : adj[v]) {
```

```
 if (!visited[to]) {
```

```
 dfs(to);
```

```
 }
```

```
 }
```

```
 ans.push_back(v); // Добавляем в момент ВЫХОДА (pos
```

```
}
```

```
void topological_sort(int n) {
```

```
 for (int i = 0; i < n; ++i) {
```

```
 if (!visited[i]) dfs(i);
```

```
 }
```

```
 reverse(ans.begin(), ans.end()); // Разворачиваем с
```

```
</pre>
```

```
</div>
```

```
 Правило: Если два цикла имеют хо

```

```
<div class="grid grid-cols-1 xl:grid-cols-2
 <div class="bg-slate-800 p-6 rounded-lg
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
```

Аналогия: Интриги в офисе

```
</div>
```

```
<p class="text-slate-300 text-sm mb
```

Сильная связность — это "клуб с  
ни в одной SCC. Добавить курьера Гошу, кото  
йти наружу (к директору), но обратно в клуб

```
</p>
```

```
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg
 <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
```

Связь с Билетом 12

```
</div>
```

```
<p class="text-slate-300 text-sm mb
```

<b>SCC (Билет 10)</b> склеивает

```
</br>

```

<b>Точки сочленения (Билет 12)</b>  
ость</strong> монолита. Вырвешь точку сочле

```
</p>
```

```
</div>
```

```
</div>
```

```
<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
```

Душный мод: Код Алгоритм Косарайю

```
</summary>
```

```
<div class="p-5 text-sm text-slate-300
```

```
<p class="text-emerald-400 font-bol
```

```
<ol class="list-decimal list-inside
```



```

 </div>
 <p class="text-slate-300 text-sm mb-6">
 ЭТОТ МОСТ, ЭКОНОМИКОЙ ОБОИХ КУСКОВ ПРИДЕТ
 </div>
 </div>
</div>
</div>
</section>`,
 },
 {
 id: "graph-articulation",
 title: "12. Графы. Точки сочленения",
 type: "html",
 description: "Вершины, удаление которых убивает связность",
 category: "Графы. Продвинутое",
 content: `
<section id="graph-articulation" class="mb-12 scroll-mt-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 12
 <h2 class="text-2xl sm:text-3xl font-bold text-slate-900">Графы. Точки сочленения
 </div>

 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-rose-400">Точки сочленения
 <p>Вершины, удаление которых убивает связность графа. Точка сочленения — это вершина, удаление которой увеличивает число компонент связности графа.
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-rose-300 italic">Пример: граф с 5 вершинами и 6 ребрами. Точка сочленения — вершина 2.
 <div class="text-left font-mono text-sm">
 <p><strong class="text-white">Точка сочленения — это вершина, удаление которой увеличивает число компонент связности графа.
 <div class="p-3 bg-slate-950 rounded">
 Пример: граф с 5 вершинами и 6 ребрами. Точка сочленения — вершина 2.
 <p class="text-xl font-bold text-slate-900">Точка сочленения — это вершина, удаление которой увеличивает число компонент связности графа.
 <p class="text-xs text-slate-400">Пример: граф с 5 вершинами и 6 ребрами. Точка сочленения — вершина 2.
 </div>
 </div>
 </div>
 </div>
 </div>
`
 }
}

```

```

<div class="p-5 text-sm text-slate-300 space-x-4">
 <p class="text-blue-400 font-bold">Особый случай (если нет ребёнка, то выше неё прыгать некуда). Поэтому для неё у него больше 1 независимого ребёнка.
 </p>
 <div class="bg-slate-950 p-4 rounded-lg">
 <pre class="text-xs font-mono text-slate-300">
void dfs(int v, int p = -1) {
 visited[v] = true;
 tin[v] = low[v] = timer++;
 int children = 0;

 for (int to : adj[v]) {
 if (to == p) continue;
 if (visited[to]) {
 low[v] = min(low[v], tin[to]);
 } else {
 dfs(to, v);
 low[v] = min(low[v], low[to]);
 if (low[to] >= tin[v] && p != -1)
 IS_CUTPOINT(v);
 ++children;
 }
 }
 if (p == -1 && children > 1)
 IS_CUTPOINT(v);
}</pre>
 </div>
 </div>
</details>

<div class="bg-indigo-950/60 p-6 rounded-xl border">
 <h4 class="text-lg font-bold text-indigo-400">
 <p class="text-slate-300 text-sm mb-4">
 Это глубокий дуальный мост между ориентацией
 </p>

```

Аналогия 1: Рисование не отры

```
 </div>
 <p class="text-slate-300 text-sm mb
е сходится нечетное число линий, значит, из
а везде должно быть четное число (зашел-выш
 </div>
 </div>
</div>
</section>`,
 },
];
```

---

## РАЗДЕЛ: ИНТЕРАКТИВНЫЕ КОМПОНЕНТЫ И СИМУЛЯТОРЫ

---

ФАЙЛ 30/96: src/components/ArticulationPointsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

---

```
import React, { useState, useEffect } from "react";
import { useVizStepSync, vizNumber } from '../data/vizS
import { Play, Pause, RotateCcw, Info } from "lucide-re
```

```
interface Node {
 id: number;
 x: number;
 y: number;
 label: string;
}
```

```
interface Edge {
 u: number;
 v: number;
}
```

```

 low: number[];
 }

export const ArticulationPointsViz: React.FC = () => {
 const [steps, setSteps] = useState<StepInfo[]>([]);
 const [currentStepIndex, setCurrentStepIndex] = useState(0);
 useVizStepSync(currentStepIndex, setCurrentStepIndex, () => {
 const v = vizNumber(vars.v);
 if (v === null) return null;
 return steps.map((step, index) => ({ step, index }));
 });
 const [isPlaying, setIsPlaying] = useState(false);

 useEffect(() => {
 const newSteps: StepInfo[] = [];
 let timer = 0;
 const visited = new Set<number>();
 const ap = new Set<number>();
 const currentTin = Array(nodes.length).fill(-1);
 const currentLow = Array(nodes.length).fill(-1);

 const recordStep = (desc: string, u: number | null) => {
 newSteps.push({
 desc,
 visited: new Set(visited),
 ap: new Set(ap),
 currentNode: u,
 tin: [...currentTin],
 low: [...currentLow],
 });
 };

 recordStep("Начало DFS (Тарьян) для поиска точек со

 const dfs = (v: number, p: number = -1) => {
 visited.add(v);
 currentTin[v] = currentLow[v] = timer++;
 let children = 0;

```

```

 } else if (p === -1) {
 recordStep(
 `Корень дерева DFS ${v} имеет <=1 ребенка, он
 v,
);
 }
 };

 for (let i = 0; i < nodes.length; i++) {
 if (!visited.has(i)) {
 dfs(i);
 }
 }

 recordStep("Алгоритм завершен. Найдены все точки со
 setSteps(newSteps);
}, []));

const currentStep = steps[currentStepIndex] || {
 desc: "",
 visited: new Set(),
 ap: new Set(),
 currentNode: null,
 tin: [],
 low: [],
};

useEffect(() => {
 let interval: NodeJS.Timeout;
 if (isPlaying && currentStepIndex < steps.length -
 interval = setInterval(() => {
 setCurrentStepIndex((prev) => prev + 1);
 }, 1500);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearInterval(interval);
}, [isPlaying, currentStepIndex, steps.length]);

```

```

<div className="grid grid-cols-1 lg:grid-cols-3 g
 <div className="lg:col-span-2 relative h-96 lg:l
 <svg className="w-full h-full" viewBox="0 0 8
 {/* Edges */}
 {edges.map((edge, i) => {
 const u = nodes.find((n) => n.id === edge
 const v = nodes.find((n) => n.id === edge

 const isBridge =
 (edge.u === 3 && edge.v === 4) ||
 (edge.u === 4 && edge.v === 3);

 return (
 <line
 key={"edge-" + i}
 x1={u.x}
 y1={u.y}
 x2={v.x}
 y2={v.y}
 stroke={isBridge ? "#f43f5e" : "#475568"}
 strokeWidth={isBridge ? "4" : "2"}
 strokeDasharray={isBridge ? "8,4" : ""}
 />
);
 })}

 {/* Nodes */}
 {nodes.map((node) => {
 const isCurrent = currentStep.currentNode
 const isVisited = currentStep.visited.has
 const isAp = currentStep.ap.has(node.id);

 const fill = isCurrent
 ? "#3b82f6"
 : isAp
 ? "#e11d48"
 : isVisited

```

```

 x={node.x}
 y={node.y - r - 15}
 textAnchor="middle"
 fill="#94a3b8"
 fontSize="10px"
 >
 tin:{" "}
 {currentStep.tin[node.id] >= 0
 ? currentStep.tin[node.id]
 : "-"}
 </text>
 <text
 x={node.x}
 y={node.y + r + 20}
 textAnchor="middle"
 fill="#94a3b8"
 fontSize="10px"
 >
 low:{" "}
 {currentStep.low[node.id] >= 0
 ? currentStep.low[node.id]
 : "-"}
 </text>
 </>
)}
</g>
);
}}
</svg>
</div>

<div className="bg-slate-950 p-6 border-t lg:bo
 <h4 className="text-sm font-bold text-slate-4
 Статус алгоритма
 </h4>

 <div className="bg-slate-900 border border-sl
 <p className="text-white font-medium min-h-
```





```

 }
 };

 const edges = getEdges(hasCycle);

 const getAdjList = (ed: Edge[]) => {
 const list: { [key: string]: { v: string; w: number } } = {};
 nodes.forEach(n => (list[n.id] = []));
 ed.forEach(e => list[e.u].push({ v: e.v, w: e.w }));
 return list;
 };
 const adjList = getAdjList(edges);

 const codeLines = [
 { line: 1, text: 'function bellman_ford(graph, start)' },
 { line: 2, text: ' dist = {v: ∞}; dist[start] = 0;' },
 { line: 3, text: ' for i from 1 to V - 1: // |V| - 1 iterations' },
 { line: 4, text: ' for (u, v, weight) in E:' },
 { line: 5, text: ' if dist[u] + weight < dist[v]:' },
 { line: 6, text: ' dist[v] = dist[u] + weight;' },
 { line: 7, text: ' for (u, v, weight) in E: // Проверка на отрицательный цикл' },
 { line: 8, text: ' if dist[u] + weight < dist[v]:' },
 { line: 9, text: ' return "ОБНАРУЖЕН ОТРИЦАТЕЛЬНЫЙ ЦИКЛ"' },
 { line: 10, text: ' return dist' },
];

 useEffect(() => {
 const simulationSteps: Step[] = [];
 let dist: { [key: string]: number } = { S: 0, A: 99 };
 let prev: { [key: string]: string | null } = { S: null };

 simulationSteps.push({
 iteration: 0, checkingEdge: null,
 distances: { ...dist }, previous: { ...prev },
 log: ' Фура заведена. Начинаем с Базы (S). Расстояние до А: 99',
 activeCodeLine: 2,
 });
 });

```

```

 if (dist[edge.u] !== 999 && dist[edge.u] + edge.w < dist[v]) {
 cycleFound = true;
 simulationSteps.push({
 iteration: vCount, checkingEdge: { u: edge.u, v: edge.v, w: edge.w },
 distances: { ...dist }, previous: { ...prevDist },
 log: `    ВНИМАНИЕ! Проверка цикла: Ребро ${edge.u} -> ${edge.v} с весом ${edge.w} приводит к уменьшению расстояния до вершины ${v} с ${prevDist[v]} до ${dist[v]}.`,
 activeCodeLine: 9,
 });
 break;
 }
 }

 if (!cycleFound) {
 simulationSteps.push({
 iteration: vCount, checkingEdge: null,
 distances: { ...dist }, previous: { ...prevDist },
 log: `    Проверка завершена. Ни одно ребро не привело к уменьшению расстояния до вершины ${v}.`,
 activeCodeLine: 10,
 });
 }
}

setSteps(simulationSteps);
setCurrentStepIndex(0);
setIsPlaying(false);
}, [hasCycle]);

useEffect(() => {
 let timer: any = null;
 if (isPlaying && currentStepIndex < steps.length - 1) {
 timer = setTimeout(() => setCurrentStepIndex((p) => p + 1), 1000);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const baseStep = steps[currentStepIndex] || null;

```

```

 <button onClick={() => { setIsPlaying(false);
 g-slate-600 text-white px-3 py-2 rounded-lg
 <button onClick={() => setIsPlaying(!isPlaying)
 sition-colors text-white ${isPlaying ? 'bg-
 <button onClick={() => { setIsPlaying(false);
 ; }} className="flex items-center gap-1 bg-l
 s">⏮ar </button>
 </div>
</div>

<div className="flex flex-col lg:flex-row gap-6 ml
 {/* Граф */}
 <div className="w-full lg:w-2/3 bg-blue-950/20
 y-center min-h-[400px]">
 <div className="absolute top-3 left-3 bg-blue
 ld shadow-sm">
 Итерация {currentStep.iteration} (⏮ar {curro
 </div>

 {currentStep.hasNegativeCycle && (
 <div className="absolute top-3 right-3 bg-r
 animate-pulse shadow-lg shadow-rose-600/50
 ▲ Отрицательный цикл!
 </div>
)}

 <svg className="w-full h-auto max-h-[500px]"
 <defs>
 <marker id="arrow-bf-normal" markerWidth=
 <path d="M0,0 L6,3 L0,6 z" fill="#64748b" />
 <marker id="arrow-bf-active" markerWidth=
 <path d="M0,0 L6,3 L0,6 z" fill="#f59e0b" />
 <marker id="arrow-bf-cycle" markerWidth=
 path d="M0,0 L6,3 L0,6 z" fill="#e11d48" />
 </defs>

 {edges.map((edge, idx) => {
 const uNode = nodes.find((n) => n.id ===

```

```

 split(' ')[0]}</text>
 </g>
);
 }}}
</svg>

<div className="w-full bg-slate-950/80 p-4 rounded" >
 <p className="font-semibold text-amber-400" >
 <p className="min-h-[40px] flex items-center" >
 </p>
</div>
</div>

{/* Список смежности */}
<div className="w-full lg:w-1/3 bg-emerald-950/80" >
 <h4 className="text-lg font-bold text-emerald-400" >
 <div className="space-y-2 text-sm font-mono text-white" >
 {Object.keys(adjList).map(u => {
 <div key={u} className="flex flex-wrap gap-2" >

 {adjList[u].map((edge, idx) => {
 const isChk = currentStep.checkingEdge === u;
 return (
 <span key={idx} className={`${px-2} ${
 edge.w < 0 ? 'bg-rose-900/80 text-rose-300' : 'bg-emerald-950/80 text-emerald-400'
 }`} >
 {edge.v} ({edge.w})

)
 })}
 </div>
 </div>
 })}
 </div>
 </div>
</div>
</div>

<div className="flex flex-col lg:flex-row gap-6" >
 {/* Таблица расстояний */}
 <div className="w-full lg:w-1/2 bg-rose-950/10" >

```

---

ФАЙЛ 32/96: src/components/BfsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import { useState, useEffect, useCallback } from 'react'
import { Play, Pause, RotateCcw, StepForward } from 'lucide-react'
```

```
const NODES = [
 { id: 0, label: '0', x: 300, y: 50 },
 { id: 1, label: '1', x: 150, y: 150 },
 { id: 2, label: '2', x: 450, y: 150 },
 { id: 3, label: '3', x: 75, y: 250 },
 { id: 4, label: '4', x: 225, y: 250 },
 { id: 5, label: '5', x: 375, y: 250 },
 { id: 6, label: '6', x: 525, y: 250 },
];
```

```
const EDGES = [
 { from: 0, to: 1 }, { from: 0, to: 2 },
 { from: 1, to: 3 }, { from: 1, to: 4 },
 { from: 2, to: 5 }, { from: 2, to: 6 }
];
```

```
const ADJ: { [key: number]: number[] } = {
 0: [1, 2], 1: [3, 4], 2: [5, 6], 3: [], 4: [], 5: [], 6: [],
};
```

```
interface Step {
 activeLine: number;
 queue: number[];
 visited: number[];
 currentNode: number | null;
 logs: string;
}
```

```

 currentNode: curr,
 logs: `Достаем из очереди (${curr}).`
 });

const neighbors = ADJ[curr];
if (neighbors.length > 0) {
 for (const n of neighbors) {
 if (!visited.has(n)) {
 steps.push({
 activeLine: 7,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,
 logs: `Рассматриваем соседа: ${n}. Он еще не
 });

 visited.add(n);
 queue.push(n);

 steps.push({
 activeLine: 9,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,
 logs: `Помечаем ${n} как посещенную и добавляем
 });
 }
 }
} else {
 steps.push({
 activeLine: 6,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,
 logs: `Соседей нет или все уже посещены.`
 });
}
}

```

```

return (
 <div className="bg-slate-900 border-2 border-slate-
 <div className="flex flex-col md:flex-row items-c
 <div>
 <h3 className="text-2xl font-bold text-white
 Ал
 </h3>
 <p className="text-slate-400 text-sm mt-1">Ви
 </div>
 <div className="flex bg-slate-800 p-1.5 rounded
 <button
 onClick={() => { setIsPlaying(false); setSt
 className="p-2 text-slate-400 hover:text-wh
 title="Сброс"
 >
 <RotateCcw className="w-5 h-5" />
 </button>
 <button
 onClick={() => setIsPlaying(p => !p)}
 className="p-2 text-blue-400 hover:text-bl
 title={isPlaying ? 'Пауза' : 'Воспроизведен
 >
 {isPlaying ? <Pause className="w-5 h-5" />
 </button>
 <button
 onClick={handleNext}
 disabled={stepIdx >= STEPS.length - 1}
 className="p-2 text-emerald-400 hover:text-
 ed:hover:bg-transparent"
 title="Шаг вперед"
 >
 <StepForward className="w-5 h-5" />
 </button>
 </div>
 </div>
 <div className="grid grid-cols-1 lg:grid-cols-2 g

```

```

 } else if (isVisited) {
 fill = '#0f172a'; // slate-900
 stroke = '#3b82f6'; // blue-500
 }

 return (
 <g key={n.id} className="transition-tran
 {isCurrent && {
 <circle cx={n.x} cy={n.y} r="30" fi
 }}
 <circle
 cx={n.x} cy={n.y}
 r="20"
 fill={fill}
 stroke={stroke}
 strokeWidth="3"
 className="transition-colors durati
 />
 <text
 x={n.x} y={n.y}
 textAnchor="middle"
 dy=".3em"
 className="fill-white font-bold tex
 >
 {n.label}
 </text>
 </g>
);
}
}
</svg>
</div>

```

```

{/* Код и Очередь */}
<div className="flex flex-col gap-4">
 <div className="bg-slate-900 border border-sl
 er">
 <div className="bg-slate-800 text-slate-400
 se tracking-wider">

```



```
 enter justify-center rounded-md shadow-lg s
 {qId}

))
 })
</div>
<div className="mt-2 text-sm text-amber-300
ms-center justify-center">
 "{step.logs}"
</div>
</div>
</div>
</div>
);
}
```

---

ФАЙЛ 33/96: src/components/ChapterImage.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import dijkstraImg from '../assets/dijkstra-kind.jpg';
import bellmanImg from '../assets/bellman-ford-truck.jp
import floydImg from '../assets/floyd-network.jpg';
```

```
const imageMap: Record<string, { src: string; alt: string }> = {
 dijkstra: {
 src: dijkstraImg,
 alt: 'Добрый Дейкстра',
 caption: 'Добрый — только позитив!',
 borderColor: 'border-blue-500/30',
 captionColor: 'text-blue-400',
 },
 'bellman-ford': {
 src: bellmanImg,
 alt: 'Фура по болоту',
```

```

 prev?: Chapter;
 next?: Chapter;
 index: number;
 total: number;
 onGo: (id: string) => void;
 compact?: boolean;
}

/**
 * Переходы «предыдущая / следующая тема» — как на Code
 * Компактный вариант живёт в шапке главы, крупный — по
 */
export const ChapterNav: React.FC<Props> = ({ prev, next,
 if (compact) {
 return (
 <div className="flex items-center gap-2 text-xs">
 <button
 disabled={!prev}
 onClick={() => prev && onGo(prev.id)}
 title={prev ? prev.title : "Это первая тема"}
 className="flex items-center gap-1 px-2.5 py-1
 :bg-slate-700 enabled:hover:text-white disabled:opacity-50"
 >
 <ChevronLeft className="w-4 h-4" />
 Назад
 </button>

 {index + 1} / {total}

 <button
 disabled={!next}
 onClick={() => next && onGo(next.id)}
 title={next ? next.title : "Это последняя тема"}
 className="flex items-center gap-1 px-2.5 py-1
 :bg-slate-700 enabled:hover:text-white disabled:opacity-50"
 >
 Вперёд
 <ChevronRight className="w-4 h-4" />
 </button>
 </div>
);
 }
 return null;
};

```

```
};
```

ФАЙЛ 35/96: src/components/ChapterView.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React, { useCallback, useEffect, useRef, useState } from "react";
import { Check, Download, Loader2, Maximize2, MousePointerClick } from "lucide-react";
import type { Chapter } from "../types";
import { useTermHints } from "../hooks/useTermHints";
import { TermPopover, type HintAnchor } from "../hints/TermHint";
import { getViz } from "../vizRegistry";
import { VizChapterContext } from "../data/vizStepBus";
import { ChapterNav } from "../ChapterNav";
import { RELATED, chapters as allChapters } from "../data/related";
```

```
/** «Читать рядом»: кликабельные ссылки на связанные библиографические записи */
const RelatedTickets: React.FC<{ chapterId: string; onGo: (id: string) => void }> = ({
 chapterId,
 onGo,
}) => {
 const related = (RELATED[chapterId] ?? []).filter((id) => id !== chapterId);
 if (related.length === 0) return null;
 const getTitle = (id: string): string => allChapters[id].title;
 return (
 <nav className="mt-10 mb-2 p-4 rounded-xl border border-indigo-500">
 <p className="text-[11px] font-bold uppercase tracking-wide">Связанные библиографические записи</p>
 <MousePointerClick className="w-3.5 h-3.5" />
 </p>
 <div className="flex flex-wrap gap-2">
 {related.map((id) => (
 <button
 key={id}
 onClick={() => onGo(id)}
 className="text-xs font-bold px-3 py-1.5 rounded border border-indigo-500 transition-colors"
 >
 {getTitle(id)}
 </button>
))}
 </div>
);
};
```

```

 hideTimer.current = window.setTimeout(() => setAnchor(
 }, [cancelHide]);

 useEffect(() => {
 setAnchor(null);
 setSaving("idle");
 }, [chapter.id]);

 const saveHtml = useCallback(async () => {
 setSaving("work");
 try {
 await downloadChapterHtml(chapter);
 setSaving("done");
 window.setTimeout(() => setSaving("idle"), 2500);
 } catch {
 setSaving("idle");
 }
 }, [chapter]);

 const open = useCallback(
 (el: HTMLElement) => {
 const termId = el.dataset.termId;
 if (!termId) return;
 cancelHide();
 setAnchor({ termId, rect: el.getBoundingClientRect() });
 },
 [cancelHide]
);

 const handlePointerOver = (e: React.MouseEvent) => {
 if (isTouch()) return;
 const el = (e.target as HTMLElement).closest<HTMLElement>(".chapter-link");
 if (el) open(el);
 };

 const handlePointerOut = (e: React.MouseEvent) => {
 if (isTouch()) return;
 const el = (e.target as HTMLElement).closest<HTMLElement>(".chapter-link");

```

```

 </button>
 <button
 onClick={() => window.print()}
 title="Распечатать или сохранить в PDF сред
 className="flex items-center gap-1.5 text-x
 300 hover:text-white hover:border-indigo-500
 >
 <Printer className="w-3.5 h-3.5" /> Печать
 </button>

 </div>

```

```

<div
 ref={contentRef}
 className="prose prose-invert max-w-none"
 onMouseOver={handlePointerOver}
 onMouseOut={handlePointerOut}
 onClick={handleClick}
 onFocus={handleFocus}
 dangerouslySetInnerHTML={{ __html: chapter.con
/>

```

```

<p className="mt-6 flex items-start gap-2 text-
 ed-lg px-3 py-2">
 <MousePointerClick className="w-4 h-4 shrink-1

 Подчёркнутые термины – интерактивные: навед
 объяснение на пальцах, мини-анимацию и кноп

</p>

```

```

{viz && (
 <section id="chapter-viz" className="mt-8 scr
 <div className="flex items-center justify-b
 <h3 className="flex items-center gap-2 te
 <Sparkles className="w-4 h-4 text-indigo
 Демонстрация: {viz.title}
 </h3>

```

```
 onOpenSimulator(id);
 }}
 onCardEnter={cancelHide}
 onCardLeave={scheduleHide}
 />
</>
);
};
```

---

ФАЙЛ 36/96: src/components/ColoringViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import { useMemo, useState } from "react";
import { Palette, RotateCcw, Wand2 } from "lucide-react";
import { useVizRuntime, vizNumber, vizRecord, vizArray, vizColor, vizText } from "viz";

/**
 * Тема 7 «Графы. Планарные. Покраска».
 * Жадная раскраска планарного графа: кликни цвет, получи цвет
 * Компилятор читает v (текущая вершина), color[] (массив цветов)
 */

interface V {
 id: number;
 label: string;
 x: number;
 y: number;
}

// Планарный граф (триангуляция): 7 вершин, все рёбра белы
const NODES: V[] = [
 { id: 0, label: "A", x: 210, y: 40 },
 { id: 1, label: "B", x: 350, y: 110 },
 { id: 2, label: "C", x: 320, y: 250 },
 { id: 3, label: "D", x: 110, y: 260 },
```

```

 return color;
}

export function ColoringViz() {
 const [colors, setColors] = useState<number[]>(NODES.length);
 const [brush, setBrush] = useState(0);

 // — Синхронизация с Python-компилятором.
 const runtime = useVizRuntime();
 const vars = runtime?.variables;
 const liveVLabel = vizString(vars?.v);
 const liveVNum = vizNumber(vars?.v);
 const liveColorRec = vizRecord(vars?.color) ?? vizRecord({});
 const liveColorArr = vizArray(vars?.color) ?? vizArray({});
 const liveK = vizNumber(vars?.k);

 const liveColors: number[] | null = (() => {
 if (liveColorArr) {
 const arr = liveColorArr.map((c) => vizNumber(c));
 if (arr.every((c) => c === null || c >= -1)) {
 const padded = [...arr.map((c) => c ?? -1)];
 while (padded.length < NODES.length) padded.push(-1);
 return padded.slice(0, NODES.length);
 }
 return null;
 }
 if (liveColorRec) {
 return NODES.map((n) => {
 const byLabel = liveColorRec[n.label];
 const byId = liveColorRec[String(n.id)];
 const raw = byLabel !== undefined ? byLabel : byId;
 const c = typeof raw === "string" ? COLORS.find((v) => v === raw) : raw;
 return c === null || c === undefined ? -1 : c;
 });
 }
 return null;
 })();
}

```

```

<div className="flex flex-wrap items-center justify-center">
 <div>
 <h3 className="text-lg font-bold text-white">
 <p className="text-xs text-slate-400">
 Смежные вершины не должны совпадать по цвету
 </p>
 </div>
 <div className="flex items-center gap-2">
 <button
 onClick={() => setColors(greedy())}
 className="flex items-center gap-1 bg-indigo-500 text-white"
 >
 <Wand2 className="w-3.5 h-3.5" /> Жадная раскраска
 </button>
 <button
 onClick={() => setColors(NODES.map((n) => -1))}
 className="flex items-center gap-1 bg-slate-500 text-white"
 >
 <RotateCcw className="w-3.5 h-3.5" /> Сброс
 </button>
 </div>
 </div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-[1fr 1fr] gap-2">
 <div className="bg-slate-950 rounded-xl border border-slate-800 p-4">
 <div className="absolute inset-0 bg-[radial-gradient(circle,transparent_1px,#475569_1px)]" />
 <svg className="w-full h-[340px] relative z-10" />
 {EDGES.map(([u, v], i) => {
 <line key={i} x1={NODES[u].x} y1={NODES[u].y} x2={NODES[v].x} y2={NODES[v].y}
 stroke="#475569" strokeWidth={2.2} />
 })}
 {NODES.map((n) => {
 const c = shownColors[n.id];
 const isConflict = conflicts.has(n.id);
 const isCurrent = liveCurrent === n.id;
 const fill = isConflict ? CONFLICT : c ? c : 'white';
 return (
 <g key={n.id} className="cursor-pointer">

```



```

 {COLORS.map((c, i) => (
 <button
 key={c.name}
 onClick={() => setBrush(i)}
 title={c.name}
 className={`h-8 rounded-lg border-2 t
opacity-70 hover:opacity-100`} `}
 style={{ backgroundColor: c.hex }}
 />
)))
 </div>
 </div>
 <div className="bg-slate-950 rounded-xl borde
 <div className="text-[10px] uppercase track
 <div className={`text-2xl font-extrabold fo
 {compilerLinked && liveK !== null ? liveK
 </div>
 <div className="text-[10px] text-slate-500
 </div>
 <div className="bg-slate-900/60 rounded-xl bo
 <b className="text-slate-200">Жадный алгори
 дей.
 На этом графе жадная раскраска даёт 4 цвета
 6 цветов (в планарном всегда есть вершина с
 еребор на компьютере).
 </div>
 </div>
 </div>
</div>
);
}

```

```
};
```

```
const CLASS_META: Record<ClassId, { title: string; color: string; note: string }> = {
 P: { title: "P", color: "#34d399", note: "решаются за полиномиальное время" },
 NP: { title: "NP", color: "#fbbf24", note: "решение проверить за полиномиальное время" },
 PSPACE: { title: "PSPACE", color: "#818cf8", note: "решаются за полиномиальную память" },
 EXPTIME: { title: "EXPTIME", color: "#f472b6", note: "решаются за экспоненциальное время" },
};
```

```
const CLASS_INFO: Record<ClassId, string> = {
 P: "P — класс задач, у которых ЕСТЬ быстрый (полиномиальный) алгоритм",
 NP: "NP — задачи, у которых быстрой проверки ПРОЧЕСТЬ решение достаточно. Если",
 "P ⊆ NP: нашёл быстро → и проверяй быстро.",
 PSPACE: "PSPACE — хватает полиномиальной ПАМЯТИ (временная сложность не важна)",
 "ветку рекурсии.",
 EXPTIME: "EXPTIME — есть алгоритм за 2^(poly n). Обобщение PSPACE",
 ". ",
};
```

```
const RECT: Record<ClassId, { x: number; y: number; w: number; h: number }> = {
 P: { x: 40, y: 90, w: 220, h: 210 },
 NP: { x: 30, y: 40, w: 400, h: 260 },
 PSPACE: { x: 15, y: 20, w: 440, h: 320 },
 EXPTIME: { x: 5, y: 5, w: 470, h: 360 },
};
```

```
export function ComplexityClassesViz() {
 const [selected, setSelected] = useState<ClassId | null>(null);
 const [selectedProblem, setSelectedProblem] = useState<string>("");

 // — Синхронизация с Python-компилятором: cls — имя класса
 // reductions — список стрелок вида ["3-SAT", "Vertex Cover", "3-SAT"]
 const runtime = useVizRuntime();
 const vars = runtime?.variables;
 const liveCls = vizString(vars?.cls);
 const liveTask = vizString(vars?.task);
 const liveRed = vizArray(vars?.reductions);
 const compilerLinked = liveCls !== null || liveTask !== null || liveRed.length > 0;
}
```

```

return (
 <div className="bg-slate-800/90 p-5 rounded-2xl border"
 <div className="flex flex-wrap items-center justify"
 <div>
 <h3 className="text-lg font-bold text-white">
 <p className="text-xs text-slate-400">Стрелки
 </div>
 <button
 onClick={reset}
 className="flex items-center gap-1 bg-slate-700
 >
 <RotateCcw className="w-3.5 h-3.5" /> Сброс
 </button>
 </div>

 <div className="grid grid-cols-1 lg:grid-cols-[1fr 1fr]
 <div className="bg-slate-950 rounded-xl border border-slate-800
 <svg className="w-full" viewBox="0 0 480 380">
 {[["EXPTIME", "PSPACE", "NP", "P"] as ClassNames]
 const r = RECT[cls];
 const dim = activeCls !== null && activeCls === cls ? 3.5 : 1;
 return (
 <g key={cls} onClick={() => setSelectedCls(cls)}>
 <rect
 x={r.x} y={r.y} width={r.w} height={r.h}
 fill={CLASS_META[cls].color + (dim > 1 ? "a" : "f")}
 stroke={CLASS_META[cls].color}
 strokeWidth={activeCls === cls ? 3.5 : 1}
 opacity={dim > 0.45 ? 1 : 0.45}
 />
 <text x={r.x + 12} y={r.y + 22} fill={CLASS_META[cls].color}
 {cls === "NP" && <tspace="10px"}>{CLASS_META[cls].title}
 </text>
 </g>
);
 }
 </div>
 </div>
 </div>
);
}

```

```

 <text key={li} x={p.x} y={p.y + 4 +
ontWeight="600">
 {line}
 </text>
 }}}
</g>
);
}}
</svg>
</div>

<div className="space-y-3">
 <div className="grid grid-cols-2 gap-2">
 {(Object.keys(CLASS_META) as ClassId[]).map
 <button
 key={cls}
 onClick={() => setSelected(selected ===
 className={`rounded-lg border px-3 py-2
 activeCls === cls ? "text-white" : "t
 }`}
 style={{
 borderColor: CLASS_META[cls].color +
 backgroundColor: activeCls === cls ?
 }}
 >
 {CLASS_META[cls].title}
 <div className="text-[9px] font-normal
 {cls === "P" ? "быстро решаем" : cls
ени"}
 </div>
 </button>
 }}}
</div>
<div className="bg-slate-900/60 rounded-xl bo
 {compilerLinked && {
 <div className="mb-2 text-[10px] font-mono
 k="${liveTask}"` : ""}</div>
 }}

```

```

 { id: 0, label: "A", x: 60, y: 60 },
 { id: 1, label: "B", x: 170, y: 30 },
 { id: 2, label: "C", x: 150, y: 130 },
 { id: 3, label: "D", x: 40, y: 160 },
 { id: 4, label: "E", x: 300, y: 60 },
 { id: 5, label: "F", x: 410, y: 90 },
 { id: 6, label: "G", x: 330, y: 160 },
 { id: 7, label: "H", x: 90, y: 260 },
 { id: 8, label: "I", x: 200, y: 240 },
 { id: 9, label: "J", x: 330, y: 270 },
];

const EDGES: [number, number][] = [
 [0, 1], [1, 2], [2, 3], [3, 0], [0, 2], // остров 1: A B C
 [4, 5], [5, 6], [6, 4], // остров 2: E F G (цикл)
 [7, 8], [8, 9], // остров 3: H I J
];

const ADJ: number[][] = (() => {
 const adj: number[][] = NODES.map(() => []);
 EDGES.forEach(([u, v]) => {
 adj[u].push(v);
 adj[v].push(u);
 });
 return adj;
})();

const COMP_COLORS = ["#6366f1", "#f59e0b", "#10b981", "#f87192"];

interface StepFrame {
 comp: (number | null)[];
 queue: number[];
 current: number | null;
 scanning: number | null;
 count: number;
 log: string;
}

```

```

 return frames;
}

const FRAMES = buildFrames();

export function ComponentsViz() {
 const [step, setStep] = useState(0);
 const [playing, setPlaying] = useState(false);

 useEffect(() => {
 if (!playing) return;
 if (step >= FRAMES.length - 1) {
 setPlaying(false);
 return;
 }
 const t = window.setTimeout(() => setStep((s) => Math.min(s + 1, FRAMES.length - 1)), 1000);
 return () => window.clearTimeout(t);
 }, [playing, step]);

 // — Синхронизация с Python-компилятором: v — текущая метка, q — очередь, count — число компонентов
 const runtime = useVizRuntime();
 const vars = runtime?.variables;
 const liveVLabel = vizString(vars?.v);
 const liveVNum = vizNumber(vars?.v);
 const liveCompRec = vizRecord(vars?.comp);
 const liveCompArr = vizArray(vars?.comp);
 const liveQ = vizArray(vars?.q);
 const liveCount = vizNumber(vars?.count);

 const liveCompByLabel = useMemo(() => {
 if (liveCompRec) return liveCompRec;
 if (liveCompArr) return Object.fromEntries(liveCompArr.entries());
 return null;
 }, [liveCompRec, liveCompArr]);

 const frame = FRAMES[Math.min(step, FRAMES.length - 1)];
 const compilerLinked =

```

```

 : frame.queue;

return (
 <div className="bg-slate-800/90 p-5 rounded-2xl border" >
 <div className="flex flex-wrap items-center justify-center" >
 <div>
 <h3 className="text-lg font-bold text-white">
 <p className="text-xs text-slate-400">
 Каждой вершине присваивается номер компонент
 </p>
 </div>
 <div className="flex items-center gap-2">
 <button
 onClick={() => { setPlaying(false); setStep(0); }}
 className="flex items-center gap-1 bg-slate-700 hover:bg-slate-600" >
 <RotateCcw className="w-3.5 h-3.5" /> Сброс
 </button>
 <button
 onClick={() => setStep((s) => Math.max(0, s + 1))}
 disabled={step === 0}
 className="bg-slate-700 hover:bg-slate-600" >
 <Next className="w-3.5 h-3.5" /> Следующий
 </button>
 <button
 onClick={() => setPlaying((p) => !p)}
 className={`flex items-center gap-1 px-3 py-2 border rounded-md ${
 playing ? "bg-emerald-600 hover:bg-emerald-500" : "bg-slate-700 hover:bg-slate-600"
 }`} >
 {playing ? <Pause className="w-3.5 h-3.5" /> : <Play className="w-3.5 h-3.5" />}
 {playing ? "Пауза" : "Пуск"}
 </button>
 <button
 onClick={() => { setPlaying(false); setStep(FRAMES.length - 1); }}
 disabled={step >= FRAMES.length - 1}
 className="flex items-center gap-1 bg-indigo-700 hover:bg-indigo-600 text-white text-xs font-medium" >
 <RotateCw className="w-3.5 h-3.5" /> Конец
 </button>
 </div>
 </div>
 </div>
);

```

```

 ">{n.label}</text>
 <text x={n.x} y={n.y - 18} textAnchor=
nts-none">
 {c === null ? "-" : `c${c}`}
 </text>
 </g>
);
 }}}}
</svg>
<div className="absolute top-2 left-2 bg-slate-
z-20">
 Компонент: <span className="font-bold text-r
</div>
{compilerLinked && (
 <div className="absolute top-2 right-2 bg-em
erald-300 z-20">
 синхронизация с Python
 </div>
)}
</div>

<div className="space-y-3">
 <div className="bg-slate-950 rounded-xl borde
 <div className="text-[10px] uppercase track
 <div className="flex flex-wrap gap-1 min-h-
 {shownQueue.length === 0 && <span classNa
 {shownQueue.map((q, i) => (
 <span key={` ${q} - ${i} `} className="px-2
-mono">
 {NODES[q].label}

)}}
 </div>
 </div>
</div>
<div className="bg-slate-950 rounded-xl borde
 <div className="text-[10px] uppercase track
 <div className="grid grid-cols-5 gap-1">
 {NODES.map((n) => {

```



```

{ line: "void dfs(int v, int p = -1) {", highlight: "normal" },
{ line: " visited[v] = true;", highlight: "normal" },
{ line: " tin[v] = low[v] = timer++;", highlight: "normal" },
{ line: " for (int to : adj[v]) {", highlight: "normal" },
{ line: " if (to == p) continue;", highlight: "normal" },
{ line: " if (visited[to]) {", highlight: "normal" },
{ line: " low[v] = min(low[v], tin[to]);", highlight: "normal" },
{ line: " } else {", highlight: "normal" },
{ line: " dfs(to, v);", highlight: "normal" },
{ line: " low[v] = min(low[v], low[to]);", highlight: "normal" },
{ line: " }", highlight: "normal" },
{ line: " if (low[to] > tin[v]) {", highlight: "normal" },
{ line: " // ЭТО МОСТ!", highlight: "normal" },
{ line: " }", highlight: "normal" },
{ line: " }", highlight: "normal" },
{ line: "}", highlight: "normal" },
];

```

```

interface DfsStep {
 currentNode: number | null;
 visitedIndices: number[];
 tin: Record<number, number>;
 low: Record<number, number>;
 stack: number[];
 bridges: string[];
 log: string;
 activeEdge: [number, number] | null;
 backEdge: [number, number] | null;
 activeCodeLine: number[];
}

```

```

const GRAPH_NODES = [
 { id: 0, label: "A", x: 100, y: 120 },
 { id: 1, label: "B", x: 260, y: 120 },
 { id: 2, label: "C", x: 180, y: 200 },

```

```

 },
 {
 currentNode: 1, visitedIndices: [0,1,6],
 tin: {0:1,1:2,6:3}, low: {0:1,1:2,6:3}, stack: [0,1],
 activeEdge: null, backEdge: null,
 log: "У G нет соседей (кроме B). Возврат. low[G]=3 : ",
 activeCodeLine: [13,14,15]
 },
 {
 currentNode: 2, visitedIndices: [0,1,6,2],
 tin: {0:1,1:2,6:3,2:4}, low: {0:1,1:2,6:3,2:4}, stack: [0,1,2],
 activeEdge: [1,2], backEdge: null,
 log: "Из B идём в C. tin[C]=4, low[C]=4.",
 activeCodeLine: [4,5,7,10,11]
 },
 {
 currentNode: 2, visitedIndices: [0,1,6,2],
 tin: {0:1,1:2,6:3,2:4}, low: {0:1,1:2,6:3,2:1}, stack: [0,1,2],
 activeEdge: null, backEdge: [2,0],
 log: "Видим соседа A (уже посещён)! Обратное ребро ",
 activeCodeLine: [7,8,9]
 },
 {
 currentNode: 3, visitedIndices: [0,1,6,2,3],
 tin: {0:1,1:2,6:3,2:4,3:5}, low: {0:1,1:2,6:3,2:1,3:2},
 activeEdge: [2,3], backEdge: null,
 log: "Идём C→D. tin[D]=5, low[D]=5.",
 activeCodeLine: [4,5,7,10,11]
 },
 {
 currentNode: 4, visitedIndices: [0,1,6,2,3,4],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6}, low: {0:1,1:2,6:3,2:1,3:2,4:3},
 activeEdge: [3,4], backEdge: null,
 log: "D→E. tin[E]=6, low[E]=6.",
 activeCodeLine: [4,5,7,10,11]
 },
 {
 currentNode: 5, visitedIndices: [0,1,6,2,3,4,5],

```

```

 activeCodeLine: [11,13,16,17]
 },
 {
 currentNode: 0, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:1},
 activeEdge: null, backEdge: null,
 log: "Возврат B→A. DFS завершён! Мосты: (B,G) и (C,D). Алгоритм завершён!",
 activeCodeLine: [11,13,16,17,18]
 },
 {
 currentNode: null, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:1},
 activeEdge: null, backEdge: null,
 log: "Готово! Найдены мосты: (B,G) и (C,D). Алгоритм завершён!",
 activeCodeLine: []
 },
],
];

export function DfsBridgesSimulator() {
 const [dfsIdx, setDfsIdx] = useState(0);
 useVizStepSync(dfsIdx, setDfsIdx, DFS_STEPS.length - 1);
 const rawV = vizString(vars.v) ?? (vizNumber(vars.v) ?? null);
 const rawTo = vizString(vars.to) ?? (vizNumber(vars.to) ?? null);
 const toId = (value: string | null) => {
 if (value === null) return null;
 const byLabel = GRAPH_NODES.find((node) => node.label === value);
 return byLabel ?? (Number.isFinite(Number(value)) ? Number(value) : null);
 };
 const v = toId(rawV);
 const to = toId(rawTo);
 if (v === null) return null;
 const candidates = DFS_STEPS
 .map((step, index) => ({ step, index }))
 .filter(({ step }) => step.currentNode === v && (step.backEdge === to || step.activeEdge === to));
 return candidates.at(-1)?.index ?? null;
});

const [dfsAuto, setDfsAuto] = useState(false);
const dfsTimer = useRef<NodeJS.Timeout | null>(null);

```

```

 <RefreshCw className="h-3.5 w-3.5" />
 </button>
 </div>
</div>

<div className="grid grid-cols-1 md:grid-cols-5 gap-4" >
 { /* Graph visualization */ }
 <div className="md:col-span-3 bg-slate-950 rounded-2xl p-4" >
 <div className="absolute inset-0 bg-[radial-gradient(circle,transparent_1px,black_1px)]" >
 <svg className="w-full h-[280px] z-10 relative" >
 {GRAPH_EDGES.map(e => {
 const u = GRAPH_NODES.find(n => n.id === e.u);
 const v = GRAPH_NODES.find(n => n.id === e.v);
 const bridge = step.bridges.includes(e.key);
 const activeTree = step.activeEdge && (
 (step.activeEdge[0]===e.u && step.activeEdge[1]===e.v) ||
 (step.activeEdge[0]===e.v && step.activeEdge[1]===e.u)
);
 const activeBack = step.backEdge && (
 (step.backEdge[0]===e.u && step.backEdge[1]===e.v) ||
 (step.backEdge[0]===e.v && step.backEdge[1]===e.u)
);
 let sc = "#475569", sw = 2, dash = "none";
 if (bridge) { sc = "#eab308"; sw = 4; }
 else if (activeTree) { sc = "#10b981"; sw = 3; }
 else if (activeBack) { sc = "#f43f5e"; sw = 3; }
 else if (step.visitedIndices.includes(e.u) || step.visitedIndices.includes(e.v)) {
 sc = "#9467bd"; sw = 2;
 }
 return <line key={e.key} x1={u.x} y1={u.y} x2={v.x} y2={v.y}
 stroke={sc} strokeWidth={sw} strokeDash={dash}
 className="transition-all duration-500" />
 })}
 {step.currentNode !== null && step.currentNode.id === step.activeNode.id ? (
 const n = GRAPH_NODES.find(x => x.id === step.currentNode.id);
 if (!n) return null;
 return <circle cx={n.x} cy={n.y} r={18} fill={step.currentNode.color} />
) : null}
 </svg>
 </div>
 </div>
</div>

```

```

<div className="p-2 overflow-x-auto">
 <pre className="text-[10px] font-mono leading-relaxed">
 {CODE_LINES.map(({cl, i}) => {
 const hl = step.activeCodeLine.includes(cl);
 return <div key={i}
 className={`px-1 transition-all duration-200
 ${hl ? "bg-indigo-600/20 text-indigo-400" : "text-slate-400"}
 `}>
 {i}
 {cl.line || ' '}
 </div>);
 </pre>
</div>

{/* Stack */}
<div className="bg-slate-900/50 rounded-xl border border-slate-800 p-2">
 <div className="text-[10px] font-bold text-slate-400">
 Стек DFS
 {step.stack.length}
 </div>
 <div className="flex flex-col-reverse gap-1">
 {step.stack.length === 0
 ? <div className="text-[10px] text-slate-400">Пустой стек</div>
 : step.stack.map((id, i) => {
 <div key={id} className={`px-2 py-0.5 transition-colors
 ${i === step.stack.length - 1
 ? "bg-emerald-600/20 text-emerald-400"
 : "bg-slate-900 text-slate-400"}
 `}>Вершина {GRAPH_NODES.find(n=>n.id === id).name}</div>
 {step.stack[i + 1]}
 </div>);
 </div>
 </div>
</div>

```

```

const nodes: Node[] = [
 { id: 'A', x: 60, y: 150, name: 'Пиццерия (A)' },
 { id: 'B', x: 160, y: 60, name: 'Дом 1 (B)' },
 { id: 'C', x: 160, y: 240, name: 'Парк (C)' },
 { id: 'D', x: 340, y: 60, name: 'Офис (D)' },
 { id: 'E', x: 340, y: 240, name: 'Дом 2 (E)' },
 { id: 'F', x: 450, y: 150, name: 'ТЦ (F)' },
 { id: 'G', x: 250, y: 150, name: 'Метро (G)' },
];

const edges: Edge[] = [
 { u: 'A', v: 'B', w: 4 },
 { u: 'A', v: 'C', w: 2 },
 { u: 'C', v: 'B', w: 1 },
 { u: 'C', v: 'G', w: 3 },
 { u: 'C', v: 'E', w: 8 },
 { u: 'B', v: 'G', w: 4 },
 { u: 'B', v: 'D', w: 5 },
 { u: 'G', v: 'D', w: 1 },
 { u: 'G', v: 'E', w: 2 },
 { u: 'D', v: 'F', w: 3 },
 { u: 'E', v: 'F', w: 1 },
];

const adjList: { [key: string]: { v: string; w: number } } = {};
nodes.forEach(n => (adjList[n.id] = []));
edges.forEach(e => {
 adjList[e.u].push({ v: e.v, w: e.w });
});

const codeLines = [
 { line: 1, text: 'function dijkstra(graph, start):' },
 { line: 2, text: ' dist = {v: ∞}; dist[start] = 0' },
 { line: 3, text: ' pq = PriorityQueue(); pq.push(start, 0)' },
 { line: 4, text: ' while not pq.empty():' },
 { line: 5, text: ' d, u = pq.pop() // Извлечение минимума' },
 { line: 6, text: ' if d > dist[u]: continue' },
 { line: 7, text: ' for v, weight in graph.neighbors(u):' },

```

```

 currentNode: u, checkingEdge: null,
 distances: { ...dist }, visited: { ...visited },
 log: ` Выбираем вершину ${u} с мин. непосещённой
 activeCodeLine: 5,
 });

// Граф неориентированный: при релаксации смотрим
const neighbors = edges.filter(e => e.u === u || e.v === u);
for (const edge of neighbors) {
 const v = edge.v;
 if (visited[v]) continue;

 simulationSteps.push({
 currentNode: u, checkingEdge: { u, v },
 distances: { ...dist }, visited: { ...visited },
 log: ` Смотрим соседа ${v}. Текущее расстояние
 ${dist[u] + edge.w}.`,
 activeCodeLine: 8,
 });

 if (dist[u] + edge.w < dist[v]) {
 dist[v] = dist[u] + edge.w;
 prev[v] = u;
 simulationSteps.push({
 currentNode: u, checkingEdge: { u, v },
 distances: { ...dist }, visited: { ...visited },
 log: ` Улучшение (Релаксация)! Новое кратчайшее
 activeCodeLine: 9,
 });
 }
}
}

simulationSteps.push({
 currentNode: null, checkingEdge: null,
 distances: { ...dist }, visited: { ...visited },
 log: ` Дейкстра всё доставил! Все возможные кратчайшие
 activeCodeLine: 11,
});

```

```

return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border"
 {/* Шапка симулятора */}
 <div className="flex flex-wrap items-center justify-between"
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-indigo-600 text-white"

 </div>
 <div>
 <h3 className="text-2xl font-bold text-white">
 <p className="text-sm text-indigo-300">Полный
 </p>
 </div>
 </div>

 <div className="flex items-center gap-2">
 <button
 onClick={() => { setIsPlaying(false); setCurrentStepIndex(0); }}
 className="flex items-center gap-1 bg-slate-800 text-white p-2 rounded-lg"
 >
 Сброс
 </button>
 <button
 onClick={() => setIsPlaying(!isPlaying)}
 className={`flex items-center gap-1 px-4 py-2 rounded-lg ${
 isPlaying ? 'bg-amber-600 hover:bg-amber-500' : 'bg-indigo-600 hover:bg-indigo-500'
 }`}
 >
 {isPlaying ? 'Пауза' : '▶ Пуск'}
 </button>
 <button
 onClick={() => { setIsPlaying(false); if (currentStepIndex < steps.length - 1) {
 disabled={currentStepIndex >= steps.length - 1}
 className="flex items-center gap-1 bg-indigo-600 text-white p-2 rounded-lg text-sm font-medium transition-colors"
 }
 >
 Шаг

```



```

 return (
 <g key={idx}>
 <line
 x1={uNode.x} y1={uNode.y}
 x2={vNode.x} y2={vNode.y}
 stroke={isChecking ? '#f59e0b' : isOpt}
 strokeWidth={isChecking ? 5 : isOpt}
 markerEnd={marker}
 className="transition-all duration-1"
 strokeDasharray={isChecking ? '4,4' : ''}
 />
 <circle
 cx={({uNode.x + vNode.x} / 2} cy={({uNode.y + vNode.y} / 2)}
 r="12" fill="#1e293b"
 stroke={isChecking ? '#f59e0b' : '#f87192'}
 />
 <text
 x={({uNode.x + vNode.x} / 2} y={({uNode.y + vNode.y} / 2)}
 fill={isChecking ? '#f59e0b' : '#f87192'}
 fontSize="11" fontWeight="bold" text={edge.w}
 />
 </g>
);
 }
}

```

```

{/* Вершины */}
nodes.map((node) => {
 const isCurrent = currentStep.currentNode === node.id
 const isVisited = currentStep.visited[node.id]
 const dist = currentStep.distances[node.id]

 return (
 <g key={node.id} className="transition-all duration-1">
 <circle
 cx={node.x} cy={node.y}

```



```

 <div className="bg-slate-950/80 rounded-lg p-4" style={isDark ? {} : {background: 'linear-gradient(to right, #1e293b 49%, #0d1b2a 49% 51%, #0d1b2a 51% 53%, #1e293b 53%)'}}>
 {codeLines.map((item) => {
 const isActive = currentStep.activeCodeLine === item.line
 return (
 <div key={item.line} className={`py-1 px-2 ${
 isActive ? 'bg-indigo-900/80 text-white' : 'text-slate-400'
 }`} style={isDark ? {} : {background: 'linear-gradient(to right, #1e293b 49%, #0d1b2a 49% 51%, #0d1b2a 51% 53%, #1e293b 53%)'}}>
 {item.line}
 {item.code}
 </div>
)
 })}
 </div>
 </div>
</div>
</div>
</div>
</div>
);
}

```

ФАЙЛ 41/96: src/components/DPVisualizer.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 10

```

import { useState, useEffect } from 'react';
import { useVizRuntime, vizNumber, vizRecord } from '../vizRuntime';
import { Play, Pause, RotateCcw, ChevronLeft, ChevronRight } from 'lucide-react';

```

```

interface DPStep {
 id: number;
 n: number;
 action: 'call' | 'memo_hit' | 'base_case' | 'calc' | 'return';
 desc: string;
 codeLine: number;
}

```

```

 id: stepId++,
 n,
 action: 'memo_hit',
 desc: `× Кэш-хит! fibМемо(${n}) уже посчитано`,
 codeLine: 2,
 memoState: { ...memoMap }
 });
 return memoMap[n];
}

if (n <= 2) {
 generatedSteps.push({
 id: stepId++,
 n,
 action: 'base_case',
 desc: `Базовый случай: n <= 2 (n=${n}). Возвр`,
 codeLine: 3,
 memoState: { ...memoMap }
 });
 return 1;
}

generatedSteps.push({
 id: stepId++,
 n,
 action: 'call',
 desc: `Вычисляем рекурсивно: fibМемо(${n - 1})`,
 codeLine: 4,
 memoState: { ...memoMap }
});

const res1 = simulateFib(n - 1);
const res2 = simulateFib(n - 2);
memoMap[n] = res1 + res2;

generatedSteps.push({
 id: stepId++,
 n,

```

```

const vars = runtime?.variables;
const liveK = vizNumber(vars?.k) ?? vizNumber(vars?.n);
const liveMemo = vizRecord(vars?.memo);
const compilerLinked = liveK !== null || !!liveMemo;
const memoState = liveMemo
 ? Object.fromEntries(Object.entries(liveMemo).flatMap(
 ([key, value]) => {
 const number = vizNumber(value);
 return number === null ? [] : [[Number(key), number]];
 }
))
 : baseStep?.memoState || {};
const currentStep: DPStep | null = baseStep && compilerLinked
 ? {
 ...baseStep,
 n: liveK ?? baseStep.n,
 memoState,
 desc: `Компилятор: k=${liveK ?? "-"}; таблица memo: ${memoState}`
 }
 : baseStep;

return (
 <div className="bg-slate-900 rounded-2xl border border-slate-800" >
 { /* Header & Controls */ }
 <div className="flex flex-col lg:flex-row lg:items-center" >
 <div>
 <div className="flex items-center gap-3 mb-2" >
 <div className="p-2 bg-emerald-500/10 border border-emerald-500" >
 <RefreshCw className="w-6 h-6" />
 </div>
 <h3 className="text-2xl font-extrabold text-white" >
 Компилятор
 </h3>
 </div>
 <p className="text-sm text-slate-400" >
 Смотри, как таблица DP кэширует результаты
 </p>
 </div>

 <div className="flex flex-wrap items-center gap-2" >
 <div className="flex items-center gap-2 bg-slate-800 p-2" >
 N:

```

```

 {isPlaying ? 'Пауза' : 'Пуск'}
 </button>
 <button
 onClick={() => { setIsPlaying(false); set
 disabled={currentStepIndex === steps.length
 className="p-2 text-slate-400 hover:text-
 title="Шаг вперед"
 >
 <ChevronRight className="w-5 h-5" />
 </button>
 </div>
</div>
</div>

{/* DP Table View */}
<div className="mb-8 bg-slate-950 p-4 md:p-6 rounded
 <h4 className="text-xs font-bold uppercase trac
 <div className="flex flex-wrap items-center gap
 {Array.from({ length: targetN }, (_, i) => i
 const isCached = num in memoState || num <=
 const val = num <= 2 ? 1 : memoState[num];
 const isCurrent = currentStep?.n === num;

 return (
 <div
 key={num}
 className={`flex flex-col items-center p
 isCurrent
 ? 'bg-emerald-950 border-emerald-500
 : isCached
 ? 'bg-slate-900 border-emerald-500/
 : 'bg-slate-900/50 border-slate-800
 }`
 >
 <span className="text-xs text-slate-400
 <span className={`text-xl md:text-2xl f
 {isCached ? val : '0'}


```

```

 }` }
 >
 <Code className="w-4 h-4" />
 Интерактивный код
 </button>
 </div>

 { /* Tab Content */ }
 <div className="min-h-[320px] flex flex-col justify-between" >
 {activeTab === 'table' && (
 <div className="bg-slate-950 p-6 rounded-2xl" >
 <h5 className="font-bold text-white text-sm" >

 </h5>
 <p className="text-sm text-slate-400" >
 В классической рекурсии для $N=\{targetN\}$ по
 вычисленное значение в таблице, мы момента
 </p>
 <div className="p-4 bg-slate-900 rounded-xl" >
 Всего шагов симуляции: {steps.length}
 <span className="text-emerald-400 font-bo
 </div>
 </div>
)}

 {activeTab === 'code' && (
 <div className="bg-slate-950 rounded-2xl border" >
 <div className="bg-slate-900 px-6 py-3 border" >

 </div>
 <pre className="p-6 font-mono text-sm space" >
 {DP_CODE_LINES.map((line, idx) => {
 const lineNum = idx + 1;
 const isActive = currentStep?.codeLine :
 return (
 <div

```

ФАЙЛ 42/96: src/components/EulerSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import { useMemo, useState } from "react";
import { Undo } from "lucide-react";
import { useVizRuntime, vizArray, vizString } from "../viz";
```

```
const C1_NODES = [
 { id: 0, label: "A", x: 190, y: 60 },
 { id: 1, label: "B", x: 280, y: 140 },
 { id: 2, label: "C", x: 100, y: 140 },
 { id: 3, label: "D", x: 280, y: 250 },
 { id: 4, label: "E", x: 100, y: 250 }
];
```

```
const C1_EDGES = [
 { u: 0, v: 1 }, { u: 0, v: 2 },
 { u: 1, v: 3 }, { u: 2, v: 4 },
 { u: 1, v: 2 },
 { u: 3, v: 4 }
];
```

```
export function EulerSimulator() {
 // — Синхронизация с Python-компилятором: path — пос.
 // вершин (["A","B","C"...]), degrees/deg — словарь с
 const runtime = useVizRuntime();
 const vars = runtime?.variables;
 const livePathLabels = vizArray(vars?.path)?.map((x) => {
 const [c1Path, setC1Path] = useState<number[]>([]);
 const [c1VisitedEdges, setC1VisitedEdges] = useState<
 const [c1Msg, setC1Msg] = useState("Клики на вершину");
 const [c1Done, setC1Done] = useState(false);

 const resetC1 = () => {
```





```
 <div className="absolute bottom-2 left-2 bg-slate-900">
 Степени: {degrees.map((d) => `${d.label}:${d.value}`)}
 </div>
 </div>

 <div className={`p-3 rounded-xl border text-sm ${
 c1Done && c1VisitedEdges.length === C1_EDGES.length
 ? "bg-emerald-950/40 border-emerald-500/30 text-emerald-50"
 : c1Done
 ? "bg-rose-950/40 border-rose-500/30 text-rose-50"
 : "bg-slate-900/50 border-slate-700 text-slate-50"
 }`}>
 {c1Msg}
 </div>
 </div>
);
}
```

---

ФАЙЛ 43/96: src/components/ExpressQuiz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useState } from 'react';
import { CheckCircle2, XCircle, Award, RotateCcw, AlertTriangle } from 'lucide-react';

interface Question {
 id: number;
 question: string;
 options: string[];
 correctAnswer: number;
 explanation: string;
}

const quizQuestions: Question[] = [
 {
 id: 1,
 question: "Какая планета является самой большой в нашей Солнечной системе?",
 options: ["Земля", "Юпитер", "Сатурн", "Уран"],
 correctAnswer: 1,
 explanation: "Юпитер — самая большая планета в нашей Солнечной системе."
 },
 {
 id: 2,
 question: "Какой элемент является самым легким?",
 options: ["Водород", "Гелий", "Литий", "Бериллий"],
 correctAnswer: 0,
 explanation: "Водород — самый легкий элемент в таблице Менделеева."
 },
 {
 id: 3,
 question: "Какая планета имеет кольца?",
 options: ["Юпитер", "Сатурн", "Уран", "Нептун"],
 correctAnswer: 1,
 explanation: "Сатурн известен своими яркими и заметными кольцами."
 },
 {
 id: 4,
 question: "Какой газ составляет основную часть воздуха?",
 options: ["Кислород", "Азот", "Водород", "Гелий"],
 correctAnswer: 1,
 explanation: "Азот составляет около 78% объема воздуха."
 },
 {
 id: 5,
 question: "Какая планета известна как 'Красная планета'?",
 options: ["Марс", "Венера", "Юпитер", "Сатурн"],
 correctAnswer: 0,
 explanation: "Марс получил название 'Красная планета' из-за своего красноватого цвета."
 },
 {
 id: 6,
 question: "Какой орган отвечает за выработку энергии в организме?",
 options: ["Сердце", "Печень", "Поджелудочная железа", "Почки"],
 correctAnswer: 0,
 explanation: "Сердце отвечает за перекачку крови и выработку энергии."
 },
 {
 id: 7,
 question: "Какая планета имеет самый высокий температурный диапазон?",
 options: ["Венера", "Меркурий", "Юпитер", "Сатурн"],
 correctAnswer: 0,
 explanation: "Венера имеет самый высокий температурный диапазон из-за своего плотного атмосферы."
 },
 {
 id: 8,
 question: "Какой элемент является самым тяжелым?",
 options: ["Водород", "Гелий", "Литий", "Бериллий"],
 correctAnswer: 0,
 explanation: "Водород — самый легкий элемент, а не самый тяжелый."
 },
 {
 id: 9,
 question: "Какая планета имеет самый высокий температурный диапазон?",
 options: ["Венера", "Меркурий", "Юпитер", "Сатурн"],
 correctAnswer: 0,
 explanation: "Венера имеет самый высокий температурный диапазон из-за своего плотного атмосферы."
 },
 {
 id: 10,
 question: "Какой орган отвечает за выработку энергии в организме?",
 options: ["Сердце", "Печень", "Поджелудочная железа", "Почки"],
 correctAnswer: 0,
 explanation: "Сердце отвечает за перекачку крови и выработку энергии."
 }
];
```

```

 делить вычисления на GPU или многопроцессорных кл
 },
 {
 id: 4,
 question: 'В чем главная суть Динамического програм
 options: [
 'Писать код очень быстро, динамично нажимая на кл
 'Запоминать (кешировать) результаты уже решенных
 'Использовать динамическую типизацию как в Python
 'Постоянно менять требования к проекту в процессе
],
 correctAnswer: 1,
 explanation: 'Точно! Главный принцип ДП – мемоизаци
 },
 {
 id: 5,
 question: 'Какая структура данных идеально помогает
 options: [
 'СНМ (Система непересекающихся множеств / Disjoin
 'Стек (LIFO).',
 'Красно-черное дерево.',
 'Хэш-таблица со списками коллизий.'
],
 correctAnswer: 0,
 explanation: 'Блестяще! СНМ (DSU) позволяет за прак
 '
 }
];

```

```

export const ExpressQuiz: React.FC = () => {
 const [currentQIndex, setCurrentQIndex] = useState<num
 const [selectedOption, setSelectedOption] = useState<
 const [isAnswered, setIsAnswered] = useState<boolean>
 const [score, setScore] = useState<number>(0);
 const [showResults, setShowResults] = useState<boolean>

 const currentQ = quizQuestions[currentQIndex];

```

```

 <div className="bg-slate-950 border border-slate-900" >
 <p className="text-slate-400 text-sm uppercase">Результат теста</p>
 <p className="text-5xl font-black text-indigo-500">{score}</p>
 <p className="text-slate-300 text-sm">
 {score === quizQuestions.length
 ? ' Идеально! Ты настоящий сеньор-раскра
 : score >= 3
 ? ' Отличный результат! Главные концепции
 : ' Стоит повторить главы пособия и еще
 }
 </p>
 </div>

 <button
 onClick={handleRestart}
 className="inline-flex items-center gap-2 px-4 py-2 rounded-xl shadow-lg shadow-indigo-600/30"
 >
 <RotateCcw className="w-5 h-5" />
 Пройти тест заново
 </button>
 </div>
);
}

```

```

return (
 <div className="bg-slate-900/90 border border-slate-900" >
 <div className="flex items-center justify-between" >
 <div className="flex items-center gap-3">

 Экспресс-тест

 <h3 className="text-xl font-bold text-white">
 Вопрос {currentQIndex + 1} / {quizQuestions.length}
 </h3>
 </div>

 Вопрос {currentQIndex + 1} / {quizQuestions.length}

 </div>
 </div>
);

```

```

 }}
 </div>
 <div className="flex-1 text-sm md:text-l
 {option}
 </div>
 </div>
);
 }}}
</div>
</div>

{isAnswered && (
 <div className="bg-slate-950 border border-slate
 <div className="flex items-center gap-2 mb-2
 <AlertCircle className="w-4 h-4" />
 Объяснение:
 </div>
 <p className="text-slate-300 text-sm leading-
 {currentQ.explanation}
 </p>
 </div>
)}

<div className="flex justify-end pt-4 border-t bo
 <button
 onClick={handleNext}
 disabled={!isAnswered}
 className={
 "px-8 py-3.5 font-bold rounded-xl transition
 (!isAnswered
 ? "bg-slate-800 text-slate-500 border bor
 : "bg-indigo-600 hover:bg-indigo-500 acti
 }
 >
 {currentQIndex + 1 < quizQuestions.length ? '
 </button>
</div>
</div>

```

```

 [999, 0, 3, 999, 999, 3, 999],
 [999, 999, 0, 999, 2, 999, 999],
 [999, 999, 999, 0, 4, 2, 999],
 [999, 999, 999, 999, 0, 999, 1],
 [999, 999, 999, 999, 999, 0, 5],
 [999, 1, 999, 999, 999, 999, 0]
];

 const adjList: { [key: number]: { v: number; w: number }[] } = {
 0: [{ v: 1, w: 4 }, { v: 3, w: 2 }],
 1: [{ v: 2, w: 3 }, { v: 5, w: 3 }],
 2: [{ v: 4, w: 2 }],
 3: [{ v: 4, w: 4 }, { v: 5, w: 2 }],
 4: [{ v: 6, w: 1 }],
 5: [{ v: 6, w: 5 }],
 6: [{ v: 1, w: 1 }],
 };

 const codeLines = [
 { line: 1, text: 'function floyd_warshall(matrix, V' },
 { line: 2, text: ' dist = copy(matrix)' },
 { line: 3, text: ' for k from 0 to V - 1: // Пос' },
 { line: 4, text: ' for i from 0 to V - 1:' },
 { line: 5, text: ' for j from 0 to V - 1' },
 { line: 6, text: ' if dist[i][k] + d' },
 { line: 7, text: ' dist[i][j] = ' },
 { line: 8, text: ' return dist' },
];

 useEffect(() => {
 const simulationSteps: Step[] = [];
 const dist = initialMatrix.map(row => [...row]);

 simulationSteps.push({
 k: -1, i: -1, j: -1, matrix: dist.map(r => [...r]),
 log: ' Инициализация матрицы прямых путей, ∞ (999)',
 activeCodeLine: 2,
 });
 });

```

```

useEffect(() => {
 let timer: any = null;
 if (isPlaying && currentStepIndex < steps.length - 1) {
 timer = setTimeout(() => setCurrentStepIndex((p) => p + 1), 1000);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const baseStep = steps[currentStepIndex] || null;
if (!baseStep) return <div>Загрузка...</div>;

const vars = runtime?.variables;
const liveK = vizNumber(vars?.k);
const liveI = vizNumber(vars?.i);
const liveJ = vizNumber(vars?.j);
const rawMatrix = vizArray(vars?.d) ?? vizArray(vars?.d);
const liveMatrix = rawMatrix
 ? baseStep.matrix.map((row, i) => {
 const values = vizArray(rawMatrix[i]);
 return row.map((_, j) => vizNumber(values?.[j]))
 })
 : undefined;
const compilerLinked = liveK !== null || liveI !== null || liveJ !== null;
const currentStep: Step = compilerLinked
 ? {
 ...baseStep,
 k: liveK ?? -1,
 i: liveI ?? -1,
 j: liveJ ?? -1,
 matrix: liveMatrix ?? baseStep.matrix,
 updated: !!runtime?.changed?.some(({name}) => name === 'currentStep'),
 log: `Компилятор: k=${liveK ?? "-"}, i=${liveI ?? "-"}, j=${liveJ ?? "-"},`
 }
 : baseStep;

```

```

<marker id="arrow-fw-via" markerWidth="6"
th d="M0,0 L6,3 L0,6 z" fill="#818cf8" /></
</defs>
{Object.keys(adjList).flatMap((uStr) => {
 const u = parseInt(uStr);
 return adjList[u].map((edge) => {
 const uNode = nodesSvg.find((n) => n.id
 const vNode = nodesSvg.find((n) => n.id
 const isTarget = currentStep.i === u &&
 const isVia = (currentStep.i === u && c
 let strokeColor = '#475569'; let marker
 if (isTarget) { strokeColor = '#10b981'
 else if (isVia) { strokeColor = '#818cf8'
 return (
 <g key={`-${u}-${edge.v}`}>
 <line x1={uNode.x} y1={uNode.y} x2=
markerEnd={marker} strokeDasharray={isTarg
 <circle cx={({uNode.x + vNode.x} / 2
isVia ? '#818cf8' : '#64748b'} strokeWidth
 <text x={({uNode.x + vNode.x} / 2} y=
8fafc'} fontSize="10" fontWeight="bold" tex
 </g>
);
 });
});
}}}
{nodesSvg.map((node) => {
 const isK = currentStep.k === node.id;
 const isI = currentStep.i === node.id;
 const isJ = currentStep.j === node.id;
 let fill = '#1e293b'; let stroke = '#64748b';
 if (isK) { fill = '#4f46e5'; stroke = '#a
 else if (isI) { fill = '#b45309'; stroke =
 else if (isJ) { fill = '#047857'; stroke =
 return (
 <g key={node.id} className="transition-
 <circle cx={node.x} cy={node.y} r={is
sJ ? 4 : 2} className="transition-colors du
 <text x={node.x} y={node.y + 5} fill=

```



```

 }}}
 </div>
);
 }}}
</div>
</div>
</div>

<div className="flex flex-col lg:flex-row gap-6">
 {/* Матрица расстояний */}
 <div className="w-full lg:w-1/2 bg-rose-950/10">
 <div className="flex justify-between items-center">
 <h4 className="text-lg font-bold text-rose-950">
 </div>
 <div className="overflow-x-auto">
 <table className="w-full text-center border">
 <thead><tr className="bg-rose-950/40 text-rose-950">
 <th className="p-2 border-r border-rose-950">
 {nodeNames.map((n, idx) => <th key={idx}
 className="p-2 border-r border-rose-950 font-bold" : ''}`}>{n.split(' ')[0]} {
 </tr></thead>
 <tbody className="text-xs font-mono">
 {currentStep.matrix.map((row, i) => (
 <tr key={i} className="border-b border-rose-950">
 <td className={`p-2 bg-rose-950/40 font-mono`}
 {currentStep.k === i ? 'text-amber-400 font-bold' : ''}`}>
 {nodeNames[i].split(' ')[0]} {i}
 </td>
 {row.map((val, j) => {
 const isUpd = currentStep.i === i && currentStep.k !== j;
 const isVia = currentStep.k !== -1 && currentStep.j === j);
 let bg = 'bg-slate-900/40 text-slate-50';
 if (isUpd) bg = 'bg-emerald-900/80 text-emerald-50';
 else if (isVia) bg = 'bg-indigo-950/60 text-indigo-50';
 else if (currentStep.i === -1 && currentStep.j === j)
 bg = 'bg-indigo-950/60 text-indigo-50';
 else if (i === j) bg = 'bg-slate-900/40 text-slate-50';

```

```

import { Play, Pause, SkipForward, Undo, RefreshCw } from
```

```
type Node = { id: string; label: string; x: number; y: number };
type Edge = { u: string; v: string };
```

```
const nodes: Node[] = [
 { id: 'A', label: 'A', x: 200, y: 40 },
 { id: 'B', label: 'B', x: 100, y: 120 },
 { id: 'C', label: 'C', x: 300, y: 120 },
 { id: 'D', label: 'D', x: 50, y: 200 },
 { id: 'E', label: 'E', x: 150, y: 200 },
 { id: 'F', label: 'F', x: 250, y: 200 },
 { id: 'G', label: 'G', x: 350, y: 200 },
];
```

```
const edges: Edge[] = [
 { u: 'A', v: 'B' }, { u: 'A', v: 'C' },
 { u: 'B', v: 'D' }, { u: 'B', v: 'E' },
 { u: 'C', v: 'F' }, { u: 'C', v: 'G' },
];
```

```
const adj: Record<string, string[]> = {
 'A': ['B', 'C'],
 'B': ['A', 'D', 'E'],
 'C': ['A', 'F', 'G'],
 'D': ['B'],
 'E': ['B'],
 'F': ['C'],
 'G': ['C'],
};
```

```
type Action = {
 type: 'visit' | 'process' | 'backtrack';
 node: string;
 desc: string;
 queueOrStack?: string[];
 visitedNodes?: string[];
};
```

```

while (q.length > 0) {
 const v = q[0];
 steps.push({ type: 'process', node: v, desc: `Добавлен в очередь`,
 q, visitedNodes: [...Array.from(vis)] });
 q.shift();

 for (const u of adj[v]) {
 if (!vis.has(u)) {
 vis.add(u);
 q.push(u);
 steps.push({ type: 'visit', node: u, desc: `Посетили`,
 stack: [...q], visitedNodes: [...Array.from(vis)] });
 }
 }
}

steps.push({ type: 'backtrack', node: '', desc: `Обратный шаг`,
 stack: [...q], visitedNodes: [...Array.from(vis)] });

return steps;
}

```

```

export function GraphTraversalViz() {
 const [mode, setMode] = useState<"dfs" | "bfs">("dfs");
 const [autoPlay, setAutoPlay] = useState(false);
 const [stepIdx, setStepIdx] = useState(0);

 const steps = useMemo(() => mode === 'dfs' ? generateDfsSteps(
 useVizStepSync(stepIdx, setStepIdx, steps.length - 1)) : generateBfsSteps(
 useVizStepSync(stepIdx, setStepIdx, steps.length - 1)), [mode, steps]);

 const v = vizString(vars.v);
 const structure = vizArray(vars.stack) ?? vizArray(vars.visited);
 const visited = vizArray(vars.visited);
 if (v === null && !structure && !visited) return null;
 const candidates = steps
 .map(({step, index}) => ({ step, index }))
 .filter(({ step }) => {
 if (step.type === 'process') return true;
 if (step.type === 'visit') return !vis.has(step.node);
 if (step.type === 'backtrack') return true;
 return false;
 });
 return candidates;
}

```

```

 DFS (В глубину)
 </button>
 <button onClick={() => setMode("bfs")}
 className={`px-4 py-2 text-xs font-l
"text-slate-400 hover:text-slate-300"}`>
 BFS (В ширину / Волна)
 </button>
 </div>

 <div className="flex flex-col md:flex-row g
 {/* SVG Graph View */}
 <div className="bg-slate-900 border bor
ive min-h-[300px]">
 <svg className="w-full h-full max-w
 {/* Edges */}
 {edges.map((e, i) => {
 const u = nodes.find(n => n
 const v = nodes.find(n => n
 // Check if edge is part of
 const edgeTraversed = isVis

 return (
 <line key={i} x1={u.x}
 className={`stroke-
500' : 'stroke-emerald-500'} : 'stroke-slate
);
)}}
 {

 {/* Nodes */}
 {nodes.map(n => {
 const visited = isVisited(n
 const current = isCurrent(n

 let fillColor = "fill-slate
 let strokeColor = "stroke-s
 let textColor = "fill-slate
 let radius = 18;

```



```

 priority: number; // 1 to 99 (Severity)
 animState: 'in' | 'idle' | 'winner' | 'out';
}

```

// Max-heap helpers for Patients

```

function siftUp(arr: Patient[]): Patient[] {
 const a = arr.map(x => ({ ...x }));
 let idx = a.length - 1;
 while (idx > 0) {
 const parent = Math.floor((idx - 1) / 2);
 if (a[parent].priority < a[idx].priority) {
 [a[parent], a[idx]] = [a[idx], a[parent]];
 idx = parent;
 } else break;
 }
 return a;
}

```

```

function siftDown(arr: Patient[]): Patient[] {
 const a = arr.map(x => ({ ...x }));
 const n = a.length;
 let idx = 0;
 while (true) {
 let largest = idx;
 const l = 2 * idx + 1;
 const r = 2 * idx + 2;
 if (l < n && a[l].priority > a[largest].priority) largest = l;
 if (r < n && a[r].priority > a[largest].priority) largest = r;
 if (largest !== idx) {
 [a[largest], a[idx]] = [a[idx], a[largest]];
 idx = largest;
 } else break;
 }
 return a;
}

```

```

function heapInsert(arr: Patient[], item: Patient): Patient[] {
 return siftUp([...arr, { ...item }]);
}

```

```

let uid = 300;

export const HeapViz: React.FC = () => {
 const [heap, setHeap] = useState<Patient[]>();
 const runtime = useVizRuntime();
 const liveHeap = vizArray(runtime?.variables?.h) ?? vizArray([]);
 const liveI = vizNumber(runtime?.variables?.i);
 const visualHeap: Patient[] = liveHeap
 ? liveHeap.map((value, index) => {
 const tuple = vizArray(value);
 const priority = vizNumber(tuple?.[0]) ?? vizNumber(0);
 return {
 id: `python-${index}`,
 name: tuple ? String(tuple[1] ?? value) : String(index),
 emoji: " ",
 symptom: `индекс ${index}`,
 priority,
 animState: "idle",
 };
 })
 : heap;
 const [customName, setCustomName] = useState('');
 const [customPriority, setCustomPriority] = useState(0);
 const [log, setLog] = useState<string[]>([]);
 const [shakeEmpty, setShake] = useState(false);
 const [busy, setBusy] = useState(false);
 const [healingPatient, setHealing] = useState<Patient>();

 const addLog = (msg: string) => setLog(prev => [msg, ...prev]);

 // --- INSERT: Новый пациент поступает в приемный покой
 const insertPatient = useCallback((name: string, priority: number) => {
 if (busy || heap.length >= 15) {
 addLog('⚠ Все палаты переполнены! Дождитесь выписки');
 setShake(true);
 setTimeout(() => setShake(false), 400);
 return;
 }
 const newPatient: Patient = {
 id: `python-${uid}`,
 name,
 emoji: "👤",
 symptom: "Новый пациент",
 priority,
 animState: "idle",
 };
 setHeap(heap.concat(newPatient));
 setBusy(true);
 setHealing(newPatient);
 uid++;
 }, [heap, busy, setBusy, setHealing, setShake, setLog]);
}

```

```

 e.preventDefault();
 if (!customName.trim()) return;
 insertPatient(customName, customPriority);
 setCustomName('');
 });

// --- EXTRACT MAX: Направить самого тяжелого в реанимацию
const extractMax = useCallback(() => {
 if (busy || heap.length === 0) {
 setShake(true);
 addLog('⚠ Нет пациентов для экстренной госпитализации');
 setTimeout(() => setShake(false), 400);
 return;
 }
 setBusy(true);
 const topPatient = heap[0];
 setHealing(topPatient);

 addLog(`СРОЧНО! Забираем самого тяжелого: ${topPatient.name}`);

 // Step 1: Root lights up as winner
 setHeap(prev => prev.map((x, i) => i === 0 ? { ...x, animStat: 1 } : x));

 setTimeout(() => {
 // Step 2: Root exits the tree, tree reorganizes
 setHeap(prev => prev.map((x, i) => i === 0 ? { ...x, animStat: 0 } : x));

 setTimeout(() => {
 // Run binary heap extraction
 setHeap(prev => {
 if (prev.length === 0) return [];
 const a = prev.map(x => ({ ...x }));
 a[0] = { ...a[a.length - 1] };
 a.pop();
 return siftDown(a).map(x => ({ ...x, animStat: 0 }));
 });

 // Patient gets healed in the bed
 });
 });
}, [busy, heap]);

```



```

 res.push(
 <line key={`er${idx}`}
 x1={`${p.x}%`} y1={`${p.y + 4}%`}
 x2={`${rp.x}%`} y2={`${rp.y - 4}%`}
 stroke="#475569" strokeWidth="2"
 />
);
 }
 return res;
 });

 return (
 <div className="flex flex-col gap-6">
 {/* МНЕМОНИКА / ПРАВИЛО */}
 <div className="bg-emerald-950/40 border border-emerald-500">
 <div className="text-3xl"> </div>
 <div>
 <h3 className="font-black text-emerald-400 text-center">
 Priority Queue</h3>
 <p className="text-xs text-slate-300 mt-1 leading-4">
 В реанимации не важно, кто пришел раньше, а кто в
 настоящее время (максимальный приоритет).
 Собака (Heap) автоматически перестраивает папку
 оказывался на самом вершине дерева (в корне).
 </p>
 </div>
 </div>

 {/* УПРАВЛЕНИЕ */}
 <div className="flex flex-col lg:flex-row items-stretch">

 {/* Добавить случайного */}
 <button
 onClick={handleRandomInsert}
 disabled={busy || heap.length >= 15}
 className="flex-1 min-w-[150px] bg-gradient-to-r from-emerald-500 to-emerald-400
 opacity-40 disabled:cursor-not-allowed text-emerald-500 font-medium
 py-2 px-3 transition-all cursor-pointer">
 Добавить
 </button>

```

```

 onClick={extractMax}
 disabled={busy || heap.length === 0}
 className="flex-1 min-w-[170px] bg-gradient-to-r
 40 disabled:cursor-not-allowed text-white fr
 transition-all cursor-pointer flex items-c
 >
 В реанимацию (EXTRACT MAX)
 </button>

 <button
 onClick={reset}
 className="px-5 py-3.5 rounded-2xl bg-slate-800
 r-pointer border border-slate-700"
 >
 Сброс
 </button>
 </div>

 {/* ИНТЕРАКТИВНОЕ ОТДЕЛЕНИЕ */}
 <div className="grid grid-cols-1 lg:grid-cols-12

 {/* ЛЕВАЯ КОЛОНКА: ДЕРЕВО ПАЦИЕНТОВ */}
 <div className={`lg:col-span-8 bg-slate-900 rou
 -[380px] overflow-hidden ${shakeEmpty ? 'anim
 <div className="absolute top-4 left-4 text-[1
 Сортировка по тяжести (Двоичная куча / Max-
 </div>

 <div className="relative w-full h-full min-h-
 <svg className="absolute inset-0 w-full h-f
 {edges}
 </svg>

 {visualHeap.map((patient, idx) => {
 const pos = nodePos(idx);
 const isRoot = idx === 0;
 const isCompilerNode = liveI === idx;
 return (

```

```

 { /* ПРАВАЯ КОЛОНКА: ОПЕРАЦИОННЫЙ СТОЛ */ }
 <div className="lg:col-span-4 bg-slate-900 rounded-2xl" >
 <div className="absolute top-4 left-4 text-white" >
 Операционная койка
 </div>

 <div className="flex-1 flex flex-col justify-between" >
 {healingPatient ? (
 <div className="flex flex-col items-center" >
 <div className="relative" >

 </div>
 </div>
 <h4 className="text-white font-black" >
 Пациент: {healingPatient.name}
 </h4>
 <p className="text-emerald-400 text-x-small" >
 Состояние: Стабилизация ({healingPatient.status})
 </p>
 <div className="flex justify-center gap-4" >

 <Heart className="w-4 h-4 text-rose-500" >
 </div>
 </div>
 </div>
 </div>
) : (
 <div className="text-center text-slate-600" >

 <p className="text-xs font-bold font-mono" >
 <p className="text-[10px] mt-1" >Ждём пациента
 </p>
 </div>
 </div>
)}
 </div>
 <div className="w-full h-3 bg-gradient-to-r from-slate-900 to-slate-800" >

```



```

width?: number;
dashed?: boolean;
}> = ({ a, b, color = C.edge, width = 2, dashed }) => (
 <line
 x1={a[0]}
 y1={a[1]}
 x2={b[0]}
 y2={b[1]}
 stroke={color}
 strokeWidth={width}
 strokeDasharray={dashed ? "4 3" : undefined}
 style={{ transition: MOVE }}
 />
);

```

---

ФАЙЛ 48/96: src/components/hints/demosExtra.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```

import React from "react";
import { C, Edge, MOVE, Node, Svg, useLoop } from "../demo";

/** Флойд: внешний цикл по «разрешённой» промежуточной
export const FloydDemo: React.FC = () => {
 const step = useLoop(3, 1200);
 // i -> j напрямую 9; через k=1 получаем 3+4=7
 const pos: Record<string, [number, number]> = { i: [4, 9], j: [3, 7] };
 const viaOpen = step >= 1;
 const relaxed = step >= 2;
 return (
 <Svg
 caption={
 step === 0
 ? "k ещё запрещена: путь i → j = 9"
 : step === 1
 ? "Разрешаем пересадку через k: 3 + 4 = 7"
 : "Разрешаем пересадку через k: 3 + 4 = 7"

```



```

 }
 >
 {raw.map((v, i) => {
 <g key={i}>
 <rect x={startX + i * (boxW + 4)} y={20} width={boxW} height={boxH} />
 <text x={startX + i * (boxW + 4) + boxW / 2} y={40} textAnchor="middle">
 {v}
 </text>
 </g>
 })}

 {step >= 1 &&
 sorted.map((v, i) => {
 <g key={v}>
 <rect x={30 + i * 52} y={62} width={46} height={40} />
 <text x={53 + i * 52} y={77} textAnchor="middle">
 {v}
 </text>
 </g>
 })}

 {step >= 2 &&
 raw.map((v, i) => {
 <g key={`c${i}`}>
 <rect x={startX + i * (boxW + 4)} y={98} width={boxW} height={boxH} />
 <text x={startX + i * (boxW + 4) + boxW / 2} y={120} textAnchor="middle">
 {sorted.indexOf(v)}
 </text>
 </g>
 })}
 </Svg>
);
};

/* ----- Splay: три случая поворота -----

type Pt = [number, number];

```

```

<SplayFrames
 captions={["р – корень, х под ним", "Один поворот: "]}
 frames=[
 { pos: { p: [130, 32], x: [78, 92] }, edges: [{"p": "x", "x": "p"}]},
 { pos: { x: [130, 32], p: [182, 92] }, edges: [{"p": "x", "x": "p"}]}
]
/>
);

/** Zig-Zig – х и р с одной стороны, крутим сначала верхнюю ветвь
export const ZigZigDemo: React.FC = () => {
 <SplayFrames
 captions={["Линия g → p → x («бамбук»)", "Поворот Ветви Влево"]}
 frames=[
 { pos: { g: [176, 24], p: [126, 70], x: [76, 112] }, edges: [{"g": "p", "p": "x", "x": "g"}]},
 { pos: { p: [130, 26], x: [78, 78], g: [186, 78] }, edges: [{"g": "p", "p": "x", "x": "g"}]},
 { pos: { x: [102, 24], p: [152, 70], g: [202, 112] }, edges: [{"g": "p", "p": "x", "x": "g"}]}
]
 />
};

/** Zig-Zag – х и р с разных сторон, крутим сначала нижнюю ветвь
export const ZigZagDemo: React.FC = () => {
 <SplayFrames
 captions={["Зигзаг: g влево, x вправо", "Поворот Нижней Ветви Влево"]}
 frames=[
 { pos: { g: [176, 24], p: [110, 70], x: [152, 112] }, edges: [{"g": "p", "p": "x", "x": "g"}]},
 { pos: { g: [176, 24], x: [110, 70], p: [64, 112] }, edges: [{"g": "p", "p": "x", "x": "g"}]},
 { pos: { x: [130, 30], p: [80, 100], g: [180, 100] }, edges: [{"g": "p", "p": "x", "x": "g"}]}
]
 />
};

```



```

 const iu = visited.indexOf(u);
 const iv = visited.indexOf(v);
 const onPath = iu >= 0 && iv >= 0 && Math.abs(iu - iv) < 2;
 return <Edge key={i} a={GRAPH_POS[u]} b={GRAPH_POS[v]} color={onPath ? "red" : "black"} />
)}
 {Object.entries(GRAPH_POS).map(([k, [x, y]]) => (
 <Node key={k} x={x} y={y} label={k} fill={k === "носки" ? "red" : "black"} />
))}
</Svg>
);
};

export const ToposortDemo: React.FC = () => {
 const nodes = [
 { id: "труссы", x: 52, y: 32 }, { id: "штаны", x: 148, y: 32 },
 { id: "носки", x: 52, y: 98 }, { id: "боты", x: 148, y: 98 }
];
 const edges: [string, string][] = [
 ["труссы", "штаны"],
 ["штаны", "боты"],
 ["носки", "боты"]
];
 const order = ["труссы", "носки", "штаны", "боты"];
 const step = useLoop(order.length + 1, 800);
 const placed = order.slice(0, step);
 const pos = Object.fromEntries(nodes.map((n) => [n.id, [n.x, n.y]]));
 return (
 <Svg caption={`Порядок: ${placed.join(" → ") || "..."} />
 {edges.map(([u, v], i) => (
 <Edge key={i} a={pos[u]} b={pos[v]} color={placed.includes(u) ? "red" : "black"} />
))}
 {nodes.map((n) => {
 const idx = placed.indexOf(n.id);
 return (
 <g key={n.id}>
 <rect x={n.x - 34} y={n.y - 12} width={68} height={24} fill={idx >= 0 ? "black" : "white"} />
 <text x={n.x} y={n.y} textAnchor="middle" dx={idx >= 0 ? -30 : 30} dy={0}>{n.id}</text>
 {idx >= 0 && (
 <text x={n.x + 30} y={n.y - 12} textAnchor="left">{placed[idx]}</text>
)}
 </g>
);
 })}
);
};

```

```

 {edges.map(([u, v], i) => {
 const isBridge = u === "C" && v === "D";
 if (isBridge && cut) return null;
 return <Edge key={i} a={pos[u]} b={pos[v]} color={cut ? "red" : "black"} />
 })}
 {Object.entries(pos).map(([k, [x, y]]) => (
 <Node key={k} x={x} y={y} label={k} fill={cut ? "red" : "black"} />
))}
 </Svg>
);
};

export const ArticulationDemo: React.FC = () => {
 const step = useLoop(2, 1300);
 const pos: Record<string, [number, number]> = {
 A: [36, 40], B: [36, 96], C: [130, 68], D: [224, 40]
 };
 const edges: [string, string][] = [["A", "B"], ["A", "C"], ["B", "D"], ["C", "D"]];
 const cut = step === 1;
 return (
 <Svg caption={cut ? "Убрали вершину C → две отдельные компоненты" : ""}>
 {edges.map(([u, v], i) => (cut && (u === "C" || v === "C") ? null : <Edge key={i} a={pos[u]} b={pos[v]} color={cut ? "red" : "black"} />))}
 {Object.entries(pos).map(([k, [x, y]]) => (cut && k === "C" ? null : <Node key={k} x={x} y={y} label={k} fill={cut ? "red" : "black"} />))}
 </Svg>
);
};

export const MstDemo: React.FC = () => {
 const pos: Record<string, [number, number]> = {
 A: [40, 35], B: [130, 22], C: [220, 45], D: [70, 100]
 };
 const edges: [string, string][] = [
 ["A", "B"], ["A", "C"], ["A", "D"], ["B", "C"], ["B", "D"], ["C", "D"]
];
 const cut = step === 1;
 return (
 <Svg caption={cut ? "Убрали вершину C → две отдельные компоненты" : ""}>
 {edges.map(([u, v], i) => (cut && (u === "C" || v === "C") ? null : <Edge key={i} a={pos[u]} b={pos[v]} color={cut ? "red" : "black"} />))}
 {Object.entries(pos).map(([k, [x, y]]) => (cut && k === "C" ? null : <Node key={k} x={x} y={y} label={k} fill={cut ? "red" : "black"} />))}
 </Svg>
);
};

```

```
 <Node key={k} x={x} y={y} label={k} fill={step} />
)}}
 </Svg>
);
};
```

ФАЙЛ 50/96: src/components/hints/demosStruct.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React from "react";
import { C, Edge, Node, Svg, useLoop } from "../demoKit";

export const HeapDemo: React.FC = () => {
 const states = [[4, 8, 9, 12, 2], [4, 2, 9, 12, 8], [4, 8, 9, 12, 2], [4, 2, 9, 12, 8]];
 const step = useLoop(states.length, 1000);
 const heap = states[step];
 const pos: [number, number][] = [[130, 24], [80, 68], [130, 100], [80, 136]];
 const moving = step === 0 ? 4 : step === 1 ? 1 : 0;
 return (
 <Svg caption="Новый элемент всплывает наверх, пока не достигнет вершины" >
 <Edge a={pos[0]} b={pos[1]} /><Edge a={pos[0]} b={pos[2]} />
 <Edge a={pos[1]} b={pos[3]} /><Edge a={pos[1]} b={pos[2]} />
 {heap.map((v, i) => (
 <Node key={i} x={pos[i][0]} y={pos[i][1]} label={v} />
))}
 </Svg>
);
};
```

```
export const StackDemo: React.FC = () => {
 const states = [["A"], ["A", "B"], ["A", "B", "C"], ["A", "B", "C", "D"], ["A", "B", "C", "D", "E"]];
 const step = useLoop(states.length, 800);
 const items = states[step];
 const popping = step >= 3;
 return (
 <Svg caption="Элементы выталкиваются из стека" >
 {items.map((v, i) => (
 <Node key={i} x={pos[i][0]} y={pos[i][1]} label={v} />
))}
 </Svg>
);
};
```

```
export const DsuDemo: React.FC = () => {
 const parents = [[0, 1, 2, 3], [0, 0, 2, 3], [0, 0, 2, 3], [0, 0, 2, 3]];
 const step = useLoop(parents.length, 950);
 const p = parents[step];
 const pos: [number, number][] = [[50, 95], [105, 95], [155, 95], [205, 95]];
 const rootColor = ["#6366f1", "#10b981", "#f59e0b", "#f59e0b"];
 const root = (i: number): number => (p[i] === i ? i : root(p[i]));
 return (
 <Svg caption="find = «кто твой староста», union = 0" >
 {p.map((par, i) =>
 par !== i ? (
 <path key={i} d={`M ${pos[i][0]} ${pos[i][1]} L ${pos[root(i)][0]} ${pos[root(i)][1]}`
 fill="none" stroke={rootColor[root(i)]} stroke-width={2} />
) : null
)}
 {pos.map(([x, y], i) => <Node key={i} x={x} y={y} />)}
 </Svg>
);
};
```

```
const TRIE_NODES = [
 { id: 0, x: 130, y: 22, ch: "•", parent: null as number },
 { id: 1, x: 72, y: 62, ch: "a", parent: 0 },
 { id: 2, x: 188, y: 62, ch: "b", parent: 0 },
 { id: 3, x: 42, y: 104, ch: "b", parent: 1 },
 { id: 4, x: 102, y: 104, ch: "c", parent: 1 },
 { id: 5, x: 188, y: 104, ch: "c", parent: 2 },
];
```

```
export const TrieDemo: React.FC = () => {
 const step = useLoop(TRIE_NODES.length + 1, 700);
 return (
 <Svg caption="Путь от корня по буквам = слово; общий путь = префикс" >
 {TRIE_NODES.filter((n) => n.parent !== null).map((n) =>
 const p = TRIE_NODES[n.parent as number];
 return <Edge key={n.id} a={[p.x, p.y]} b={[n.x, n.y]} />
)}
 </Svg>
);
};
```

```

 <path key={i} d={`M ${a.x} ${a.y} Q ${(a.x + 1) ${a.y}
 fill="none" stroke={C.warn} strokeWidth={1.5}
 `}
);
 }}}
 {TRIE_NODES.map((n) => {
 <Node key={n.id} x={n.x} y={n.y} label={n.ch} f={n.f}
 })}
</Svg>
);
};

export const SegmentTreeDemo: React.FC = () => {
 const step = useLoop(3, 1000);
 const nodes = [
 { x: 130, y: 24, w: 226, label: "[0..7]", lvl: 0 },
 { x: 72, y: 60, w: 110, label: "[0..3]", lvl: 1 },
 { x: 188, y: 60, w: 110, label: "[4..7]", lvl: 1 },
 { x: 44, y: 96, w: 52, label: "[0..1]", lvl: 2 },
 { x: 100, y: 96, w: 52, label: "[2..3]", lvl: 2 },
 { x: 160, y: 96, w: 52, label: "[4..5]", lvl: 2 },
 { x: 216, y: 96, w: 52, label: "[6..7]", lvl: 2 },
];
 return (
 <Svg caption="Запрос собирается из $O(\log n)$ готовых"
 {nodes.map((n, i) => {
 <g key={i}>
 <rect x={n.x - n.w / 2} y={n.y - 10} width={n.w} height={20}
 fill={n.lvl === step ? C.active : C.idle} stroke={C.warn}
 />
 <text x={n.x} y={n.y} textAnchor="middle" dom
 fill={C.idle} stroke={C.warn}
 />
 </g>
 })}
 </Svg>
);
};

export const PrefixSumDemo: React.FC = () => {
 const a = [3, 1, 4, 1, 5, 9];
 const pref = a.reduce<number[]>((acc, v) => [...acc, v], []);

```

```

 <text x={25 + i * 30} y={29} textAnchor="middle"
 </g>
)))
 {Array.from({ length: count }).map((_, i) => (
 <rect key={i} x={12 + i * 30} y={52 + (i % 4) * 10}
 fill={C.active} opacity={i === 0 ? 1 : 0.4} stroke={C.active}
 >
 </Svgs>
);
};

```

```

export const AmortizedDemo: React.FC = () => {
 const costs = [1, 1, 1, 8, 1, 1, 1, 1];
 const step = useLoop(costs.length + 1, 550);
 const total = costs.slice(0, step).reduce((a, b) => a + b, 0);
 const avg = step ? (total / step).toFixed(2) : "-";
 return (
 <Svgs caption={`Сделано ${step} операций, суммарно $
 {costs.map((c, i) => (
 <rect key={i} x={14 + i * 30} y={104 - c * 10}
 fill={i < step ? (c > 2 ? C.bad : C.done) : C.done}
 >
 </rect>
 </Svgs>
 <line x1={8} y1={106} x2={252} y2={106} stroke={C.active}
 <text x={130} y={20} textAnchor="middle" fontSize=16>
 </Svgs>
);
};

```

```

export const NpDemo: React.FC = () => {
 const step = useLoop(2, 1400);
 return (
 <Svgs caption={step === 0 ? "Найти решение – долго (N^2)" : "Решение найдено"}
 <ellipse cx={130} cy={64} rx={112} ry={48} fill={C.active}
 <ellipse cx={92} cy={64} rx={52} ry={32} fill="#333"
 <text x={92} y={64} textAnchor="middle" dominantBaseline="middle">
 <text x={206} y={34} textAnchor="middle" fontSize=16>
 <circle cx={196} cy={84} r={13} fill={step === 1 ? C.done : C.active}
 <text x={196} y={84} textAnchor="middle" dominantBaseline="middle">
 </Svgs>
);
};

```

```

<Svg caption={`Заполняем таблицу подзадач: готово $
 {Array.from({ length: rows }).map((_, r) =>
 Array.from({ length: cols }).map((_, c) => {
 const idx = r * cols + c;
 const filled = idx < step;
 return (
 <g key={idx}>
 <rect x={40 + c * 38} y={22 + r * 32} wid
 fill={idx === step - 1 ? C.active : fil
 {filled && (
 <text x={57 + c * 38} y={36 + r * 32} t
 fontSize={10} fontWeight="700" fill="
)}
 </g>
);
 })
)}
 <text x={130} y={122} textAnchor="middle" fontSiz
 каждая клетка считается один раз и переиспользу
 </text>
</Svg>
);
};

```

ФАЙЛ 51/96: src/components/hints/MiniDemo.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useState } from "react";
import { DemoPauseContext } from "../demoKit";
import { ArticulationDemo, BfsDemo, BridgeDemo, DfsDemo
import {
 AmortizedDemo, DsuDemo, HeapDemo, NpDemo, PrefixSumDemo,
 SparseTableDemo, StackDemo, SuffixLinkDemo, TrieBfsDemo
} from "../demosStruct";
import {

```

```
 zig: ZigDemo,
 "zig-zig": ZigZigDemo,
 "zig-zag": ZigZagDemo,
 };

export const MiniDemo: React.FC<{ kind: DemoKind }> = (
 const [paused, setPaused] = useState(false);
 const Cmp = REGISTRY[kind];
 if (!Cmp) return null;
 return (
 <DemoPauseContext.Provider value={paused}>
 <div
 className="bg-slate-950/70 rounded-lg border bo
 onMouseEnter={() => setPaused(true)}
 onMouseLeave={() => setPaused(false)}
 >
 <Cmp />
 </div>
 </DemoPauseContext.Provider>
);
};
```

---

ФАЙЛ 52/96: src/components/hints/SimulatorModal.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

---

```
import React, { useEffect } from "react";
import { createPortal } from "react-dom";
import { X } from "lucide-react";
import { getViz } from "../vizRegistry";

interface Props {
 vizId: string | null;
 onClose: () => void;
}
```



```
 <Component />
 </div>
 </div>
 </div>,
 document.body
);
};
```

---

ФАЙЛ 53/96: src/components/hints/TermPopover.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useEffect, useLayoutEffect, useRef, use
import { createPortal } from "react-dom";
import { Play, X } from "lucide-react";
import { GLOSSARY_BY_ID } from "../../data/glossary";
import { MiniDemo } from "../MiniDemo";
import { getViz } from "../vizRegistry";
```

```
export interface HintAnchor {
 termId: string;
 rect: DOMRect;
}
```

```
interface Props {
 anchor: HintAnchor | null;
 onClose: () => void;
 onOpenSimulator: (vizId: string) => void;
 onCardEnter: () => void;
 onCardLeave: () => void;
}
```

```
const CARD_W = 320;
const GAP = 10;
```

```
export const TermPopover: React.FC<Props> = ({ anchor,
```

```

const viz = getViz(entry.simulator);
const simulatorId = entry.simulator;

return createPortal(
 <div
 ref={cardRef}
 onMouseEnter={onCardEnter}
 onMouseLeave={onCardLeave}
 className="fixed z-[100] term-popover"
 style={{
 top: pos ? pos.top : -9999,
 left: pos ? pos.left : -9999,
 width: Math.min(CARD_W, typeof window !== "unde
 visibility: pos ? "visible" : "hidden",
 }}
 role="dialog"
 >
 <div className="bg-slate-900 border border-indigo
 <div className="flex items-start gap-2 px-3 pt-
 <h4 className="text-sm font-bold text-indigo-
 <button
 onClick={onClose}
 className="text-slate-500 hover:text-white
 aria-label="Заккрыть подсказку"
 >
 <X className="w-4 h-4" />
 </button>
 </div>

 <div className="p-3 space-y-2.5 max-h-[60vh] ov
 <p className="text-[13px] leading-relaxed tex

 {entry.demo && <MiniDemo kind={entry.demo} />

 {entry.formal && (
 <p className="text-[11.5px] leading-relaxed
 {entry.formal}
 </p>

```

```

 const edge = `${vizString(vars.u) ?? ""}${vizString(vars.v) ?? ""}`;
 return edge === "AB" ? 4 : edge === "BC" ? 5 : 0;
 });

const nodes = [
 { id: 'S', x: 200, y: 150, label: 'S' },
 { id: 'A', x: 200, y: 50, label: 'A' },
 { id: 'B', x: 320, y: 220, label: 'B' },
 { id: 'C', x: 80, y: 220, label: 'C' },
];

const edges = [
 { id: 'SA', u: 'S', v: 'A', weight: 0, labelDx: 0, labelDy: 0 },
 { id: 'SB', u: 'S', v: 'B', weight: 0, labelDx: 0, labelDy: 0 },
 { id: 'SC', u: 'S', v: 'C', weight: 0, labelDx: 0, labelDy: 0 },
 { id: 'AB', u: 'A', v: 'B', weight: -2, newWeight: 0, labelDx: 0, labelDy: 0 },
 { id: 'BC', u: 'B', v: 'C', weight: 1, newWeight: 0, labelDx: 0, labelDy: 0 },
 { id: 'CA', u: 'C', v: 'A', weight: 2, newWeight: 0, labelDx: 0, labelDy: 0 },
];

const isNodeVisible = (id: string) => {
 if (id === 'S' && (step === 0 || step === 7)) return true;
 return false;
};

const getPotential = (id: string) => {
 if (step >= 2 && step < 7) {
 if (id === 'S') return 0;
 if (id === 'A') return 0;
 if (id === 'B') return -2;
 if (id === 'C') return -1;
 }
 return null;
};

const getEdgeColorClass = (e: any) => {
 if (e.u === 'S') {
 if (step === 0 || step === 7) return "opacity: 0.5;";
 }
 return "";
};

```

```

 };

 const getEdgeContent = (e: any) => {
 if (e.u === 'S') return e.weight;

 if (step < 4) return e.weight;

 if (e.id === 'AB') {
 if (step === 4) return `${e.weight} + (${0}`
 return e.newWeight;
 }
 if (e.id === 'BC') {
 if (step < 5) return e.weight;
 if (step === 5) return `${e.weight} + (${-2}`
 return e.newWeight;
 }
 if (e.id === 'CA') {
 if (step < 6) return e.weight;
 if (step === 6) return `${e.weight} + (${-1}`
 return e.newWeight;
 }
 return e.weight;
 };

 const isEdgeTextHighlighted = (e: any) => {
 if (e.id === 'AB' && step === 4) return true;
 if (e.id === 'BC' && step === 5) return true;
 if (e.id === 'CA' && step === 6) return true;
 return false;
 };

 const isEdgeTextEmerald = (e: any) => {
 if (e.id === 'AB' && step > 4) return true;
 if (e.id === 'BC' && step > 5) return true;
 if (e.id === 'CA' && step > 6) return true;
 return false;
 };

```

```

);
 case 5: return (
 <div>
 Шаг 3.2: Пересчет ребра B → C.
 Старый вес: 1.

 Вычисляем: $1 + (-2) - (-1) =$
 </div>
);
 case 6: return (
 <div>
 Шаг 3.3: Пересчет ребра C → A.
 Старый вес: 2.

 Вычисляем: $2 + (-1) - 0 =$
 </div>
);
 case 7: return (
 <div className="text-emerald-300">
 Готово! Фиктивную вершину S мы
 Все новые веса ребер в графе стали
 При этом старые кратчайшие пути оста
 </div>
);
 default: return "";
}
};

return (
 <div className="bg-slate-800 p-4 rounded-xl border">
 <h4 className="text-sm md:text-base font-bold">

 Визуализация: Перевзвешивание алгоритмом
 </h4>

 <div className="flex flex-col md:flex-row gap-4">
 <div className="bg-[#0f172a] flex-1 min-h-100 p-4">
 <svg width="400" height="300" class="

```

```

 return (
 <g key={i}>
 <line
 x1={u.x} y1={u.y}
 className={`stroke-${u.id}`}
 markerEnd={getMarkerEnd(u.id)}
 />

 <text
 x={midX} y={midY}
 textAnchor="middle"
 dominantBaseline="bottom"
 className={`text-${u.id}`}
 style={{textHeight: 1.2, textEmergence: 1}}
 isS ? 'fill' : 'stroke'
 />
 {getEdgeContent(u.id)}
 </text>
 </g>
)
 }
 }
}

{/* Nodes */}
{nodes.map(n => {
 if (!isNodeVisible(n.id)) return null
 const pot = getPotential(n.id)

 return (
 <g key={n.id} className={`node-${n.id}`}>
 <circle
 cx={n.x} cy={n.y}
 className={`stroke-${n.id}`}
 style={{strokeWidth: 2, fill: n.id === 'S' ? 'fill-[#2e8b57]' : 'fill-slate-500'}}
 />
 <text x={n.x} y={n.y}>

```

```

 <Undo strokeWidth={3} size=
 </button>
 <button onClick={() => setStep(
 className={`flex-1 font-bol
 ${step === MAX_STEP
 ? 'bg-emerald-600/5
 : 'bg-indigo-600 ho
 {step === MAX_STEP ? "Готов
 {step !== MAX_STEP && <Skip
 </button>
 </div>

 {/* Progress Dots */}
 <div className="flex justify-center
 {Array.from({length: MAX_STEP +
 <div key={i} className={`w-
125' : i < step ? 'bg-indigo-900' : 'bg-sla
 }}}
 </div>
 </div>
 </div>
</div>
);
}

```

ФАЙЛ 55/96: src/components/KosarajuViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useState, useEffect } from "react";
import { useVizStepSync, vizArray, vizNumber } from "...
import {
 Play,
 Pause,
 RotateCcw,
 Layers,

```

```

 sccMap: Record<number, number>; // node -> sccId
 currentSccId: number;
 activeEdges: GEdge[];
}

export const KosarajuViz: React.FC = () => {
 const [steps, setSteps] = useState<AlgoStep[]>([]);
 const [currentStepIdx, setCurrentStepIdx] = useState(0);
 useVizStepSync(currentStepIdx, setCurrentStepIdx, steps);
 const v = vizNumber(vars.v);
 const orderLength = vizArray(vars.order)?.length;
 if (v === null && orderLength === undefined) return null;
 const candidates = steps
 .map((step, index) => ({ step, index }))
 .filter(({ step }) => (v === null || step.currentNode === v));
 return candidates.at(-1)?.index ?? null;
});
const [isPlaying, setIsPlaying] = useState(false);

useEffect(() => {
 // Generate step-by-step Kosaraju steps
 const generatedSteps: AlgoStep[] = [];
 const visited = new Set<number>();
 const order: number[] = [];
 const sccMap: Record<number, number> = {};

 const recordStep = (
 phase: AlgoStep["phase"],
 desc: string,
 currentNode: number | null,
 transposed: boolean,
 currentSccId: number,
) => {
 generatedSteps.push({
 phase,
 desc,
 visited: new Set(visited),
 currentNode,
 });
 };

```



```

 order.push(u);
 recordStep(
 "dfs1",
 `DFS 1: вершина ${u} полностью обработана. Запи-
 u,
 false,
 -1,
);
 };

// Run DFS1 on all unvisited
for (let i = 0; i < 5; i++) {
 if (!visited.has(i)) {
 dfs1(i);
 }
}

const topoOrder = [...order].reverse();
recordStep(
 "transpose",
 `Первый проход завершен! Стек выхода в топологиче-
 ебра.` ,
 null,
 true,
 -1,
);

// Transpose adj
const adjT: number[][] = Array.from({ length: 5 },
initialEdges.forEach((e) => adjT[e.v].push(e.u));

// Reset visited for phase 2
visited.clear();
let sccId = 0;

const dfs2 = (u: number, currentScc: number) => {
 visited.add(u);

```

```

 }

 recordStep(
 "finished",
 `Алгоритм успешно завершен! Найдено ${sccId} компонента,
 null,
 true,
 sccId,
);

 setSteps(generatedSteps);
 setCurrentStepIdx(0);
 }, []));

const step = steps[currentStepIdx] || {
 phase: "init",
 desc: "Загрузка...",
 visited: new Set(),
 currentNode: null,
 order: [],
 transposed: false,
 sccMap: {},
 currentSccId: -1,
 activeEdges: initialEdges,
};

useEffect(() => {
 let timer: NodeJS.Timeout;
 if (isPlaying && currentStepIdx < steps.length - 1) {
 timer = setInterval(() => {
 setCurrentStepIdx((prev) => prev + 1);
 }, 1500);
 } else {
 setIsPlaying(false);
 }
 return () => clearInterval(timer);
}, [isPlaying, currentStepIdx, steps.length]);

```

```

 >
 <RotateCcw className="w-4 h-4" />
 </button>
</div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-12" >
 {/* Playfield SVG */}
 <div className="lg:col-span-8 bg-[#0B1120] relative" >
 <svg className="w-full h-full max-w-[650px]" >
 {/* Markers for directed arrows */}
 <defs>
 <marker
 id="arrow"
 viewBox="0 0 10 10"
 refX="22"
 refY="5"
 markerWidth="6"
 markerHeight="6"
 orient="auto-start-reverse"
 >
 <path d="M 0 1 L 10 5 L 0 9 z" fill="#66BB6A" />
 </marker>
 <marker
 id="arrow-active"
 viewBox="0 0 10 10"
 refX="22"
 refY="5"
 markerWidth="6"
 markerHeight="6"
 orient="auto-start-reverse"
 >
 <path d="M 0 1 L 10 5 L 0 9 z" fill="#FFC107" />
 </marker>
 <marker
 id="arrow-transposed"
 viewBox="0 0 10 10"
 refX="22"

```

```

 }

 // Adjust slightly for bidirectional link
 const isBidi =
 (edge.u === 3 && edge.v === 4) ||
 (edge.u === 4 && edge.v === 3);
 const dy = isBidi ? (edge.u === 3 ? -10 :

 return (
 <line
 key={`edge-${idx}`}
 x1={x1}
 y1={y1 + dy}
 x2={x2}
 y2={y2 + dy}
 stroke={stroke}
 strokeWidth="2.5"
 markerEnd={`url(#${markerId})`}
 className="transition-all duration-300"
 />
);
 }
}

{/* Nodes */}
initialNodes.map((node) => {
 const isCurrent = step.currentNode === node.id;
 const isVisited =
 step.visited.has(node.id) ||
 (step.phase === "dfs1" && step.currentNode === node.id);
 const sccId = step.sccMap[node.id];

 let classes = "fill-slate-800 stroke-slate-800";
 if (sccId) {
 classes = getSccColor(sccId);
 } else if (isCurrent) {
 classes = "fill-amber-600 stroke-amber-600";
 } else if (isVisited && step.phase === "dfs1") {
 classes = "fill-emerald-700 stroke-emerald-700";
 }
 return (
 <div
 key={node.id}
 className={classes}
 style={{
 width: node.w,
 height: node.h,
 position: 'relative',
 margin: '10px 0',
 }}
 >
 {node.label}
 </div>
);
});

```



```

 onClick={() => setCurrentStepIdx((prev)
 className="bg-slate-900 border border-s
 >
 Назад
</button>
<button
 disabled={currentStepIdx >= steps.length
 onClick={() => setCurrentStepIdx((prev)
 className="bg-slate-900 border border-s
 >
 Вперед
</button>
</div>
</div>
</div>
</div>
</div>
);
};

```

ФАЙЛ 56/96: src/components/KruskalSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```

import React, { useState } from 'react';
import { Play, RotateCcw, SkipForward, HelpCircle, Chec
 ide-react';
import { useVizRuntime, vizNumber, vizString } from '...

interface Vertex {
 id: number;
 label: string;
 x: number;
 y: number;
 color: string; // 'none', 'red', 'blue', 'purple', 'm
}

```

```

 { id: '2-5', u: 2, v: 5, weight: 9, status: 'pending'
 { id: '6-7', u: 6, v: 7, weight: 10, status: 'pending'
 { id: '4-7', u: 4, v: 7, weight: 11, status: 'pending'
 { id: '7-8', u: 7, v: 8, weight: 12, status: 'pending'
 { id: '5-8', u: 5, v: 8, weight: 13, status: 'pending'
];

```

```

export const KruskalSimulator: React.FC<{ defaultAlgo?:
 const [activeAlgo, setActiveAlgo] = useState<'kruskal'
 const [vertices, setVertices] = useState<Vertex[]>(in
 const [edges, setEdges] = useState<Edge[]>(initialEdg
 const [stepIndex, setStepIndex] = useState<number>(0)
 const [heapQueue, setHeapQueue] = useState<string[]>(
 const [activeCheatTab, setActiveCheatTab] = useState<

```

// — Синхронизация с Python-компилятором: i — индекс  
 // списке, u/v — метки вершин, w — вес. Подсвечиваем

```

const runtime = useVizRuntime();
const vars = runtime?.variables;
const liveI = vizNumber(vars?.i);
const liveW = vizNumber(vars?.w);
const liveU = vizString(vars?.u);
const liveV = vizString(vars?.v);
const liveEdges: Edge[] = (() => {
 if (liveI === null && liveW === null && liveU === n
 const target =
 (liveI !== null && initialEdges[liveI]) ||
 (liveU && liveV
 ? initialEdges.find(
 (e) =>
 (initialVertices[e.u].label === liveU &&
 initialVertices[e.v].label === liveV &&
)
 : null) ||
 (liveW !== null ? initialEdges.find((e) => e.weight === liveW) : null)
 if (!target) return edges;
 return edges.map((e) => (e.id === target.id && e.status === 'pending' ? e : null));
})();

```

```

 type: 'info'
 }, ...prev]);
 },
 // Step 4: Include D-E
 () => {
 setVertices(prev => prev.map(v => v.id === 3 || v
 setEdges(prev => prev.map(e => e.id === '3-4' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро D-E в осто́ве',
 description: 'Вершины D и E окрашены в синий цвет',
 type: 'success'
 }, ...prev]));
 },
 // Step 5: Inspect B-C (4)
 () => {
 setEdges(prev => prev.map(e => e.id === '1-2' ? {
 setLogs(prev => [{
 title: 'Шаг 3: Берем ребро B-C (вес 4)',
 description: 'Оно соединяет красную вершину B и
 type: 'info'
 }, ...prev]));
 },
 // Step 6: Include B-C
 () => {
 setVertices(prev => prev.map(v => v.id === 2 ? {
 setEdges(prev => prev.map(e => e.id === '1-2' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро B-C в осто́ве',
 description: 'Вершина C перекрашена в красный цвет',
 type: 'success'
 }, ...prev]));
 },
 // Step 7: Inspect A-D (5)
 () => {
 setEdges(prev => prev.map(e => e.id === '0-3' ? {
 setLogs(prev => [{
 title: 'Шаг 4: Берем ребро A-D (вес 5)',
 description: 'ВНИМАНИЕ! Ребро соединяет КРАСНУЮ

```



```

 type: 'info'
 }, ...prev]);
},
// Step 12: Include E-F
() => {
 setVertices(prev => prev.map(v => v.id === 5 ? {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро E-F в осто́ве',
 description: 'Вершина F окрашена в фиолетовый ц
 type: 'success'
 }, ...prev]));
},
// Step 13: Inspect D-G (8)
() => {
 setEdges(prev => prev.map(e => e.id === '3-6' ? {
 setLogs(prev => [{
 title: 'Шаг 7: Берем ребро D-G (вес 8)',
 description: 'Оно соединяет фиолетовую вершину
 type: 'info'
 }, ...prev]));
},
// Step 14: Include D-G
() => {
 setVertices(prev => prev.map(v => v.id === 6 ? {
 setEdges(prev => prev.map(e => e.id === '3-6' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро D-G в осто́ве',
 description: 'Вершина G окрашена в фиолетовый.'
 type: 'success'
 }, ...prev]));
},
// Step 15: Inspect C-F (9) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '2-5' ? {
 setLogs(prev => [{
 title: 'Шаг 8: Берем ребро C-F (вес 9)',
 description: 'Оно соединяет вершины C и F, кото

```

```

 }, ...prev]));
 },
 // Step 20: Reject E-H
 () => {
 setEdges(prev => prev.map(e => e.id === '4-7' ? {
 setLogs(prev => [{
 title: 'Отказ: Цикл обнаружен!',
 description: 'Выбрасываем ребро E-H.',
 type: 'error'
 }, ...prev]));
 },
 // Step 21: Inspect H-I (12)
 () => {
 setEdges(prev => prev.map(e => e.id === '7-8' ? {
 setLogs(prev => [{
 title: 'Шаг 11: Берем ребро H-I (вес 12)',
 description: 'Соединяет фиолетовую H и последнюю',
 type: 'info'
 }, ...prev]));
 },
 // Step 22: Include H-I
 () => {
 setVertices(prev => prev.map(v => v.id === 8 ? {
 setEdges(prev => prev.map(e => e.id === '7-8' ? {
 setLogs(prev => [{
 title: 'Успех: MST построено Краскалом!',
 description: 'Все 9 вершин окрашены в единый фиолетовый цвет.
 + 4 + 5 + 7 + 8 + 10 + 12 = 51.',
 type: 'success'
 }, ...prev]));
 },
];

 // --- PRIM STEPS ---
 const primSteps = [
 // Step 1: Start at E
 () => {
 setVertices(prev => prev.map(v => v.id === 4 ? {

```

```

 },
 // Step 5: Include A-D, add A's edges to heap
 () => {
 setVertices(prev => prev.map(v => v.id === 0 ? {
 setEdges(prev => prev.map(e => e.id === '0-3' ? {
 eap' } : e));
 setHeapQueue(['A-B (2)', 'B-E (6)', 'E-F (7)', 'D-
 setLogs(prev => [{
 title: 'Успех: Плесень поглотила вершину A',
 description: 'Новое ребро A-B(2) добавлено в кучу',
 type: 'success'
 }, ...prev]));
 },
 // Step 6: Pop A-B (2)
 () => {
 setEdges(prev => prev.map(e => e.id === '0-1' ? {
 setLogs(prev => [{
 title: 'Шаг 4: Куча выдает ребро A-B (вес 2)',
 description: 'Плесень стремительно бежит к вершине A',
 type: 'info'
 }, ...prev]));
 },
 // Step 7: Include A-B, add B's edges
 () => {
 setVertices(prev => prev.map(v => v.id === 1 ? {
 setEdges(prev => prev.map(e => e.id === '0-1' ? {
 eap' } : e));
 setHeapQueue(['B-C (4)', 'B-E (6 - цикл)', 'E-F (7)']);
 setLogs(prev => [{
 title: 'Успех: Вершина B захвачена плесенью',
 description: 'В кучу добавлено B-C(4). Ребро B-E(6) - цикл',
 type: 'success'
 }, ...prev]));
 },
 // Step 8: Pop B-C (4)
 () => {
 setEdges(prev => prev.map(e => e.id === '1-2' ? {
 setLogs(prev => [{

```

```

 setEdges(prev => prev.map(e => e.id === '4-5' ? {
 setLogs(prev => [{
 title: 'Шаг 7: Куча выдает ребро E-F (вес 7)',
 description: 'Плесень из центра Токио (E) тянет',
 type: 'info'
 }, ...prev]));
},
// Step 13: Include E-F, add F's edges
() => {
 setVertices(prev => prev.map(v => v.id === 5 ? {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {
 eap' } : e));
 setHeapQueue(['D-G (8)', 'C-F (9 - цикл)', 'E-H (
 setLogs(prev => [{
 title: 'Успех: Вершина F захвачена',
 description: 'Ребро F-I(13) добавлено в кучу. P
 type: 'success'
 }, ...prev]));
},
// Step 14: Pop D-G (8)
() => {
 setEdges(prev => prev.map(e => e.id === '3-6' ? {
 setLogs(prev => [{
 title: 'Шаг 8: Куча выдает ребро D-G (вес 8)',
 description: 'Плесень расширяется вниз к G.',
 type: 'info'
 }, ...prev]));
},
// Step 15: Include D-G, add G's edges
() => {
 setVertices(prev => prev.map(v => v.id === 6 ? {
 setEdges(prev => prev.map(e => e.id === '3-6' ? {
 eap' } : e));
 setHeapQueue(['C-F (9 - цикл)', 'G-H (10)', 'E-H
 setLogs(prev => [{
 title: 'Успех: Вершина G захвачена',
 description: 'В кучу добавлено G-H(10).',
 type: 'success'

```

```

 description: 'В кучу добавлено H-I(12).',
 type: 'success'
 }, ...prev]);
},
// Step 20: Pop E-H (11) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '4-7' ? {
 setLogs(prev => [{
 title: 'Шаг 11: Куча выдает E-H (вес 11)',
 description: 'Обе вершины E и H уже в плесени.',
 type: 'warning'
 }, ...prev]));
},
// Step 21: Reject E-H
() => {
 setEdges(prev => prev.map(e => e.id === '4-7' ? {
 setHeapQueue(['H-I (12)', 'F-I (13)']));
 setLogs(prev => [{
 title: 'Отказ: Игнорируем E-H',
 description: 'Выбрасываем из кучи.',
 type: 'error'
 }, ...prev]));
},
// Step 22: Pop H-I (12)
() => {
 setEdges(prev => prev.map(e => e.id === '7-8' ? {
 setLogs(prev => [{
 title: 'Шаг 12: Куча выдает H-I (вес 12)',
 description: 'Плесень тянется к последней чистой',
 type: 'info'
 }, ...prev]));
},
// Step 23: Include H-I
() => {
 setVertices(prev => prev.map(v => v.id === 8 ? {
 setEdges(prev => prev.map(e => e.id === '7-8' ? {
 setHeapQueue(['F-I (13 - цикл)']));
 setLogs(prev => [{

```

```

 setVertices(prev => prev.map(v =>
 [0, 1, 2].includes(v.id) ? { ...v, color: 'red'
 }));
 setEdges(prev => prev.map(e => {
 if (['0-1', '1-2'].includes(e.id)) {
 return { ...e, status: 'included', color: 'red'
 }
 if (['3-4', '4-5', '3-6', '6-7', '7-8'].includes(e.id)) {
 return { ...e, status: 'included', color: 'blue'
 }
 return e;
 }));
 setLogs(prev => [{
 title: 'Слияние: Деревни объединились в колхозы',
 description: 'Все выбранные дороги построены разномасштабно',
 type: 'success'
 }, ...prev]);
 },
 // Step 3: Five-Year Plan 2 (Inspecting bridge between two villages)
 () => {
 // Now both kolkhozes look at all outgoing roads
 // Outgoing roads between {A, B, C} and {D, E, F, G, H, I}.
 // A-D (5), B-E (6), C-F (9).
 // Cheapest is A-D (5).
 setEdges(prev => prev.map(e => e.id === '0-3' ? {
 status: 'included', color: 'red'
 } : e));
 setLogs(prev => [{
 title: 'Вторая пятилетка: Колхозы выбирают мосты',
 description: 'Теперь Красный и Синий колхозы одобрены мосты A-D с весом 5. Остальные мосты B-E(6) и C-F(9) не одобрены',
 type: 'warning'
 }, ...prev]);
 },
 // Step 4: Concurrently merge into single Kolkhoz
 () => {
 setVertices(prev => prev.map(v => ({ ...v, color: 'blue'
 })));
 setEdges(prev => prev.map(e =>
 e.status === 'included' || e.id === '0-3' ? {
 status: 'included', color: 'red'
 } : e));
 setLogs(prev => [{
 title: 'Слияние: Колхозы объединились в один',
 description: 'Все одобренные дороги построены',
 type: 'success'
 }, ...prev]);
 }
);

```

```

 setLogs([
 {
 title: 'Введение в раскраску (Краскал)',
 description: 'Представь, что все 9 вершин графа
 type: 'info',
 },
]));
 } else if (algo === 'prim') {
 setLogs([
 {
 title: 'Введение в Плесень (Прима)',
 description: 'Логика «Плесени»: выбираем стар
 едь (Heap) и захватываем соседей!',
 type: 'info',
 },
]));
 } else {
 setLogs([
 {
 title: 'Введение в Коллективизацию (Борувка)',
 description: 'Логика «Борувки»: Каждая из 9 д
 чтобы объединиться в колхоз.',
 type: 'info',
 },
]));
 }
 }
};

```

```

const getColorClass = (color: string, id: number) =>
 switch (color) {
 case 'red': return 'bg-red-500 border-red-300 tex
 case 'blue': return 'bg-blue-500 border-blue-300
 case 'purple': return 'bg-purple-600 border-purple
 case 'mold': return 'bg-emerald-500 border-emerald
 default:
 if (activeAlgo === 'prim' && id === 4 && stepIn
 return 'bg-slate-700 border-emerald-500 text-
 }
 }

```

```

<div className="flex items-center bg-slate-500" style={{width: '100%', height: '100%'}}>
 <button
 onClick={() => handleReset('kruskal')}
 className={
 "flex items-center gap-1.5 px-3 py-2"
 }
 +
 (activeAlgo === 'kruskal'
 ? 'bg-indigo-600 text-white shadow-md'
 : 'text-slate-400 hover:text-slate-300')
 >
 <GitGraph className="w-3.5 h-3.5" />
 Краскал (Билет 18)
 </button>
 <button
 onClick={() => handleReset('prim')}
 className={
 "flex items-center gap-1.5 px-3 py-2"
 }
 +
 (activeAlgo === 'prim'
 ? 'bg-emerald-600 text-white shadow-md'
 : 'text-slate-400 hover:text-slate-300')
 >
 <Layers className="w-3.5 h-3.5" />
 Прима (Билет 19)
 </button>
 <button
 onClick={() => handleReset('boruvka')}
 className={
 "flex items-center gap-1.5 px-3 py-2"
 }
 +
 (activeAlgo === 'boruvka'
 ? 'bg-rose-600 text-white shadow-md'
 : 'text-slate-400 hover:text-slate-300')
 >
 <Users className="w-3.5 h-3.5" />

```





```

 strokeDasharray={isRejected ? '4,4' : ''}
 className={"transition-all duration-300"}
 />
 <text
 x={({u.x + v.x} / 2 + "%")}
 y={({u.y + v.y} / 2 + "%")}
 fill={isInspecting ? '#f59e0b' : '#94a3b8'}
 fontSize="4.5"
 fontFamily="monospace"
 fontWeight="bold"
 textAnchor="middle"
 dominantBaseline="middle"
 className="select-none filter drop-shadow"
 dy="-1.5"
 >
 {edge.weight}
 </text>
 </g>
);
}}}
</svg>

```

```

{/* HTML Overlay for Vertices */}
<div className="absolute inset-0 w-full h-full" >
 {vertices.map(vertex => {
 <div
 key={vertex.id}
 style={{ top: vertex.y + "%", left: vertex.x + "%", width: 100px, height: 100px }}
 className={"absolute -translate-x-1/2 -translate-y-1/2 font-bold text-base border-2 transition-all duration-300"}
 id={vertex.id}
 >
 {vertex.label}
 {vertex.id === 4 && activeAlgo === 'path' ? (

 4

) : null}
 </div>
)}

```

```


))
 })
 </div>
</div>
})

<div className="flex-1 p-4 overflow-y-auto"
 {logs.map((log, idx) => {
 <div
 key={idx}
 className={
 "p-4 rounded-xl border transition-a
 {log.type === 'success'
 ? 'bg-emerald-950/40 border-emera
 : log.type === 'warning'
 ? 'bg-amber-950/40 border-amber-5
 : log.type === 'error'
 ? 'bg-rose-950/40 border-rose-500
 : 'bg-slate-900/60 border-slate-7
 }
 }
 >
 <div className="flex items-center gap
 {log.type === 'success' && <CheckCi
 {log.type === 'warning' && <HelpCir
 {log.type === 'error' && <XCircle c
 {log.type === 'info' && <Play class
 <h5 className="font-bold text-sm le
 </div>
 <p className="text-xs text-slate-300
 </div>
)})
</div>
</div>
</div>
</div>

```

/\* Cheat Sheet: Complexity & Code for all 3 algo

```

 </p>
 <p className="text-2xl font-black text-violet-500">
 <p className="text-slate-400 text-xs mt-1">
 </div>
 <div className="bg-slate-950 p-4 rounded-3xl">
 <p className="text-slate-400 text-xs font-medium">
 <Award className="w-3.5 h-3.5 text-indigo-500">
 </p>
 <p className="text-2xl font-black text-violet-500">
 <p className="text-slate-400 text-xs mt-1">
 </div>
 <div className="bg-slate-950 p-4 rounded-3xl">
 <p className="text-slate-400 text-xs font-medium">
ождении:</p>
 <p className="text-xs text-slate-300 leading-5">
ди и красим вершины в один цвет, бракуя циклы.
 </div>
</div>
<div className="lg:col-span-2">
 <div className="bg-slate-950 p-4 rounded-3xl">
 <p className="text-slate-400 text-xs font-medium">
 <Code className="w-3.5 h-3.5 text-indigo-500">
 </p>
 <pre className="text-xs text-emerald-400 font-mono">
80 flex-1">

```

```
{`# 1. Инициализируем DSU`}
```

```
def find(i):
 if parent[i] == i: return i
 parent[i] = find(parent[i])
 return parent[i]
```

```
def union(i, j):
 r_i, r_j = find(i), find(j)
 if r_i != r_j:
 parent[r_i] = r_j
 return True
 return False
```

```

</p>
<Code className="w-3.5 h-3.5 text-emerald-400" style="border: 1px solid #333; padding: 10px; margin: 10px 0; font-family: monospace; font-size: 0.9em; background-color: #f9f9f9;">
80 flex-1">
{`import heapq

def prim(graph, start):
 visited = [False] * len(graph)
 heap = [(0, start, -1)] # (weight, node, parent)
 mst = []

 while heap:
 w, u, prev = heapq.heappop(heap)
 if visited[u]: continue
 visited[u] = True
 if prev != -1: mst.append((prev, u, w))

 for next_w, v in graph[u]:
 if not visited[v]:
 heapq.heappush(heap, (next_w, v, u))
 return mst`}
</pre>
</div>
</div>
</div>
</div>
))

{activeCheatTab === 'boruvka' && (
 <div className="grid grid-cols-1 lg:grid-cols-2" style="background-color: #f9f9f9; padding: 10px; margin: 10px 0;">
 <div className="lg:col-span-1 space-y-4">
 <div className="bg-slate-950 p-4 rounded-2xl" style="background-color: #1a202c; color: white; padding: 10px; margin-bottom: 10px;">
 <p className="text-slate-400 text-xs font-mono" style="margin: 0 0 10px 0;">
 <Clock className="w-3.5 h-3.5 text-rose-500" style="float: right; margin-left: 10px;"/>
 </p>
 <p className="text-2xl font-black text-white" style="margin: 0 0 10px 0;">
 <p className="text-slate-400 text-xs font-mono" style="margin: 0 0 10px 0;">

```

```
 </div>
 </div>
 </div>
 })
</div>

</div>
);
};
```

---

ФАЙЛ 57/96: src/components/MnemonicCards.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import dijkstraImg from '../assets/dijkstra-kind.jpg';
import bellmanImg from '../assets/bellman-ford-truck.jpg';
import floydImg from '../assets/floyd-network.jpg';

const cards = [
 {
 img: dijkstraImg,
 alt: 'Добрый робот Дейкстра',
 title: ' Добрый (Дейкстра)',
 desc: 'Быстрый, жадный, но работает только с',
 highlight: 'положительными',
 highlightColor: 'text-emerald-400',
 rest: 'весами. Подсунь отрицательное ребро – сломает',
 complexity: 'O((V+E) log V)',
 border: 'border-blue-500/30 hover:border-blue-400/60',
 titleColor: 'text-blue-400',
 badge: 'text-blue-400/70 bg-blue-950/40',
 },
 {
 img: bellmanImg,
 alt: 'Фура по болоту Форд-Беллман',
 title: ' Фура по Болоту (Ф-Б)',
```

```
 <div className={`mt-3 text-xs font-mono px-3`} >
 </div>
 </div>
)})
 </div>
);
}
```

---

ФАЙЛ 58/96: src/components/MobileToc.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useEffect, useState } from "react";
import { List, X } from "lucide-react";
import { Sidebar } from "../Sidebar";

interface Props {
 selectedChapterId: string;
 setSelectedChapterId: (id: string) => void;
 /** Заголовок текущей темы — показываем в кнопке, что */
 currentTitle: string;
 index: number;
 total: number;
}

/**
 * Мобильное содержание: липкая панель снизу-сверху + в
 * На десктопе не рендерится (там обычный сайдбар).
 */
export const MobileToc: React.FC<Props> = ({ selectedChapterId,
 const [open, setOpen] = useState(false);

 useEffect(() => {
 document.body.style.overflow = open ? "hidden" : "";
 return () => {
 document.body.style.overflow = "";
 };
 });
}
```

```
 className="p-2 -mr-2 text-slate-400 hover:cursor-pointer"
 aria-label="Заккрыть содержание"
 >
 <X className="w-5 h-5" />
 </button>
 </div>
 <div className="flex-1 min-h-0">
 <Sidebar
 selectedChapterId={selectedChapterId}
 setSelectedChapterId={setSelectedChapterId}
 onNavigate={() => setOpen(false)}
 />
 </div>
 </div>
</div>
 </>
);
};
```

---

ФАЙЛ 59/96: src/components/Navbar.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useState } from 'react';
import { BookOpen, Sparkles, FileDown, FileText, Code2,
import { chapters } from '../data/content';
import { downloadBookHtml } from '../utils/exportHtml';
```

```
interface NavbarProps {
 /** Открыта ли боковая панель компилятора. */
 compilerOpen: boolean;
 onToggleCompiler: () => void;
}
```

```
export const Navbar: React.FC<NavbarProps> = ({ compilerOpen, onToggleCompiler }) => {
```



```

>
 <BookOpen className="w-4 h-4" /> Учебное по
</button>
<button
 id="tab-compiler-btn"
 onClick={onToggleCompiler}
 aria-pressed={compilerOpen}
 title="Боковая панель Python-компилятора (на
 страницей"
 className={`flex-1 lg:flex-none flex items-
 uration-200 whitespace-nowrap ${
 compilerOpen
 ? 'bg-emerald-600 text-white shadow-lg
 : 'text-slate-400 hover:text-white'
 }`}
>
 <Terminal className="w-4 h-4" /> Python
</button>
<a
 id="download-guide-pdf-btn"
 href={` ${import.meta.env.BASE_URL}export/gu
 download="guide_all_topics.pdf"
 target="_blank"
 rel="noreferrer"
 className="flex items-center justify-center
 ver:bg-indigo-600/20 border border-indigo-5
 title="Скачать PDF-конспект: текст всех тем
>
 <GraduationCap className="w-4 h-4" /> PDF (

<a
 id="download-pdf-btn"
 href={` ${import.meta.env.BASE_URL}export/fu
 download="full_code_all_pages.pdf"
 target="_blank"
 rel="noreferrer"
 className="flex items-center justify-center
 over:bg-emerald-600/20 border border-emeral

```

```

export function PlanarityDemo() {
 return (
 <div className="space-y-4">
 <h4 className="text-base font-bold text-white">K5
 <p className="text-xs text-slate-400">Просто смотри

 <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
 { /* K5 */ }
 <div className="bg-slate-950 rounded-xl p-4 border border-slate-800">
 <div className="absolute top-2 left-2 bg-slate-900/30 w-auto z-20">K5 – НЕПЛАНАРЕН</div>
 <svg className="w-full h-full absolute inset-0">
 {(() => {
 const pts = [
 {id:0,x:160,y:40},
 {id:1,x:270,y:100},
 {id:2,x:230,y:230},
 {id:3,x:90,y:230},
 {id:4,x:50,y:100}
];
 const edges = [
 [0,1],[0,2],[0,3],[0,4],
 [1,2],[1,3],[1,4],
 [2,3],[2,4],
 [3,4]
];
 function findPt(id:number) { return pts.find(p => p.id === id); }
 function intersect(a:number,b:number,c:number,d:number) {
 if (a===c||a===d||b===c||b===d) return true;
 const p1=findPt(a),p2=findPt(b),p3=findPt(c),p4=findPt(d);
 function ccw(ax:number,ay:number,bx:number,by:number,cx:number,cy:number) {
 return (cy-ay)*(bx-ax)>(by-ay)*(cx-ax);
 }
 return ccw(p1.x,p1.y,p3.x,p3.y,p4.x,p4.y)<0
 && ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y)<0;
 }
 const crosses: number[] = [];
 for(let i=0;i<edges.length;i++) {

```

```

 {id:3,x:60,y:220},{id:4,x:160,y:220},{id:5,x:260,y:220}
];
 const edges = [
 [0,3],[0,4],[0,5],
 [1,3],[1,4],[1,5],
 [2,3],[2,4],[2,5]
];
 function findPt(id:number) { return pts.f[id]; }
 function intersect(a:number,b:number,c:number,d:number) {
 if (a===c||a===d||b===c||b===d) return true;
 const p1=findPt(a),p2=findPt(b),p3=findPt(c),p4=findPt(d);
 if (!p1||!p2||!p3||!p4) return false;
 function ccw(ax:number,ay:number,bx:number,by:number,cx:number,cy:number) {
 return (cy-ay)*(bx-ax)>(by-ay)*(cx-ax);
 }
 return ccw(p1.x,p1.y,p3.x,p3.y,p4.x,p4.y)<0
 && ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y)<0;
 }
 const crosses:number[]=[];
 for(let i=0;i<edges.length;i++) for(let j=i+1;j<edges.length;j++) {
 if (intersect(edges[i][0],edges[i][1],edges[j][0],edges[j][1])) {
 if (!crosses.includes(i)) crosses.push(i);
 if (!crosses.includes(j)) crosses.push(j);
 }
 }
 const lines = []; const circles = [];
 for(let i=0;i<edges.length;i++) {
 const p1=findPt(edges[i][0]),p2=findPt(edges[i][1]);
 const xing=crosses.includes(i);
 lines.push(<line key={`l${i}`} x1={p1.x} y1={p1.y} x2={p2.x} y2={p2.y}
 stroke={xing?"#f43f5e":"#4f46e5"} stroke-width={2}/>);
 }
 const labels = ["\u04141","\u04142","\u04143","\u04144","\u04145"];
 for(const p of pts) {
 circles.push(<g key={`c${p.id}`}>
 <circle cx={p.x} cy={p.y} r={8} fill="#4f46e5" stroke="#4f46e5" stroke-width={2}/>
 <text x={p.x} y={p.y+3} textAnchor="middle">{labels[p.id]}</text>
 </g>);
 }

```

```

 Play,
 Plus,
 RotateCcw,
 SkipBack,
 SkipForward,
 Square,
 Terminal,
 Unlink,
 Variable,
 X,
} from "lucide-react";
import { buildInitTemplate, buildSyncTemplate, getPageS
import { clearVizState, emitVizState, onVizDemo } from
import {
 baseVarName,
 buildDebugRunner,
 changedVars,
 DEBUG_STEP_CAP,
 orderVars,
 type DebugResult,
} from "../data/debugRunner";
import { Tooltip } from "../Tooltip";

const PYODIDE_VERSION = "0.27.7";
const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/py
const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyo
const PLAYBACK_SPEED_KEY = "pythonCompilerPlaybackSpeed
const PLAYBACK_SPEEDS = [0.25, 0.5, 1, 1.5, 2, 4] as co

interface PythonCompilerProps {
 /** Страница, с которой синхронизируется редактор. */
 chapterId: string;
 chapterTitle: string;
 /** Размер панели (px) управляется родителем и сохран
 width?: number;
 height?: number | null;
 onWidthChange?: (w: number) => void;
 onHeightChange?: (h: number) => void;

```

```
/* -----
/* Шпаргалка Python: команды сортировки, циклы, структ
/* «Вставить» кладёт снippet в редактор под курсор.
/* -----
```

```
interface Snippet {
 label: string;
 tip: string;
 code: string;
}
```

```
const SNIPPETS: { group: string; items: Snippet[] }[] =
{
 group: "Сортировка",
 items: [
 {
 label: "arr.sort() / sorted()",
 tip: "In-place сортировка по возрастанию и новая",
 code: 'arr = [5, 2, 8, 1]\narr.sort()
) # копия по убыванию\n',
 },
 {
 label: "sorted(key=...)",
 tip: "Сортировка по ключу: рёбра по весу, пары",
 code: 'edges = [(3, "A", "B"), (1, "B", "C"), (
 es)\n',
 },
],
},
{
 group: "Циклы",
 items: [
 {
 label: "for i in range(n)",
 tip: "Вложенные циклы по i и j – как обход матри",
 code: "n, m = 3, 4\nfor i in range(n):",
 },
],
}
```

```

 tip: "Посещённые вершины: in/add за $O(1)$.",
 code: 'visited = set()\nvisited.add("A")\nprint
 },
],
},
{
 group: "Вывод",
 items: [
 {
 label: "f-строка",
 tip: "Подставить значения переменных шага в строку",
 code: 'i, j, k = 1, 2, 0\nprint(f"dist[{i}][{j}]"
 },
],
},
];

```

```

/** Переживает сккрытие панели: код пользователя не теряется
let savedEditor: { code: string; template: string; chapterId: string; } = {

```

```

export function PythonCompiler({ chapterId, chapterTitle, chapterProps, compilerProps }) {
 /** Активная вкладка демонстрации страницы (у sparse-матрицы)
 const [demoId, setDemoId] = useState<string | null>(null)
 const sync = useMemo(() => getPageSync(chapterId, demoId))
 const syncVarBases = useMemo(
 () =>
 [...new Set(
 sync.variables.flatMap((v) =>
 v.name
 .split(/\\s*(?:,|\\/)\s*/)
 .map(baseVarName)
 .filter((name) => /^[A-Za-z_][A-Za-z0-9_]*$/))
)],
 [sync]
);
 const syncVarSet = useMemo(() => new Set(syncVarBases

```

```

/** Автопроход включается пользователем; первый шаг п
const [playing, setPlaying] = useState(false);
const [playbackSpeed, setPlaybackSpeed] = useState<num
 if (typeof window === "undefined") return 1;
 const saved = Number(window.localStorage.getItem(PL
 return PLAYBACK_SPEEDS.includes(saved as (typeof PL
));
const listRef = useRef<HTMLDivElement>(null);

useEffect(() => {
 window.localStorage.setItem(PLAYBACK_SPEED_KEY, Str
}, [playbackSpeed]);

useEffect(() => {
 savedEditor = { code, template: initTemplate, chapt
}, [code, initTemplate, chapterId]);

// демонстрация сообщает, какая вкладка открыта – код
useEffect(() => setDemoId(null), [chapterId]);
useEffect(() => onVizDemo(chapterId, (d) => setDemoId
useEffect(() => () => clearVizState(chapterId), [chap

// Смена страницы (или вкладки демо): нетронутый реда
// инициализацию; свой код не затираем – предлагаем с
useEffect(() => {
 const prev = prevTplRef.current;
 if (initTemplate === prev.base && template === prev
 prevTplRef.current = { base: initTemplate, full: te
 setPlaying(false);
 setEditHold(null);
 setDebug(null); // трасса ссылается на строки старо
 if (code === prev.base || code === prev.full || cod
 setCode(initTemplate);
 setDesyncedFrom(null);
 } else {
 setDesyncedFrom(chapterId);
 }
 // eslint-disable-next-line react-hooks/exhaustive-

```

```

const beforeValues = step?.values ?? {};
const line = step?.line ? source.split("\n")[step.l
// Нас интересуют только простые скалярные цели (`i
// `i +=`). Контейнеры и так всегда берутся из after
// внутри print(f"i={i}") не будет ошибочно принят
const assignment = line.match(
 /^\\s*([A-Za-z_][A-Za-z0-9_]*(?:\\s*,\\s*[A-Za-z_][A
);
const assigned = new Set(assignment?.[1].split(",")

const variables = { ...afterValues };
const locals = { ...afterLocals };
for (const [name, value] of Object.entries(beforeVa
 const scalar = value === null || typeof value !==
 if (scalar && !assigned.has(name) && name in after
 variables[name] = value;
 if (step?.locals[name] !== undefined) locals[na
 }
}
const changed = changedVars(step?.locals, locals);
return {
 line: step?.line,
 func: step?.func,
 locals,
 variables,
 changed: Object.keys(changed).filter((name) => ch
};
}, []));

```

```

const publishDebug = useCallback(
 (result: DebugResult, idx: number, source: string) :
 if (!hasViz || result.steps.length === 0) return;
 const snapshot = snapshotAt(result, idx, source);
 emitVizState(chapterId, {
 line: snapshot.line,
 func: snapshot.func,
 variables: snapshot.variables,
 changed: snapshot.changed,

```



```

 setDebug(null);
 clearVizState(chapterId);
}, [chapterId, code]);

// Базовый интервал 650 мс; множитель скорости хранитс
useEffect(() => {
 if (!playing || !debug) return;
 if (debug.idx >= debug.result.steps.length - 1) {
 setPlaying(false);
 return;
 }
 const t = window.setTimeout(() => stepBy(1), 650 / playbackSpeed);
 return () => window.clearTimeout(t);
}, [playing, debug, stepBy, playbackSpeed]);

const debugRun = useCallback(async (autoPlay = false) => {
 if (running) return;

 const source = code;
 const input = stdin;
 const request = traceRequest;
 setRunning(true);
 setPlaying(false);
 setEditHold(null);
 setDebug(null);
 clearVizState(chapterId);
 setOutputOpen(false);
 setRuntimeMessage(runtimeReady ? "Обновляю трассу..." : "");

 const stdout: string[] = [];
 const stderr: string[] = [];
 const inputLines = input.split(/\r?\n/);

 try {
 const runtime = await getPythonRuntime();
 setRuntimeReady(true);
 runtime.setStdout({ batched: (message) => stdout.push(message)});
 runtime.setStderr({ batched: (message) => stderr.push(message)});
 }

```

```

 if (target && /^(INPUT|TEXTAREA|SELECT)$/.test(target)) {
 if (e.metaKey || e.ctrlKey || e.altKey) return;
 if (e.key === "ArrowRight") {
 e.stopPropagation();
 e.preventDefault();
 setPlaying(false);
 stepBy(1);
 }
 if (e.key === "ArrowLeft") {
 e.stopPropagation();
 e.preventDefault();
 setPlaying(false);
 stepBy(-1);
 }
 if (e.key === " ") {
 e.stopPropagation();
 e.preventDefault();
 setPlaying((v) => !v);
 }
 if (e.key === "Escape") stopDebug();
 };
 // capture-фаза: чтобы глобальный обработчик навигации
 window.addEventListener("keydown", onKey, true);
 return () => window.removeEventListener("keydown", onKey);
 }, [debug, stepBy, stopDebug]);

 // держим текущую строку листинга в видимой области
 const curDebug = debug ? { step: debug.result.steps[debug.result.step] } : undefined;
 const curDebugLine = curDebug?.step?.line;
 useEffect(() => {
 if (curDebugLine === undefined || !listRef.current) return;
 listRef.current
 .querySelector(`[data-line="${curDebugLine}"]`)
 ?.scrollIntoView({ block: "nearest" });
 }, [curDebugLine]);

 // производные данные шага: что изменилось, в каком порядке
 // Строка, которую назначает текущий шаг, подсвечивается

```

```

 return next;
 });
}, []);

return (
 <aside
 aria-label="Python-компилятор"
 className="fixed z-40 inset-x-0 bottom-0 h-[58dvh]
 -[620px] flex flex-col bg-slate-900 border-t lg
 style={typeof window !== "undefined" ? {
 ...(width && window.innerWidth >= 1024 ? { width:
 ...(height ? { height: Math.min(height, window.h
 } : undefined)}
 >
 {/* Верхний край меняет высоту на телефоне и деск
 {onHeightChange && {
 <div
 role="separator"
 aria-orientation="horizontal"
 aria-label="Потяните, чтобы изменить высоту т
 title="Потяните, чтобы изменить высоту термин
 className="absolute -top-1.5 left-0 right-0 h
 onPointerDown={(event) => {
 const el = event.currentTarget;
 el.setPointerCapture(event.pointerId);
 const startY = event.clientY;
 const startH = el.parentElement?.getBoundin
 const onMove = (moveEvent: PointerEvent) =>
 const maxHeight = window.innerHeight - (w
 const minHeight = Math.min(260, maxHeight
 const next = Math.round(startH + startY -
 onHeightChange(Math.max(minHeight, Math.m
 });
 const onUp = () => {
 el.removeEventListener("pointermove", onM
 el.removeEventListener("pointerup", onUp
 el.removeEventListener("pointercancel", on
 };
 }

```

```

 })
 { /* — Заголовок панели
 <div className="shrink-0 flex items-center gap-2"
 <span className={`w-2 h-2 rounded-full shrink-0
 <Terminal className="w-4 h-4 text-emerald-400 s
 <span className="text-[13px] font-bold text-whi
 <span className="hidden sm:inline text-[10px] t

 <div className="ml-auto flex items-center gap-1"
 <Tooltip
 side="bottom"
 content={running ? "Python выполняется..." :
 "запустить автопроход"}
 >
 <button
 type="button"
 onClick={runOrTogglePlayback}
 disabled={running}
 className={`inline-flex items-center gap-
0 ${
 playing
 ? "bg-amber-500/15 text-amber-300 hov
 : "bg-emerald-500/15 text-emerald-300
 }`}
 aria-label={running ? "Python выполняется
 >
 {running ? <Loader2 className="h-4 w-4 an
 " />}
 {playbackSpeed}*
 </button>
 </Tooltip>
 <Tooltip side="bottom" content="Скопировать к
 <button
 type="button"
 onClick={() => void copyCode()}
 className="p-1.5 rounded-lg text-slate-40
 aria-label="Скопировать код"
 >

```

```

 aria-label="Вставить эталонный код"
 >
 <Eye className="w-4 h-4" />
 </button>
 </Tooltip>
 <label
 className="ml-1 inline-flex items-center gap-1 border border-slate-700 hover:border-emerald-500 rounded px-1"
 title="Скорость автоматического проигрывания"
 >
 {running ? <Loader2 className="w-3.5 h-3.5" style={{width: 400px}} /> : null}
 {running ? "авто..." : "авто"}
 <select
 aria-label="Скорость автопрохода"
 value={playbackSpeed}
 onChange={(event) => setPlaybackSpeed(Number(event.target.value))}
 className="cursor-pointer rounded bg-slate-800"
 >
 {PLAYBACK_SPEEDS.map((speed) => (
 <option key={speed} value={speed}>{speed}
))}
 </select>
 </label>
 </div>
</div>

/* — Мини-вкладки: переменные страницы / шпаргалки
<div className="shrink-0 border-b border-slate-800" style={{width: 100px; height: 30px; margin-bottom: 10px;}}>
 <div className="flex items-center gap-1 px-2 pt-1" style={{width: 100%; height: 100%;}}>
 <button
 type="button"
 onClick={() => setStripTab("vars")}
 className={`inline-flex items-center gap-1.5 px-2 py-1 rounded-sm ${
 stripTab === "vars" ? "bg-slate-800 text-white" : "text-slate-400"
 }`}
 >
 <Variable className="w-3.5 h-3.5" /> Переменные
 </button>
 </div>
</div>

```

```

 type="button"
 onClick={onOpenGuide}
 className="inline-flex items-center gap-1.5
700 text-slate-300 hover:text-white hover:backgroun
 >
 {chapterTitle}
 </button>
 </Tooltip>
 </div>
 {desynchedFrom !== null && (
 <Tooltip
 side="bottom"
 content="В редакторе ваш код, а страница
т заменён)."
 >
 <button
 type="button"
 onClick={resync}
 className="flex items-center gap-1.5
0/40 text-amber-400 hover:bg-amber-500/20 t
 >
 <Unlink className="w-3.5 h-3.5 shrink-0">
 Страница сменилась – подставить её пер
 </button>
 </Tooltip>
)}
 {hasViz && (
 <div className="flex flex-wrap items-center"
 {sync.variables.map((v) => (
 <Tooltip
 key={v.name}
 side="bottom"
 content={

 <code className="font-mono text-sm">{v.name}</code>

 {v.role}

 }
 </Tooltip>
)}
 >
)}

```

```

 onClick={() => insertSnippet(s,
 className="inline-flex items-center
 -[10.5px] text-indigo-300 hover:text-white
 title=""
 >
 <Plus className="w-3 h-3 text-slate-500
 {s.label}
 </button>
 </Tooltip>
)}}
 </div>
 </div>
)}}
 <p className="text-[10px] text-slate-600">H
 </div>
)}
</div>

```

```

{ /* — Редактор + ввод
<div className="flex-1 min-h-0 flex flex-col">
 {debug ? (
 <div className="flex-1 min-h-[80px] flex flex-col">
 { /* панель управления шагами */
 <div className="shrink-0 flex items-center">
 <Tooltip content={playing ? `Пауза автопр
 роиграть трассу заново со скоростью ${playba
 л)`}>
 <button
 type="button"
 onClick={runOrTogglePlayback}
 className={`mx-0.5 inline-flex items-center
 r-500/15 hover:bg-amber-500/25`} : "text-eme
 aria-label={playing ? `Пауза автопрох
 >
 {playing ? <Pause className="w-4 h-4"
 <span className="text-[9px] font-bold
 </button>
 </Tooltip>

```

```


 ЛИМИТ

 </Tooltip>
)}
 <Tooltip content={hasViz ? "Визуализация " : "Свободный текст не требуется." : "У страницы нет визуализации"}>
 <span
 tabIndex={0}
 className={`cursor-help inline-flex items-center justify-center gap-1 border-emerald-500/40 bg-emerald-500/10 text-emerald-500`}
 >
 {hasViz ? <Link2 className="w-3 h-3" href="#">Визуализация</Link2> : "Визуализация"}
 {hasViz ? "имена → демо" : "свободный текст"}

 </Tooltip>
 <Tooltip content="Вернуться в редактор (Еще раз нажать на кнопку)">>
 <button type="button" aria-label="Редактор" className="p-1 rounded-md text-rose-400 hover:text-white">
 <Square className="w-3.5 h-3.5" />
 Вернуться в редактор (Еще раз нажать на кнопку)
 </button>
 </Tooltip>
 </div>

```

```

{/* листинг с подсветкой текущей строки */}
<div ref={listRef} className="flex-1 min-h-[200px] p-1">
 {debug.src.split("\n").map((line, i) => {
 const ln = i + 1;
 const active = ln === curDebug?.step?.line;
 return (
 <div
 key={i}
 data-line={ln}
 className={`flex whitespace-pre px-1 py-1 text-[10px] ${
 active ? "bg-emerald-500/15 text-emerald-500" : "text-emerald-500"
 }`}
 >

```



```

 : synced
 ? "border-emerald-500/40 bg-white"
 : "border-slate-700 bg-slate-500"
 }` }
 >

 </div>
 </div>
</div>
) : (
 <textarea
 ref={editorRef}
 value={code}
 onChange={(event) => {
 setEditHold(null);
 setCode(event.target.value);
 }}
 onKeyDown={(event) => {
 if ((event.ctrlKey || event.metaKey) && event.key === 'Enter') {
 event.preventDefault();
 void debugRun();
 }
 event.stopPropagation();
 }}
 spellCheck={false}
 aria-label="Код Python"
 className="flex-1 min-h-[90px] w-full resize-none focus:ring-2 focus:ring-inset focus:ring-emerald-500"
 placeholder="Напишите Python-код..."
 />
)
}

```

```
 </pre>
)}
 <div className="px-3 py-1.5 border-t border-sla
 </div>
 </aside>
);
}
```

---

ФАЙЛ 62/96: src/components/QueueViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useState, useCallback } from 'react';
import { Sparkles } from 'lucide-react';
import { useVizRuntime, vizArray, vizString } from '../v'
```

```
interface Customer {
 id: string;
 name: string;
 emoji: string;
 color: string;
 bubbleText: string;
 animState: 'in' | 'idle' | 'eating' | 'out';
}
```

```
const PRESETS = [
 { name: 'Студент Вася', emoji: '👨‍🎓', color: 'from-blue',
 bubbleText: 'Матана!' },
 { name: 'Кодер Семён', emoji: '👨‍💻', color: 'from-violet',
 bubbleText: 'Рацию борща!' },
 { name: 'Кот Борис', emoji: '🐈', color: 'from-amber',
 bubbleText: 'Плиз.' },
 { name: 'Бабушка Люба', emoji: '👵', color: 'from-rose',
 bubbleText: '' },
 { name: 'Инопланетянин', emoji: '👽', color: 'from-emerald',
 bubbleText: 'Пливо для нашего НЛО!' },
];
```

```

const enqueue = useCallback(() => {
 if (isServing) return;
 if (queue.length >= 6) {
 addLog('⚠ Хвост очереди уже на улице, повару тяже.);
 setShake(true);
 setTimeout(() => setShake(false), 400);
 return;
 }

 const preset = PRESETS[counter % PRESETS.length];
 counter++;

 const newGuest: Customer = {
 id: Date.now().toString(),
 ...preset,
 animState: 'in',
 };

 setQueue(prev => [...prev, newGuest]);
 addLog(`- ENQUEUE: ${newGuest.name} ${newGuest.emoj);

 setTimeout(() => {
 setQueue(prev => prev.map(c => c.id === newGuest.
 }, 450);
}, [queue, isServing]);

// DEQUEUE: Выдать суп первому (голова / Head, FIFO)
const dequeue = useCallback(() => {
 if (isServing) return;
 if (queue.length === 0) {
 setShake(true);
 addLog('⚠ DEQUEUE: В очереди никого нет! Повар ск);
 setTimeout(() => setShake(false), 400);
 return;
 }

 setIsServing(true);
 const headGuest = queue[0];

```

```

return (
 <div className="flex flex-col gap-6">
 {/* МНЕМОНИКА / ПРАВИЛО */}
 <div className="bg-indigo-950/40 border border-indigo-500">
 <div className="text-3xl"> </div>
 <div>
 <h3 className="font-black text-indigo-400 text-2xl">ПРАВИЛО</h3>
 <p className="text-xs text-slate-300 mt-1 leading-4">
 В очереди за супом всё честно. Кто первый встал, тот и ест.
 Новые гости встают строго в конец очереди.
 Здесь нет обхода и блата — только строгий порядок.
 </p>
 </div>
 </div>

 {/* УПРАВЛЕНИЕ */}
 <div className="flex items-center gap-3 flex-wrap">
 <button
 onClick={enqueue}
 disabled={isServing}
 className="flex-1 min-w-[150px] bg-gradient-to-r from-indigo-500 to-indigo-600
 text-white opacity-40 disabled:cursor-not-allowed text-center font-medium
 hover:scale-95 transition-all cursor-pointer">
 Встать в очередь (ENQUEUE)
 </button>

 <button
 onClick={dequeue}
 disabled={isServing || queue.length === 0}
 className="flex-1 min-w-[150px] bg-gradient-to-r from-indigo-500 to-indigo-600
 text-white opacity-40 disabled:cursor-not-allowed text-center font-medium
 hover:scale-95 transition-all cursor-pointer flex-shrink-0">
 Налить суп первому (DEQUEUE)
 </button>

 <button

```

```

</div>

{/* РАЗДЕЛИТЕЛЬНАЯ СТРЕЛКА */}
<div className="flex flex-col items-center" >

 </div>

{/* СПИСОК ОЧЕРЕДИ */}
<div className="flex-1 flex items-end gap-4" >
 {displayQueue.length === 0 ? (
 <div className="flex-1 flex flex-col" >
 </
 <p className="text-xs font-bold font" >
 <p className="text-[10px]" >Все сыты
 </div>
) : (
 displayQueue.map(({customer, idx}) => {
 const isHead = idx === 0;
 const isTail = idx === displayQueue.length - 1;
 return (
 <div
 key={customer.id}
 className={`
 flex flex-col items-center gap-4
 ${customer.animState === 'in' ? 'animate-in' : ''}
 ${customer.animState === 'eat' ? 'animate-eat' : ''}
 ${customer.animState === 'out' ? 'animate-out' : ''}
 `}
 >
 {/* Облачко мыслей */}
 <div className={`
 absolute -top-16 left-1/2 -tr
 8 text-center text-slate-200 leading-tight
 ${isHead ? 'opacity-100 border' : ''}
 `} >
 <div className="font-bold text" >
 {customer.bubbleText}
 </div>
 </div>
 </div>
)
)
)
}

```

```

<div className="absolute top-4 left-4 text-[10px] font-mono">
 Сытые гости (Архив FIFO)
</div>

<div className="absolute top-4 right-4 flex items-center justify-center"
 order-emerald-800/80 text-[9px] font-mono">
 <Sparkles className="w-3.5 h-3.5 animate-pulse">
 СЫТЫ
 </div>

<div className="flex-1 flex flex-col justify-between"
 {servedHistory.length === 0 ? {
 <div className="flex-1 flex flex-col item-center justify-center"

 <p className="text-xs font-bold font-mono">
 <p className="text-[10px] mt-1">Здесь 6
 </div>
) : {
 <div className="flex flex-col gap-1.5 w-full"
 {servedHistory.map((item, idx) => {
 <div
 key={idx}
 className="w-full py-1.5 px-3 rounded"
 0 flex items-center gap-2 text-[11px] text-
 >
 {item}
 </div>
)}}
 </div>
)}
 </div>

 {/* Пол */}
 <div className="w-full h-3 bg-gradient-to-r from-
</div>
</div>

{/* ЛОГ ДЕЙСТВИЙ */}

```

```

export const SalmonAutomatonWidget: React.FC = () => {
 const [words, setWords] = useState<string[]>(['CAR',
 const [newWord, setNewWord] = useState('');
 const [nodes, setNodes] = useState<{ [key: number]: T
 const [simulationStep, setSimulationStep] = useState<
 const [bfsQueue, setBfsQueue] = useState<number[]>([]
 const [currentNode, setCurrentNode] = useState<number
 const [speechText, setSpeechText] = useState<string>('
 'Привет! Я Эхо-Лосось Сеня! Введи слова в словарь и
 буль!'
);
 const [isTalking, setIsTalking] = useState<boolean>(f
 const [isBlinking, setIsBlinking] = useState<boolean>

 // Регулярное моргание рыбы
 useEffect(() => {
 const blinkInterval = setInterval(() => {
 setIsBlinking(true);
 setTimeout(() => setIsBlinking(false), 200);
 }, 4000);
 return () => clearInterval(blinkInterval);
 }, []);

 // Эффект разговора при смене текста
 useEffect(() => {
 setIsTalking(true);
 const timer = setTimeout(() => setIsTalking(false),
 return () => clearTimeout(timer);
 }, [speechText]);

 // Построение начального бора (Trie) без fail-ссылок
 const buildInitialTrie = (wordList: string[]) => {
 const newNodes: { [key: number]: TrieNode } = {
 0: { id: 0, char: 'ROOT', parent: 0, children: {}
 };
 let count = 0;

```

```

 } else {
 let leftX = -1;
 let rightX = -1;
 childKeys.forEach(v => {
 layoutDFS(v, depth + 1);
 if (leftX === -1) leftX = newNodes[v].x!;
 rightX = newNodes[v].x!;
 });
 newNodes[u].x = leftX === -1 ? 40 + currentX * 2 : rightX;
 if (leftX === -1) currentX++;
 }
 };

 layoutDFS(0, 0);

 setNodes(newNodes);
 setSimulationStep(0);
 setBfsQueue([]);
 setCurrentNode(null);
 setSpeechText(
 `Отлично! Я построил базовое префиксное дерево (БПД) для слова "fail".

 S, чтобы расставить fail-ссылки!`
);
};

useEffect(() => {
 buildInitialTrie(words);
}, []);

const handleAddWord = (e: React.FormEvent) => {
 e.preventDefault();
 const cleanWord = newWord.trim().toUpperCase().replace(' ', '');
 if (!cleanWord) return;
 if (words.includes(cleanWord)) {
 setSpeechText(`Слово «${cleanWord}» уже есть в слове "fail".`);
 return;
 }
 const updated = [...words, cleanWord];

```



```

 setCurrentNode(null);
 setSpeechText(
 'Ура! Очередь пуста! Мы успешно построили полную таблицу переходов!'
);
 return;
 }

 const r = bfsQueue[0];
 const newQueue = bfsQueue.slice(1);
 const updatedNodes = { ...nodes };
 const rNode = updatedNodes[r];
 const childrenIds = Object.values(rNode.children);

 let logs = `Рассматриваем вершину #${r} (${rNode.char})`;

 for (const [char, u] of Object.entries(rNode.children)) {
 newQueue.push(u);
 let v = rNode.fail;
 while (v !== 0 && updatedNodes[v].children[char] === undefined) {
 v = updatedNodes[v].fail;
 }
 const failTarget = updatedNodes[v].children[char];
 updatedNodes[u].fail = failTarget;

 // Рассчитываем терминальную ссылку (term_link)
 if (updatedNodes[failTarget].is_terminal) {
 updatedNodes[u].term_link = failTarget;
 } else {
 updatedNodes[u].term_link = updatedNodes[failTarget].term_link;
 }

 logs += `Для #${u} ('${char}') fail = #${failTarget}`;
 }

 if (childrenIds.length === 0) {
 logs += 'У этой вершины нет детей, просто переходим к следующему';
 }

```

```

<div className="w-40 h-40 relative flex items
 full border border-blue-500/30 shadow-inner
 {/* SVG Лосося */}
 <svg
 viewBox="0 0 200 200"
 className={`w-36 h-36 transition-transform
 isTalking ? 'scale-105 drop-shadow-[0_0_
]`}
 >
 {/* Тело лосося */}
 <ellipse cx="100" cy="100" rx="70" ry="45"
 <path d="M 40 85 Q 100 55 160 85 Q 100 115
 40 85" fill="#e67e22"
 className="origin-[35px_100px] animate-pulse"
 />

 {/* Плавники */}
 <path d="M 90 55 L 70 30 L 110 55 Z" fill="#2980b9"
 <path d="M 90 145 L 70 170 L 110 145 Z" fill="#2980b9"
 <path
 d="M 120 115 L 100 135 L 130 130 Z"
 fill="#f87171"
 className="origin-[120px_115px] animate-pulse"
 />

 {/* Глаз (с морганием) */}
 <circle cx="145" cy="85" r="12" fill="#fff"
 {isBlinking ? (
 <line x1="133" y1="85" x2="157" y2="85"
) : (
 <circle cx="148" cy="85" r="6" fill="#fff"
 <circle cx="151" cy="82" r="2" fill="#fff"
)}
 </>
 </div>

```

```

<div className="mt-4 pt-3 border-t border-slate-200">
 <div className="text-xs text-slate-400 flex justify-between">
 <Info className="w-3.5 h-3.5 text-blue-400" />
 Статус: {simulationStep === 0 ? 'Бор готов к началу!' : 'Бор готов к началу!'}
 </div>

 <div className="flex gap-2">
 {simulationStep === 0 && (
 <button
 onClick={startBFS}
 className="bg-emerald-600 hover:bg-emerald-500 text-white px-1 shadow-lg shadow-emerald-900/40 transition-colors"
 >
 <Play className="w-4 h-4" /> Запустить
 </button>
)}
 {simulationStep === 1 && (
 <button
 onClick={stepBFS}
 className="bg-blue-600 hover:bg-blue-500 text-white px-1 shadow-lg shadow-blue-900/40 transition-colors"
 >
 Шаг BFS

 </button>
)}
 {simulationStep === 2 && (
 <button
 onClick={() => buildInitialTrie(words)}
 className="bg-indigo-600 hover:bg-indigo-500 text-white px-1 shadow-lg shadow-indigo-900/40 transition-transform active:scale-95"
 >
 <RotateCcw className="w-4 h-4" /> Перезагрузить
 </button>
)}
 </div>

```

```

 return (
 <g key={`edge-${node.id}`}>
 <line
 x1={parent.x}
 y1={parent.y + 16}
 x2={node.x}
 y2={node.y - 16}
 stroke={isCurrent ? '#60a5fa' : '#ccc'}
 strokeWidth="2"
 markerEnd="url(#arrowhead)"
 />
 <text
 x={({parent.x + node.x} / 2 - 10)}
 y={({parent.y + node.y} / 2)}
 fill="#94a3b8"
 fontSize="12"
 fontWeight="bold"
 >
 {node.char}
 </text>
 </g>
);
 });
 });

 /* Draw Fail Links */
 {simulationStep > 0 && Object.values(nodes).some(
 node => {
 if (node.id === 0 || node.fail === 0) return false;
 // n draw all! But maybe wait, we can just draw fail link
 // Wait, all nodes get fail link. Let's draw it at next simula-
 // tion step >= 1 and computed it
 // To avoid too many lines, let's draw only one fail link
 const target = nodes[node.fail];
 if (!target || !node.x || !node.y || !target.x || !target.y) return false;

 // Simple bezier curve for fail link
 const isCurrent = currentNode === node.id;

```

```

 cx={node.x}
 cy={node.y}
 r="16"
 fill={fill}
 stroke={stroke}
 strokeWidth="2"
 />
 <text
 x={node.x}
 y={node.y + 4}
 fill="white"
 fontSize={isRoot ? "10" : "12"}
 fontWeight="bold"
 textAnchor="middle"
 >
 {isRoot ? 'R' : node.char}
 </text>
 {!isRoot && {
 <text
 x={node.x + 12}
 y={node.y - 12}
 fill="#94a3b8"
 fontSize="9"
 >
 {node.id}
 </text>
 }}
 </g>
);
 }}}
</svg>
);
}}({})
</div>

{/* Управление словарем */}
<div className="grid grid-cols-1 lg:grid-cols-3 g
 <div className="bg-slate-900/50 p-5 rounded-xl

```

```

 >
 <Plus className="w-3.5 h-3.5" /> Добавить
 </button>
 </form>
 </div>

 { /* Таблица Вершин и суффиксных ссылок */ }
 <div className="lg:col-span-2 bg-slate-900/50 p-4" >
 <h5 className="text-white font-bold mb-3 flex" >

 Таблица переходов и fail-

 Всего вершин: {Object.keys(nodes).length}

 </h5>

 <div className="max-h-[220px] overflow-y-auto" >
 <table className="w-full text-left border-collapse" >
 <thead>
 <tr className="border-b border-slate-700" >
 <th className="py-2 px-3" >ID Вершины</th>
 <th className="py-2 px-3" >Символ</th>
 <th className="py-2 px-3" >Родитель</th>
 <th className="py-2 px-3" >fail-ссылка</th>
 <th className="py-2 px-3" >term_link</th>
 <th className="py-2 px-3" >Дети</th>
 <th className="py-2 px-3" >Слова (dict
 </tr>
 </thead>
 <tbody className="divide-y divide-slate-800" >
 {Object.values(nodes).map((node) => {
 const isCurrent = currentNode === node
 const isInQueue = bfsQueue.includes(n

 return (
 <tr
 key={node.id}

```

```

 }}
 </td>
 <td className="py-2.5 px-3">
 {node.id === 0 || node.term_link
 <span className="text-slate-500"
) : (
 <span className="bg-rose-950 text-white transition-colors cursor-help" title={`Ск
 #{node.term_link}

 }}
 </td>
 <td className="py-2.5 px-3 text-slate-500">
 {Object.keys(node.children).length === 0 ? 'Ø' : Object.entries(node.children)
 .map(([c, id]) => `${c}→#${id}`)
 .join(', ')}
 </td>
 <td className="py-2.5 px-3">
 {node.words.length === 0 ? (

) : (

 {node.words.join(', ')}

)}
 </td>
</tr>
</tbody>
</table>
</div>
</div>
</div>
</div>
);

```

```
 return tree;
}
```

```
// ===== STEP GENERATORS =====
```

```
// 1. Build steps (recursive top-down)
```

```
function genBuildSteps(arr: number[]): Step[] {
 const n = arr.length;
 const steps: Step[] = [];
 let id = 0;
 const tree: Record<number, number> = {};
```

```
 const CODE = [
 "build(v, l, r) {", // 1
 " if (l === r) {", // 2
 " tree[v] = A[l]; return", // 3
 " }", // 4
 " mid = (l + r) >> 1", // 5
 " build(2v, l, mid)", // 6
 " build(2v+1, mid+1, r)", // 7
 " tree[v] = tree[2v]+tree[2v+1]", // 8
 "}", // 9
];
```

```
};
void CODE;
```

```
function go(v: number, l: number, r: number) {
 if (l === r) {
 steps.push({ id: id++, desc: `build(${v}, ${l}, ${r}`,
 , highlightNodes: [v], arrayHighlight: [l, r] });
 tree[v] = arr[l];
 steps.push({ id: id++, desc: `tree[${v}] = ${arr[l]}`,
 , highlightNodes: [v], arrayHighlight: [l, r] });
 return;
 }
 const m = (l + r) >> 1;
 steps.push({ id: id++, desc: `build(${v}, ${l}, ${r}`,
 { ...tree }, highlightNodes: [v], arrayHighlight: [l, r] });
 go(2 * v, l, m);
 go(2 * v + 1, m, r);
}
```



```

 rgeChildren: [2 * v, 2 * v + 1], arrayHighlight:
 return res;
}
const ans = go(1, 0, n - 1, qL, qR);
steps.push({ id: id++, desc: `Запрос sum([${qL}..${qR}], result:
 highlightNodes: [1], arrayHighlight: [qL, qR], result:
return steps;
}

```

// 3. Bottom-Up Query steps (iterative on implicit tree

```

function genQueryBottomUpSteps(arr: number[], qL: number, qR: number) {
 const n = arr.length;
 // Build iterative tree: leaves at [n..2n-1]
 const tree: Record<number, number> = {};
 for (let i = 0; i < n; i++) tree[n + i] = arr[i];
 for (let i = n - 1; i > 0; --i) tree[i] = (tree[2 * i] + tree[2 * i + 1]) / 2;

 const steps: Step[] = [];
 let id = 0;
 let l = qL + n;
 let r = qR + n + 1; // half-open [l, r)
 let res = 0;

```

```

 steps.push({ id: id++, desc: `Старт: l = qL + n = ${qL + n}, r = qR + n + 1 = ${qR + n + 1}, codeLine: 3, treeState: tree, highlightNodes: [l, r)`, codeLine: 3, treeState: tree, highlightNodes: [l, r)

```

```

while (l < r) {
 const highlights: number[] = [];
 if (l & 1) {
 res += tree[l] ?? 0;
 steps.push({ id: id++, desc: `l=${l} нечётный → l`, codeLine: 4, treeState: tree, highlightNodes: [l, l + 1), codeLine: 5, treeState: tree, highlightNodes: [l, l + 1),
 highlights.push(l);
 l++;
 }
 if (r & 1) {
 r--;
 res += tree[r] ?? 0;
 }
}

```

```
const QUERY_TD_CODE = [
 "query(v, l, r, ql, qr) {",
 " if (ql > r || qr < l) return 0;",
 " // -----",
 " if (ql <= l && r <= qr) return tree[v];",
 " // -----",
 " mid = (l + r) >> 1;",
 " // спускаемся в обоих детей",
 " return query(left) + query(right);",
 "}"
```

```
];
```

```
const QUERY_BU_CODE = [
 "queryBU(ql, qr) {",
 " res = 0;",
 " l = ql + n; r = qr + n + 1;",
 " // -----",
 " while (l < r) {",
 " if (l & 1) res += tree[l++];",
 " // -----",
 " if (r & 1) res += tree[--r];",
 " l >>= 1; r >>= 1;",
 " }",
 " return res; }"
```

```
];
```

// ===== STEP GENERATOR: массовое обновление с Lazy Prop

```
const LAZY_CODE = [
 "update(v, l, r) {", // 1
 " if (r < uL || uR < l) return", // 2
 " if (uL <= l && r <= uR) {", // 3
 " tree[v] += add * (r - l + 1)", // 4
 " lazy[v] += add // обещание", // 5
 " return", // 6
 " }", // 7
 " push(v) // старое обещание вниз", // 8
 " m = (l + r) >> 1", // 9
 " update(2v, l, m)", // 10
 " update(2v+1, m+1, r)", // 11
 " tree[v] = tree[2v]+tree[2v+1]", // 12
```

```

 tree[2 * v] = (tree[2 * v] ?? 0) + d * (m - l + 1);
 lazy[2 * v] = (lazy[2 * v] ?? 0) + d;
 tree[2 * v + 1] = (tree[2 * v + 1] ?? 0) + d * (r - m);
 lazy[2 * v + 1] = (lazy[2 * v + 1] ?? 0) + d;
 lazy[v] = 0;
 snap(`Обещание ${v} исполнено и снято (lazy[${v}])`);
 }
}

function update(v: number, l: number, r: number) {
 if (r < uL || uR < l) {
 snap(`update(v=${v}, [${l}..${r}]): отрезок не пересек`);
 return;
 }
 if (uL <= l && r <= uR) {
 tree[v] = (tree[v] ?? 0) + add * (r - l + 1);
 lazy[v] = (lazy[v] ?? 0) + add;
 snap(`update(v=${v}, [${l}..${r}]): отрезок ЦЕЛИКОМ`);
 обещание lazy[v] = lazy[v]. Детям сюда с...
 return;
 }
 pushDown(v, l, r);
 const m = (l + r) >> 1;
 snap(`update(v=${v}, [${l}..${r}]): частичное пересечение`);
 update(2 * v, l, m);
 update(2 * v + 1, m + 1, r);
 tree[v] = (tree[2 * v] ?? 0) + (tree[2 * v + 1] ?? 0);
 snap(`Возврат в v=${v}: tree[${v}] = tree[${2 * v}]`);
 });
}

update(1, 0, n - 1);
snap(`Массовое обновление [${uL}..${uR}] += ${add}
окрывает целый подотрезок.`, 13, [1]);

// --- фаза 2: запрос суммы, который "собирает" обещания
let res = 0;
function query(v: number, l: number, r: number) {

```

```

const step = steps[idx] || steps[0];
if (!step) return null;

const clr = { indigo: { btn: 'bg-indigo-600 hover:bg-indigo-500 border-indigo-500 bg-indigo-600', childRing: 'ring-indigo-900' }, emerald: { btn: 'bg-emerald-600 hover:bg-emerald-500 border-emerald-500 bg-emerald-600', childRing: 'ring-emerald-900' }, amber: { btn: 'bg-amber-600 hover:bg-amber-500 border-amber-500 bg-amber-600', childRing: 'ring-rose-500' }, [color]: { btn: 'bg-' + color + '-600 hover:bg-' + color + '-500 border-' + color + '-500 bg-' + color + '-600', childRing: 'ring-' + color + '-900' } };

return { step, idx, setIdx, playing, setPlaying, speed };
}

// ===== SIMULATION PANEL =====
function SimPanel({ title, icon, steps, code, color, arr, liveNode }) {
 const p = Player({ steps, color });
 if (!p) return null;
 const { step, idx, setIdx, playing, setPlaying, speed } = p;
 const n = arr.length;

 const vTree = useMemo(() => buildVisualTree(step.tree, n), [step, n]);

 const renderNode = useCallback((nd: VNode): React.ReactNode => {
 const isHigh = step.highlightNodes.includes(nd.v);
 const isChild = step.mergeChildren?.includes(nd.v);
 const isLive = liveNode !== null && liveNode !== undefined;
 const has = nd.val !== null;
 const promise = step.lazyState?.[nd.v] ?? 0;

 let cls = 'bg-slate-800 border-slate-700 text-slate-200';
 if (isHigh) cls = `${clr.ring} text-white ring-2 shadow-2`;
 else if (isChild) cls = `${clr.childRing} text-amber-500 ring-2 shadow-2`;
 else if (has) cls = `${clr.filled} text-slate-200`;
 if (isLive) cls += ' outline outline-2 outline-cyan';

 return (

```

```

 {arr.map((v, i) => {
 const inRange = step.arrayHighlight && i >= s
 return (
 <div key={i} className={`flex flex-col item
go' ? 'bg-indigo-950 border-indigo-500' : co
er-500'} -translate-y-0.5` : 'bg-slate-900 l
 <span className="text-[7px] text-slate-500
 <span className="text-[11px] font-bold fo
 </div>
);
 })}
</div>

{/* Tree */}
<div className="flex-1 p-2 overflow-x-auto bg-sla
 <div className="scale-[0.85] origin-top min-w-m
</div>

{/* Code */}
<pre className="px-3 py-2 text-[9px] md:text-[10px]
 x">
 {code.map((line, i) => {
 const ln = i + 1;
 const active = step.codeLine === ln;
 return (
 <div key={ln} className={`px-2 py-0.5 round
2 border-indigo-500 text-indigo-200' : colo
 ' : 'bg-amber-600/20 border-l-2 border-ambe
 <span className="inline-block w-4 text-sl
 </div>
);
 })}
</pre>

{/* Description + result */}
<div className="px-3 py-2.5 bg-slate-900/60 borde
 px]">
 <span className={`mt-0.5 inline-block w-2 h-2 r

```

```

}

// =====
// MAIN EXPORTED COMPONENT
// =====
export function SegmentTreeVisualizer() {
 const runtime = useVizRuntime();
 const liveI = vizNumber(runtime?.variables?.i);
 const liveV = vizNumber(runtime?.variables?.v);
 const liveAdd = vizNumber(runtime?.variables?.add);
 const [arr, setArr] = useState<number[]>([5, 8, 3, 12]);
 const [customInput, setCustomInput] = useState('5, 8,');
 const [qL, setQL] = useState(1);
 const [qR, setQR] = useState(4);
 const [uL, setUL] = useState(1);
 const [uR, setUR] = useState(3);
 const [uAdd, setUAdd] = useState(10);
 const [view, setView] = useState<'build' | 'queries'>('build');

 // Clamp query range when array changes
 useEffect(() => {
 setQL(prev => Math.min(prev, arr.length - 1));
 setQR(prev => Math.min(prev, arr.length - 1));
 }, [arr]);

 const handleApply = (e: React.FormEvent) => {
 e.preventDefault();
 const parsed = customInput.split(',').map(s => parseInt(s));
 if (parsed.length >= 2 && parsed.length <= 16) {
 setArr(parsed);
 }
 };

 const randomize = () => {
 const size = arr.length;
 const a = Array.from({ length: size }, () => Math.floor(Math.random() * 100));
 setArr(a);
 setCustomInput(a.join(', '));
 };
}

```

```

 ext-amber-200 ring-2 ring-amber-400/30" : "
 <span className={liveI === i ? "text-

)}}
 <span className="text-xs text-slate-500 f
 </div>
 </form>

 {/* Query range */}
 <div className="bg-slate-950 p-4 rounded-xl b
 <label className="text-[10px] font-bold upp
 <div className="flex items-center gap-2">
 <div className="flex items-center gap-1">
 <span className="text-xs text-slate-400
 <input
 type="number"
 min={0}
 max={arr.length - 1}
 value={qL}
 onChange={e => setQL(Math.min(Number(
 className="w-14 bg-slate-900 border b
 focus:border-indigo-500"
 />
 </div>
 <div className="flex items-center gap-1">
 <span className="text-xs text-slate-400
 <input
 type="number"
 min={0}
 max={arr.length - 1}
 value={qR}
 onChange={e => setQR(Math.min(Number(
 className="w-14 bg-slate-900 border b
 focus:border-indigo-500"
 />
 </div>
</div>
<div className="text-[10px] text-slate-500

```

```

 color="indigo"
 arr={arr}
 />
 })

{view === 'queries' && (
 <div className="grid grid-cols-1 xl:grid-cols-2">
 <SimPanel
 key={`qtd-${arr.join(' - ')}-${qL}-${qR}`}
 title={`Запрос sum([${qL}..${qR}]) – Сверху`}
 icon="↑"
 steps={queryTDSteps}
 code={QUERY_TD_CODE}
 color="emerald"
 arr={arr}
 />
 <SimPanel
 key={`qbu-${arr.join(' - ')}-${qL}-${qR}`}
 title={`Запрос sum([${qL}..${qR}]) – Снизу`}
 icon="↓"
 steps={queryBUSteps}
 code={QUERY_BU_CODE}
 color="amber"
 arr={arr}
 />
 </div>
)}

{view === 'lazy' && (
 <div className="space-y-4">
 <div className="bg-slate-950 p-4 rounded-xl border">
 <div className="flex items-center gap-2">
 <label className="text-[10px] uppercase">
 <input type="number" min={0} max={arr.length}
 value={e.target.value} />
 ...
 <input type="number" min={0} max={arr.length}

```



```

к получает <code className="bg-slate-900 px-4 py-4 rounded font-mono text-emerald-500">
- slate-900 px-1 rounded font-mono text-emerald-500
екая часть забирает на 1 больше (округление)
</p>
<div className="bg-slate-900 p-4 rounded-lg">
 <div className="text-slate-400">Корень: L
 <div className="text-slate-300">mid = l(0
 <div className="text-indigo-300">→ Левый:
 <div className="text-emerald-300">→ Правый
h - l) >> l)} эл.)</div>
 <div className="text-slate-500 mt-2">Всего
} внутренних.</div>
</div>
</div>

{/* Comparison cards */}
<div className="grid grid-cols-1 xl:grid-cols-2">
 <div className="bg-slate-950 p-5 rounded-xl">
 <div className="absolute -top-3 left-4 bg-emerald-500 border border-emerald-500 shadow-md">
 Запрос Сверху-вниз (Рекурсия)
 </div>
 <p className="text-xs text-slate-300 mb-3">
 Начинаем с корня. Если отрезок узла полн
 наче спускаемся в обоих детей.
 </p>
 <div className="space-y-1.5 text-xs">
 <div className="flex justify-between border">
 0(
 <div className="flex justify-between border">

 <div className="flex justify-between">
 ✓ легко</div>
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
 <div className="bg-slate-950 p-5 rounded-xl">
 <div className="absolute -top-3 left-4 bg-amber-500 border border-amber-500 shadow-md">

```

```

import React, { useMemo, useState } from "react";
import { ChevronRight, Compass, Search, Sparkles, X } from "react-icons";
import { chapters } from "../data/content";
import { VIZ_REGISTRY } from "../vizRegistry";

interface SidebarProps {
 selectedChapterId: string;
 setSelectedChapterId: (id: string) => void;
 /** Мобильный режим: список живёт в выдвижной панели */
 onNavigate?: () => void;
}

export const Sidebar: React.FC<SidebarProps> = ({ selectedChapterId, setSelectedChapterId, onNavigate }) => {
 const [query, setQuery] = useState("");

 const groups = useMemo(() => {
 const q = query.trim().toLowerCase();
 const filtered = q ? chapters.filter((c) => c.title.toLowerCase().includes(q)) : chapters;
 const map = new Map<string, typeof chapters>();
 for (const c of filtered) {
 const key = c.category?.trim() || "Прочие темы";
 if (!map.has(key)) map.set(key, []);
 (map.get(key) as typeof chapters).push(c);
 }
 return Array.from(map.entries());
 }, [query]);

 return (
 <aside className="w-full bg-slate-900/80 lg:bg-transparent lg:rounded-r-2xl">
 <div className="p-4 pb-3 shrink-0">
 <h3 className="text-xs font-bold text-slate-400">
 <Compass className="w-4 h-4 text-indigo-400" />
 </h3>
 <div className="relative">
 <Search className="w-4 h-4 text-slate-500 absolute top-0 right-0" />
 <input
 type="text"
 value={query}
 onChange={(e) => setQuery(e.target.value)}
 />
 </div>
 </div>
 <div>
 {groups.map(([category, chapters]) => (
 <div key={category} className="text-slate-400">
 <div>{category}</div>
 <div>
 {chapters.map((c) => (
 <div key={c.id} className="text-slate-500">
 <div>{c.title}</div>
 <div>{c.description}</div>
 </div>
))}
 </div>
 </div>
))}
 </div>
 </aside>
);
};

```

```

 }` }
 >
 {hasViz && {
 <span
 className="w-1.5 h-1.5 rounded-
 title="Есть интерактивная визуа
 />
 }}
 <span className="font-semibold text
 <ChevronRight
 className={`w-4 h-4 shrink-0 mt-0
 isSelected ? "text-indigo-400"
 }` }
 />
 </button>
);
 }}}
</div>
</div>
)})
</div>

<div className="shrink-0 m-3 mt-0 bg-gradient-to-
] text-slate-300 space-y-1.5 hidden lg:block">
 <div className="font-bold text-indigo-400 flex
 <Sparkles className="w-3.5 h-3.5" /> Как поль
 </div>
 <p className="leading-relaxed">
 Подчёркнутые термины в тексте раскрываются по
 симулятора.
 </p>
 <p className="text-slate-400 italic">Стрелки ←
 </div>
</aside>
);
};

```

```

 {[
 { id: "cases" as Tab, label: "1 · Механика: Z" },
 { id: "big" as Tab, label: "2 · Большое дерево" },
 { id: "play" as Tab, label: "3 · Собери сам" },
]}.map((t) => {
 <button
 key={t.id}
 onClick={() => setTab(t.id)}
 className={`px-4 py-2 rounded-lg text-xs font-medium
 ${tab === t.id ? "bg-indigo-600 text-white" : ""}
 `}
 >
 {t.label}
 </button>
 })
 </div>

 {tab === "cases" && <SplayRotationsViz />}
 {tab === "big" && <SplayWalkViz />}
 {tab === "play" && <SplayRotationSandbox />}

 {tab === "play" && {
 <p className="text-xs text-slate-500 mt-3">
 Порядок поворотов в песочнице сверяется с правым
 </p>
 }}
</div>
);
};

export default SplayPlayground;

```

```

 hintCase: "Zig-Zag",
 target: 3,
 build: () => {
 const n3 = leaf(3);
 const n2: TNode = { key: 2, left: leaf(1), right:
 const n4: TNode = { key: 4, left: n2, right: leaf
 return { key: 6, left: n4, right: leaf(7) };
 },
},
{
 name: "Один уровень",
 hintCase: "Zig",
 target: 2,
 build: () => ({ key: 4, left: { key: 2, left: leaf(
},
{
 name: "Смешанный",
 hintCase: "Zig-Zag, затем Zig-Zig",
 target: 5,
 build: () => {
 const n5 = leaf(5);
 const n6: TNode = { key: 6, left: n5, right: leaf
 const n4: TNode = { key: 4, left: leaf(3), right:
 return { key: 8, left: n4, right: leaf(9) };
 },
},
},
];

/* ----- операции над деревом -----

export const findPath = (root: TNode | null, key: number
const path: TNode[] = [];
let cur = root;
while (cur) {
 path.push(cur);
 if (key === cur.key) return path;
 cur = key < cur.key ? cur.left : cur.right;
}

```

```

 if (xLeft === pLeft) return { node: p.key, kind: "Zig"
 return { node: x.key, kind: "Zig-Zag (сначала нижняя
}

```

```

/* ----- раскладка -----

```

```

interface Placed {
 key: number;
 x: number;
 y: number;
 depth: number;
}

```

```

function layout(root: TNode | null): { nodes: Placed[];
 const nodes: Placed[] = [];
 const edges: [number, number][] = [];
 let order = 0;

```

```

 const walk = (n: TNode | null, depth: number) => {
 if (!n) return;
 walk(n.left, depth + 1);
 nodes.push({ key: n.key, x: order++, y: depth, depth: depth });
 if (n.left) edges.push([n.key, n.left.key]);
 if (n.right) edges.push([n.key, n.right.key]);
 walk(n.right, depth + 1);
 };
 walk(root, 0);

```

```

 const cols = Math.max(1, order);
 const rows = Math.max(1, Math.max(...nodes.map((n) => n.depth)));
 const colW = 46;
 const rowH = 62;
 return {
 nodes: nodes.map((n) => ({ ...n, x: n.x * colW + colW / 2, y: n.y * rowH + rowH / 2 })),
 edges,
 width: cols * colW,
 height: rows * rowH + 20,
 };
}

```

```

 setMoves((m) => [...m, { node: key, correct: want?.correct }]);
 setTree((t) => rotateUp(clone(t) as TNode, key));
 });

```

```

const undo = () => {
 if (history.length === 0) return;
 setTree(history[history.length - 1]);
 setHistory((h) => h.slice(0, -1));
 setMoves((m) => m.slice(0, -1));
};

```

```

const wrong = moves.filter((m) => !m.correct).length;
const minimal = useMemo(() => findPath(preset.build()))

```

```

return (
 <div className="w-full bg-slate-950 rounded-2xl border" >
 <div className="flex items-start justify-between" >
 <div>
 <h4 className="text-sm sm:text-base font-bold" >
 <p className="text-[12px] text-slate-400 leading-4" >
 Клик по узлу = один поворот с его родителем

 поворотов выбираешь сам, разбор покажем в к
 </p>
 </div>
 <button
 onClick={() => reset((presetIdx + 1) % PRESETS.length)}
 className="flex items-center gap-1.5 px-3 py-1 text-sm font-medium text-slate-400 transition-colors shrink-0"
 >
 <Dices className="w-3.5 h-3.5" /> Другое дерево
 </button>
 </div>

```

```

 <div className="flex gap-2 mb-3 overflow-x-auto" >
 {PRESETS.map((p, i) => (
 <button
 key={p.name}

```

```

 const clickable = !solved && n.depth > 0;
 return (
 <g
 key={n.key}
 onClick={() => clickable && clickNode(n)}
 style={{
 transform: `translate(${n.x}px, ${n.y}px)`,
 transition: "transform 700ms cubic-bezier(0.4, 0, 0.2, 1)",
 cursor: clickable ? "pointer" : "default",
 }}
 >
 {isTarget && (
 <circle r={22} fill="none" stroke="#6666ff" stroke-width={2} />
)}
 <circle
 r={16}
 fill={isTarget ? "#6666ff" : path.has(n.key) ? "#a5b4fc" : "#475568"}
 stroke={isTarget ? "#a5b4fc" : "#475568"}
 stroke-width={2}
 />
 <text textAnchor="middle" dominantBaseline="middle">
 {n.key}
 </text>
 </g>
);
 }
 }
 </svg>
</div>

<div className="flex flex-wrap items-center gap-2">
 <button
 onClick={undo}
 disabled={history.length === 0}
 className="flex items-center gap-1.5 px-3 py-1.5 text-white disabled:opacity-40 text-xs font-bold"
 >
 <Undo2 className="w-3.5 h-3.5" /> ОТМЕНИТЬ
 </button>

```



```

 >
 <div className="flex items-center gap-2 font-l
 {wrong === 0 ? <CheckCircle2 className="w-4
 {wrong === 0
 ? `Узел ${target} в корне за ${moves.leng
 : `Узел ${target} в корне за ${moves.leng
 </div>
 {wrong > 0 && (
 <ul className="list-disc pl-5 space-y-0.5">
 {moves.map(
 (m, i) =>
 !m.correct && (
 <li key={i}>
 ход {i + 1}: крутили <span classN
 {m.expected}

)
)
 }

)}
 {wrong === 0 && moves.length > minimal && (
 <div className="opacity-80">Порядок верный,
)}
 </div>
)}
</div>
);
};

```

---

ФАЙЛ 68/96: src/components/SplayRotationsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

---

```

import React, { useEffect, useMemo, useState } from "re
import { useVizRuntime, vizNumber, vizString } from "...
import { Gauge, Pause, Play, RotateCcw } from "lucide-r

```

```

keys: Partial<Record<NodeId, number>>;
/** Обход слева направо – одинаковый на всех кадрах.
inorder: string[];
/** Что означают поддеревья A..D. */
ranges: string;
frames: Frame[];
}

const VB = { w: 360, h: 285 };
const AIM_MS = 2600;
const RESULT_MS = 3400;
const MOVE = "transform 1200ms cubic-bezier(.34,.01,.2,

/* ----- Zig -----
const ZIG_BEFORE = {
 pos: { p: [195, 50], x: [105, 130], C: [280, 130], A:
 edges: [["p", "x"], ["p", "C"], ["x", "A"], ["x", "B"]
} as unknown as Pick<Frame, "pos" | "edges">;

const ZIG: Case = {
 key: "zig",
 label: "Zig",
 when: "родитель x – уже корень, деда нет",
 rotations: "1 поворот",
 keys: { x: 20, p: 40 },
 inorder: ["A", "20", "B", "40", "C"],
 ranges: "A < 20 · B между 20 и 40 · C > 40",
 frames: [
 {
 ...ZIG_BEFORE,
 kind: "start",
 title: "Было: 20 висит под корнем 40",
 text: "Нам нужен ключ 20, а он на глубине 1. Деду
 ms: RESULT_MS,
 },
 {
 ...ZIG_BEFORE,
 kind: "aim",

```

```

{
 ...ZZ_S0,
 kind: "start",
 title: "Было: прямая линия 60 → 40 → 20",
 text: "Три узла выстроились в «бамбук»: каждый сл
 зглаживает.",
 ms: RESULT_MS,
},
{
 ...ZZ_S0,
 kind: "aim",
 title: "Прицел 1: ВЕРХНЕЕ ребро 60 – 40",
 text: "Главная хитрость этого случая: первым крути
 focus: ["g", "p"],
 ms: AIM_MS,
},
{
 ...ZZ_S1,
 kind: "result",
 title: "Поворот 1: 40 поднялось над 60",
 text: "40 встало в корень, 60 опустилось к нему п
 убине 1, а не 2.",
 moved: ["p", "g", "C"],
 ms: RESULT_MS,
},
{
 ...ZZ_S1,
 kind: "aim",
 title: "Прицел 2: ребро 40 – 20",
 text: "А теперь обычный одиночный поворот – тот ж
 focus: ["p", "x"],
 ms: AIM_MS,
},
{
 pos: { x: [100, 45], A: [45, 118], p: [188, 118],
 edges: [["x", "A"], ["x", "p"], ["p", "B"], ["p",
 kind: "result",
 title: "Стало: 20 в корне, бамбук стал кустом",

```

```

 kind: "aim",
 title: "Прицел 1: НИЖНЕЕ ребро 20 – 40",
 text: "Дед (60) стоит на месте и в этом повороте",
 focus: ["p", "x"],
 ms: AIM_MS,
 },
 {
 ...ZG_S1,
 kind: "result",
 title: "Поворот 1: змейка выпрямилась",
 text: "40 заняло место 20 и забрало его к себе ле",
 moved: ["x", "p", "C"],
 ms: RESULT_MS,
 },
 {
 ...ZG_S1,
 kind: "aim",
 title: "Прицел 2: ребро 60 – 40",
 text: "Остался одиночный поворот вокруг деда – и",
 focus: ["g", "x"],
 ms: AIM_MS,
 },
 {
 pos: { x: [180, 50], p: [90, 130], g: [275, 130],
 edges: [["x", "p"], ["x", "g"], ["p", "A"], ["p",
 kind: "result",
 title: "Стало: 20 и 60 – два сына сорока",
 text: "Дерево стало идеально симметричным: все че
 очку». ",
 moved: ["x", "p", "g", "C"],
 ms: RESULT_MS,
 },
],
};

```

```
const CASES: Case[] = [ZIG, ZIGZIG, ZIGZAG];
```

```
const COLOR: Record<string, { fill: string; stroke: str
```

```

<circle cx={from[0]} cy={from[1]} r={r} fill="white"
{len > pad * 2 + 6 && {
 <line
 x1={from[0] + (dx / len) * pad}
 y1={from[1] + (dy / len) * pad}
 x2={to[0] - (dx / len) * pad}
 y2={to[1] - (dy / len) * pad}
 stroke="#a5b4fc"
 strokeWidth={1.5}
 strokeDasharray="4 3"
 markerEnd="url(#splay-arrow)"
 />
}}
</g>
);
}})

```

```

{edges.map(([a, b], i) => {
 const pa = pos[a];
 const pb = pos[b];
 if (!pa || !pb) return null;
 const hot = Boolean(focus && focus.includes(a));
 return (
 <line
 key={` ${a} - ${b} - ${i} `}
 x1={pa[0]}
 y1={pa[1]}
 x2={pb[0]}
 y2={pb[1]}
 stroke={hot ? "#fbbf24" : "#475569"}
 strokeWidth={hot ? 5 : 2}
 opacity={focus && !hot ? 0.25 : 1}
 style={{ transition: "all 1200ms cubic-bezier(0.4, 0, 0.2, 1)" }}
 />
);
})}

```

```

{{Object.keys(pos) as NodeId[]}.map((id) => {

```

```

export const SplayRotationsViz: React.FC = () => {
 const runtime = useVizRuntime();
 const vars = runtime?.variables;
 const liveCase = vizString(vars?.case);
 const liveFrame = vizNumber(vars?.i);
 const [caseIdx, setCaseIdx] = useState(0);
 const [frame, setFrame] = useState(0);
 const [playing, setPlaying] = useState(false);
 const [slow, setSlow] = useState(false);

 // Компилятор: case = "zig" / "zig-zig" / "zig-zag",
 useEffect(() => {
 if (liveCase) {
 const ci = CASES.findIndex((c) => c.key.toLowerCase === liveCase);
 if (ci >= 0) setCaseIdx(ci);
 }
 }, [liveCase]);
 useEffect(() => {
 if (liveFrame !== null) setFrame(Math.min(liveFrame, CASES[caseIdx].frames.length));
 }, [liveFrame, caseIdx]);

 const active = CASES[caseIdx];
 const total = active.frames.length;
 const idx = Math.min(frame, total - 1);
 const current = active.frames[idx];
 const prevPos = active.frames[Math.max(idx - 1, 0)].pos;
 const duration = useMemo(() => Math.round(current.ms - prevPos.ms));
 const last = idx === total - 1;

 useEffect(() => {
 setFrame(0);
 }, [caseIdx]);

 useEffect(() => {
 if (!playing) return;
 const t = setTimeout(() => setFrame((f) => (f + 1) % total), duration);
 return () => clearTimeout(t);
 }, [playing, frame, duration, total]);
}

```

```


 </div>
 <p className="text-[13px] text-slate-400 leading-6">
 </div>

 <div className="bg-slate-900 rounded-xl border border-slate-800">
 <Tree frame={current} prev={prevPos} keys={activeKeys}>

 {/* обход слева направо – одинаковый на всех кадрах */}
 <div className="mt-2 pt-2 border-t border-slate-800">
 <div className="flex items-center gap-1.5 flex-wrap">
 Обход
 {active.inorder.map((t, i) => (
 <span
 key={i}
 className={`px-1.5 py-0.5 rounded font-mono text-sm
 /\d/.test(t) ? "bg-emerald-500/15 text-emerald-400" : ""
 }`}
 >
 {t}

))}
 Всего: 1
 </div>
 <p className="text-[11px] text-slate-500 mt-1">
 </div>
 </div>

 {/* индикатор автопрокрутки */}
 <div className="h-1 w-full bg-slate-800 rounded-full">
 <div
 key={` ${caseIdx} - ${frame} - ${playing} - ${slow} `}
 className="h-full bg-indigo-500"
 style={{
 playing
 ? { animation: `demoProgress ${duration}ms linear 0s 1` }
 : { width: "100%", background: "#334155" }
 }}
 </div>
 </div>
 </div>
 </div>
</div>

```

```

 ? "bg-emerald-600/20 border-emerald-500 text-slate-500"
 : "bg-slate-800 border-slate-700 text-slate-500"
)`}
 >
 <Gauge className="w-3.5 h-3.5" /> {slow ? "0.5" : "1"}
 </button>
 <button
 onClick={() => {
 setFrame(0);
 setPlaying(false);
 }}
 className="flex items-center gap-1.5 px-3 py-3 rounded-full
 ext-xs font-bold transition-colors"
 >
 <RotateCcw className="w-3.5 h-3.5" /> Сброс
 </button>

 <div className="flex items-center gap-3 ml-auto" >
 {[["x", "p", "g"] as const].map((id) =>
 active.keys[id] === undefined ? null : (

 <span className="w-3 h-3 rounded-full" style={
 {background-color: active[id]}
 } />

)
)}
 </div>
</div>

<p className="mt-2 text-[11px] text-slate-500">
 Листай кнопкой «Дальше» в своём темпе – кадры не пропустят
</p>
</div>
);
};

```



```

 }
 return n;
}

export function bstInsert(root: SN | null, key: number) {
 if (!root) return { key, left: null, right: null, parent: null };
 const r = cloneWithParents(root)!;
 let cur: SN = r;
 for (;;) {
 if (key < cur.key) {
 if (!cur.left) {
 cur.left = { key, left: null, right: null, parent: cur };
 return r;
 }
 cur = cur.left;
 } else {
 if (!cur.right) {
 cur.right = { key, left: null, right: null, parent: cur };
 return r;
 }
 cur = cur.right;
 }
 }
}

```

/\*\* Поднять child над его родителем (один атомарный поворот)

```

export function rotateUp(child: SN): void {
 const p = child.parent;
 if (!p) return;
 const g = p.parent;
 if (p.left === child) {
 p.left = child.right;
 if (child.right) child.right.parent = p;
 child.right = p;
 p.parent = child;
 } else {
 p.right = child.left;
 if (child.left) child.left.parent = p;
 }
}

```

```

 root: SN | null;
 aimEdge?: [number, number];
 /** Кадр cut: ребро, по которому отстёгивается среднее */
 cutEdge?: [number, number];
 cutLabel?: string;
 /** Куда β переедет: сторона у верхнего узла. */
 cutDest?: string;
 pathKeys?: number[];
 caseLabel?: string;
 note: string;
 rotCount: number;
}

/** Полный пошаговый прогон splay(key): спуск → (прицел
export function splaySteps(
 rootIn: SN | null,
 key: number,
 isInsert = false,
): { steps: SplayStep[]; found: boolean } {
 let root: SN | null = cloneWithParents(rootIn);
 const steps: SplayStep[] = [];
 if (!root)
 return { steps: [{ kind: "done", root: null, note:

// --- спуск ---
let cur: SN | null = root;
const pathKeys: number[] = [];
let found = false;
let last: SN = root;
while (cur) {
 last = cur;
 pathKeys.push(cur.key);
 if (key === cur.key) {
 found = true;
 break;
 }
 cur = key < cur.key ? cur.left : cur.right;
}

```

```

 note: mid
 ? `Отстёгиваем среднее поддереву $\beta = \{\text{mid.key}\}$
 и у них общий — после поворота переедет $\{\text{p.l}\}$
 : `Среднего поддерева нет (β пусто): $\{\text{child.key}\}$
 rotCount,
 });
 const before = collectEdges(root);
 rotateUp(child);
 fixRoot();
 rotCount += 1;
 const diff = diffEdges(before, collectEdges(root));
 steps.push({
 kind: "rot",
 root: cloneWithParents(root)!,
 caseLabel: label,
 note: `Поворот: $\{\text{child.key}\}$ встал над $\{\text{p.key}\}$.
 rotCount,
 });
};

while (target.parent) {
 const p = target.parent;
 const g = p.parent;
 if (!g) {
 emitRotation(
 target,
 "Zig",
 [p.key, target.key],
 `Zig: у $\{\text{target.key}\}$ нет деда — хватит одного
);
 continue;
 }
 const pIsLeft = g.left === p;
 const xIsLeft = p.left === target;
 if (pIsLeft === xIsLeft) {
 const side = pIsLeft ? "оба слева" : "оба справа"
 emitRotation(
 p,

```

```

 [target.parent.key, target.key],
 `Доводим: ${target.key} над ${target.parent.key}
);
 }
}

steps.push({
 kind: "done",
 root: cloneWithParents(root)!,
 note: ` ${target.key} в корне за ${rotCount} ${rotCount}
 rotCount,
});
return { steps, found };
}

/* ----- служебное -----

const depthOf = (t: SN | null, key: number): number =>
 let d = 0;
 let cur = t;
 while (cur) {
 if (key === cur.key) return d;
 cur = key < cur.key ? cur.left : cur.right;
 d += 1;
 }
 return -1;
};

const heightOf = (t: SN | null): number => (t ? 1 + Math.max(
const inorderKeys = (t: SN | null): number[] => (t ? [...
 ...

export const INITIAL_KEYS = [50, 40, 35, 32, 30, 28, 12, 10, 8, 7, 6, 5, 4, 3, 2, 1];

const buildInitial = (): SN => {
 let t: SN | null = null;
 for (const k of INITIAL_KEYS) t = bstInsert(t, k);
 return t!;
};

```

```

<circle cx="30" cy="86" r="15" fill="#78350f"
<text x="30" y="90" textAnchor="middle" fill=
<circle cx="12" cy="126" r="13" fill="#0f172a"
<text x="12" y="130" textAnchor="middle" fill=
<circle
 cx="48" cy="126" r="13"
 fill={isCut ? "#78350f" : "#0f172a"}
 stroke={isCut ? "#f59e0b" : "#64748b"}
 strokeWidth={isCut ? 2.5 : 1.5}
 strokeDasharray={isCut ? "4,3" : "none"}
/>
<text x="48" y="130" textAnchor="middle" fill=
<circle cx="80" cy="126" r="13" fill="#0f172a"
<text x="80" y="130" textAnchor="middle" fill=
<text x="98" y="92" fill={isCut ? "#f59e0b" :
 {isCut ? " β отстёрнута" : "β при x"}}
</text>
<text x="16" y="160" fill="#64748b" fontSize=
</g>
<text x="163" y="96" fill="#10b981" fontSize="20"
<g transform="translate(186,6)">
 <text y="10" fill="#94a3b8" fontSize="11" font=
 <line x1="70" y1="46" x2="45" y2="86" stroke=
 <line x1="70" y1="46" x2="95" y2="86" stroke=
 <line x1="45" y1="86" x2="27" y2="126" stroke=
 <line
 x1="45" y1="86" x2="63" y2="126"
 stroke={isRot ? "#10b981" : "#64748b"}
 strokeWidth={isRot ? 3 : 2}
 />
 <circle cx="70" cy="46" r="15" fill="#059669"
 <text x="70" y="50" textAnchor="middle" fill=
 <circle cx="45" cy="86" r="15" fill="#1e293b"
 <text x="45" y="90" textAnchor="middle" fill=
 <circle cx="27" cy="126" r="13" fill="#0f172a"
 <text x="27" y="130" textAnchor="middle" fill=
 <circle
 cx="63" cy="126" r="13"

```

```

 return pos;
};

const SplayWalkViz: React.FC = () => {
 const initial = useMemo(buildInitial, []);
 const [steps, setSteps] = useState<SplayStep[] | null>();
 const [idx, setIdx] = useState(0);
 const [playing, setPlaying] = useState(false);
 const [input, setInput] = useState("");
 const [treeVers, setTreeVers] = useState(0);
 const baseRef = useRef<SN>(initial);
 const timer = useRef<ReturnType<typeof setTimeout> | null>();

 // — Синхронизация с Python-компилятором: key (или x)
 // i — перейти к кадру расшейвливания.
 const runtime = useVizRuntime();
 const liveKey = vizNumber(runtime?.variables?.key) ?? 0;
 const liveIdx = vizNumber(runtime?.variables?.i);
 const appliedKey = useRef<number | null>(null);
 useEffect(() => {
 if (liveKey !== null && appliedKey.current !== liveKey) {
 appliedKey.current = liveKey;
 runKeyRef.current?.(liveKey);
 }
 }, [liveKey]);
 useEffect(() => {
 if (liveIdx !== null && steps) setIdx(Math.min(liveIdx, steps.length - 1));
 }, [liveIdx, steps]);

 const runKey = (key: number, isInsert = false) => {
 const { steps: st } = splaySteps(baseRef.current, key, isInsert);
 setSteps(st);
 setIdx(0);
 setPlaying(true);
 };
 const runKeyRef = useRef(runKey);
 runKeyRef.current = runKey;

```

```
// переехавшие рёбра: разница с предыдущим кадром
const prevEdgeSet = useMemo(() => {
 if (!steps || !step || idx <= 0) return null;
 return new Set(collectEdges(steps[idx - 1].root));
}, [steps, step, idx]);
const movedCount = useMemo(() => {
 if (!prevEdgeSet) return 0;
 return edges.filter((e) => !prevEdgeSet.has(`${e.a}`))
 .length;
}, [prevEdgeSet, edges]);

const ino0 = useMemo(() => (steps ? inorderKeys(steps) : 0));
const inoNow = inorderKeys(viewRoot).join(",");
const checkOk = !steps || ino0 === inoNow;

const doInsert = () => {
 const v = parseInt(input, 10);
 if (Number.isNaN(v)) return;
 const withNew = bstInsert(baseRef.current, v);
 baseRef.current = withNew;
 setTreeVers((t) => t + 1);
 runKey(v, true);
 setInput("");
};

const stepColor = (k: number): string => {
 if (!step) return "#0f172a";
 if (step.kind === "search" && step.pathKeys?.includes(k)) return "#0f172a";
 if ((step.kind === "aim" || step.kind === "cut") &&
 const e = step.aimEdge ?? step.cutEdge!;
 if (e[0] === k) return "#1e293b";
 if (e[1] === k) return "#78350f";
)
 if (step.kind === "rot" && step.pathKeys?.includes(k)) return "#0f172a";
 return "#0f172a";
};

const strokeColor = (k: number): string => {
 if (!step) return "#64748b";
 if (step.kind === "search" && step.pathKeys?.includes(k)) return "#0f172a";
 if ((step.kind === "aim" || step.kind === "cut") &&
 const e = step.aimEdge ?? step.cutEdge!;
 if (e[0] === k) return "#1e293b";
 if (e[1] === k) return "#78350f";
)
 if (step.kind === "rot" && step.pathKeys?.includes(k)) return "#0f172a";
 return "#0f172a";
};
```

```

 className="w-28 bg-slate-950 border border-
s:border-indigo-500"
 />
 <button
 onClick={() => {
 baseRef.current = buildInitial();
 setSteps(null);
 setIdx(0);
 setPlaying(false);
 setTreeVers((v) => v + 1);
 }}
 className="px-3 py-2 rounded-lg text-xs fon
 >
 Исходное
 </button>
 <button
 onClick={() => {
 const shuffled = [...INITIAL_KEYS].sort((
 let r: SN | null = null;
 for (const k of shuffled) r = bstInsert(r
 baseRef.current = r!;
 setSteps(null);
 setIdx(0);
 setPlaying(false);
 setTreeVers((v) => v + 1);
 }}
 className="px-3 py-2 rounded-lg text-xs fon
 >
 Перемешать
 </button>
 </div>
</div>

{/* Плеер */}
{steps && (
 <div className="flex flex-wrap items-center gap
 <button onClick={() => { setIdx(0); setPlayin
 ld bg-slate-700 text-white disabled:opacity

```



```

 if (!a || !b) return null;
 const isAim = step?.aimEdge && step.aimEdge
 const isCutEdge =
 step?.kind === "cut" && step.cutEdge && step
 .cutEdge[1];
 const isMoved = !isAim && !isCutEdge && pre
 return (
 <g key={`e-${e.a}-${e.b}`}>
 <line
 x1={a.px} y1={a.py} x2={b.px} y2={b.py}
 stroke={isAim ? "#f59e0b" : isCutEdge ? "#f00"}
 strokeWidth={isAim ? 4 : isCutEdge ? 2 : 1}
 strokeDasharray={isAim ? "6,4" : isCutEdge ? "4,4" : ""}
 className="transition-all duration-500"
 />
 {isCutEdge && (
 <text x={(a.px + b.px) / 2 + 14} y={(a.py + b.py) / 2}
 nospace" className="transition-all duration-500"
 {step?.cutLabel} → {step?.cutDest}
 </text>
)}
 </g>
);
 })}
}
{nodes.map((n) => {
 const aimE = step?.aimEdge ?? (step?.kind === "aim" ? step.aimEdge : null);
 const isAimNode = aimE?.includes(n.key);
 return (
 <g
 key={`n-${n.key}-${treeVers}`}
 transform={`translate(${n.px}, ${n.py})`}
 onClick={() => {
 if (playing) return;
 const has = inorderKeys(baseRef.current).includes(n.key);
 if (has) runKey(n.key);
 }}
 className="cursor-pointer"
 >

```

```

 {movedCount > 0 ? ` ` · переехало рёбер: `
 </p>
 })
 </div>
 <div className="w-full lg:w-[300px] shrink-0">
 <MechanismInset step={step} />
 </div>
</div>
</div>

{/* Счётчики + код */}
<div className="flex flex-col lg:flex-row gap-6 m-6">
 <div className="w-full lg:w-1/2 bg-rose-950/10">
 <h4 className="text-sm font-bold text-rose-300">Шаги
 {steps ? (
 <table className="w-full text-sm">
 <tbody className="font-mono">
 <tr className="border-b border-slate-800">
 <td className="py-2 text-slate-400">г.
 <td className="py-2 text-right text-slate-400">
 </tr>
 <tr className="border-b border-slate-800">
 <td className="py-2 text-slate-400">в.
 <td className="py-2 text-right text-slate-400">
 </tr>
 <tr>
 <td className="py-2 text-slate-400">п.
 <td className="py-2 text-right text-slate-400">
 </tr>
 </tbody>
 </table>
) : (
 <p className="text-xs text-slate-400">Записи
)}
 <p className="text-xs text-slate-500 mt-3">Одн
 щений дешевлеет.</p>
 </div>
 <div className="w-full lg:w-1/2 bg-slate-900/60">

```



```
// POP: Помыть верхнюю тарелку (LIFO)
const popPlate = useCallback(() => {
 if (isWashing) return;
 if (dirtyStack.length === 0) {
 setShake(true);
 addLog('⚠ POP: Нечего мыть! Все тарелки чистые.')
 setTimeout(() => setShake(false), 400);
 return;
 }

 setIsWashing(true);
 const topPlate = dirtyStack[dirtyStack.length - 1];

 addLog(`  POP: Начинаем мыть верхнюю тарелку с ${topPlate.emoji}`);

 // Step 1: Start washing animation (sponge and soap)
 setDirtyStack(prev => prev.map(p => p.id === topPlate.id ? { ...p, isDirty: true } : p));
 setShowSoap(true);

 setTimeout(() => {
 // Step 2: Wash complete. Plate becomes clean and goes back to the rack
 const cleanPlate: Plate = {
 ...topPlate,
 isDirty: false,
 animState: 'clean',
 };

 setCleanRack(prev => [cleanPlate, ...prev].slice(0, cleanRack.length));
 setDirtyStack(prev => prev.slice(0, prev.length - 1));
 setShowSoap(false);
 setIsWashing(false);
 addLog(`  Готово! Чистая тарелка ${topPlate.emoji}`);
 }, 850);
}, [dirtyStack, isWashing]);

const reset = () => {
 counter = 0;
```

```

 ale-95 transition-all cursor-pointer flex i
 >
 Положить грязную (PUSH)
</button>

<button
 onClick={popPlate}
 disabled={isWashing || dirtyStack.length === 0}
 className="flex-1 min-w-[150px] bg-gradient-to-r from-slate-800 to-slate-900
 opacity-40 disabled:cursor-not-allowed text-white font-mono font-weight-normal
 ve:scale-95 transition-all cursor-pointer flex items-center justify-center"
 >
 Помыть верхнюю (POP)
</button>

<button
 onClick={reset}
 className="px-5 py-3.5 rounded-2xl bg-slate-800 text-white font-mono font-weight-normal
 cursor-pointer border border-slate-700"
 >
 Сброс
</button>
</div>

{/* ИНТЕРАКТИВНАЯ КУХНЯ */}
<div className="grid grid-cols-1 md:grid-cols-12 gap-4">

 {/* ЛЕВАЯ КОЛОНКА: СТОПКА ГРЯЗНЫХ ТАРЕЛОК */}
 <div className="md:col-span-7 bg-slate-900 rounded-3xl p-4 min-h-[380px]">
 <div className="absolute top-4 left-4 text-[10px] font-mono font-weight-normal">
 Грязная стопка (Стек вызовов / LIFO)
 </div>

 {/* Визуализация раковины */}
 <div className="absolute top-4 right-4 flex items-center justify-center">


```

```

 {
 /* Тарелки (отрисовываем в обратном порядке) */
 {displayStack.slice().reverse().map((plate) => {
 const idx = displayStack.length - 1 - plate.index;
 const isTop = idx === topIndex;
 return (
 <div
 key={plate.id}
 className={`
 w-full max-w-[280px] h-10 rounded
 shadow-md select-none transition-all duration-200
 ${plate.color}
 ${plate.animState === 'in' ? 'animate-in' : ''}
 ${plate.animState === 'washing' ? 'animate-wash' : ''}
 ${isTop ? 'ring-4 ring-blue-500/50' : ''}
 `}
 >
 {plate.name}

 {isTop && (

 LIFO

)}

 </div>
);
 })}
 </div>
);
}

/* Столешница раковины */
<div className="w-full h-4 bg-gradient-to-r from-slate-400 to-slate-600">
 </div>

/* ПРАВАЯ КОЛОНКА: СУШИЛКА ДЛЯ ЧИСТЫХ ТАРЕЛОК */
<div className="md:col-span-5 bg-slate-900 rounded-2xl p-4">
 <div className="absolute top-4 left-4 text-[10px] text-white">
 Сушилка для чистой посуды
 </div>

```

```

 {log.map((entry, idx) => (
 <div
 key={idx}
 className={`text-xs font-mono flex items-center`}
 >
 {idx === 0 ?
 {entry}
 </div>
)}}
 </div>
 </div>
);
};

```

ФАЙЛ 71/96: src/components/StringAlgorithmsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import { useState, useEffect, useRef, useMemo } from "react";
import { useVizRuntime, vizArray, vizNumber, vizString } from "viz-runtime";
import { Play, Pause, SkipForward, Undo, RefreshCw } from "lucide-react";

```

```

function generateKmpSteps(pattern: string, text: string) {
 if (!pattern) pattern = " ";
 const s = pattern + "#" + text;
 const n = s.length;
 const pi = new Array(n).fill(0);
 const steps: any[] = [];

 steps.push({ i: 0, j: 0, pi: [...pi], desc: `Начало` });

 for (let i = 1; i < n; i++) {
 let j = pi[i - 1];

 while (s[i] !== s[j]) j = pi[j - 1];

 steps.push({ i, j, pi: [...pi], highlight: [i, j] });
 }
}

```

```

 if (i <= r) {
 steps.push({ i, l, r, z: [...z], desc: `Внимание!` });
 z[i] = Math.min(r - i + 1, z[i - l]);
 steps.push({ i, l, r, z: [...z], desc: `Префикс-функция. Копипаста сработала.` });
 }

 steps.push({ i, l, r, zTarget: z[i], zSource: i });
 while (i + z[i] < n && s[z[i]] === s[i + z[i]]) {
 steps.push({ i, l, r, zTarget: z[i], zSource: i + z[i], desc: `Окно падает! Окно растёт.` });
 z[i]++;
 }

 if (i + z[i] < n) {
 steps.push({ i, l, r, zTarget: z[i], zSource: i + z[i], desc: `Окно падает! Окно растёт.` });
 } else {
 steps.push({ i, l, r, z: [...z], desc: `Упс!` });
 }

 steps.push({ i, l, r, z: [...z], desc: `Фиксируем` });

 if (i + z[i] - 1 > r) {
 l = i;
 r = i + z[i] - 1;
 steps.push({ i, l, r, z: [...z], desc: `Окно растёт!` });
 }

 if (z[i] === pattern.length) {
 steps.push({ i, l, r, z: [...z], found: true });
 }
}
return { s, steps, patternLength: pattern.length };
}

```



```

 timer.current = setInterval(() => {
 setStepIdx(prev => {
 if (prev >= steps.length - 1) { set
 return prev + 1;
 });
 }, 1200);
 }
 return () => { if (timer.current) clearInterval
}, [autoPlay, steps.length]);

const playPause = () => setAutoPlay(!autoPlay);
const nextStep = () => { setAutoPlay(false); setSte
const prevStep = () => { setAutoPlay(false); setSte
const reset = () => { setAutoPlay(false); setStepId

const baseStep = steps[stepIdx] || steps[0];
const vars = runtime?.variables;
const liveValues = vizArray(mode === "kmp" ? vars?.l
const hasLiveValues = !!vars && Object.prototype.ha
const liveI = vizNumber(vars?.i);
const liveJ = vizNumber(vars?.j);
const liveL = vizNumber(vars?.l);
const liveR = vizNumber(vars?.r);
const compilerLinked = liveI !== null || hasLiveVal
const step = compilerLinked
 ? {
 ...baseStep,
 i: liveI ?? baseStep.i,
 j: liveJ ?? baseStep.j,
 l: liveL ?? baseStep.l,
 r: liveR ?? baseStep.r,
 [mode === "kmp" ? "pi" : "z"]: hasLiveValues
 desc: `Компилятор: i=${liveI ?? "-"}${mode ===
 }
 : baseStep;

return (
 <div className="space-y-4">

```

```

 // KMP Logic
 if (mode === "kmp") {
 if (step.highlight?.include
 borderColor = "border-a
 bgColor = step.match ===
-900/40");
 }
 }

 // Z Logic
 if (mode === "z") {
 if (index >= step.l && index
 bgColor = "bg-emerald-900
 borderColor = "border-e
 }
 if (step.zTarget === index
 borderColor = "border-a
 bgColor = step.match ===
-900/40");
 }
}

return {
 <div key={index} className=
 {/* L/R markers for Z */}
 {mode === "z" && step.l
 <div className="abs
 }}
 {mode === "z" && step.r
 <div className="abs
 }}

 {/* Index */}
 <span className={`text-

 {/* Char Box */}
 <div className={`w-8 h-

```

```

 <span className=
 </div>
 }}
 {mode === "z" && step.z}
 <div className="abs
 <span className=
 </div>
 }}

 {/* Found flash effect
 {step.found && mode ===
 <div className="abs
ulse z-0 bg-emerald-500/20 shadow-[0_0_15px
 }}
 {step.found && mode ===
 <div className="abs
ulse z-0 bg-emerald-500/20 shadow-[0_0_15px
 }}
 </div>
)
 }}}}
 </div>
 </div>

<div className="flex flex-col sm:flex-row g
 {/* Controls */}
 <div className="flex bg-slate-900 borde
 <button onClick={prevStep} disabled
rounded-lg disabled:opacity-30 disabled:hov
 <Undo strokeWidth={3} size={16}
 </button>
 <button onClick={playPause} classNa
x items-center justify-center min-w-[80px]"
 {autoplay ? <><Pause size={16} <
 </button>
 <button onClick={nextStep} disabled
r:bg-slate-800 rounded-lg disabled:opacity-
 <SkipForward strokeWidth={3} si

```

```
 Код C++ ({mode === 'kmp' ? 'КМП'
 </div>
 <pre className="p-4 text-xs font-mono te
{mode === 'kmp' ? `vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
 int j = pi[i-1];
 while (j > 0 && s[i] != s[j]) j = pi[j-1];
 if (s[i] == s[j]) j++;
 pi[i] = j;
}` : `int l = 0, r = 0;
for (int i = 1; i < n; i++) {
 if (i <= r) z[i] = min(r - i + 1, z[i - l]);
 while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
 if (i + z[i] - 1 > r) {
 l = i;
 r = i + z[i] - 1;
 }
}`}
 </pre>
 </div>
 </div>
);
}
```

---

ФАЙЛ 72/96: src/components/Tooltip.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useState } from "react";

interface TooltipProps {
 /** Содержимое подсказки (можно JSX). */
 content: React.ReactNode;
 /** Элемент, на который наводят курсор / фокус. */
 children: React.ReactNode;
 /** С какой стороны показывать. По умолчанию — сверху
```

```
 });
 };
};
```

---

ФАЙЛ 73/96: src/components/TopologicalSortViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useState, useEffect } from "react";
import { useVizStepSync, vizArray, vizNumber } from "...";
import {
 Play,
 Pause,
 RotateCcw,
 Clock,
} from "lucide-react";

interface TNode {
 id: number;
 x: number;
 y: number;
 label: string;
 name: string;
}

interface TEdge {
 u: number;
 v: number;
}

const dagNodes: TNode[] = [
 { id: 0, x: 100, y: 150, label: "0", name: "Матан 1 (Математика)" },
 { id: 1, x: 280, y: 70, label: "1", name: "Дискретка (Дискретная математика)" },
 { id: 2, x: 280, y: 230, label: "2", name: "Линейный алгебра (Линейная алгебра)" },
 { id: 3, x: 460, y: 150, label: "3", name: "Алгоритмы (Алгоритмы)" },
 { id: 4, x: 640, y: 150, label: "4", name: "Машинное обучение (Машинное обучение)" },
];
```

```

const dfsStack: number[] = [];

const recordStep = (desc: string, currentNode: number) => {
 listSteps.push({
 desc,
 visited: new Set(visited),
 fullyProcessed: new Set(fullyProcessed),
 currentNode,
 topoList: [...topoList],
 dfsStack: [...dfsStack],
 });
};

recordStep(
 "Начало топологической сортировки. Используем кла",
 null,
);

// Adj list
const adj: number[][] = Array.from({ length: 5 }, () => []);
dagEdges.forEach((e) => adj[e.u].push(e.v));

const dfs = (u: number) => {
 visited.add(u);
 dfsStack.push(u);
 recordStep(
 `DFS: зашли в вершину ${u} ("${dagNodes[u].name}`,
 u,
);

 for (const to of adj[u]) {
 if (!visited.has(to)) {
 dfs(to);
 } else if (!fullyProcessed.has(to)) {
 // Explaining potential cycle detected (not in
 recordStep(
 `⚠ Внимание: Рёбра в серую вершину ${to} о`,
 u,
);
 }
 }
 fullyProcessed.add(u);
 topoList.push(u);
};

dfs(0);

return topoList;
}

export default topologicalSort;

```

```

 topoList: [],
 dfsStack: [],
 };

 useEffect(() => {
 let timer: NodeJS.Timeout;
 if (isPlaying && currentStepIdx < steps.length - 1) {
 timer = setInterval(() => {
 setCurrentStepIdx((prev) => prev + 1);
 }, 1600);
 } else {
 setIsPlaying(false);
 }
 return () => clearInterval(timer);
 }, [isPlaying, currentStepIdx, steps.length]);

 const togglePlay = () => setIsPlaying(!isPlaying);
 const reset = () => {
 setIsPlaying(false);
 setCurrentStepIdx(0);
 };

 const getTopoListString = () => {
 return [...step.topoList]
 .reverse()
 .map((id) => dagNodes[id].name)
 .join(" → ");
 };

 return (
 <div className="bg-slate-900 rounded-xl border border-
 <div className="bg-slate-800 p-4 border-b border-
 <h3 className="font-bold text-white flex items-
 <Clock className="w-5 h-5 text-indigo-400" />
 Топологическая сортировка (Зависимости обучен
 </h3>

 <div className="flex items-center gap-2 bg-slate-

```

```

 </marker>
 <marker
 id="dag-arrow-active"
 viewBox="0 0 10 10"
 refX="24"
 refY="5"
 markerWidth="6"
 markerHeight="6"
 orient="auto-start-reverse"
 >
 <path d="M 0 1 L 10 5 L 0 9 z" fill="#8"
 </marker>
 </defs>

 {/* Edges */}
 {dagEdges.map(({edge, idx}) => {
 const u = dagNodes.find((n) => n.id === e
 const v = dagNodes.find((n) => n.id === e

 const isActive =
 (step.currentNode === edge.u || step.cu
 step.visited.has(edge.v));

 return (
 <line
 key={`edge-${idx}`}
 x1={u.x}
 y1={u.y}
 x2={v.x}
 y2={v.y}
 stroke={isActive ? "#818cf8" : "#3341
 strokeWidth={isActive ? "3" : "2"}
 markerEnd={`url(#${isActive ? "dag-ar
 className="transition-all duration-300
 />
);
 })}
)}
)}

```



```

 <text
 x={node.x}
 y={node.y - 4}
 textAnchor="middle"
 fill="white"
 fontSize="11px"
 fontWeight="bold"
 className="pointer-events-none"
 >
 ({node.label}) {node.name.split(" "
 </text>
 <text
 x={node.x}
 y={node.y + 12}
 textAnchor="middle"
 fill="#94a3b8"
 fontSize="9px"
 className="pointer-events-none"
 >
 {node.name.split(" ").slice(1).join
 </text>
 </g>
);
 }}}
</svg>
</div>

```

```

{/* Console column */}
<div className="lg:col-span-4 bg-slate-950 p-5
 00 overflow-y-auto">
 <h4 className="text-xs font-bold text-slate-400">
 Статус сортировки
 </h4>

 <div className="bg-slate-900 border border-slate-800">
 <p className="text-xs text-indigo-300 min-h-[100px]">
 {step.desc}
 </p>
 </div>

```

```

 </div>

 <div className="mt-4 pt-3 border-t border-slate-200">

 Шаг {currentStepIdx + 1} из {steps.length}

 <div className="flex gap-1">
 <button
 disabled={currentStepIdx === 0}
 onClick={() => setCurrentStepIdx((prev) => prev - 1)}
 className="bg-slate-900 border border-slate-700 px-2 py-1 rounded text-white"
 >
 Назад
 </button>
 <button
 disabled={currentStepIdx >= steps.length - 1}
 onClick={() => setCurrentStepIdx((prev) => prev + 1)}
 className="bg-slate-900 border border-slate-700 px-2 py-1 rounded text-white"
 >
 Вперед
 </button>
 </div>
 </div>
 </div>
 </div>
 </div>
);
};

```

ФАЙЛ 74/96: src/components/TreapBuildViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useEffect, useMemo, useState } from "react";
```

```

 { key: 9, grade: 3 },
 { key: 7, grade: 2 },
 { key: 12, grade: 0 },
 { key: 1, grade: -4 },
];

/** Порядок ввода (как в черновике) – намеренно НЕ отсортирован
export const INPUT_ORDER: Pair[] = [
 { key: 3, grade: 5 },
 { key: 45, grade: 8 },
 { key: 3, grade: 6 },
 { key: 1, grade: -4 },
 { key: 7, grade: 2 },
 { key: 6, grade: 5 },
 { key: 2, grade: 4 },
 { key: 9, grade: 3 },
 { key: 12, grade: 0 },
];
// (3,5) дубль ключа/приоритета – уберём: ввод без повторов
export const INPUT_CLEAN: Pair[] = INPUT_ORDER.filter((p, i) => !INPUT_ORDER.some((p2, j) => j < i && p2.key === p.key));

const sortedByY = (ps: Pair[]): Pair[] => [...ps].sort((a, b) => a.grade - b.grade);

/* ----- чистые алгоритмы (экспортируются для тестов) -----

export function insert(root: TNode | null, p: Pair): TNode | null {
 if (!root) return { id: pairId(p.key, p.grade), key: p.key, grade: p.grade, left: null, right: null };
 if (p.key <= root.key) root.left = insert(root.left, p);
 else root.right = insert(root.right, p);
 return root;
}

export function buildTree(ps: Pair[]): TNode | null {
 let root: TNode | null = null;
 for (const p of sortedByY(ps)) root = insert(root, p);
 return root;
}

```

```

 if (a.grade > b.grade) {
 trace.push({ aId: a.id, bId: b.id, chosenId: a.id,
 `ем его правое поддереву с (${b.key}, ${b.grade})`
 });
 a.right = go(a.right, b);
 return a;
 }
 trace.push({ aId: a.id, bId: b.id, chosenId: b.id,
 ` (${a.key}, ${a.grade}) с его левым поддеревом.`
 });
 b.left = go(a, b.left);
 return b;
 };
 return { root: go(a, b), trace };
}

export function eraseNode(root: TNode | null, key: number): TNode | null {
 const trace: { aId: NodeId | null; note: string }[] = [];
 const go = (t: TNode | null): TNode | null => {
 if (!t) return null;
 if (key < t.key) {
 trace.push({ aId: t.id, note: `${key} < ${t.key}` });
 t.left = go(t.left);
 return t;
 }
 if (key > t.key) {
 trace.push({ aId: t.id, note: `${key} > ${t.key}` });
 t.right = go(t.right);
 return t;
 }
 trace.push({ aId: t.id, note: `Нашли (${t.key}, ${t.grade})` });
 const m = mergeTree(t.left, t.right);
 trace.push(...m.trace.map((s) => ({ aId: s.chosenId, note: s.note })));
 return m.root;
 };
 return { root: go(root), trace };
}

const cloneTree = (t: TNode | null): TNode | null =>

```

```

const es: { from: NodeId; to: NodeId }[] = [];
const walk = (n: TNode | null) => {
 if (!n) return;
 if (n.left) { es.push({ from: n.id, to: n.left.id });
 if (n.right) { es.push({ from: n.id, to: n.right.id });
};
walk(root);
return es;
};

```

```

interface TreeViewProps {
 root: TNode | null;
 width?: number;
 height?: number;
 nodeColor?: (id: NodeId) => string;
 nodeStroke?: (id: NodeId) => string;
 edgeColor?: (from: NodeId, to: NodeId) => string;
 edgeDash?: (from: NodeId, to: NodeId) => boolean;
 pulseId?: NodeId | null;
 subtitle?: string;
}

```

```

const TreeView: React.FC<TreeViewProps> = ({ root, width, height, ...props }) => {
 const pos = useMemo(() => layoutTree(root), [root]);
 const nodes = useMemo(() => [...pos.entries()].map(([id, node]) => node), [pos]);
 const edges = useMemo(() => treeEdges(root), [root]);
 const autoW = Math.max(420, ...nodes.map((n) => n.px));
 const autoH = Math.max(260, ...nodes.map((n) => n.py));
 const w = width ?? autoW;
 const h = height ?? autoH;
 const byId = (id: NodeId) => nodes.find((n) => n.id === id);
 return (
 <div>
 <svg viewBox={`0 0 ${w} ${h}`} className="w-full h-full">
 {edges.map((e) => {
 const a = byId(e.from);
 const b = byId(e.to);
 if (!a || !b) return null;

```

```

const PX = (key: number) => 42 + (key - 1) * ((PL_W - 6) / 4);
const PY = (grade: number) => 24 + (8 - grade) * ((PL_H - 16) / 8);

interface PlaneProps {
 points: Pair[];
 drawnEdges: { a: Pair; b: Pair }[];
 pending: Pair | null;
 interactive?: boolean;
 selected?: Pair | null;
 cursor?: { x: number; y: number } | null;
 badEdge?: { a: Pair; b: Pair } | null;
 onPointDown?: (p: Pair) => void;
 onPointUp?: (p: Pair) => void;
 onMove?: (e: React.PointerEvent<SVGSVGElement>) => void;
}

const Plane: React.FC<PlaneProps> = ({ points, drawnEdges, pending,
 }) => {
 const byPair = (p: Pair) => ({ x: PX(p.key), y: PY(p.grade) });
 return (
 <svg
 viewBox={`0 0 ${PL_W} ${PL_H}`}
 className={`w-full ${interactive ? "touch-none select-none" : ""}`}
 onPointerMove={onMove}
 style={interactive ? { cursor: "crosshair" } : undefined}
 >
 <defs>
 <pattern id="pl-grid" width="14" height="14" patternUnits="userSpaceOnUse">
 <path d="M 14 0 L 0 0 0 14" fill="none" stroke="black" stroke-width="1"/>
 </pattern>
 </defs>
 <rect x="34" y="16" width={PL_W - 46} height={PL_H - 16} fill="white" stroke="black" stroke-width="1"/>
 <line x1="34" y1={PY(0)} x2={PL_W - 12} y2={PY(0)} stroke="black" stroke-width="1"/>
 <line x1={PX(1) - 8} y1="16" x2={PX(1) - 8} y2={PL_H - 16} stroke="black" stroke-width="1"/>
 {[0, 10, 20, 30, 40].map((k) => {
 <text key={`tx-${k}`} x={PX(k)} y={PY(0) + 16} text-align="center">{k}</text>
 })}
 {[{-4, 0, 4, 8}.map((g) => {
 <text key={`ty-${g}`} x={PX(1) - 14} y={PY(g) + 16} text-align="right">{g}</text>
 })}
 </svg>
);
};

```

```

 });
 }}}

 {pending && (
 <g className="transition-all duration-500">
 <circle cx={PX(pending.key)} cy={PY(pending.g
 >
 <text x={PX(pending.key)} y={PY(pending.grade
 ly="monospace">{pending.key}</text>
 </g>
)}
 </svg>
)};
};

/* ===== ИГРА: соединение мышкой =====

export type GEdge = { a: Pair; b: Pair }; // a – родите

export const CANON_GAME: GEdge[] = (() => {
 const es: GEdge[] = [];
 const walk = (n: TNode | null) => {
 if (!n) return;
 if (n.left) { es.push({ a: { key: n.key, grade: n.g
 if (n.right) { es.push({ a: { key: n.key, grade: n.g
 };
 walk(buildTree(POINTS));
 return es;
})();

const sameP = (p: Pair, q: Pair) => p.key === q.key &&
const gpid = (p: Pair) => `${p.key}:${p.grade}`;

/** Цепочка родителей от точки вверх. */
const parentOf = (edges: GEdge[], id: string): GEdge |
 edges.find((e) => gpid(e.b) === id);
const reaches = (edges: GEdge[], from: Pair, target: Pa

```

```

}

/** Попытка хода: почему нельзя – или ОК. */
export function tryConnect(edges: GEdge[], a: Pair, b: Pair): boolean {
 if (sameP(a, b)) return { ok: false, reason: "Это одна и та же точка" };
 if (a.grade === b.grade)
 return { ok: false, reason: "«!» равны – «кто выше?»" };
 const parent = a.grade > b.grade ? a : b;
 const child = a.grade > b.grade ? b : a;
 if (parentOf(edges, gpId(child)))
 return { ok: false, reason: `У точки ${child.key} уже есть родительская линия.` };
 if (reaches(edges, parent, child))
 return { ok: false, reason: "Цикл: линия замкнёт саму себя" };
 const next = [...edges, { a: parent, b: child }];
 if (!forestIsBst(next, POINTS))
 return { ok: false, reason: "Линия вниз по у допустима только в левом поддереве" };
 return { ok: true };
}

/* ===== РЕЖИМ BUILD ===== */

type BuildPhase = "sort" | "connect" | "game";
type SortAlgo = "none" | "quicksort" | "counting";
type ConnectOrder = "y-desc" | "left-right";

const parsePairs = (raw: string): { pairs: Pair[]; errors: string[] } => {
 const pairs: Pair[] = [];
 const errors: string[] = [];
 const seen = new Set<string>();
 const chunks = raw.split(/[,;\n]+/).map((c) => c.trim());
 for (const chunk of chunks) {
 const nums = chunk.split(/[\sxx*]+/).filter((v) => !Number.isNaN(Number(v)));
 if (nums.length !== 2 || nums.some((v) => !Number.isInteger(Number(v)))) {
 errors.push(`«${chunk}» – нужно ровно два числа: целые и не меньше 0`);
 continue;
 }
 const id = `${nums[0]}:${nums[1]}`;
 if (seen.has(id)) {
 errors.push(`«${chunk}» – такая пара уже была`);
 continue;
 }
 pairs.push({ key: id, grade: Math.min(...nums) });
 seen.add(id);
 }
 return { pairs, errors };
}

const parseGame = (raw: string): { game: Game; errors: string[] } => {
 const game: Game = {
 edges: [],
 points: new Map(),
 sortAlgo: "none",
 connectOrder: "y-desc",
 };
 const errors: string[] = [];
 const chunks = raw.split(/[,;\n]+/).map((c) => c.trim());
 for (const chunk of chunks) {
 const [key, value] = chunk.split(":");
 if (!key || !value) {
 errors.push(`«${chunk}» – некорректная запись`);
 continue;
 }
 if (key === "sortAlgo") {
 if (value !== "none" && value !== "quicksort" && value !== "counting") {
 errors.push(`«${chunk}» – некорректное значение`);
 continue;
 }
 game.sortAlgo = value;
 } else if (key === "connectOrder") {
 if (value !== "y-desc" && value !== "left-right") {
 errors.push(`«${chunk}» – некорректное значение`);
 continue;
 }
 game.connectOrder = value;
 } else if (key === "edges") {
 const pairs = parsePairs(value);
 if (pairs.errors.length) {
 errors.push(`«${chunk}» – некорректная запись`);
 continue;
 }
 game.edges.push(...pairs.pairs);
 } else {
 errors.push(`«${chunk}» – некорректная запись`);
 continue;
 }
 }
 return { game, errors };
}

const parse = (raw: string): { game: Game; pairs: Pair[]; errors: string[] } => {
 const { game, errors: gameErrors } = parseGame(raw);
 const { pairs, errors: pairsErrors } = parsePairs(raw);
 return { game, pairs, errors: [...gameErrors, ...pairsErrors] };
}

const main = (raw: string): string => {
 const { game, pairs, errors } = parse(raw);
 if (errors.length) {
 return errors.join("\n");
 }
 const { forest, errors: forestErrors } = buildForest(game, pairs);
 if (forestErrors.length) {
 return forestErrors.join("\n");
 }
 const { game, errors: gameErrors } = playGame(forest);
 if (gameErrors.length) {
 return gameErrors.join("\n");
 }
 return "Игра окончена. Победитель: " + (game.winner ? game.winner : "ничья");
}

const run = (raw: string): string => {
 const result = main(raw);
 return result;
}

```



```

// цикл: родитель уже сидит в компоненте ребёнка (реб
const childComp = new Set<string>([pid(child)]);
let grew = true;
while (grew) {
 grew = false;
 for (const e of edges) {
 if (childComp.has(pid(e.b)) && !childComp.has(pid
 childComp.add(pid(e.a));
 grew = true;
 }
 }
}
if (childComp.has(pid(parent))) return { ok: false, r
return {
 ok: true,
 edge: { a: parent, b: child },
 reason:
 x.grade > y.grade ? undefined : "линия в treap вс
};
}

```

```

export function checkGameState(
 points: Pair[],
 edges: GameEdge[],
): { heapOk: boolean; singleParent: boolean; connected:
 const n = points.length;
 if (n < 2) return { heapOk: true, singleParent: true,
 const heapOk = edges.every((e) => e.a.grade > e.b.gra
 const parentCount = new Map<string, number>();
 for (const e of edges) parentCount.set(pid(e.b), (par
 const singleParent = [...parentCount.values()].every(
 // связность: компоненты через union-find
 const comp = new Map<string, string>(points.map((p) =>
 const find = (x: string): string => {
 while (comp.get(x) !== x) {
 comp.set(x, comp.get(x)!);
 x = comp.get(x)!;
 }
 }
}

```

```

 right = sorted[1];
 } else if (sorted.length === 1) {
 // одиночный ребёнок: сторона важна – определяем
 if (sorted[0].key <= p.key) left = sorted[0];
 else right = sorted[0];
 }
 if (left) walk(left);
 inorder.push(p.key);
 if (right) walk(right);
};
walk(roots[0]);
if (seen.size !== n) bstOk = false;
for (let i = 1; i < inorder.length; i++)
 if (inorder[i] < inorder[i - 1]) bstOk = false;
}
return { heapOk, singleParent, connected, countOk, bstOk };
}

```

```

/** Дерево из проведённых линий – для показа результата
export function edgesToTree(edges: GameEdge[]): TNode | null {
 const parentOf = new Map<string, GameEdge>();
 for (const e of edges) parentOf.set(pid(e.b), e);
 const roots = edges.filter((e) => !parentOf.has(pid(e.a)));
 if (!roots.length && edges.length) return null;
 const nodes = new Map<string, TNode>();
 const make = (p: Pair): TNode => ({
 id: pid(p),
 key: p.key,
 grade: p.grade,
 left: null,
 right: null,
 });
 const rootPair = roots.length ? roots[0].a : edges[0];
 if (!rootPair) return null;
 const build = (p: Pair): TNode => {
 const node = make(p);
 nodes.set(pid(p), node);
 const kids = edges.filter((e) => pid(e.a) === pid(p) &&

```

```

 walk(userTree);
 return es;
 }, [userTree]);
const leftRightEdges = useMemo(() => [...canonEdges].

useEffect(() => {
 if (!playing || phase === "game") return;
 const t = setTimeout(() => {
 if (phase === "sort") {
 if (algo === "none") { setPlaying(false); return; }
 if (sortIdx < 3) setSortIdx(sortIdx + 1);
 else setPlaying(false);
 } else {
 const total = order === "y-desc" ? canonEdges.l
 if (connIdx < total) setConnIdx(connIdx + 1);
 else setPlaying(false);
 }
 }, 900);
 return () => clearTimeout(t);
}, [playing, phase, sortIdx, connIdx, algo, order, ca

const startConnect = (ord: ConnectOrder) => {
 setOrder(ord);
 setPhase("connect");
 setConnIdx(0);
 setPlaying(true);
};

// Компилятор: i двигает демонстрацию – шаг сортировки
useEffect(() => {
 if (live.i === null) return;
 if (phase === "sort") setSortIdx(Math.min(live.i, 3)
 else if (phase === "connect") setConnIdx(Math.min(l
}, [live.i, phase, leftRightEdges.length]);

const comparisons = algo === "quicksort" ? (n > 1 ? M
const connEdgesAll = order === "y-desc" ? canonEdges
const connEdges = connEdgesAll.slice(0, connIdx);

```

```

 }
 };
 const pointDown = (p: Pair) => {
 if (phase !== "game") return;
 if (!sel) {
 setSel(p);
 setGameMsg(`Выбрана (${p.key}; ${p.grade}) – теперь...`);
 } else if (sel.key === p.key && sel.grade === p.grade) {
 setSel(null);
 setGameMsg(null);
 } else {
 attemptConnect(sel, p);
 setSel(null);
 }
 };
 const pointUp = (p: Pair) => {
 if (phase !== "game" || !sel) return;
 if (sel.key === p.key && sel.grade === p.grade) return;
 attemptConnect(sel, p);
 setSel(null);
 setCursor(null);
 };
 const planeMove = (e: React.PointerEvent<SVGSVGElement>) => {
 if (!sel) return;
 const rect = (e.currentTarget as SVGSVGElement).getBoundingClientRect();
 setCursor({ x: ((e.clientX - rect.left) / rect.width) * 100 });
 };

 const undoEdge = () => {
 setGameEdges((prev) => prev.slice(0, -1));
 setGameMsg(null);
 };

 const showSolution = () => {
 setGameEdges(canonEdges.map((e) => ({ a: e.a, b: e.b })));
 setHintUsed(true);
 setGameMsg("Решение показано – собери сам в следующий раз");
 };

```

```

<h4 className="text-base font-bold text-emera
<div className="flex items-center gap-2">
 {phase === "sort" ? (
 <>
 <span className="text-xs text-slate-400
 {["quicksort", "counting"] as const }.m
 <button
 key={a}
 onClick={() => { setAlgo(a); setSor
Math.round(n * Math.log2(n)) : 0} сравнений
 disabled={n < 2}
 className={ `px-3 py-1.5 rounded-lg
600 text-white" : "bg-slate-700 text-slate-
 >
 {a === "quicksort" ? "quicksort (ср
 </button>
)}}
 </>
) : (
 <>
 <span className="text-xs text-slate-400
 <button
 onClick={() => startConnect("y-desc")
 className={ `px-3 py-1.5 rounded-lg te
bg-emerald-600 text-white" : "bg-slate-700
 >
 по y ↓ (алгоритм)
 </button>
 <button
 onClick={() => startConnect("left-right
 className={ `px-3 py-1.5 rounded-lg te
"bg-emerald-600 text-white" : "bg-slate-700
 >
 слева направо
 </button>
 <button
 onClick={startGame}
 disabled={n < 2}

```

```

) : phase === "game" ? (
 gameWon ? (
 <> Готово! {hintUsed ? "Решение подсказано": "Решение найдено"}
 ево: линии вниз по y, обход по x отсортировано
) : gameMsg ? (
 <>△ {gameMsg}</>
) : (
 <>Кликни точку-родителя (выше по «!»), за
)
) : connDone ? (
 <> Проведено {connEdges.length} линий в по
 : точки прибиты, дерево одно. А теперь
) : (
 <>Линий проведено: {connIdx} из {connEdgesA
 })
 </div>
</div>

{/* Чек-лист инвариантов */}
<div className={`rounded-xl p-4 border transition
 er-slate-700 bg-slate-900/40`}>
 <h4 className="text-sm font-bold text-slate-300">
 <ul className="space-y-1.5 text-sm">
 {phase === "game" ? (
 <>
 {verdict.heapOk ? " " : " "} каждое р
 {verdict.singleParent ? " " : " "} у
 {verdict.connected ? " " : " "} все т
 {verdict.countOk ? " " : " "} линий р
 {verdict.bstOk ? " " : " "} обход сле
 </>
) : (
 <>
 {connDone ? " " : " "} каждое ребро в
 {connDone ? " " : " "} обход слева-на
 {connDone ? " " : " "} линий ровно
 </>
)
 }
}

```



```

const { l, r } = useMemo(() => splitTree(buildTree(P0
const merged = useMemo(() => mergeTree(cloneTree(l),
const steps = merged.trace;
const idx = Math.min(stepIdx, steps.length - 1);
const done = idx >= steps.length - 1;
const cur = steps[idx];

return (
 <div className="space-y-4">
 <div className="flex items-center gap-3">
 <button onClick={() => setStepIdx(Math.max(0, i
 bg-slate-700 text-white disabled:opacity-30">
 <button onClick={() => setStepIdx(Math.min(step
 font-bold bg-indigo-600 text-white disabled:op
 mar {i
 </div>

 <div className="grid grid-cols-1 md:grid-cols-2 g
 <div className="bg-sky-950/20 rounded-xl p-3 bo
 <h5 className="text-sm font-bold text-sky-300
 <TreeView root={l} width={480} height={220} n
 </div>
 <div className="bg-rose-950/20 rounded-xl p-3 b
 <h5 className="text-sm font-bold text-rose-300
 <TreeView root={r} width={480} height={220} n
 </div>
 </div>

 <div className="text-xs text-slate-300 bg-slate-9
 <LiveChip live={live} />{' '}
 {cur?.chosenId ? (
 <>⊗ {cur.note}</>
) : (
 <>Готово: {done ? "деревья склеены обратно в
 />
)}
 </div>

```



```

<div className="space-y-4">
 <div className="flex flex-wrap items-center gap-3">
 Удалить
 <LiveChip live={live} />
 <select value={key} onChange={(e) => { setKey(N
 te-700 rounded-lg px-3 py-1.5 text-sm font-mo
 {POINTS.map((p) => <option key={p.key} value=
 </select>
 <button onClick={() => setStepIdx(Math.max(0, i
 bg-slate-700 text-white disabled:opacity-30">
 <button onClick={() => setStepIdx(Math.min(step
 font-bold bg-indigo-600 text-white disabled:op
 шаг {i
 </div>

 {!done ? (
 <div className="bg-slate-900/40 rounded-xl p-4"
 <TreeView
 root={fullTree}
 nodeColor={(id) => (pathSet.has(id) ? "#783
 nodeStroke={(id) => (pathSet.has(id) ? "#f5
 subtitle="Куча сама умеет удалять только вер
 />
 </div>
) : (
 <div className="bg-slate-900/40 rounded-xl p-4"
 <h4 className="text-sm font-bold text-slate-300">
 <TreeView root={res.root} subtitle="Валидность"
 <ul className="mt-2 space-y-1 text-sm">
 обход слева-направо по-прежнему сортир
 каждое ребро вниз по y (куча целая) —

 </div>
)}

 <div className="text-xs text-slate-300 bg-slate-900/40 p-2">
 {steps[idx]?.note}
 </div>

```

```

{pick === "heap" ? {
 <div className="bg-slate-900/40 rounded-xl p-4" >
 <h4 className="text-sm font-bold text-slate-300" >
 4>
 <div className="flex gap-2 overflow-x-auto mb-2" >
 {heapArr.slice(1).map((v, i) => {
 <div key={i} className="px-3 py-2 rounded" >
 <div className="text-slate-500 text-[10px]" >
 <div className="text-white font-bold">{v}</div>
 <div className="text-emerald-400 text-[10px]" >
 {i}</div>
 </div>
 </div>
 })}
 </div>
 <p className="text-xs text-slate-400">Полное $2i/2i+1$ попадает в узел.</p>
 </div>
) : {
 <div className="bg-slate-900/40 rounded-xl p-4" >
 <h4 className="text-sm font-bold text-rose-300" >
 >
 <TreeView root={full} subtitle="Попробуй «посмотреть»" />
 <div className="flex gap-2 overflow-x-auto mt-2" >
 {[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13].map((v, i) => {
 const id = slots[i];
 return (
 <div key={i} className={`px-2.5 py-2 rounded border-slate-700`} : "bg-rose-950/40 border-rose-950" >
 <div className="text-slate-500 text-[10px]" >
 {id ? <div className="text-white font-bold">{v}</div> : v}
 </div>
 </div>
);
 })}
 </div>
 <p className="text-xs text-rose-300 mt-2">Смотри, что левая ветка ушла на 4 уровня и заняла два уровня указателями (left/right), а не арифметикой.</p>
 </div>
}
}

```

```

 нашли за ${steps.length + 1} сравнение(й).` }));
 if (sortedByKey[mid].key < target) { steps.push({
 } — вся левая половина выброшена.` })); lo = mid
 else { steps.push({ lo, hi, mid, found: false, no
 ` })); hi = mid - 1; }
 }
 return steps;
}, [sortedByKey]);

```

```

const wrongMid = Math.floor((shuffled.length - 1) / 2)
const comparisons = stage === 2 ? bidx + 1 : 0;

```

```

return (
 <div className="space-y-4">
 <div className="flex gap-2">
 {stage < 2 && (
 <button onClick={() => { setStage(stage === 0
 digo-600 text-white">
 {stage === 0 ? "Найти y для x=7 бинарным по
 </button>
)}
 {stage === 2 && (
 <button onClick={() => setBidx(Math.min(bsStep
 -1.5 rounded-lg text-xs font-bold bg-indigo
 Шаг поиска →
 </button>
)}
 <button onClick={() => { setStage(0); setBidx(0
 Сброс</button>
 </div>

```

```

<div className="flex gap-1.5 overflow-x-auto">
 {(stage === 2 ? sortedByKey : shuffled).map((p,
 const highlight = stage === 1 && i === wrongMid
 const inBs = stage === 2 && (() => { const s
 const isMid = stage === 2 && (() => { const s
 const isFound = stage === 2 && bidx >= bsStep
 return (

```



```

 <text x={PX(p.key)} y={PY(p.grade) + 24}
 </text>
 </g>
);
 }}}
</svg>
</div>

{/* Судья – условия, как у Ахо–Корасика */}
<div className={`rounded-xl p-4 border transition
 ate-900/40`} `>
 <h4 className="text-sm font-bold text-slate-300">
 <ul className="space-y-1.5 text-sm">
 {v.heap ? " " : " "} каждое ребро – вниз
 {v.oneParent ? " " : " "} у каждой точки
 {v.acyclic ? " " : " "} циклов нет
 {v.bst ? " " : " "} обход слева-направо –
 {v.complete ? " " : " "} линий $n-1 = \{POI$

 {win && {
 <p className="mt-3 text-sm font-bold text-еме
 ения просто не существует.</p>
 }}
</div>

 <div className="text-xs text-slate-300 bg-slate-9
</div>
);
};

/* ===== КОРПУС ===== */

type Tab = "build" | "split" | "merge" | "erase" | "lay

const TABS: { id: Tab; label: string }[] = [
 { id: "build", label: "Собрать" },
 { id: "split", label: "Split ≈ " },
 { id: "merge", label: "Merge " },

```

```

 </div>
 </div>
 </div>

 <div className="flex flex-wrap gap-2 mb-5">
 {TABS.map((t) => {
 <button
 key={t.id}
 onClick={() => setTab(t.id)}
 className={`px-4 py-2 rounded-lg text-xs font-medium border
 900/60 text-slate-400 hover:text-white border
 >
 {t.label}
 </button>
)}}
 </div>

 {tab === "build" && <BuildMode />}
 {tab === "split" && <SplitMode />}
 {tab === "merge" && <MergeMode />}
 {tab === "erase" && <EraseMode />}
 {tab === "layers" && <LayersMode />}
 {tab === "search" && <SearchMode />}
 {tab === "game" && <GameMode />}
 </div>
);
};

```

---

ФАЙЛ 75/96: src/components/vizRegistry.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React from "react";
```

```
import { ArticulationPointsViz } from "../ArticulationPointsViz";
```

```
import BellmanFordViz from "../BellmanFordViz";
```

- \* Единый реестр интерактивных демонстраций.
  - \* Используется и для панели под главой, и для всплывающей подсказки
  - \* и для модального окна «Открыть симулятор».
- \*/

```
export const VIZ_REGISTRY: Record<string, VizEntry> = {
 "stack-dfs": {
 title: "Стек и обход в глубину",
 hint: "Кладите и снимайте тарелки – это ровно то, что нужно для обхода в глубину",
 Component: StackViz,
 },
 "queue-bfs": {
 title: "Очередь и обход в ширину",
 hint: "Очередь слева, живой BFS по графу справа.",
 Component: () => (
 <div className="space-y-10">
 <QueueViz />
 <BfsViz />
 </div>
),
 },
 "heap-beam-search": {
 title: "Куча (priority queue)",
 hint: "Добавляйте гипотезы – куча сама держит лучшую",
 Component: HeapViz,
 },
 "segment-trees": {
 title: "Дерево отрезков",
 hint: "Запросы и обновления на отрезке за $O(\log n)$.",
 Component: SegmentTreeVisualizer,
 },
 "sparse-table": {
 title: "Разрезанная таблица (1D и 2D)",
 hint: "Наведите курсор на любую ячейку – подсветится",
 Component: SparseTableViz,
 },
 treap: {
 title: "Treap: плоскость → дерево → Split/Merge/Erase",
 hint: "7 вкладок: Собрать (сортировка + соединение)
```

```

 hint: "Слева – непланарные графы с подсветкой пересечений",
 Component: () => (
 <div className="space-y-10">
 <PlanarityDemo />
 <ColoringViz />
 </div>
),
 },
 dijkstra: {
 title: "Дейкстра",
 hint: "Жадно забираем ближайшую вершину и релаксируем",
 Component: () => (
 <>
 <ChapterImage vizType="dijkstra" />
 <DijkstraViz />
 </>
),
 },
 "bellman-ford": {
 title: "Форд–Беллман",
 Component: () => (
 <>
 <ChapterImage vizType="bellman-ford" />
 <BellmanFordViz />
 </>
),
 },
 floyd: {
 title: "Флойд–Уоршелл",
 Component: () => (
 <>
 <ChapterImage vizType="floyd" />
 <FloydViz />
 </>
),
 },
 "johnson-algo": { title: "Алгоритм Джонсона", Component: () => (
 "mst-kruskal": { title: "Краскал и DSU", hint: "Сортировка по весу",

```



```

// Словарь: ["HE", "SHE", "HIS", "HERS"]
interface WNode {
 id: number;
 label: string; // путь от корня
 char: string;
 x: number; // Координаты для SVG водопада
 y: number;
 depth: number;
 fail: number;
 term_link: number;
 is_terminal: boolean;
 words: string[];
}

const WATERFALL_NODES: { [key: number]: WNode } = {
 0: { id: 0, label: 'ROOT', char: 'ø', x: 400, y: 60, depth: 0, fail: 0, term_link: 0, is_terminal: false, words: [] },
 // Ветка H -> E -> R -> S
 1: { id: 1, label: 'H', char: 'H', x: 280, y: 160, depth: 1, fail: 0, term_link: 0, is_terminal: false, words: [] },
 2: { id: 2, label: 'HE', char: 'E', x: 200, y: 260, depth: 2, fail: 0, term_link: 0, is_terminal: false, words: [] },
 3: { id: 3, label: 'HER', char: 'R', x: 150, y: 360, depth: 3, fail: 0, term_link: 0, is_terminal: false, words: [] },
 4: { id: 4, label: 'HERS', char: 'S', x: 100, y: 460, depth: 4, fail: 0, term_link: 0, is_terminal: true, words: ['HERS'] },
 // Ветка H -> I -> S
 5: { id: 5, label: 'HI', char: 'I', x: 350, y: 260, depth: 2, fail: 0, term_link: 0, is_terminal: false, words: [] },
 6: { id: 6, label: 'HIS', char: 'S', x: 320, y: 360, depth: 3, fail: 0, term_link: 0, is_terminal: true, words: ['HIS'] },
 // Ветка S -> H -> E
 7: { id: 7, label: 'S', char: 'S', x: 550, y: 160, depth: 1, fail: 0, term_link: 0, is_terminal: false, words: [] },
 8: { id: 8, label: 'SH', char: 'H', x: 580, y: 260, depth: 2, fail: 0, term_link: 0, is_terminal: false, words: [] },
 9: { id: 9, label: 'SHE', char: 'E', x: 610, y: 360, depth: 3, fail: 0, term_link: 0, is_terminal: true, words: ['SHE'] },
 // Ветка E -> H -> S
 10: { id: 10, label: 'E', char: 'E', x: 600, y: 160, depth: 1, fail: 2 (HE), term_link: 0, is_terminal: false, words: [] },
 11: { id: 11, label: 'EH', char: 'H', x: 630, y: 260, depth: 2, fail: 0, term_link: 0, is_terminal: false, words: [] },
 12: { id: 12, label: 'EHS', char: 'S', x: 660, y: 360, depth: 3, fail: 0, term_link: 0, is_terminal: true, words: ['EHS'] },
};

// Нормальные переходы бора
const TRIE_TRANSITIONS: { [key: number]: { [char: string]: number } } = {
 0: { H: 1, S: 7 },
 1: { E: 2, I: 5 },
 2: { R: 3 },
 3: { S: 4 },
 4: {},
 5: { S: 6 },
 6: {},
 7: {},
 8: {},
 9: {},
 10: { H: 11 },
 11: { S: 12 },
 12: {},
 13: { H: 14 },
 14: { S: 15 },
 15: {},
 16: { H: 17 },
 17: { S: 18 },
 18: {},
 19: { H: 20 },
 20: { S: 21 },
 21: {},
 22: { H: 23 },
 23: { S: 24 },
 24: {},
 25: { H: 26 },
 26: { S: 27 },
 27: {},
 28: { H: 29 },
 29: { S: 30 },
 30: {},
 31: { H: 32 },
 32: { S: 33 },
 33: {},
 34: { H: 35 },
 35: { S: 36 },
 36: {},
 37: { H: 38 },
 38: { S: 39 },
 39: {},
 40: { H: 41 },
 41: { S: 42 },
 42: {},
 43: { H: 44 },
 44: { S: 45 },
 45: {},
 46: { H: 47 },
 47: { S: 48 },
 48: {},
 49: { H: 50 },
 50: { S: 51 },
 51: {},
 52: { H: 53 },
 53: { S: 54 },
 54: {},
 55: { H: 56 },
 56: { S: 57 },
 57: {},
 58: { H: 59 },
 59: { S: 60 },
 60: {},
 61: { H: 62 },
 62: { S: 63 },
 63: {},
 64: { H: 65 },
 65: { S: 66 },
 66: {},
 67: { H: 68 },
 68: { S: 69 },
 69: {},
 70: { H: 71 },
 71: { S: 72 },
 72: {},
 73: { H: 74 },
 74: { S: 75 },
 75: {},
 76: { H: 77 },
 77: { S: 78 },
 78: {},
 79: { H: 80 },
 80: { S: 81 },
 81: {},
 82: { H: 83 },
 83: { S: 84 },
 84: {},
 85: { H: 86 },
 86: { S: 87 },
 87: {},
 88: { H: 89 },
 89: { S: 90 },
 90: {},
 91: { H: 92 },
 92: { S: 93 },
 93: {},
 94: { H: 95 },
 95: { S: 96 },
 96: {},
 97: { H: 98 },
 98: { S: 99 },
 99: {}
};

```

```

 }, [liveText]);
 useEffect(() => {
 if (liveI !== null) setCurrentIndex(Math.max(0, Math.min(liveI, textToScan.length)));
 }, [liveI, textToScan.length]);
 useEffect(() => {
 if (liveNode !== null && WATERFALL_NODES[liveNode]) {
 setCurrentNode(liveNode);
 }
 }, [liveNode]);

 const handleTextChange = (e: React.ChangeEvent<HTMLInputElement>) => {
 const clean = e.target.value.toUpperCase().replace(/[^a-zA-Z0-9_]/g, '');
 setTextToScan(clean);
 resetSearchSimulation();
 };

 const resetSearchSimulation = () => {
 setCurrentIndex(0);
 setCurrentNode(0);
 setIsSearchDone(false);
 setSearchLog('Симуляция сброшена. Лосось вернулся на старт');
 setJumpPath(null);
 setFoundMatches([]);
 setActiveSearchCodeLine(1);
 };

 const stepSearchSimulation = () => {
 if (currentIndex >= textToScan.length) {
 setIsSearchDone(true);
 setSearchLog('Мы проплыли весь текст! Лосось доволен');
 setActiveSearchCodeLine(4);
 return;
 }

 const char = textToScan[currentIndex];
 let curr = currentNode;
 let logMsg = `[Символ '${char}']`;

 let jumpType: 'normal' | 'fail' | null = null;
 void jumpType;

```

```

 });
 }
 temp = WATERFALL_NODES[temp].term_link;
}

if (matchedWordsForLog.length > 0) {
 logMsg += ` БИНГО! Найдено слово: ${matchedWords}
 setActiveSearchCodeLine(4);
}

setCurrentNode(curr);
setCurrentIndex(currentIndex + 1);
setSearchLog(logMsg);
if (currentMatches.length > 0) {
 setFoundMatches((prev) => [...prev, ...currentMat
}
};

// ПОЛНАЯ ПОШАГОВАЯ ИНФОРМАЦИЯ О ПОСТРОЕНИИ ДЕРЕВА (В
const TRAINING_STEPS_INFO = [
 {
 targetNodeId: 0,
 title: 'Шаг 0 (Depth 0): Вершина #0 (ROOT)',
 desc: 'Начало алгоритма BFS. Корень (ROOT) не обра
 рние вершины сразу инициализируются с fail = RO
 codeLine: 1,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'ø',
 checkingBranchesFrom: null,
 },
 {
 targetNodeId: 1,
 title: 'Шаг 1 (Depth 1): Вершина #1 (H)',
 desc: 'Мы обрабатываем первую вершину из очереди -
 просто не существует. fail[H] автоматически нап
 codeLine: 5,
 scoutPos: 0,
 },

```

```

 targetNodeId: 8,
 title: 'Шаг 5 (Depth 2): Вершина #8 (SH) — КАК СТ
 desc: ' ВАЖНЫЙ ВОПРОС! Обрабатываем #8 (SH) на D
). Рыба плавает в ROOT и ищет ветку "H". У корня
 суффикс "H"!',
 codeLine: 4,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'H',
 checkingBranchesFrom: 0,
},
{
 targetNodeId: 3,
 title: 'Шаг 6 (Depth 3): Вершина #3 (HER)',
 desc: 'Переходим на Depth 3 к #3 (HER). Родитель
 е подходят. fail[HER] = ROOT.',
 codeLine: 3,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'R',
 checkingBranchesFrom: 0,
},
{
 targetNodeId: 6,
 title: 'Шаг 7 (Depth 3): Вершина #6 (HIS)',
 desc: 'Обрабатываем #6 (HIS) на Depth 3. Родитель
 но ветка "S" → #7 (S) совпадает! fail[HIS] = #7
 codeLine: 4,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'S',
 checkingBranchesFrom: 0,
},
{
 targetNodeId: 9,
 title: 'Шаг 8 (Depth 3): Вершина #9 (SHE) — КАК СТ
 desc: ' МАГИЯ АХО-КОРАСИК! Обрабатываем #9 (SHE)
 ба плавает в #1 (H) и ищет ветку "E". У вершины

```

```

 {/* Переключатель режимов */}
 <div className="bg-slate-900 p-1.5 rounded-xl b
 <button
 onClick={() => setActiveMode('training')}
 className={`flex items-center space-x-1.5 p
 activeMode === 'training'
 ? 'bg-indigo-600 text-white shadow-lg sl
 : 'text-slate-400 hover:text-white'
 `}
 >
 <GitBranch className="w-4 h-4" />
 Режим 1: Полное обучение (BFS от корн
 </button>
 <button
 onClick={() => setActiveMode('search')}
 className={`flex items-center space-x-1.5 p
 activeMode === 'search'
 ? 'bg-blue-600 text-white shadow-lg sha
 : 'text-slate-400 hover:text-white'
 `}
 >
 <Play className="w-4 h-4" />
 Режим 2: Поиск в тексте
 </button>
 </div>
</div>

```

```

{/* ПАНЕЛЬ УПРАВЛЕНИЯ В ЗАВИСИМОСТИ ОТ РЕЖИМА */}
{activeMode === 'training' ? {
 /* ПАНЕЛЬ РЕЖИМА ОБУЧЕНИЯ */
 <div className="grid grid-cols-1 lg:grid-cols-3
 {/* Управление шагами обучения */}
 <div className="bg-slate-900/90 p-5 rounded-x
 <div>
 <h5 className="text-indigo-300 font-bold
 Полный обход в ширину (В
 </h5>

```

```

 {trainStep === TRAINING_STEPS_INFO.length
 </button>
 </div>
</div>

/* Блок с кодом построения fail-ссылки и опи
<div className="lg:col-span-2 bg-slate-900/90
 >
 <div>
 <h5 className="text-white font-bold mb-3
 <span className="flex items-center gap-
 Код BFS построения fail

 <span className="text-xs bg-indigo-950
 Вершина: #{targetTrainingNode.id} ({t

 </h5>
 <div className="bg-slate-950 p-4 rounded-
 <div className={`p-1.5 rounded flex ite
 -l-4 border-indigo-500 text-indigo-300 font
 1. let u = trie[r][char]; // Те
 {currentTrainInfo.codeLine === 1 && <
 тут}
 </div>
 <div className={`p-1.5 rounded flex ite
 -l-4 border-indigo-400 text-indigo-200 font
 2. let v = fail[r]; // Подъем к
 ainInfo.inspectedParentFail}.label}}
 {currentTrainInfo.codeLine === 2 && <
 к родителю}
 </div>
 <div className={`p-1.5 rounded flex ite
 l-4 border-amber-500 text-amber-300 font-bo
 3. while (v !== root && trie[v]
 {currentTrainInfo.codeLine === 3 && <
 т, возврат в корень}
 </div>
 <div className={`p-1.5 rounded flex ite

```

```

 type="text"
 value={textToScan}
 onChange={handleTextChange}
 maxLength={15}
 placeholder="ВВЕДИТЕ ТЕКСТ..."
 className="w-full bg-slate-800 border border-blue-500 focus:outline-none focus:border-blue-500"
 />
 </div>

 <div>
 <div className="mb-4 flex flex-wrap gap-1">

 {textToScan.split('').map((char, idx) =>
 <span
 key={idx}
 className={`w-7 h-8 flex items-center justify-center
 ${idx === currentIndex ? 'bg-blue-600 text-white shadow' :
 idx < currentIndex ? 'bg-emerald-950/60 text-emerald-500' :
 'bg-slate-800 text-slate-500'}`}
 >
 {char}

)}}

 ГOTOB0 ✓

 </div>

 <button
 onClick={stepSearchSimulation}
 disabled={isSearchDone || textToScan.length === 0}
 className={`w-full py-2.5 rounded-lg font-medium text-white`}
 >
 Поиск
 </button>
 </div>
)}
)

```

```

 4. if (is_terminal[curr]) report
 {activeSearchCodeLine === 4 && <span <
овпадение!}
 </div>
 </div>
</div>

<div>
 <h5 className="text-slate-300 font-bold m
 <AlertCircle className="w-3.5 h-3.5 tex
 </h5>
 <div className="bg-slate-800 p-3.5 rounded
tems-center">
 {searchLog}
 </div>
</div>
</div>
</div>
))

{/* SVG Анимация водопада */}
<div className="bg-gradient-to-b from-blue-950 vi
 -hidden shadow-2xl">
 {/* Водопадные фоновые эффекты */}
 <div className="absolute inset-0 opacity-10 bg-
 o-900 to-transparent pointer-events-none"></d
 <div className="absolute top-0 left-1/2 -transl
 <div className="w-8 bg-blue-400 h-full animat
 <div className="w-12 bg-blue-300 h-full anima
 <div className="w-6 bg-blue-500 h-full animat
 </div>

 <div className="flex items-center justify-betwe
 <h5 className="text-white font-bold text-base
 Ветвящийся синий водопад (Бо
 </h5>
 <div className="flex flex-wrap gap-4 text-xs"
 <span className="flex items-center gap-1 te

```



```

 /* Рендер прямых переходов (потоков воды)
 {Object.entries(TRIE_TRANSITIONS).map(([fromNode, toNode]) => {
 const fromNode = WATERFALL_NODES[Number(fromNode)];
 return Object.entries(children).map(([childNode, toNode]) => {
 const toNode = WATERFALL_NODES[toNode];

 // Подсветка в режиме поиска
 let isHighlighted = activeMode === 'search' || activeMode === 'normal';

 // Подсветка в режиме обучения (когда ищем)
 let isTrainingExamined = activeMode === 'train' || activeMode === 'normal';
 let isMatchBranch = isTrainingExamined && childNode === toNode;

 return (
 <g key={`edge-${fromNode.id}-${toNode.id}`} >
 /* Линия течения воды */
 <line
 x1={fromNode.x}
 y1={fromNode.y}
 x2={toNode.x}
 y2={toNode.y}
 stroke={isHighlighted ? '#3b82f6' : '#ccc'}
 strokeWidth={isHighlighted || isTrainingExamined ? 2 : 1}
 className={isHighlighted || isTrainingExamined ? 'highlighted' : ''}
 />
 /* Символ перехода */
 <circle cx={{fromNode.x + toNode.x} / 2} cy={{fromNode.y + toNode.y} / 2}
 r={10} fill={isTrainingExamined ? (isMatchBranch ? '#10b981' : '#f43f5b') : '#ccc'}
 />
 <text
 x={{(fromNode.x + toNode.x) / 2}}
 y={{(fromNode.y + toNode.y) / 2} + 15}
 fill={isTrainingExamined ? (isMatchBranch ? '#10b981' : '#f43f5b') : '#ccc'}
 fontSize="12"
 fontWeight="bold"
 textAnchor="middle"
 />
 </g>
);
 });
 });
);
}

```

```

 const cy = node.y + dy / 2 - 30;

 return (
 <g key={`fail-${node.id}`}>
 <path
 d={`M ${node.x} ${node.y} Q ${cx} $
 fill="none"
 stroke={isHighlighted ? '#f43f5e' :
 strokeWidth={isHighlighted || isJust
 strokeDasharray="6,6"
 className={isHighlighted || isJustE
 opacity={isHighlighted || isJustEst
 />
 {isJustEstablished && (
 <text x={cx} y={cy - 10} fill="#10b
 >
 fail[{node.label}] = {targetNode.
 </text>
)}
 </g>
);
 }
 }
}

```

```

 { /* Рендер вершин (бассейнов водопада) */ }
 {Object.values(WATERFALL_NODES).map((node) => {
 const isCurrent = activeFishPosNode.id === node.id;
 const isTargetChild = activeMode === 'target' && node.id === targetNode.id;
 const hasWords = node.words.length > 0;

```

```

 return (
 <g key={`node-${node.id}`} className={isCurrent ? 'active' : ''}>
 { /* Внешнее свечение для активной вершины */ }
 {isCurrent && (
 <circle cx={node.x} cy={node.y} r={node.radius} fill="none" stroke="#f43f5e" stroke-width={2} />
)}
 {isTargetChild && (
 <circle cx={node.x} cy={node.y} r={node.radius} fill="none" stroke="#f43f5e" stroke-width={2} />
)}

```

```

 <text x="0" y="3" fontSize="10" f
 *
 </text>
 </g>
 })
</g>
);
}}}
```

**{/\* ОДИН ПЛАВНЫЙ АНИМИРОВАННЫЙ ЭХО-ЛОСОСЬ \*/}**

```

<g>
 { /* Круг-подложка под рыбу */ }
 <circle
 cx={activeFishPosNode.x + 15}
 cy={activeFishPosNode.y - 25}
 r="18"
 fill={activeMode === 'training' ? '#8b5'
 stroke="#ffffff"
 strokeWidth="2"
 style={{ transition: 'cx 0.8s cubic-bez
 className="shadow-lg"
 />
 { /* Сама рыба */ }
 <text
 x={activeFishPosNode.x + 15}
 y={activeFishPosNode.y - 19}
 fontSize="18"
 textAnchor="middle"
 style={{ transition: 'x 0.8s cubic-bezi
 className="animate-float select-none"
 >
 {activeMode === 'training' ? ' ♂ ' :
 </text>
</g>

</svg>
</div>
</div>
```

---

ФАЙЛ 77/96: src/components/visualizers/PrefixSumViz.tsx

КАТЕГОРИЯ: Визуализаторы (вложенные) | СТРОК: 415 | РАЗМ: 10.5 КБ

---

```
import React, { useContext, useEffect, useMemo, useState } from "react";
import {
 emitVizDemo,
 useVizRuntime,
 vizArray,
 vizNumber,
 VizChapterContext,
} from "../../data/vizStepBus";
```

```
/**
```

```
 * Префиксные суммы: 1D и 2D.
```

```
 *
```

```
 * Главная фишка – наведение на число:
```

```
 * * в 1D наводим на P[i] – подсвечивается кусок массива
```

```
 * * в 1D наводим на a[i] – видно, в какие префиксы он входит
```

```
 * * в 2D наводим на ячейку – подсвечивается прямоугольник
```

```
 * а при выбранном запросе показывается формула включения-исключения
```

```
 */
```

```
const A1 = [3, 1, 4, 1, 5, 9, 2, 6];
```

```
const A2 = [
 [1, 2, 3, 4],
 [5, 6, 7, 8],
 [9, 1, 2, 3],
 [4, 5, 6, 7],
];
```

```
const cellBase =
```

```
 "flex items-center justify-center rounded-md font-mono text-sm";
```

```
 "w-[clamp(30px,9vw,46px)] h-[clamp(30px,9vw,46px)] text-center";
```

```

 []
);

 /** Что сейчас под курсором: префикс P[i] или элемент
const [hover, setHover] = useState<{ kind: "pref" | "a" }>({ kind: "pref" });
const [range, setRange] = useState<{ l: number; r: number }>({ l: 0, r: 0 });
 /** Снимок i/P из автоматически выполненного Python-кода
const runtime = useVizRuntime();
const vars = runtime?.variables;
const liveI = vizNumber(vars?.i);
const liveP = vizArray(vars?.P);
const hasLiveP = !!vars && Object.prototype.hasOwnProperty.call(vars, "P");
const compilerLinked = liveI !== null || hasLiveP;
 // i/P управляют ячейками напрямую; индекс Python-строки
const shownDriven = compilerLinked
 ? liveI !== null
 ? Math.max(0, Math.min(A1.length, Math.trunc(liveI)))
 : Math.max(0, Math.min(A1.length, (liveP?.length || 0)))
 : null;

const covered =
 hover?.kind === "pref"
 ? { from: 0, to: hover.i - 1 }
 : hover?.kind === "a"
 ? { from: hover.i, to: hover.i }
 : null;

const sum = pref[range.r + 1] - pref[range.l];

return (
 <div className="space-y-5">
 <div className="overflow-x-auto pb-1">
 <div className="inline-block min-w-full">
 {/** индексы */}
 <div className="flex gap-1.5 mb-1 pl-[52px]">
 {A1.map((_, i) => (
 <div key={i} className={` ${cellBase} !h-5`}>
 {i}
 </div>
))}
 </div>
 </div>
 </div>
 </div>
);

```

```

const active = hover?.kind === "pref" &&
const isL = i === range.l;
const isR = i === range.r + 1;
const liveValue = liveP?.[i];
const blank = hasLiveP && (liveValue ===
const masked = !hasLiveP && shownDriven != null
const isDrivenCur = shownDriven !== null
const shownValue = hasLiveP ? liveValue :
return (
 <div
 key={i}
 onMouseEnter={() => setHover({ kind:
 onMouseLeave={() => setHover(null)}
 className={`${cellBase} cursor-help $
 isDrivenCur
 ? "bg-emerald-600 text-white ring
 : blank
 ? "bg-slate-950 text-slate-600
 : masked
 ? "bg-slate-800/50 text-slate-7
 : active
 ? "bg-emerald-500 text-white
 : isR
 ? "bg-emerald-700 text-white
 : isL
 ? "bg-rose-700 text-white
 : "bg-slate-900 text-slate-
 }`}
 title={blank || masked ? `P[${i}] - n
 >
 {blank || masked ? "." : String(shown
 </div>
);
}}}
</div>
</div>
</div>

```

```

 Наведите курсор на любое число: y <b className=
 y <b className="text-indigo-400">a – ку
 </p>
 })
</div>

{/* Запрос */}
<div className="bg-slate-900 border border-indigo
 <div className="text-xs text-slate-400 mb-2">
 Запрос суммы на отрезке (кликайте по массиву
 </div>
 <p className="font-mono text-sm sm:text-base te
 sum(a[{range.l}..{range.r}]) = P[{range.r + 1
 {pref[range
 {pref[range.l
 {
 </p>
 <p className="text-[11px] text-slate-500 mt-1">
 Один вычитание вместо цикла: запрос за O(1) п
 </p>
</div>
</div>
});
};

/* ----- 2D -----
const Prefix2D: React.FC = () => {
 const n = A2.length;
 const m = A2[0].length;
 const runtime = useVizRuntime();
 const vars = runtime?.variables;
 const liveI = vizNumber(vars?.i);
 const liveJ = vizNumber(vars?.j);
 const liveS = vizArray(vars?.S);
 const hasLiveS = !!vars && Object.prototype.hasOwnProperty.call(
 vars, 'S');

 const P = useMemo(() => {
 const p = Array.from({ length: n + 1 }, () => Array

```

```

 if (i === rect.r2 + 1 && j === rect.c1) return "C";
 if (i === rect.r1 && j === rect.c1) return "D";
 return null;
 };

 return (
 <div className="space-y-5">
 <div className="grid gap-5 lg:grid-cols-2">
 {/* Исходная матрица */}
 <div className="overflow-x-auto">
 <div className="text-xs text-slate-400 mb-2">
 <div className="inline-block">
 {A2.map((row, i) => {
 <div key={i} className="flex gap-1.5 mb-1">
 {row.map((v, j) => {
 const inRect = i >= rect.r1 && i <= r
 return (
 <div
 key={j}
 onClick={() =>
 setRect((r) =>
 i < r.r1 || j < r.c1 ? { r1:
)
 }
 className={` ${cellBase} cursor-po
 inRect ? "bg-indigo-500 text-wh
 }`}
 >
 {v}
 </div>
)
 </div>
)
 </div>
 </div>
 </div>
 </div>
 </div>

 {/* Матрица префиксных сумм */}

```



```

 {corner && !isHover && (
 <span className="absolute -top-
 ht">
 {corner}

)}
 </div>
);
 }}}}
 </div>
}&}}
</div>
</div>
</div>

```

```

<div className="min-h-[54px] bg-slate-900 border
 {hover ? (
 <p className="text-slate-300">
 <b className="text-amber-400">
 $P[\{hover.i\}][\{hover.j\}] = \{P[hover.i][hov$
 {" "}
 – сумма всего прямоугольника от левого верх
 </p>
) : (
 <p className="text-slate-500">Наведите курсор
)}
</div>

```

```

<div className="bg-slate-900 border border-indigo
 <p className="font-mono text-sm sm:text-base te
 сумма = A<
 C + <s
 {
 </p>
 <p className="text-[11px] text-slate-500 mt-1">
 Вычли два «хвоста» и вернули дважды вычтенный
 </p>
</div>

```

```

 st1D[i][j] = Math.min(st1D[i][j - 1], st1D[i + (1 <
 }
}

```

// Data for 2D Matrix (Sparse Table 2D)

```

const val2D = [
 [45, 12, 88, 34, 11, 76, 23, 90],
 [67, 19, 44, 55, 33, 21, 65, 87],
 [14, 51, 99, 13, 22, 64, 43, 76],
 [89, 32, 54, 71, 15, 88, 29, 60],
 [25, 41, 16, 92, 9, 17, 56, 31],
 [59, 18, 77, 24, 61, 82, 35, 12],
 [73, 8, 38, 85, 47, 95, 19, 58],
 [39, 81, 62, 28, 51, 42, 85, 14]
];

```

```

const ST2D: number[][][][] = Array.from({length: 8}, () =>
 Array.from({length: 8}, () =>
 Array.from({length: 4}, () =>
 Array(4).fill(0)
)
)
);

```

```

for (let r = 0; r < 8; r++) {
 for (let c = 0; c < 8; c++) {
 ST2D[r][c][0][0] = val2D[r][c];
 }
}

```

```

for (let kx = 0; kx <= 3; kx++) {
 for (let ky = 0; ky <= 3; ky++) {
 if (kx === 0 && ky === 0) continue;
 for (let r = 0; r + (1 << kx) <= 8; r++) {
 for (let c = 0; c + (1 << ky) <= 8; c++) {
 if (kx > 0) {
 ST2D[r][c][kx][ky] = Math.min(
 ST2D[r][c][kx-1][ky],

```

```

 >
 2. Сворачивание 2D (Построение)
 </button>
 <button
 onClick={() => setTab("2d_query")}
 className={`px-3 sm:px-4 py-2 rounded-lg text-
 se-600 text-white" : "bg-slate-800 text-sla
 >
 3. Запрос 2D (Формула)
 </button>
 </div>

 <AnimatePresence mode="wait">
 {tab === "1d" ? (
 <Viz1D key="1d" />
) : tab === "2d_build" ? (
 <Viz2DBuild key="2d_build" />
) : (
 <Viz2DQuery key="2d_query" />
)}
 </AnimatePresence>
</div>
);
};

```

```

const Viz1D: React.FC = () => {
 const [step, setStep] = useState(0);
 const [hover, setHover] = useState<{ i: number; j: number}>();
 const runtime = useVizRuntime();
 const runtimeVars = runtime?.variables;
 const liveI = vizNumber(runtimeVars?.i);
 const liveJ = vizNumber(runtimeVars?.j);
 const liveSource = vizArray(runtimeVars?.a);
 const liveRows = vizArray(runtimeVars?.st);
 const hasLiveSource = !!runtimeVars && Object.prototype.hasOwnProperty.call(runtimeVars, 'a');
 const hasLiveTable = !!runtimeVars && Object.prototype.hasOwnProperty.call(runtimeVars, 'st');
 const compilerLinked = liveI !== null || liveJ !== null;
 const shownSource = hasLiveSource

```

```

 disabled={step === 0}
 >
 <ChevronLeft size={20} />
 </button>
 <button
 onClick={() => setStep(Math.min(maxStep, step + 1))}
 className="p-2 bg-indigo-600 hover:bg-indigo-500"
 disabled={step === maxStep}
 >
 <ChevronRight size={20} />
 </button>
 <button
 onClick={() => setStep(0)}
 className="p-2 bg-slate-800 hover:bg-slate-700"
 >
 <RotateCcw size={20} />
 </button>
 </div>
 </div>

 <div className="bg-slate-900 rounded-xl border border-slate-800" >
 <div className="text-[11px] uppercase tracking-wider" >
 <div className="flex gap-1 sm:gap-1.5 overflow-x-hidden" >
 {shownSource.map((v, idx) => {
 const inCover = !!covered && idx >= covered
 const display = v === undefined || v === null
 ? "."
 : typeof v === "object"
 ? JSON.stringify(v)
 : String(v);
 return (
 <div key={idx} className="flex flex-col items-center justify-center" >
 <div
 className={`flex items-center justify-center w-[44px] h-[clamp(26px,8vw,44px)] text-[clamp(10px,1.2em,14px)] ${
 inCover ? "bg-indigo-500 text-white" : "bg-slate-800 text-slate-300"
 }`}
 >

```

```

 rap">
 2^0 (L=1)
 </div>
 <div className="text-center font-bold text-
 rap">
 2^1 (L=2)
 </div>
 <div className="text-center font-bold text-
 rap">
 2^2 (L=4)
 </div>
 <div className="text-center font-bold text-
 rap">
 2^3 (L=8)
 </div>

```

```

{/* Matrix */}
{Array.from({ length: N }).map((_, i) => (
 <React.Fragment key={i}>
 <div className="text-slate-500 font-mon
 {i}
 </div>
 {Array.from({ length: LOG }).map((_, j) => {
 const isValid = i + (1 << j) <= N;
 const liveRow = vizArray(liveRows?.[i]
 const liveValue = liveRow?.[j];
 // Уровень j=0 – это исходный a[i], о
 // очищаемого рабочего массива и не д
 const sourceValue = j === 0 ? liveSou
 const runtimeValue = liveValue ?? sou
 const runtimeHasValue = runtimeValue
 const manualVisible =
 isValid &&
 (j === 0 || computeSteps.findIndex(
 const isVisible = compilerLinked ? ru

 let isActive = !!liveTarget && liveTa
 let isSrc1 = false;

```

```

 scale: compilerLinked || is
 }}
 className={`w-[clamp(30px,9vw,
clamp(11px,3vw,17px))] font-bold transition-
 hover && hover.i === i && h
 ? "bg-sky-500 text-white
 : isActive
 ? "bg-indigo-500 text-whi
 : isSrc1
 ? "bg-emerald-500 text-r
 : isSrc2
 ? "bg-amber-500 text-r
 : isVisible
 ? "bg-slate-800 tex
 : compilerLinked
 ? "bg-slate-950 t
 : "bg-transparent
 `}
 title={compilerLinked ? `st[$
>
 {displayValue}
</motion.div>
)}

```

```

{/* Arrows visualization */}
{(isSrc1 || isSrc2) && (
 <motion.svg
 initial={{ opacity: 0 }}
 animate={{ opacity: 1 }}
 className="absolute pointer-e
 style={{
 width: 100,
 height: isSrc2 ? 40 + (1 <<
 right: -50,
 top: 24,
 overflow: "visible",
 }}
 >

```

```

 ST[{c.i}][{c.j - 1}]

 ,
 <span className="text-amber-400 font-bo
 ST[{c.i + half}][{c.j - 1}]

) → min(
 {stl

 {stlD[c.i + half][c.j - 1]}

) ={" "}
 <span className="text-indigo-400 font-b
 {stlD[c.i][c.j]}

 </p>
);
 })(
) : (
 <p className="text-slate-500 italic">
 Нажмите далее, чтобы увидеть, как блоки сво
 </p>
)}
</div>
</div>
);
};

```

```

const Viz2DBuild: React.FC = () => {
 const [k, setK] = useState(1);
 // Пока пользователь сам не выбрал уровень, вкладки с
 // после клика – локальный выбор главнее (Python не «
 const [userPicked, setUserPicked] = useState(false);
 const [hover, setHover] = useState<{ r: number; c: num
 const [locked, setLocked] = useState<{ r: number; c: n
 const runtime = useVizRuntime();
 const vars = runtime?.variables;
 const liveK = vizNumber(vars?.k);

```

```

 <button
 key={step}
 onClick={() => { setK(step); setUserPicked(
 className={`px-4 py-2 rounded-md text-sm
 'text-slate-400 hover:text-white hover:bg-
 >
 {step === 0 ? "Оригинал (1x1)" : `Шаг ${s
 </button>
)}}
 </div>
 {userPicked && (
 <button
 onClick={() => { setUserPicked(false); setK
 className="mb-4 text-[11px] font-bold px-3
 hite hover:border-indigo-500 transition-col
 title="Снова следовать за переменной k из P
 >
 ↪ снова вести уровень из Python (k = {python
 </button>
)}

 <div className="flex flex-col items-center gap-
 scrollbar">
 {[...Array(shownK + 1)].map((_, i) => shownK
 const stepL = 1 << step;
 const stepSize = 8 - stepL + 1;
 const isCurr = step === shownK;

 return (
 <React.Fragment key={step}>
 {idx > 0 && <div className="text-slate
 <div className={`flex flex-col items-c
 <h4 className={`font-bold mb-3 flex
 }>
 Матрица блоков {stepL}x{step
 <span className={`text-[10px] font
 rder-indigo-800/40 text-indigo-300' : 'bg-s
 </h4>

```



```

const prevL = 1 << (shownK
const isTop = r < activeCel
const isLeft = c < activeCe

if (isTop && isLeft) {
 highlightClass = 'bg-ros
} else if (isTop && !isLeft
 highlightClass = 'bg-blur
} else if (!isTop && isLeft
 highlightClass = 'bg-eme
} else {
 highlightClass = 'bg-amb
}
zIndex = 'z-10';
}
}

return (
 <div
 key={idx}
 onMouseEnter={() => { if(is
 onMouseLeave={() => { if(is
 onClick={() => {
 if(isCurr) {
 if(locked && locked.
 else setLocked({r, c
 }
 }}
 className={`flex items-centr
sCurr ? 'w-[clamp(24px,6.5vw,38px)] h-[clamp
clamp(20px,5.2vw,30px)] text-[clamp(8px,2vw
 title={hasLiveTable ? `st2[
 >
 {isBlank ? "•" : String(shor
 </div>
);
 }}}
</div>

```

```

 </div>
) : (
 <div className="flex flex-col gap-4 animate-
 <div className="bg-[#0a0fle] rounded-xl
ate-300 shadow-inner overflow-x-auto">
 // Те
 int<
="text-amber-300">1}

 // Кв
 ST</
 <span className="text-rose
e="text-amber-300">1], <sp
 <span className="text-blue
ext-amber-300">1][k-<span className=
pan>

 <span className="text-emer
="text-amber-300">1][k-<span classNa
span>

 <span className="text-ambe
white">c + pL][k-<span className="te
ame="text-slate-500">// Правый Низ<b
 {'}'}));
 </div>

 {activeCell ? (
 <div className="bg-slate-950 p-5 round
.2)] mt-2 transition-all">
 <p className="text-slate-300 mb-3 t
 Пример для ячейки:
 {locked && <span className="text
ded border border-rose-500/20"><Lock size={
 </p>
 <p className="text-indigo-300 borde
 <span className="font-bold text-w
 }][{shownK}][{shownK}]

 </p>

```

```

 <p className="mt-4 text-[12px] font
 список матриц (слева) вниз, чтобы увидеть,
 </div>
) : {
 <div className="bg-slate-950/40 border
 mt-2 flex items-center justify-center anim
 Наведите курсор на матрицу {L}x{L}
 </div>
)}
 </div>
 </div>
</div>
);
};

```

```

const Viz2DQuery: React.FC = () => {
 const [r1, setR1] = useState(1);
 const [c1, setC1] = useState(1);
 const [r2, setR2] = useState(5);
 const [c2, setC2] = useState(6);
 const runtime = useVizRuntime();

 // Координаты запроса – те же захардкоженные имена,
 useEffect(() => {
 const vars = runtime?.variables;
 const nr1 = vizNumber(vars?.r1);
 const nc1 = vizNumber(vars?.c1);
 const nr2 = vizNumber(vars?.r2);
 const nc2 = vizNumber(vars?.c2);
 if (nr1 !== null) setR1(Math.max(0, Math.min(7, M
 if (nc1 !== null) setC1(Math.max(0, Math.min(7, M
 if (nr2 !== null) setR2(Math.max(0, Math.min(7, M
 if (nc2 !== null) setC2(Math.max(0, Math.min(7, M
 }, [runtime]);

 const H = Math.max(1, r2 - r1 + 1);

```

```

 left: `${(c1 / 8) * 100}%`,
 marginTop: '8px',
 marginLeft: '8px',
 height: `calc(' + (H/8*100) +`
 width: `calc(' + (W/8*100) +`
 }}
 />

 {/* Показываем 4 угла в самой матрице */}
 <div className="absolute pointer-events-none shadow-[0_0_10px_#f43f5e]" style={{ top: `calc(${(Ly / 8) * 100}% - 16px)`, height: `calc(${(Lx / 8) * 100}% - 16px)`}}>
 <div className="absolute pointer-events-none shadow-[0_0_10px_#3b82f6]" style={{ top: `calc(${(Ly / 8) * 100}% - 16px)`, height: `calc(${(Lx / 8) * 100}% - 16px)`}}>
 <div className="absolute pointer-events-none shadow-[0_0_10px_#10b981]" style={{ top: `calc(${(Ly / 8) * 100}% - 16px)`, height: `calc(${(Lx / 8) * 100}% - 16px)`}}>
 <div className="absolute pointer-events-none shadow-[0_0_10px_#f59e0b]" style={{ top: `calc(${(Ly / 8) * 100}% - 16px)`, height: `calc(${(Lx / 8) * 100}% - 16px)`}}>
 </div>
 </div>
 </div>
 </div>
 </div>

 <div className="bg-slate-950 p-4 rounded-lg border border-slate-800 shadow-[inset_0_2px_10px_rgba(0,0,0,0.5)]">
 <label className="flex items-center justify-between gap-2">
 Степень размытия
 <div className="flex gap-2">
 <input type="number" min={1} max={10} value={1}>
 <input type="number" min={1} max={10} value={1}>
 </div>
 </label>
 <label className="flex items-center justify-between gap-2">
 Глубина размытия
 <div className="flex gap-2">
 <input type="number" min={1} max={10} value={1}>
 <input type="number" min={1} max={10} value={1}>
 </div>
 </label>
 </div>

```

```

В запроса)

 <span className="text-rose
r1][<span className="text-white bg-r

 <span className="text-slate
во, чтобы правый край уперся в c2)<b
 <span className="text-blue
r1][<span className="text-white bg-b

 <span className="text-slate
px, чтобы нижний край уперся в r2)<b
 <span className="text-emerg
nded">r2 - Lx + 1][<span className="

 <span className="text-slate
и влево от c2)

 <span className="text-amber
">r2 - Lx + 1][<span className="text
{'}}')});
</div>

<div className="w-full flex justify-cent
<div className="flex flex-col items-cent
<h4 className="text-slate-400 font-bo
 Откуда берутся эти 4 числа: матрица
border-slate-700 shadow-sm">ST[][][{{kx}}][{k
</h4>
 <div className="grid gap-[2px] p-2.5
Columns: `repeat(${matCols}, 1fr)`}}>
 {Array.from({length: matRows * ma
 const r = Math.floor(idx / ma
 const c = idx % matCols;
 let isTL = r === r1 && c ===
 let isTR = r === r1 && c ===
 let isBL = r === r2 - Lx + 1
 let isBR = r === r2 - Lx + 1

 let color = "bg-slate-800 tex

```

```

 + 1][c1][kx][ky], ST2D[r2 - Lx + 1][c2 - Ly
 </div>
 </div>

 </div>
</div>
)
}
```

=====

## РАЗДЕЛ: HTML-РАЗМЕТКА (LAYOUT)

=====

-----

ФАЙЛ 79/96: src/layout/footer.html

КАТЕГОРИЯ: HTML-разметка (layout) | СТРОВОК: 137 | РАЗМЕР

-----

```
 </div>
 </main>
</div>

<script>
 function setMode(mode) {
 document.getElementById('btn-mode-normal').cla
 ? 'px-3 py-1 rounded text-sm font-bold l
 : 'px-3 py-1 rounded text-sm font-bold

 document.getElementById('btn-mode-wtf').cla
 ? 'px-3 py-1 rounded text-sm font-bold l
 : 'px-3 py-1 rounded text-sm font-bold

 const aside = document.querySelector('aside
 const headerMain = document.querySelector('
 const questionsContainer = document.getElem
 const wtfContainer = document.getElementById
```

```

 document.getElementById('btn-theme-light').addEventListener('click', () => {
 document.body.classList.remove('text-white transition-colors');
 document.getElementById('btn-theme-notebook').classList.add('text-white transition-colors');
 });

// Initialize theme and mode
setTheme('dark');

// Setup intersection observer for TOC highlighting
document.addEventListener('DOMContentLoaded', () => {
 const mobileMenuBtn = document.getElementById('mobile-menu-btn');
 const mobileDrawer = document.getElementById('mobile-drawer');
 const closeDrawerBtn = document.getElementById('close-drawer');
 const drawerOverlay = document.getElementById('drawer-overlay');

 function toggleDrawer() {
 const isClosed = mobileDrawer.classList.contains('closed');
 if (isClosed) {
 mobileDrawer.classList.remove('closed');
 if (drawerOverlay) {
 drawerOverlay.classList.remove('hidden');
 // Timeout allows display:block to take effect
 setTimeout(() => drawerOverlay.classList.remove('hidden'), 10);
 }
 document.body.style.overflow = "hidden";
 } else {
 mobileDrawer.classList.add('closed');
 if (drawerOverlay) {
 drawerOverlay.classList.add('hidden');
 setTimeout(() => drawerOverlay.classList.add('hidden'), 10);
 }
 document.body.style.overflow = "";
 }
 }

 mobileMenuBtn.addEventListener('click', toggleDrawer);
 closeDrawerBtn.addEventListener('click', toggleDrawer);
 drawerOverlay.addEventListener('click', toggleDrawer);
});

```

```
 });
 }, observerOptions);

 document.querySelectorAll('section[id^="que
 observer.observe(section);
 });
});
</script>
</body>
</html>
```

---

ФАЙЛ 80/96: src/layout/header.html

КАТЕГОРИЯ: HTML-разметка (layout) | СТРОВОК: 1169 | РАЗМЕР:

---

```
<!DOCTYPE html>
<html lang="ru" class="scroll-smooth">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width,
 <title>Пособие по алгебре: От пиццы к предикатам</t
 <script src="https://cdn.tailwindcss.com"></script>
 <script src="https://polyfill.io/v3/polyfill.min.js
 <script id="MathJax-script" async src="https://cdn.
 <script>
 window.MathJax = {
 tex: {
 inlineMath: [['$', '$'], ['\\(', '\\)']]
 displayMath: [['$$', '$$'], ['\\[', '\\
 }
 };
 </script>
 <style>
 @import url('https://fonts.googleapis.com/css2?
 @reference './src/index.css';
 .math-font { font-family: 'Fira Code', monospac
```



```

 -webkit-overflow-scrolling: touch;
 }
 mjx-container[display="true"]::-webkit-scrollbar {
 display: none;
 }

 @media (max-width: 640px) {
 html { font-size: 13px; }

 .uno-card { width: 40px; height: 60px; font-size: 1.2rem; }
 .uno-card::before, .uno-card::after { font-size: 1.2rem; }
 .uno-card::before { top: 1px; left: 2px; }
 .uno-card::after { bottom: 1px; right: 2px; }

 .uno-card.mini { width: 28px !important; height: 28px !important; font-size: 4px; }
 .uno-card.mini::before, .uno-card.mini::after { font-size: 4px; }

 .uno-equation .text-2xl { font-size: 1.2rem; }
 .uno-equation.gap-4 { gap: 0.5rem !important; }

 /* Allow horizontal scroll for equations in formula scroll */
 .formula-scroll { flex-wrap: nowrap !important; }
 }

 .uno-inner {
 position: absolute; width: 110%; height: 75%;
 border-radius: 50%; transform: rotate(-30deg);
 box-shadow: inset 0 0 5px rgba(0,0,0,0.3);
 }

 .uno-val { position: relative; z-index: 2; }
 .card-red { background-color: #e52521; } .card-blue { background-color: #004dcf; } .card-green { background-color: #339e35; } .card-yellow { background-color: #ffc400; } .card-black { background-color: #222; }
 .card-black::before, .card-black::after { text-align: center; }
 .info-block { @apply bg-slate-800/80 border-l-4

```

```

 }
 body.theme-notebook .text-slate-200, body.theme
 ant; }
 body.theme-notebook .text-slate-400 { color: #5
 body.theme-notebook .border-slate-600, body.them
 : #94a3b8 !important; }
 body.theme-notebook .font-mono { font-family: "
 body.theme-notebook #top-controls { background-

/* CREEPY BLINKING */
@keyframes creepy-blink {
 0%, 95%, 98%, 100% { transform: scaleY(1);
 96.5% { transform: scaleY(0.1); }
}
.creepy-eye {
 transform-origin: center;
 transform-box: fill-box;
 animation: creepy-blink 4s infinite;
}
.creepy-eye-alt {
 transform-origin: center;
 transform-box: fill-box;
 animation: creepy-blink 5.2s infinite;
 animation-delay: 1.5s;
}
.creepy-emoji {
 display: inline-block;
 transform-origin: center;
 animation: creepy-blink 3.8s infinite;
}
</style>
</head>
<body class="bg-slate-900 text-slate-200 font-sans lead
 <div class="sticky top-0 z-50 w-full bg-slate-950/9
 center items-center gap-4 sm:gap-8 transition-col
 <div class="flex items-center gap-3">

```

```

</marker>
<marker id="arrow-lime" markerWidth="6" markerHeight="6"
 <path d="M0,0 L6,3 L0,6 Z" fill="#a3e63d" stroke="black" stroke-width="1" />
</marker>
<marker id="arrow-green" markerWidth="6" markerHeight="6"
 <path d="M0,0 L6,3 L0,6 Z" fill="#4ade80" stroke="black" stroke-width="1" />
</marker>

<!-- ЛЕГО БЛОКИ -->
<g id="lego-yellow">
 <rect x="0" y="0" width="40" height="20" fill="#f1c40f" />
 <rect x="6" y="-5" width="8" height="6" fill="#f1c40f" />
 <rect x="26" y="-5" width="8" height="6" fill="#f1c40f" />
 <line x1="2" y1="2" x2="38" y2="2" stroke="black" stroke-width="1" />
</g>
<g id="lego-red">
 <rect x="0" y="0" width="40" height="20" fill="#e74c3c" />
 <rect x="6" y="-5" width="8" height="6" fill="#e74c3c" />
 <rect x="26" y="-5" width="8" height="6" fill="#e74c3c" />
 <line x1="2" y1="2" x2="38" y2="2" stroke="black" stroke-width="1" />
</g>
<g id="lego-rem">
 <rect x="0" y="0" width="40" height="10" fill="#f1c40f" />
 <rect x="6" y="-5" width="8" height="6" fill="#f1c40f" />
 <rect x="26" y="-5" width="8" height="6" fill="#f1c40f" />
 <line x1="2" y1="2" x2="38" y2="2" stroke="black" stroke-width="1" />
</g>

<!-- МЯСОРУБКА -->
<g id="meat-grinder">
 <path d="M 10 40 L 15 20 Q 25 15 35 20 L 40 40" fill="black" stroke="black" stroke-width="1" />
 <rect x="5" y="40" width="40" height="20" fill="black" stroke="black" stroke-width="1" />
 <!-- Ручка -->
 <path d="M 45 50 Q 60 50 60 30 L 75 30" fill="black" stroke="black" stroke-width="1" />
 <circle cx="75" cy="30" r="4" fill="black" stroke="black" stroke-width="1" />
 <!-- Выход -->
 <path d="M 5 45 L -5 45 L -5 55 L 5 55" fill="black" stroke="black" stroke-width="1" />
 <circle cx="25" cy="50" r="15" fill="black" stroke="black" stroke-width="1" />
</g>

```

```
<button id="close-drawer-btn" class="
 <svg xmlns="http://www.w3.org/20
 <path stroke-linecap="round
 </svg>
</button>
</div>
<nav class="space-y-3 text-sm font-medium">

 <details class="group" open>
 <summary class="list-none cursor-pointer text-blue-500 pl-3 font-bold text-blue-300">
 1. Алгебраические структуры

 </summary>
 <div class="pl-6 space-y-2 text-sm">

 </div>
 </details>

 <details class="group">
 <summary class="list-none cursor-pointer text-indigo-500 pl-3 font-bold text-indigo-300">
 2. Комплексные числа

 </summary>
 <div class="pl-6 space-y-2 text-sm">

 </div>
 </details>

 <details class="group">
```

```


 </div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer text-lime-500 pl-3 font-bold text-lime-300">

 6. Делимость, НОД и Евклид

 </summary>
 <div class="pl-6 space-y-2 text-lime-300">

 </div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer text-pink-500 pl-3 font-bold text-pink-300">

 7. Взаимно простые числа

 </summary>
 <div class="pl-6 space-y-2 text-pink-300">


```

```

 <a href="#question-10" class=
der-rose-500 pl-3 font-bold text-rose-400">
 10. Делимость и Схема Г

 </summary>
 <div class="pl-6 space-y-2 text
 <a href="#q10-1" class="blo
 <a href="#q10-2" class="blo
 <a href="#q10-3" class="blo
a>
 <a href="#q10-4" class="blo
 <a href="#q10-5" class="blo
 </div>
</details>

<details class="group">
 <summary class="list-none curso
 <a href="#question-11" clas
der-cyan-500 pl-3 font-bold text-cyan-400">
 11. НОД и НОК многочлен

 </summary>
 <div class="pl-6 space-y-2 text
 <a href="#q11-1" class="blo
 <a href="#q11-2" class="blo
нственности НОД
 <a href="#q11-3" class="blo
ПИРАЛЬ)
 <a href="#q11-4" class="blo
вращается!)
 </div>
</details>

<details class="group">
 <summary class="list-none curso
 <a href="#question-12" clas
der-blue-500 pl-3 font-bold text-blue-400">
 12. Взаимно простые мно
```

```
 </div>
 </details>

 <details class="group">
 <summary class="list-none cursor-pointer text-cyan-500 pl-3 font-bold text-cyan-400">
 15. Производная. Критерий

 </summary>
 <div class="pl-6 mt-2 space-y-2">

 </div>
 </details>

 <details class="group">
 <summary class="list-none cursor-pointer text-red-500 pl-3 font-bold text-red-400">
 16. Неприводимые многоч.

 </summary>
 <div class="pl-6 mt-2 space-y-2">

 </div>
 </details>

 <div class="mt-4 mb-2 text-xs font-l
 <a href="#question-17" class="block
-500 pl-3 text-slate-400">
 <span class="font-bold text-tea

 <a href="#question-18" class="block
```

```

 // Scroll spy logic
 document.addEventListener('DOMContentLoaded', () => {
 const sections = document.querySelectorAll(
 "h2[id^='q5-'], div[id^='q6-'], div[id^='q7-'], div[id^='q8-'], div[id^='q9-'], div[id^='q10-'], div[id^='q11-'], div[id^='q12-'], div[id^='q13-'], div[id^='q14-'], div[id^='q15-']"
);
 let isScrolling = false;

 document.querySelectorAll('.group-item a').forEach(link => {
 link.addEventListener('click', () => {
 isScrolling = true;
 setTimeout(() => isScrolling = false, 300);
 });
 });

 const observer = new IntersectionObserver(entries => {
 if (isScrolling) return;

 let visibleId = null;
 entries.forEach(entry => {
 if (entry.isIntersecting) {
 visibleId = entry.target.id;
 }
 });

 // Highlight active link
 document.querySelectorAll('a').forEach(a => {
 a.style.fontWeight = 'normal';
 if (a.href.endsWith(visibleId)) {
 a.style.fontWeight = 'bold';
 }
 });

 // Open the parent details
 const correspondingLi = document.querySelector(`li[id^='${visibleId}']`);
 if (correspondingLi) {
 const details = correspondingLi.querySelector('details');
 if (details && !details.open) {
 details.open();
 }
 }

 document.querySelector(`div[id^='${visibleId}']`)?.scrollIntoView({
 behavior: 'smooth',
 block: 'center'
 });
 });
 });

```



```
box.style.borderRadius = '10px';
box.style.display = 'flex';
box.style.flexDirection = 'column';
box.style.color = '#fff';

const header = document.createElement('h2');
header.innerText = 'Экспорт в ' + fileName;
header.style.marginBottom = '10px';

const subHeader = document.createElement('h3');
subHeader.innerText = 'Из-за особенностей веб-студии в новой вкладке (через меню)';
subHeader.style.marginBottom = '10px';
subHeader.style.color = '#f87171';
subHeader.style.fontSize = '0.9em';

const textarea = document.createElement('div');
textarea.value = contentRaw;
textarea.style.flex = '1';
textarea.style.width = '100%';
textarea.style.color = '#000';
textarea.style.padding = '10px';
textarea.style.fontFamily = 'monospace';

const btnContainer = document.createElement('div');
btnContainer.style.display = 'flex';
btnContainer.style.gap = '10px';
btnContainer.style.marginTop = '10px';

const copyBtn = document.createElement('button');
copyBtn.innerText = 'Копировать';
copyBtn.style.padding = '10px';
copyBtn.style.backgroundColor = '#fff';
copyBtn.style.color = '#fff';
copyBtn.style.border = 'none';
copyBtn.style.borderRadius = '5px';
copyBtn.style.cursor = 'pointer';
copyBtn.onclick = () => {
```

```

 // Smart markdown parsing based
 clone.querySelectorAll('h2').fo
 clone.querySelectorAll('h3').fo
 clone.querySelectorAll('.analog
 clone.querySelectorAll('li').fo
 clone.querySelectorAll('svg').fo
 let caption = '';
 if (el.nextElementSibling &
 caption = el.nextElement
 }
 el.textContent = '\n\n[Илл
 });
 clone.querySelectorAll('.img-cap

let text = clone.innerText;
text = text.replace(/\n{3,}/g,

if (window !== window.top) {
 fallbackModal(text, 'Markdow
 return;
}
const blob = new Blob([text], {
const url = URL.createObjectURL
const link = document.createEle
link.href = url;
link.download = 'vyisshaya_algel
document.body.appendChild(link)
link.click();
document.body.removeChild(link)
}

function setTheme(theme) {

 const body = document.body;
 body.classList.remove('bg-slate

);

document.getElementById('theme-

```

```

body.bg-white .img-capt

/* Tweak card blocks */
body.bg-white .card-red

body.bg-white aside { b
#334155 !important; }
body.bg-white .text-whi

/* Ensure active links
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a:h

body.bg-white table tr.
body.bg-white table tr.
body.bg-white .text-tea
body.bg-white .text-ros
body.bg-white .text-eme
body.bg-white .text-cya
body.bg-white .text-amb
body.bg-white .text-fuch
body.bg-white .text-lim
body.bg-white .text-blur
body.bg-white .text-gre
body.bg-white .text-ind

```

```

`
} else if (theme === 'terminal'
body.classList.add('bg-blac
document.getElementById('th
style.innerHTML = `
body.bg-black .bg-slate

```

```

.bg-slate-950 {

```

```

 <button onclick="downloadHTML()"
t-sm font-bold flex items-center justify-center
 HTML
 </button>
 </div>

 <!-- Смена темы -->
 <div>
 <div class="text-xs text-slate-400">
 <div class="flex items-center gap-2">
 <button id="theme-dark" onclick="toggleTheme('dark')"
justify-center text-amber-300 hover:text-white
line-amber-500" title="Темная тема"> </button>
 <button id="theme-light" onclick="toggleTheme('light')"
r justify-center text-amber-50 hover:text-white
">* </button>
 <button id="theme-terminal" onclick="toggleTheme('terminal')"
-center justify-center text-green-400 hover:text-green-500
терминала"> </button>
 </div>
 </div>
 </div>
</div>

</div>
</aside>

<!-- Основной контент -->
<main class="flex-1 bg-slate-800 p-4 sm:p-8 md:p-12 lg:p-16"
-base min-w-0">
 <header class="text-center mb-12 border-b border-slate-700">
 <h1 class="text-4xl md:text-5xl font-bl
4">
 ВЫСШАЯ АЛГЕБРА
 </h1>
 <p class="text-slate-400 italic text-lg">
 Универсальное пособие
 </p>

 <!-- Print Table of Contents -->
 <div class="hidden print:block mt-8 text-slate-400">
 <table border="0">
 <tr>
 <td>Глава 1. Введение</td>
 <td>1</td>
 </tr>
 <tr>
 <td>Глава 2. Основы алгебры</td>
 <td>10</td>
 </tr>
 <tr>
 <td>Глава 3. Матрицы</td>
 <td>25</td>
 </tr>
 <tr>
 <td>Глава 4. Группы</td>
 <td>40</td>
 </tr>
 <tr>
 <td>Глава 5. Поля</td>
 <td>55</td>
 </tr>
 <tr>
 <td>Глава 6. Кольца</td>
 <td>70</td>
 </tr>
 <tr>
 <td>Глава 7. Модули</td>
 <td>85</td>
 </tr>
 <tr>
 <td>Глава 8. Идеалы</td>
 <td>100</td>
 </tr>
 <tr>
 <td>Глава 9. Факториальные кольца</td>
 <td>115</td>
 </tr>
 <tr>
 <td>Глава 10. Локальные кольца</td>
 <td>130</td>
 </tr>
 <tr>
 <td>Глава 11. Кольца вычетов</td>
 <td>145</td>
 </tr>
 <tr>
 <td>Глава 12. Кольца целых чисел</td>
 <td>160</td>
 </tr>
 <tr>
 <td>Глава 13. Кольца многочленов</td>
 <td>175</td>
 </tr>
 <tr>
 <td>Глава 14. Кольца дробей</td>
 <td>190</td>
 </tr>
 <tr>
 <td>Глава 15. Кольца целых чисел</td>
 <td>205</td>
 </tr>
 <tr>
 <td>Глава 16. Кольца целых чисел</td>
 <td>220</td>
 </tr>
 <tr>
 <td>Глава 17. Кольца целых чисел</td>
 <td>235</td>
 </tr>
 <tr>
 <td>Глава 18. Кольца целых чисел</td>
 <td>250</td>
 </tr>
 <tr>
 <td>Глава 19. Кольца целых чисел</td>
 <td>265</td>
 </tr>
 <tr>
 <td>Глава 20. Кольца целых чисел</td>
 <td>280</td>
 </tr>
 <tr>
 <td>Глава 21. Кольца целых чисел</td>
 <td>295</td>
 </tr>
 <tr>
 <td>Глава 22. Кольца целых чисел</td>
 <td>310</td>
 </tr>
 <tr>
 <td>Глава 23. Кольца целых чисел</td>
 <td>325</td>
 </tr>
 <tr>
 <td>Глава 24. Кольца целых чисел</td>
 <td>340</td>
 </tr>
 <tr>
 <td>Глава 25. Кольца целых чисел</td>
 <td>355</td>
 </tr>
 <tr>
 <td>Глава 26. Кольца целых чисел</td>
 <td>370</td>
 </tr>
 <tr>
 <td>Глава 27. Кольца целых чисел</td>
 <td>385</td>
 </tr>
 <tr>
 <td>Глава 28. Кольца целых чисел</td>
 <td>400</td>
 </tr>
 <tr>
 <td>Глава 29. Кольца целых чисел</td>
 <td>415</td>
 </tr>
 <tr>
 <td>Глава 30. Кольца целых чисел</td>
 <td>430</td>
 </tr>
 <tr>
 <td>Глава 31. Кольца целых чисел</td>
 <td>445</td>
 </tr>
 <tr>
 <td>Глава 32. Кольца целых чисел</td>
 <td>460</td>
 </tr>
 <tr>
 <td>Глава 33. Кольца целых чисел</td>
 <td>475</td>
 </tr>
 <tr>
 <td>Глава 34. Кольца целых чисел</td>
 <td>490</td>
 </tr>
 <tr>
 <td>Глава 35. Кольца целых чисел</td>
 <td>505</td>
 </tr>
 <tr>
 <td>Глава 36. Кольца целых чисел</td>
 <td>520</td>
 </tr>
 <tr>
 <td>Глава 37. Кольца целых чисел</td>
 <td>535</td>
 </tr>
 <tr>
 <td>Глава 38. Кольца целых чисел</td>
 <td>550</td>
 </tr>
 <tr>
 <td>Глава 39. Кольца целых чисел</td>
 <td>565</td>
 </tr>
 <tr>
 <td>Глава 40. Кольца целых чисел</td>
 <td>580</td>
 </tr>
 <tr>
 <td>Глава 41. Кольца целых чисел</td>
 <td>595</td>
 </tr>
 <tr>
 <td>Глава 42. Кольца целых чисел</td>
 <td>610</td>
 </tr>
 <tr>
 <td>Глава 43. Кольца целых чисел</td>
 <td>625</td>
 </tr>
 <tr>
 <td>Глава 44. Кольца целых чисел</td>
 <td>640</td>
 </tr>
 <tr>
 <td>Глава 45. Кольца целых чисел</td>
 <td>655</td>
 </tr>
 <tr>
 <td>Глава 46. Кольца целых чисел</td>
 <td>670</td>
 </tr>
 <tr>
 <td>Глава 47. Кольца целых чисел</td>
 <td>685</td>
 </tr>
 <tr>
 <td>Глава 48. Кольца целых чисел</td>
 <td>700</td>
 </tr>
 <tr>
 <td>Глава 49. Кольца целых чисел</td>
 <td>715</td>
 </tr>
 <tr>
 <td>Глава 50. Кольца целых чисел</td>
 <td>730</td>
 </tr>
 <tr>
 <td>Глава 51. Кольца целых чисел</td>
 <td>745</td>
 </tr>
 <tr>
 <td>Глава 52. Кольца целых чисел</td>
 <td>760</td>
 </tr>
 <tr>
 <td>Глава 53. Кольца целых чисел</td>
 <td>775</td>
 </tr>
 <tr>
 <td>Глава 54. Кольца целых чисел</td>
 <td>790</td>
 </tr>
 <tr>
 <td>Глава 55. Кольца целых чисел</td>
 <td>805</td>
 </tr>
 <tr>
 <td>Глава 56. Кольца целых чисел</td>
 <td>820</td>
 </tr>
 <tr>
 <td>Глава 57. Кольца целых чисел</td>
 <td>835</td>
 </tr>
 <tr>
 <td>Глава 58. Кольца целых чисел</td>
 <td>850</td>
 </tr>
 <tr>
 <td>Глава 59. Кольца целых чисел</td>
 <td>865</td>
 </tr>
 <tr>
 <td>Глава 60. Кольца целых чисел</td>
 <td>880</td>
 </tr>
 <tr>
 <td>Глава 61. Кольца целых чисел</td>
 <td>895</td>
 </tr>
 <tr>
 <td>Глава 62. Кольца целых чисел</td>
 <td>910</td>
 </tr>
 <tr>
 <td>Глава 63. Кольца целых чисел</td>
 <td>925</td>
 </tr>
 <tr>
 <td>Глава 64. Кольца целых чисел</td>
 <td>940</td>
 </tr>
 <tr>
 <td>Глава 65. Кольца целых чисел</td>
 <td>955</td>
 </tr>
 <tr>
 <td>Глава 66. Кольца целых чисел</td>
 <td>970</td>
 </tr>
 <tr>
 <td>Глава 67. Кольца целых чисел</td>
 <td>985</td>
 </tr>
 <tr>
 <td>Глава 68. Кольца целых чисел</td>
 <td>1000</td>
 </tr>
 </table>
 </div>
 </main>

```

```

 олько полиномов</div>
 <div class="bg-cyan-900/40 border-4 border-yellow-500 hover:bg-slate-700 transition" style="border: 4px solid yellow; background-color: cyan; color: yellow; padding: 10px; text-align: center;">
 сные числа</div>

 <!-- Row 1: База -->

 -4 border-yellow-500 hover:bg-slate-700 transition" style="border: 4px solid yellow; background-color: cyan; color: yellow; padding: 10px; text-align: center;">
 <div class="text-xs text-yellow" style="margin-bottom: 5px;">
 <div class="font-bold text-slate-900" style="margin-bottom: 5px;">

 -4 border-orange-500 hover:bg-slate-700 transition" style="border: 4px solid orange; background-color: cyan; color: orange; padding: 10px; text-align: center;">
 <div class="text-xs text-orange" style="margin-bottom: 5px;">
 <div class="font-bold text-slate-900" style="margin-bottom: 5px;">

 -4 border-cyan-500 hover:bg-slate-700 transition" style="border: 4px solid cyan; background-color: cyan; color: cyan; padding: 10px; text-align: center;">
 <div class="text-xs text-cyan-500" style="margin-bottom: 5px;">
 <div class="font-bold text-slate-900" style="margin-bottom: 5px;">

 <!-- Row 2: Базовые операции / Евклид -->

 -4 border-yellow-500 hover:bg-slate-700 transition" style="border: 4px solid yellow; background-color: cyan; color: yellow; padding: 10px; text-align: center;">
 <div class="text-xs text-yellow" style="margin-bottom: 5px;">
 <div class="font-bold text-slate-900" style="margin-bottom: 5px;">

 -1-4 border-orange-500 hover:bg-slate-700 transition" style="border: 4px solid orange; background-color: cyan; color: orange; padding: 10px; text-align: center;">
 <div class="text-xs text-orange" style="margin-bottom: 5px;">
 <div class="font-bold text-slate-900" style="margin-bottom: 5px;">

 -4 border-cyan-500 hover:bg-slate-700 transition" style="border: 4px solid cyan; background-color: cyan; color: cyan; padding: 10px; text-align: center;">
 <div class="text-xs text-cyan-500" style="margin-bottom: 5px;">
 <div class="font-bold text-slate-900" style="margin-bottom: 5px;">


```

```

 <a href="#question-4" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans
 <div class="text-xs text-cyan-500"
 <div class="font-bold text-slate-700"

 <!-- Row 6: Кратность -->
 <div class="hidden md:block"></div>
 <a href="#question-14" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t
 <div class="text-xs text-orange-500"
 <div class="font-bold text-slate-700"

 <div class="hidden md:block"></div>

 <!-- Row 7: Производная -->
 <div class="hidden md:block"></div>
 <a href="#question-15" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t
 <div class="text-xs text-orange-500"
 <div class="font-bold text-slate-700"

 <div class="hidden md:block"></div>

 <!-- КЛАСТЕР 3: НЕПРИВОДИМОСТЬ -->
 <div class="col-span-1 md:col-span-3"
b-2 border-b border-emerald-500/30 pb-2">Кл

 <a href="#question-17" onclick="setM
-1-4 border-teal-500 hover:bg-slate-700 tra
 <div class="text-xs text-teal-500"
 <div class="font-bold text-slate-700"

 <a href="#question-18" onclick="setM
-1-4 border-blue-500 hover:bg-slate-700 tra
 <div class="text-xs text-blue-500"
 <div class="font-bold text-slate-700"


```

```

 <svg class="w-8 h-8 text-red-500" data-bbox="110 110 190 190">
rg/2000/svg">

```

```

 <path stroke-linecap="round" data-bbox="110 110 190 190">
110 4m-6 8a2 2 0 100-4m0 4a2 2 0 110-4m0 4a2 2 0 100-4m0 4a2 2 0 110-4m0 4
</svg>

```

```

 </div>
 <h2 class="text-3xl pr-4 border-l-2 border-l-blue-500" data-bbox="110 110 190 190">
</div>

```

```

 <div class="prose prose-invert max-w-4xl" data-bbox="110 110 190 190">
 <p class="text-lg">
 Хватит тупить. Доказательств
 </p>
 <p>
 есь для того, чтобы жать на кнопки. Вот жес
 </p>

```

```

 <h2 class="text-2xl font-bold border-l-2 border-l-blue-500" data-bbox="110 110 190 190">
(Меметика)</h2>

```

```

 <!-- Прямое -->
 <div class="bg-slate-800/80 p-6" data-bbox="110 110 190 190">
 <h3 class="text-xl text-blue-400" data-bbox="110 110 190 190">
 <p class="text-sm text-slate-400" data-bbox="110 110 190 190">
 <ul class="list-disc pl-5" data-bbox="110 110 190 190">

k$).

ово.

 </div>

```

```

 <!-- От противного -->
 <div class="bg-slate-800/80 p-6" data-bbox="110 110 190 190">
 <h3 class="text-xl text-fuchsia-400" data-bbox="110 110 190 190">
 <p class="text-sm text-slate-400" data-bbox="110 110 190 190">
 <ul class="list-disc pl-5" data-bbox="110 110 190 190">

i>

```

```
<ul class="list-disc pl-5 s
 <span class="text-w
вверх по уравнениям).
 <span class="text-w
рху вниз, показывая, что $\Delta \vdots r_i$
 <span class="text-w
х. Твой d больше.

</div>
```

```
<h2 class="text-2xl font-bold b
оритмы экзекуции)</h2>
```

```
<div class="grid grid-cols-1 md
 <div class="bg-slate-800 p-
 <h4 class="text-red-400
 <p class="text-sm text-
 <p class="text-xs font-
со следующим. Если в конце 0 – пациент мертв
 </div>
```

```
 <div class="bg-slate-800 p-
 <h4 class="text-red-400
 <p class="text-sm text-
 <p class="text-xs font-
 c . Пока получается 0 – корень жив. Как то
корня минус один.</p>
 </div>
```

```
 <div class="bg-slate-800 p-
 <h4 class="text-red-400
 <p class="text-sm text-
 <p class="text-xs font-
ициентов (они делятся на p), кроме старше
зложить.</p>
 </div>
```



```

 <h4 class="text-cyan">
 <p class="text-sm">
есть сложение — есть ноль. Если есть умноже
 <code class="text-x
 </div>

 <li class="bg-slate-800/50">

 <div>
 <h4 class="text-cyan">
 <p class="text-sm">
авь их драться. Умножение всегда врывается
 <code class="text-x
 </div>

</div>
</section>
</div>

<div id="questions-container">
<!-- ===== ВОПРОС 1 =====
```

---

## РАЗДЕЛ: УТИЛИТЫ

---

---

ФАЙЛ 81/96: src/utils/cn.ts

КАТЕГОРИЯ: Утилиты | СТРОК: 7 | РАЗМЕР: 169 В

---

```
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";
```

```

 } catch {
 /* чужой домен – пропускаем */
 }
 }
 }
 return parts.join("\n");
 }

const EXTRA_CSS = `
/* – правки для автономного файла – */
html { color-scheme: dark; }
body { background: #020617; color: #e2e8f0; font-family:
a.term-hint { text-decoration: none; }
.export-shell { max-width: 62rem; margin: 0 auto; padding:
.export-head { border-bottom: 1px solid #1e293b; padding:
.export-note { background: #0f172a; border: 1px solid #
 border-radius: .5rem; padding: .85rem 1rem; font-size:
.export-note a { color: #a5b4fc; }
.export-gloss { margin-top: 3rem; border-top: 1px solid
.export-gloss h2 { color: #fff; font-size: 1.25rem; font
.export-term { background: #0f172a; border: 1px solid #
.export-term h3 { color: #c7d2fe; font-size: .95rem; font
.export-term p { margin: 0 0 .3rem; font-size: .85rem;
.export-term .formal { color: #94a3b8; font-size: .8rem
.export-term .cx { color: #6ee7b7; font-size: .78rem; font
.export-foot { margin-top: 3rem; border-top: 1px solid #
@media print {
 body { background: #fff; color: #0f172a; }
 .export-note, details { break-inside: avoid; }
 details { display: block; }
 details > div, details > p { display: block !important; }
 pre { white-space: pre-wrap; }
}
`;

const escapeHtml = (s: string) =>
 s.replace(/&/g, "&").replace(/</g, "<").replace(/>/g, ">");

```



```

const css = await collectCss();
const { body, termIds } = prepareBody(chapter.content);
const html = wrap({
 title: chapter.title,
 subtitle: chapter.category || "Универсальное пособие",
 css,
 body,
 gloss: glossarySection(termIds),
 origin: window.location.origin,
});
triggerDownload(html, safeName(chapter.title, chapter
}

```

/\*\* Выгрузить всё пособие одним файлом со сквозным оглавлением \*/

```

export async function downloadBookHtml(chapters: Chapter[]) {

```

```

 const css = await collectCss();
 const allTerms: string[] = [];
 const bodies: string[] = [];

 const toc = chapters
 .map((c) => `<li style="margin:.2rem 0"><a href="#${c.id}">${c.title}</a></li>`)
 .join("\n");

 chapters.forEach((c) => {
 const { body, termIds } = prepareBody(c.content);
 termIds.forEach((t) => {
 if (!allTerms.includes(t)) allTerms.push(t);
 });
 bodies.push(`<div id="ch-${escapeHtml(c.id)}" style="background-color:#f9f9f9; padding:1rem 0 0 1.5rem;">
 <p style="text-align:right;font-size:.72rem;margin:0 0 10px 0">
 ${c.title}
 </p>
 ${body}
 </div>`);
 });

```

```

 const body = `<nav id="toc" style="background:#0f172a;color:#fff;padding:1rem 0 0 1.5rem;">
 <h2 style="color:#fff;font-size:1.05rem;font-weight:800;margin:0 0 10px 0">Оглавление
 <ol style="margin:0;padding-left:1.1rem;font-size:.85rem;">
 ${toc}

 </nav>
 ${bodies.join("\n")}`;

```

```

const bundle = path.join(TMP, "content.bundle.mjs");

await build({
 entryPoints: [path.join(ROOT, "src/data/content.ts")],
 bundle: true,
 format: "esm",
 platform: "node",
 outfile: bundle,
 logLevel: "error",
});

const { chapters } = await import(`file://${bundle}`);

// Список интерактивов берём из реестра визуализаторов
const registry = fs.readFileSync(path.join(ROOT, "src/c
const registryBody = registry.slice(registry.indexOf("V
const vizIds = new Set([...registryBody.matchAll(/^\\s{2

const rows = chapters.map((c) => {
 const html = c.content ?? "";
 const analogies = (html.match(/Аналогия/gi) ?? []).length;
 return {
 id: c.id,
 title: c.title,
 num: Number.parseInt(c.title.match(/^(\d+)\.\/)?.[1]
 analogies,
 hasAnalogy: analogies > 0,
 inlineSvg: /<svg[\\s>]/i.test(html),
 image: /<img[\\s>]/i.test(html),
 interactive: vizIds.has(c.id),
 chars: html.length,
 };
});

const has2D = (r) => r.interactive || r.inlineSvg || r.
const gapBoth = rows.filter((r) => !r.hasAnalogy && !ha
const gapAnalogy = rows.filter((r) => !r.hasAnalogy && l
const gapViz = rows.filter((r) => r.hasAnalogy && !has2

```

## Универсальное пособие • полный исходный код

```

* в каждой ли теме находятся термины из глоссария.
*
* node scripts/audit-terms.mjs
*/
import { build } from "esbuild";
import path from "node:path";

const ROOT = process.cwd();
await build({
 entryPoints: ["src/data/content.ts", "src/data/glossa",
 bundle: true, format: "esm", platform: "node",
 outdir: "tmp/audit-terms", outExtension: { ".js": ".m",
 external: ["react", "react-dom", "motion", "lucide-re",
});

const { chapters } = await import(path.join(ROOT, "tmp/");
const { GLOSSARY_LABELS, GLOSSARY } = await import(path

const esc = (s) => s.replace(/[\.*\+\?\^\$\{\}\(\)|[\]\[\]]/g, "\\");
const re = new RegExp(
 `(?<![\\p{L}\\p{N}_-])(${GLOSSARY_LABELS.map((l) => e
 "giu"
);
const map = new Map(GLOSSARY_LABELS.map((l) => [l.label

// те же лимиты, что и в src/hooks/useTermHints.ts
const MAX_PER_SURFACE = 2;
const MAX_PER_TERM = 8;

const bad = [];
const used = new Set();
console.log("тем | подсв. | глава");
for (const c of chapters) {
 const text = c.content
 .replace(/<(script|style|code|pre)[\s\S]*?<\/\1>/gi
 .replace(/<[^\>]+>/g, " ");

 const perTerm = new Map();
```

```

import { build } from "esbuild";
import { pathToFileURL } from "node:url";
import path from "node:path";
import fs from "node:fs";

const root = path.resolve(import.meta.dirname, "../..");
const outfile = path.join(root, "tmp/guide-pdf-bundle.c

await build({
 entryPoints: [path.join(root, "scripts/export/guide-p
 outfile,
 bundle: true,
 platform: "node",
 format: "cjs",
 target: "node20",
 jsx: "automatic",
 loader: { ".jpg": "dataurl", ".png": "dataurl", ".css
 external: ["pdfkit", "svg-to-pdfkit", "pdfkit/js/pdfk
 logLevel: "warning",
});

const outArg = process.argv[2];
await import(pathToFileURL(outfile).href);

```

ФАЙЛ 86/96: scripts/export/build-pdf.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 64 | РАЗМ

```

/**
 * ШАГ 3: PNG -> PDF
 *
 * Склеивает отрендеренные страницы tmp/export-pages/pa
 * в единый документ public/export/full_code_all_pages.p
 */
import fs from "node:fs";
import path from "node:path";
```

```
 }

 doc.end();
 await new Promise((resolve, reject) => {
 stream.on("finish", resolve);
 stream.on("error", reject);
 });

 console.log("[3/3] png -> pdf");
 console.log(`      страниц: ${pages.length}`);
 console.log(`      выход:   ${path.relative(ROOT, PDF)}`);
 }

main().catch((err) => {
 console.error("Ошибка сборки PDF:", err.message);
 process.exit(1);
});
```

---

ФАЙЛ 87/96: scripts/export/collect-code.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 196 | РАЗМ: 10.5 КБ

---

```
/**
 * ШАГ 1: КОД -> TXT
 *
 * Собирает ВСЕГДА исходный код репозитория в один текстовый файл
 * public/export/full_code_all_pages.txt
 *
 * Список файлов берётся из git (чтобы ничего не забыть)
 * с фоллбэком на обход файловой системы, если git недоступен
 */
import fs from "node:fs";
import path from "node:path";
import { execFileSync } from "node:child_process";
import { ROOT, OUT_DIR, TXT_PATH, HR, SUBHR, ensureDir,
```



```

 "Прочее",
];

function listFiles() {
 let files;
 try {
 files = execFileSync("git", ["ls-files", "--cached"], {
 cwd: ROOT,
 encoding: "utf8",
 })
 .split("\n")
 .filter(Boolean);
 } catch {
 files = [];
 const walk = (dir) => {
 for (const e of fs.readdirSync(dir, { withFileType: true })) {
 const abs = path.join(dir, e.name);
 const rel = path.relative(ROOT, abs).split(path.sep);
 if (EXCLUDE_DIRS.some((d) => rel === d || rel.startsWith(d + path.sep))) continue;
 if (e.isDirectory()) walk(abs);
 else files.push(rel);
 }
 };
 walk(ROOT);
 }

 return files
 .filter((rel) => !EXCLUDE_DIRS.some((d) => rel.startsWith(d)))
 .filter((rel) => !EXCLUDE_FILES.has(path.basename(rel)))
 .filter((rel) => CODE_EXT.has(path.extname(rel)))
 .filter((rel) => fs.existsSync(path.join(ROOT, rel)))
 .sort((a, b) => {
 const ia = ORDER.indexOf(categoryOf(a));
 const ib = ORDER.indexOf(categoryOf(b));
 return ia !== ib ? ia - ib : a.localeCompare(b);
 });
}

```

```
push(`Дата генерации:                ${stamp}`);
push(`Файлов исходного кода:         ${files.length}`);
push(`Суммарный объём кода: ${humanSize(totalByteSize)});
push(`Глав/билетов в пособии:        ${chapters.length}`);
push();
```

```
push(HR);
push(" ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ");
push(HR);
chapters.forEach((c, i) => {
 push(`  ${String(i + 1).padStart(3, " ")} ${c.title}`);
});
push();
```

```
push(HR);
push(" ОГЛАВЛЕНИЕ: ФАЙЛЫ ИСХОДНОГО КОДА");
push(HR);
let current = null;
files.forEach((rel, i) => {
 const cat = categoryOf(rel);
 if (cat !== current) {
 current = cat;
 push();
 push(`  # ${cat}`);
 }
 push(`  ${String(i + 1).padStart(3, " ")} ${rel}`);
});
push();
push();
```

```
current = null;
files.forEach((rel, i) => {
 const cat = categoryOf(rel);
 if (cat !== current) {
 current = cat;
 push(HR);
 push(`РАЗДЕЛ: ${cat.toUpperCase()}`);
 push(HR);
 }
 push(`  ${String(i + 1).padStart(3, " ")} ${rel}`);
});
push();
```

```

*
* код -> full_code_all_pages.txt (collect-code.mjs)
* txt -> page-XXXX.png (render-pages.mjs)
* png -> full_code_all_pages.pdf (build-pdf.mjs)
*/
import fs from "node:fs";
import path from "node:path";
import { fileURLToPath } from "node:url";
import { execFileSync } from "node:child_process";

export const ROOT = path.resolve(path.dirname(fileURLToPath(import.meta.url)), "..");

/** Куда кладём финальные артефакты (эта папка отдаётся клиенту) */
export const OUT_DIR = path.join(ROOT, "public", "export");
/** Промежуточные PNG-страницы (в .gitignore, в CI уезжают в кэш) */
export const PAGES_DIR = path.join(ROOT, "tmp", "export");

export const TXT_PATH = path.join(OUT_DIR, "full_code_all_pages.txt");
export const PDF_PATH = path.join(OUT_DIR, "full_code_all_pages.pdf");

/** Параметры вёрстки страницы (подобраны под A4 @150dpi) */
export const PAGE = {
 /** DPI раstra. Можно переопределить: EXPORT_DENSITY=100 */
 density: Number(process.env.EXPORT_DENSITY ?? 150),
 /** 1-bit ч/б страницы весят в 2-4 раза меньше. EXPORT_GRAYSCALE=0 */
 monochrome: process.env.EXPORT_GRAYSCALE !== "1",
 size: "A4",
 font: "DejaVu-Sans-Mono",
 fontFile: "/usr/share/fonts/truetype/dejavu/DejaVuSansMono.ttf",
 pointsize: 8,
 /** Максимум символов в строке (далее – жёсткий перенос) */
 cols: 138,
 /** Строк содержимого на странице (+2 служебные: шапка и подвал) */
 rows: 76,
};

export const HR = "=".repeat(PAGE.cols);
export const SUBHR = "-".repeat(PAGE.cols);
```

```

* {renderToStaticMarkup}), из разметки берутся SVG-с
* • НИКАКОГО кода компилятора и интерфейсных кнопок.
*
* Результат: public/export/guide_all_topics.pdf, ~40–70
*/
```

```
import * as fs from "node:fs";
import * as path from "node:path";
import { createElement } from "react";
import { renderToStaticMarkup } from "react-dom/server";
import PDFDocument from "pdfkit";
// @ts-ignore - у пакета нет типов
import SVGtoPDF from "svg-to-pdfkit/source.js";
import { chapters } from "../../src/data/content";
import { VizChapterContext } from "../../src/data/vizSt
import { VIZ_REGISTRY, getViz } from "../../src/componen
```

```
// ----- шрифты -----
```

```
const FONT_DIRS = ["/usr/share/fonts/truetype/dejavu",
function findFont(file: string): string | null {
 for (const dir of FONT_DIRS) {
 const p = path.join(dir, file);
 if (fs.existsSync(p)) return p;
 }
 return null;
}
const F_REG = findFont("DejaVuSans.ttf");
const F_BOLD = findFont("DejaVuSans-Bold.ttf");
const F_MONO = findFont("DejaVuSansMono.ttf");
```

```
// ----- HTML → блоки текста -----
```

```
type Block =
 | { kind: "h"; level: 2 | 3 | 4; text: string }
 | { kind: "p"; text: string }
 | { kind: "li"; text: string }
 | { kind: "code"; text: string }
```

```

 let depth = 1;
 let m: RegExpExecArray | null;
 while ((m = re.exec(html))) {
 depth += m[0][1] === "/" ? -1 : 1;
 if (depth === 0) return m.index;
 }
 return html.length;
}

function findCloseAny(html: string, openEnd: number, tag: string) {
 const close = html.toLowerCase().indexOf(`</${tag}`);
 if (close < 0) return { end: html.length, inner: html };
 const inner = html.slice(openEnd, close);
 const end = html.indexOf(">", close) + 1;
 return { end, inner };
}

/** Разобрать HTML главы на учебные блоки (текст → 90%)
export function htmlToBlocks(html: string): Block[] {
 const blocks: Block[] = [];
 let i = 0;
 const skipTo = ["<pre", "<h2", "<h3", "<h4", "<summary"];
 while (i < html.length) {
 const lower = html.toLowerCase();
 const nexts = skipTo.map((t) => lower.indexOf(t, i));
 const liRe = /<li(\s[>]*)?>/gi;
 liRe.lastIndex = i;
 const liM = liRe.exec(lower);
 if (liM) nexts.push(liM.index);
 const trRe = /<tr(\s[>]*)?>/gi;
 trRe.lastIndex = i;
 const trM = trRe.exec(lower);
 if (trM) nexts.push(trM.index);
 const pRe = /<p(\s[>]*)?>/gi;
 pRe.lastIndex = i;
 const pM = pRe.exec(lower);
 if (pM) nexts.push(pM.index);
 if (!nexts.length) break;
 }
}

```

```

 continue;
 }
 if (lower.startsWith("<tr", at)) {
 const openEnd = html.indexOf(">", at) + 1;
 const close = html.toLowerCase().indexOf("</tr>",
 const rowHtml = html.slice(openEnd, close < 0 ? h
 const cells: string[] = [];
 const tdRe = /<(t[dh])(\s[^\>]*)?>([\s\S]*?)<\/\1>
 let tm: RegExpExecArray | null;
 while ((tm = tdRe.exec(rowHtml))) cells.push(stri
 if (cells.some(Boolean)) blocks.push({ kind: "tab
 i = close < 0 ? html.length : close + 5;
 continue;
 }
 // <p>
 const openEnd = html.indexOf(">", at) + 1;
 const close = html.toLowerCase().indexOf("</p>", op
 const text = stripTags(html.slice(openEnd, close < 0
 if (text) blocks.push({ kind: "p", text });
 i = close < 0 ? html.length : close + 4;
}
return blocks;
}

```

// ————— SSR визуализаций → SVG —————

```

interface VizArt {
 svgs: string[];
 images: string[]; // dataURL
 grids: string[][]; // текстовые матрицы (div-таблицы
 error?: string;
}

```

```

/** Отрендерить визуализатор в статику и вытащить картин
export function renderVizArt(chapterId: string): VizArt
 const entry = getViz(chapterId);
 if (!entry) return null;
 try {

```

```

* префиксные суммы). Собираем такие «таблицы» в текст
*/
function extractDivGrids(html: string): string[][] {
 const grids: string[][] = [];
 const rows: string[] = [];
 const rowMark = /<div class="[^"]*(?:flex gap-1\.5|gr
 let m: RegExpExecArray | null;
 while ((m = rowMark.exec(html))) {
 // сбалансированно находим конец строки-контейнера
 let depth = 1;
 let pos = m.index + m[0].length;
 while (depth > 0 && pos < html.length) {
 const no = html.indexOf("<div", pos);
 const nc = html.indexOf("</div>", pos);
 if (nc < 0) break;
 if (no >= 0 && no < nc) { depth++; pos = no + 4;
 else { depth--; pos = nc + 6; }
 }
 const inner = html.slice(m.index + m[0].length, pos);
 // теги → разделители: порядок значений сохраняется
 const cells = decodeEntities(inner.replace(/<[>]+>/g, ' '))
 .split(/\s+/)
 .filter((v) => v.length > 0 && v.length <= 6);
 if (cells.length >= 2 && cells.length <= 24) rows.push(cells);
 }
 // подряд идущие строки одинаковой длины = матрица; о
 let cur: string[] = [];
 const flush = () => {
 if (cur.length >= 2) grids.push(cur);
 else if (cur.length === 1 && cur[0].split("\u2009").length > 1)
 cur = [];
 };
 for (const r of rows) {
 if (cur.length && r.length !== cur[cur.length - 1].length)
 cur.push(r);
 }
 flush();
 return grids.slice(0, 6);
}

```

```
 if (this.y + need > this.pageBottom) this.doc.addPageBreak();
 }
 footerAll(skipCover = true) {
 const range = this.doc.bufferedPageRange();
 const oldBottom = this.doc.page.margins.bottom;
 for (let p = range.start; p < range.start + range.count; p++) {
 if (skipCover && p === range.start) continue;
 this.doc.switchToPage(p);
 this.doc.page.margins.bottom = 0;
 this.doc.font(F_REG ? "Body" : "Helvetica").fontSize(10)
 .text(String(p + 1), 0, A4[1] - 30, { width: A4[1] - 30 });
 this.doc.page.margins.bottom = oldBottom;
 }
 }
}
```

```
const CH_LINE = 13.4; // межстрочный в теле
const BODY_SIZE = 9.6;
```

```
function drawTextBlock(d: Doc, text: string, opts: { size: number, color: string, indent: number, gap: number, align: string, lineGap: number, width: number }) {
 const size = opts.size ?? BODY_SIZE;
 const lead = size * 1.42;
 d.ensureSpace(lead * 2);
 d.doc.fontSize(size)
 .fillColor(opts.color ?? "#1f2937")
 .text(text, M.left + (opts.indent ?? 0), d.y, {
 width: CONTENT_W - (opts.indent ?? 0),
 lineGap: lead - size,
 align: "left",
 });
 d.doc.moveDown((opts.gap ?? 2) / lead);
}
```

```
function drawCodeBlock(d: Doc, code: string) {
 const size = 7.7;
 const lead = size * 1.5;
 const lines = code.split("\n");
```



```

 d.doc.moveDown(2.2);
}

function drawGrid(d: Doc, rows: string[]) {
 const size = 8;
 const lead = size * 1.6;
 d.ensureSpace(rows.length * lead + 16);
 const top = d.y;
 d.monoFont().fontSize(size).fillColor("#111827");
 const cellW = Math.min(
 34,
 (CONTENT_W - 16) / Math.max(...rows.map((r) => r.sp
));
 rows.forEach((row, ri) => {
 const cells = row.split("\u2009");
 cells.forEach((cval, ci) => {
 const x = M.left + 6 + ci * cellW;
 const y = top + 4 + ri * lead;
 d.doc.save().rect(x - 2, y - 2, cellW - 4, lead -
 d.doc.text(cval, x + 2, y + lead * 0.12, { width:
 });
 });
 d.y = top + 8 + rows.length * lead + 8;
 d.doc.moveDown(2);
}

function drawSvg(d: Doc, svg: string) {
 const wAttr = Number(/width="([\d.]+)"/.exec(svg)?.[1]
 const hAttr = Number(/height="([\d.]+)"/.exec(svg)?.[1]
 const vb = /viewBox="([\d.\-\s]+)"/.exec(svg)?.[1]??.t
 const natW = wAttr || (vb ? vb[2] : 480);
 const nath = hAttr || (vb ? vb[3] : 320);
 const maxW = CONTENT_W - 10;
 const maxH = 330;
 let w = Math.min(natW, maxW);
 let h = (nath / natW) * w;
 if (h > maxH) { h = maxH; w = (natW / nath) * h; }
 const avail = d.pageBottom - d.y;

```

```
 return chapters.map((c) => ({ id: c.id, title: c.title }
}
```

```
function tocPagesNeeded(): number {
 return Math.max(1, Math.ceil(tocEntries().length / FR
}
```

```
interface BuildOpts {
 outPath: string;
 record: boolean;
 /** id → стартовая страница (для второго прохода) */
 pageMap?: Map<string, number>;
 frontPages: number;
}
```

```
function buildOne(opts: BuildOpts): Stats {
 return new Promise<Stats>((resolve, reject) => {
 const stream = opts.record ? fs.createWriteStream("
 const d = new Doc(stream as unknown as fs.WriteStre
 const doc = d.doc;
```

```
 const finish = (stats: Stats) => {
 d.footerAll(!opts.record);
 doc.end();
 stream.on("close", () => resolve(stats));
 stream.on("error", reject);
 };
};
```

```
if (!opts.record) {
 // — обложка (первая страница уже создана) —
 d.bodyFont();
 doc.fillColor("#312e81").fontSize(26).font(F_BOLD
 .text("Алгоритмы и структуры данных", M.left, 1
 doc.moveDown(0.6);
 doc.fontSize(15).fillColor("#4b5563").font(F_REG
 .text("Универсальное пособие: билеты 1–24", { w
 doc.moveDown(1.2);
 doc.fontSize(10.5).fillColor("#6b7280")
}
```

```

 d.y += 16.2;
 line++;
 });
 // добиваем до расчётного числа страниц содержания
 const used = Math.ceil(entries.length / FRONT_TOC);
 for (let p = used; p < opts.frontPages - 1; p++) {
 doc.addPage();
 if (!opts.record) console.log(`TOC: ${entries.length}
 }

 // — главы —
 const stats: Stats = { chapterStart: new Map(), total: 0 };

 chapters.forEach((ch, ci) => {
 const startPageOf = () => {
 // текущая страница = pages (0-based) + 1
 return doc.bufferedPageRange().start + doc.bufferedPageRange().end;
 };
 if (ci > 0) doc.addPage();
 const pageNow = opts.record ? doc.bufferedPageRange().end : startPageOf();
 d.chapterStart.set(ch.id, pageNow);
 if (!opts.record && ci < 3) console.log(`глава ${ci + 1}: ${ch.title}`);
 d.bodyFont();
 doc.y = M.top;
 d.y = M.top;

 // заголовок главы
 doc.fontSize(16).font(F_BOLD ? "BodyBold" : "Helvetica");
 d.text(ch.title, M.left, M.top, { width: CONTENT_WIDTH });
 if (ch.description) {
 doc.moveDown(0.2);
 doc.fontSize(9).font(F_REG ? "Body" : "Helvetica");
 d.text(ch.description, M.left, M.top + 10, { width: CONTENT_WIDTH });
 }
 doc.moveDown(0.6);
 d.y = doc.y;

 // текст главы
 const blocks = htmlToBlocks(ch.content);

```

```

 drawImage(d, img);
 drawn++;
 }
 for (const svg of art.svggs) {
 if (drawn >= 5) break;
 drawSvg(d, svg);
 drawn++;
 }
 for (const grid of art.grids) {
 if (drawn >= 7) break;
 drawGrid(d, grid);
 drawn++;
 }
} else if (art?.error) {
 // SSR упал – рисуем текстовую заглушку, PDF
}
}
});
});
stats.chapterStart = d.chapterStart;
stats.total = doc.bufferedPageRange().count;
finish(stats);
});
}

```

```

async function main() {
 const outPath = process.argv[2] || path.join(process.cwd(), 'out');
 if (!F_REG) throw new Error("Не найден шрифт DejaVuSans");
 const frontPages = 1 + tocPagesNeeded(); // обложка + оглавление

 // проход 1: считаем, на какой странице начинается каждая глава
 const stats = await buildOne({ outPath: "/dev/null",
 console.log(`Проход 1: ${stats.total} стр. тела`);

 // проход 2: обложка + содержание с номерами страниц
 await buildOne({
 outPath,
 record: false,
 });
}

```

```
import path from "node:path";
import os from "node:os";
import { execFile } from "node:child_process";
import { promisify } from "node:util";
import {
 ROOT, PAGES_DIR, TXT_PATH, PAGE, ensureDir, humanSize
} from "../common.mjs";
```

```
const execFileAsync = promisify(execFile);
```

```
/** Раскрываем табы и жёстко переносим длинные строки, */
```

```
function wrap(line) {
 const expanded = line.replace(/\t/g, " ").replace(
 if (expanded.length <= PAGE.cols) return [expanded];
 const parts = [];
 const indent = " ".repeat(Math.min((expanded.match(/^
 let rest = expanded;
 let first = true;
 while (rest.length > 0) {
 const width = first ? PAGE.cols : PAGE.cols - indent;
 parts.push((first ? "" : indent) + rest.slice(0, width));
 rest = rest.slice(width);
 first = false;
 }
 return parts;
}
```

```
function paginate(txt) {
 const source = txt.replace(/\r\n/g, "\n").split("\n");
 const pages = [];
 for (let i = 0; i < source.length; i += PAGE.rows) {
 pages.push(source.slice(i, i + PAGE.rows));
 }
 return pages;
}
```

```
async function main() {
 if (!fs.existsSync(TXT_PATH)) {
```

```

let done = 0;
let cursor = 0;

async function worker() {
 while (cursor < jobs.length) {
 const job = jobs[cursor++];
 await execFileAsync(im, args(job));
 fs.rmSync(job.txtFile, { force: true });
 done += 1;
 if (done % 25 === 0 || done === total) {
 process.stdout.write(` отрендерено ${done}.
 }
 }
}

await Promise.all(Array.from({ length: concurrency },
process.stdout.write("\n");

const missing = jobs.filter((j) => !fs.existsSync(j.p
if (missing.length) throw new Error(`Не отрендерены с

const bytes = jobs.reduce((s, j) => s + fs.statSync(j
console.log("[2/3] txt -> png");
console.log(`      страниц: ${total}`);
console.log(` выход: ${path.relative(ROOT, PAG
}

main().catch((err) => {
 console.error("Ошибка рендера страниц:", err.message)
 process.exit(1);
});

```

```
// 6. Краскал/Прим/Борувка: 9 вершин и вес 51
for (const id of ["mst-kruskal", "mst-boruvka"]) {
 const c = PAGE_SYNC[id].code ?? "";
 ok(/ABCDEFGH/ .test(c), `${id}: 9 вершин A..I`);
}
ok((PAGE_SYNC["mst-prima"].code ?? "").match(/"I":/g)? .
ok((PAGE_SYNC["mst-kruskal"].code ?? "").includes('(13,

// 7. prefix-sums-2d#2d: пометка 1-индексных клеток
const p2 = PAGE_SYNC["prefix-sums-2d#2d"].code ?? "";
ok(p2.includes("клетки 1-индексные"), "prefix#2d: комме

// 8. графы top-sort/scc/graph-dfs-bfs/graph-components
ok((PAGE_SYNC["top-sort"].code ?? "").includes("0: [1,
ok((PAGE_SYNC["scc-kosaraju"].code ?? "").includes("2:
 емо");
ok((PAGE_SYNC["graph-dfs-bfs"].code ?? "").includes('ad
ok((PAGE_SYNC["graph-components"].code ?? "").includes(

// 9. каждый шаблон непуст и у каждого id есть variable
for (const [id, sync] of Object.entries(PAGE_SYNC)) {
 if (sync.code) {
 ok(sync.code.trim().length > 0, `${id}: код непуст`
 ok(sync.variables.length > 0, `${id}: variables опи
 }
}
```

```
console.log(`\nBUNDLE SELFTEST: ${n} проверок, ${proces
```

---

ФАЙЛ 92/96: scripts/selftest/harness-dump.ts

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 14 | РАЗМ

---

```
// Прогон КАЖДОГО шаблона через настоящий харнесс дебаг
// полный код и init-версия; проверяем error/truncated/
import { PAGE_SYNC } from "../../src/data/vizSync";
```

```

// query по дереву с обещаниями: pending накапливается
function q(step: { treeState: Record<number, number>; lazyState: Record<number, number>; pending: number; l: number, r: number, ql: number, qr: number; }) {
 if (qr < l || r < ql) return 0;
 // Собственный lazy[v] уже учтён в tree[v] (обещание)
 // добавляем только накопленные обещания предков (pending)
 if (ql <= l && r <= qr) return (step.treeState[v] ?? 0) + step.pending;
 const m = (l + r) >> 1;
 const d = pending + (step.lazyState[v] ?? 0);
 return q(step, 2 * v, l, m, ql, qr, d) + q(step, 2 * v + 1, m, r, ql, qr, d);
}

let fails = 0;
for (const [uL, uR, add] of cases) {
 const steps = genLazySteps(base, uL, uR, add);
 const last = steps[steps.length - 1];
 const wantRoot = sum(0, 5) + add * (uR - uL + 1);
 const wantQ = sum(uL, uR) + add * (uR - uL + 1);
 const gotRoot = last.treeState[1];
 const gotQ = q(last, 1, 0, 5, uL, uR, 0);
 const okR = gotRoot === wantRoot, okQ = gotQ === wantQ;
 console.log(`(uL,uR,add)=(${uL},${uR},${add}) root=${gotRoot} want=${wantRoot} q=${gotQ} wantQ=${wantQ} ${okR ? 'OK' : 'FAIL'} | шагов=${steps.length}`);
 if (!okR || !okQ) fails++;
}

// query без обновлений = обычная сумма
const q0 = q({ treeState: buildTreeFull(base), lazyState: {}, pending: 0, l: 0, r: 5, ql: 1, qr: 3 });
console.log("query(1,3) без обновлений =", q0, q0 === 23);
if (q0 !== 23) fails++;
console.log(fails === 0 ? "LAZY SELFTEST: ALL OK" : `LAZY SELFTEST: FAILS=${fails}`);
process.exit(fails === 0 ? 0 : 1);

```



```
uses: actions/checkout@v6
```

- name: Setup Node.js  
uses: actions/setup-node@v7  
with:  
 node-version: 24  
 cache: npm
- name: Install dependencies  
run: npm ci --no-audit --no-fund
- name: Build for the repository subpath  
env:  
 VITE\_BASE: /algv0/  
run: npm run build
- name: Make GitHub Pages fallback  
run: cp dist/index.html dist/404.html
- name: Upload Pages artifact  
uses: actions/upload-pages-artifact@v5  
with:  
 path: ./dist

deploy:

```
name: Publish to GitHub Pages
needs: build
runs-on: ubuntu-latest
environment:
 name: github-pages
 url: ${ steps.deployment.outputs.page_url }
```

steps:

- # Ранняя и понятная ошибка вместо невнятного «Fail»
- # когда в репозитории Pages не переключён на сбор
- name: Preflight – Pages собирается через GitHub  
env:  
 GH\_TOKEN: \${ secrets.GITHUB\_TOKEN }  
run: |

```
 paths-ignore:
 # чтобы коммит самого workflow не запускал бесконечный цикл
 - "public/export/**"
 - "**/*.md"
workflow_dispatch:
 inputs:
 density:
 description: "DPI растеризации страниц (по умолчанию 150)"
 required: false
 default: "150"

concurrency:
 group: rebuild-pdf-${{ github.ref }}
 cancel-in-progress: true

permissions:
 contents: write

jobs:
 rebuild-pdf:
 name: код → txt → png → pdf
 runs-on: ubuntu-latest
 # не реагируем на собственный коммит бота
 if: "!contains(github.event.head_commit.message, '[bot]')"

 steps:
 - name: Checkout
 uses: actions/checkout@v6
 with:
 fetch-depth: 0
 persist-credentials: true

 - name: Setup Node.js
 uses: actions/setup-node@v7
 with:
 node-version: "24"
 cache: npm
```

```
 echo ""
 echo "| Артефакт | Размер |"
 echo "| --- | --- |"
 echo "| \`public/export/full_code_all_pages"
 echo "| \`public/export/full_code_all_pages"
 echo "| \`public/export/guide_all_topics.pdf"
 echo "Страниц в конспекте: $(node -e "conso"
 echo "| PNG-страниц | $(ls tmp/export-pages."
 } >> "$GITHUB_STEP_SUMMARY"

- name: Upload artifacts (PDF + TXT)
 uses: actions/upload-artifact@v7
 with:
 name: full-code-export
 path: |
 public/export/full_code_all_pages.pdf
 public/export/full_code_all_pages.txt
 public/export/guide_all_topics.pdf
 if-no-files-found: error
 retention-days: 30

- name: Upload artifacts (PNG-страницы)
 uses: actions/upload-artifact@v7
 with:
 name: full-code-pages-png
 path: tmp/export-pages/*.png
 if-no-files-found: error
 retention-days: 7

- name: Commit rebuilt export back to the branch
 run: |
 git config user.name "github-actions[bot]"
 git config user.email "41898282+github-action"
 git add -- public/export/full_code_all_pages.
 if git diff --cached --quiet; then
 echo "Экспорт не изменился — коммит не нужен"
 exit 0
 fi
```